



ĐẠI HỌC THÁI NGUYÊN
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ
TRUYỀN THÔNG

Chủ biên: TS. Nguyễn Văn Núi,
Đồng chủ biên: ThS. Trần Hải Thanh,
Thành viên: TS. Tô Hữu Nguyên,
Thành viên: ThS. Phạm Thị Thương

GIÁO TRÌNH
LẬP TRÌNH JAVA

Nhà xuất bản.....



ĐẠI HỌC THÁI NGUYÊN
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ
TRUYỀN THÔNG

Chủ biên: TS. Nguyễn Văn Núi,
Đồng chủ biên: ThS. Trần Hải Thanh,
Thành viên: TS. Tô Hữu Nguyên,
Thành viên: ThS. Phạm Thị Thương

GIÁO TRÌNH
LẬP TRÌNH JAVA

Nhà xuất bản.....
Thái nguyên - 2024

Giáo Trình Lập trình Java

TS. Nguyễn Văn Núi (Chủ biên)

ThS. Trần Hải Thanh (Đồng chủ biên)

TS. Tô Hữu Nguyên

ThS. Phạm Thị Thương

Nhà xuất Bản:

Năm Xuất bản:

IBSN:

MỤC LỤC

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ.....	7
DANH MỤC HÌNH ẢNH.....	8
DANH MỤC BẢNG BIỂU.....	10
LỜI NÓI ĐẦU.....	11
<i>Chương 1</i> TỔNG QUAN.....	12
1.1. GIỚI THIỆU PHƯƠNG PHÁP TIẾP CẬN TRONG LẬP TRÌNH.....	12
1.1.1. Lập trình tuyến tính.....	12
1.1.2. Lập trình hướng chức năng.....	13
1.1.3. Lập trình hướng đối tượng.....	16
1.2. CÁC KHÁI NIỆM CƠ BẢN CỦA LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG.....	16
1.2.1. Đối tượng.....	16
1.2.2. Lớp đối tượng.....	18
1.2.3. Trừu tượng hóa.....	19
1.2.4. Bao bọc và che giấu thông tin.....	20
1.2.5. Kế thừa.....	21
1.2.6. Đa hình.....	22
1.3. CÁCH TIẾP CẬN VÀ GIẢI QUYẾT BÀI TOÁN THEO PHƯƠNG PHÁP HƯỚNG ĐỐI TƯỢNG.....	23
1.3.1. Mô tả bài toán.....	23
1.3.2. Đặc tả yêu cầu.....	24
1.3.3. Trích chọn đối tượng.....	24
1.3.4. Mô hình hoá lớp đối tượng.....	25
1.3.5. Thiết kế tổng quát.....	26
1.3.6. Thiết kế chi tiết.....	26
1.4. MỘT SỐ NGÔN NGỮ LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG.....	31
1.5. GIỚI THIỆU MỘT SỐ EDITOR BIÊN SOẠN CHƯƠNG TRÌNH JAVA.....	32
1.5.1. Eclipse.....	32
1.5.2. NetBeans.....	35
1.5.3. IntelliJ IDEA.....	37
1.6. TỔNG KẾT CHƯƠNG.....	39

1.7. CÂU HỎI TRẮC NGHIỆM	40
<i>Chương 2</i> GIỚI THIỆU NGÔN NGỮ LẬP TRÌNH JAVA	43
2.1. GIỚI THIỆU CHUNG	43
2.1.1. Lịch sử hình thành và phát triển của Java.....	43
2.1.2. Ngõ cản các ứng dụng của ngôn ngữ Java	43
2.2. MÔI TRƯỜNG LẬP TRÌNH JAVA	45
2.2.1. Máy ảo Java (JVM - Java Virtual Machine).....	45
2.2.2. Bộ công cụ phát triển JDK.....	47
2.2.3. Java Runtime Environment (JRE)	48
2.2.4. Java Integrated Development Environment (IDE)	50
2.2.5. Java Enterprise Edition (Java EE)	51
2.2.6. Java Database Connectivity (JDBC).....	53
2.3. CẤU TRÚC CỦA MỘT CHƯƠNG TRÌNH JAVA	54
2.4. BIÊN DỊCH VÀ THÔNG DỊCH CHƯƠNG TRÌNH JAVA	55
2.5. CÁC DẠNG CHƯƠNG TRÌNH ỨNG DỤNG CỦA JAVA	56
2.5.1. Phân loại theo cách vận hành, dịch và chạy chương trình.....	56
2.5.2. Phân loại theo dạng chương trình ứng dụng, mục đích sử dụng	58
2.8. TỔNG KẾT CHƯƠNG.....	60
2.9. CÂU HỎI TRẮC NGHIỆM	61
2.10. BÀI TẬP THỰC HÀNH.....	65
<i>Chương 3</i> CÁC THÀNH PHẦN CƠ SỞ CỦA JAVA.....	67
3.1. CÁC PHẦN TỬ CƠ SỞ CỦA JAVA.....	67
3.2. CÁC KIỂU DỮ LIỆU TRONG JAVA.....	68
3.3. BIẾN, HẰNG, BIỂU THỨC VÀ CÁC PHÉP TOÁN CƠ BẢN.....	69
3.3.1. Biến (variable)	69
3.3.2. Hằng (constant).....	71
3.3.3. Các phép toán cơ bản trong Java	71
3.4. CÁC CÂU LỆNH NHẬP XUẤT DỮ LIỆU CƠ BẢN QUA CÁC THIẾT BỊ VÀO RA CHUẨN	74
3.4.1. Nhập dữ liệu từ bàn phím	74
3.4.2. Xuất dữ liệu ra màn hình	74

3.5. CÁC CẤU TRÚC ĐIỀU KHIỂN CHƯƠNG TRÌNH TRONG JAVA.....	76
3.5.1. Câu lệnh rẽ nhánh	76
3.5.2. Các câu lệnh lặp.....	79
3.5.3. Các lệnh chuyển vị.....	83
3.6. CẤU TRÚC MẢNG, XÂU KÝ TỰ TRONG JAVA	86
3.6.1. Cấu trúc mảng.....	86
3.6.2. Xâu ký tự	90
3.7. NGOẠI LỆ VÀ XỬ LÝ NGOẠI LỆ TRONG JAVA	95
3.7.1. Cấu trúc phân cấp của các lớp xử lý ngoại lệ	96
3.7.2. Khối lệnh try, catch và finally	100
3.7.3. Ném ngoại lệ bằng lệnh ‘throw’	102
3.7.4. Mệnh đề throws	103
3.7.5. Tự định nghĩa ngoại lệ	104
3.7.6. Làn vết ngoại lệ StackTrace.....	106
3.8. CASE STUDY	107
3.9. VÍ DỤ ÁP DỤNG	109
3.10. TỔNG KẾT CHƯƠNG	112
3.11. CÂU HỎI TRẮC NGHIỆM	112
3.12. BÀI TẬP THỰC HÀNH.....	117
<i>Chương 4</i> LỚP ĐỐI TƯỢNG VÀ CÁC THÀNH PHẦN CỦA LỚP	118
4.1. LỚP ĐỐI TƯỢNG.....	118
4.1.1. Định nghĩa lớp đối tượng và các thành phần của lớp	118
4.1.2. Các thuộc tính dữ liệu của lớp	119
4.1.3. Định nghĩa hàm thành phần	119
4.1.4. Truyền tham số và lời gọi hàm	122
4.1.5. Toán tử tạo lập đối tượng.....	125
4.2. CÁC THUỘC TÍNH KIỂM SOÁT TRUY NHẬP CÁC THÀNH PHẦN CỦA LỚP	128
4.3. TẠO VÀ SỬ DỤNG ĐỐI TƯỢNG.....	130
4.4. CASE STUDY	132
4.5. VÍ DỤ ÁP DỤNG	133

4.6. TỔNG KẾT CHƯƠNG.....	138
4.7. CÂU HỎI TRẮC NGHIỆM	138
4.8. BÀI TẬP THỰC HÀNH.....	143
<i>Chương 5</i> GÓI CÁC LỚP VÀ CÁC LỚP CƠ SỞ TRONG GÓI JAVA.LANG	145
5.1. GÓI.....	145
5.1.1 Tạo gói cho chương trình và cách sử dụng gói.....	145
5.1.2 Giới thiệu một số gói quan trọng của Java	146
5.2. CÁC LỚP CƠ SỞ TRONG GÓI JAVA.LANG	147
5.2.1. Lớp Object	147
5.2.2. Các lớp nguyên thủy	149
5.2.3. Lớp Math	152
5.2.4. Lớp String	154
5.2.5. Lớp StringBuffer.....	157
5.3. CASE STUDY	160
5.4. VÍ DỤ ÁP DỤNG	162
5.5. TỔNG KẾT CHƯƠNG.....	164
5.6. CÂU HỎI TRẮC NGHIỆM	164
5.7. BÀI TẬP THỰC HÀNH.....	169
<i>Chương 6</i> KẾ THỪA.....	172
6.1. QUAN HỆ KẾ THỪA	172
6.2. THIẾT KẾ CÂY THỪA KẾ.....	174
6.3. VIẾT ĐỀ	176
6.4. QUAN HỆ “IS – A” VÀ “HAS – A”	178
6.4.1 Quan hệ “is – a”	178
6.4.2 Quan hệ “has – a”	179
6.5. CASE STUDY	181
6.6. Ví dụ áp dụng.....	181
6.7. TỔNG KẾT CHƯƠNG.....	185
6.8. CÂU HỎI TRẮC NGHIỆM	185
6.9. BÀI TẬP THỰC HÀNH.....	190
<i>Chương 7</i> ĐA HÌNH VÀ LỚP TRỪU TƯỢNG.....	192

7.1. ĐA HÌNH	192
7.1.1. Đa hình tĩnh	192
7.1.2. Đa hình động.....	193
7.1.3. Đặc điểm của đa hình.....	195
7.2. LỚP TRỪU TƯỢNG	196
7.2.1. Khái niệm.....	196
7.2.2. Lớp trừu tượng và lớp cụ thể	197
7.2.3. Phương thức trừu tượng.....	198
7.2.4. Lớp Object	200
7.3. CASE STUDY	201
7.4. VÍ DỤ ỨNG DỤNG	201
7.5. TỔNG KẾT CHƯƠNG.....	203
7.6. CÂU HỎI TRẮC NGHIỆM	204
7.7. BÀI TẬP THỰC HÀNH.....	208
<i>Chương 8</i> GIAO DIỆN VÀ CÁC LỚP TRONG	210
8.1. GIAO DIỆN (INTERFACES)	210
8.1.1. Định nghĩa.....	210
8.1.2. Các tính chất của giao diện	213
8.1.3. Sử dụng giao diện	213
8.2. LỚP TRONG (INNER CLASSES)	216
8.2.1. Định nghĩa.....	216
8.2.2. Tính chất của lớp nội	216
8.2.3. Phân loại lớp nội	216
8.3. CASE STUDY	220
8.4. VÍ DỤ ỨNG DỤNG	222
8.5. TỔNG KẾT CHƯƠNG.....	224
8.6. CÂU HỎI TRẮC NGHIỆM	225
8.7. BÀI TẬP THỰC HÀNH.....	230
<i>Chương 9</i> MỘT SỐ CHỦ ĐỀ NÂNG CAO	231
9.1. MỘT SỐ CẤU TRÚC DỮ LIỆU TRONG JAVA.....	231
9.1.1. ArrayList.....	231

9.1.2. LinkedList.....	233
9.1.3. HashMap.....	235
9.1.4. TreeSet.....	236
9.1.5. Stack và Queue	238
9.1.6. Lớp Collection	240
9.2. CÁC LUỒNG VÀO/RA DỮ LIỆU TRONG JAVA	244
9.2.1. Khái niệm.....	244
9.2.2. Input Streams	245
9.2.2. Output Streams	247
9.2.3. Character Streams	248
9.2.4. Object Streams.....	251
9.3. TỔ CHỨC MÃ NGUỒN THEO MÔ HÌNH HƯỚNG ĐỐI TƯỢNG	254
9.3.1. Nhóm các lớp.....	254
9.3.2. Áp dụng các nguyên tắc hướng đối tượng.....	257
9.3.3. Sử dụng JavaDoc	259
9.4. CASE STUDY	267
9.5. VÍ DỤ ÁP DỤNG	269
9.6. TỔNG KẾT CHƯƠNG.....	272
9.7. CÂU HỎI TRẮC NGHIỆM	273
9.8. BÀI TẬP THỰC HÀNH.....	278
TÀI LIỆU THAM KHẢO	281

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ

TT	Từ viết tắt	Ý nghĩa
1	API	Application Programming Interface - tập hợp các quy tắc hoặc giao thức cho phép các ứng dụng phần mềm giao tiếp với nhau
2	AWT	Abstract Window Toolkit - một bộ công cụ phát triển các ứng dụng dựa trên GUI
3	CLI	Command Line Interface - Giao diện dòng lệnh
4	FR	Functional Requirements - Yêu cầu chức năng
5	GUI	Graphical User Interface - Giao diện đồ họa người dùng
6	IDE	Integrated Development Environment - Môi trường phát triển tích hợp
7	Java EE	Java Enterprise Edition - Công cụ phát triển phần mềm doanh nghiệp
8	Java ME	Java Micro Edition - Công cụ phát triển các ứng dụng trên điện thoại di động với Java
9	JavaSE	Java Standard Edition - Công cụ cơ bản phát triển phần mềm Java
10	JDBC	Java Database Connectivity - API chuẩn trong Java được sử dụng để kết nối và thực hiện các truy vấn với cơ sở dữ liệu
11	JDK	Java Develop Kit - Bộ công cụ phát triển ứng dụng Java
12	JRE	Java Runtime Environment - Môi trường thực thi cho các ứng dụng Java
13	JVM	Java Virtual Machine - Máy ảo Java
14	ODBC	Open Database Connectivity - giao thức tiêu chuẩn để kết nối và truy xuất dữ liệu từ các cơ sở dữ liệu
15	OOP	Object-oriented programming - Lập trình hướng đối tượng
16	PP	Procedural Programming - Lập trình thủ tục
17	SWT	Standard Widget Toolkit - một bộ công cụ đồ họa cho Java

DANH MỤC HÌNH ẢNH

Hình 1.1	Cấu trúc của chương trình theo hướng chức năng.....	13
Hình 1.2	Sơ đồ chức năng của Hệ thống quản lý thư viện.....	14
Hình 1.3	Quan hệ giữa dữ liệu và hàm trong lập trình hướng chức năng	15
Hình 1.4	Tập các đối tượng của hệ thống quản lý thư viện.....	16
Hình 1.5	Lớp đối tượng.....	19
Hình 1.6	Trừu tượng hoá.....	19
Hình 1.7	Sử dụng trừu tượng hoá để xây dựng lớp.....	20
Hình 1.8	Ví dụ kế thừa	22
Hình 1.9	Đa hình	22
Hình 1.10	Ví dụ đa hình	23
Hình 1.11	Tải Eclipse.....	33
Hình 1.12	Lựa chọn cài đặt Eclipse.....	34
Hình 1.13	Giao diện mở đầu của Eclipse	34
Hình 1.14	NetBeans.....	35
Hình 1.15	Cài đặt NetBeans	37
Hình 1.16	IntelliJ IDEA	37
Hình 1.17	Sử dụng IntelliJ IDEA	39
Hình 2.1	Chương trình java chạy trên mọi hệ điều hành	44
Hình 2.2	Kiến trúc máy ảo Java	46
Hình 2.3	Java Runtime Environment	48
Hình 2.4	Integrated Development Environment	50
Hình 2.5	Kiến trúc JDBC	53
Hình 2.6	Chạy và biên dịch chương trình java.....	56
Hình 3.1	Các kiểu dữ liệu nguyên thủy trong Java	69
Hình 3.2	Giá trị mặc định của các kiểu dữ liệu nguyên thủy	69
Hình 3.3	Mở rộng kiểu	73
Hình 3.4	Câu lệnh if đơn giản	76
Hình 3.5	Câu lệnh if ... else	77
Hình 3.6	Câu lệnh switch	78
Hình 3.7	Mảng.....	86
Hình 3.8	Ngoại lệ	97
Hình 3.9	Quá trình xử lý ngoại lệ.....	99

Hình 3.10 Khối Try catch.....	100
Hình 3.11 Hoạt động của khối Try catch	101
Hình 4.1 Truyền các giá trị tham chiếu đối tượng.....	124
Hình 5.1 Lớp object.....	147
Hình 6.1 Cây kế thừa.....	175
Hình 6.2 Xây dựng cây kế thừa.....	176
Hình 6.3 Quan hệ “is – a”, hay quan hệ kế thừa	178
Hình 6.4 Ví dụ minh họa quan hệ “Association”	180
Hình 6.5 Quan hệ “Aggregation”	180
Hình 6.6 Ví dụ về quan hệ hợp thành (composition)	181
Hình 9.1 OutputStream và InputStream	245
Hình 9.2 InputStream Hierarchy	245
Hình 9.3 OutputStream Hierarchy.....	247
Hình 9.4 Character Streams Hierarchy.....	249
Hình 9.5 Ví dụ minh họa về sử dụng JavaDoc.....	264
Hình 9.6 Tạo JavaDoc với Eclipse IDE	265
Hình 9.7 Tạo JavaDoc với IntelliJ IDE	266

DANH MỤC BẢNG BIỂU

Bảng 1.1 Bảng đặc tả yêu cầu của bài toán quản lý thư viện	24
Bảng 3.1 Các từ khóa của Java.....	68
Bảng 3.2 Các kí tự định dạng xuất dữ liệu	68
Bảng 3.3 Toán tử số học.....	71
Bảng 3.4 Toán tử quan hệ & logic	72
Bảng 3.5 Toán tử gán	72
Bảng 3.6 Dạng câu lệnh return	85
Bảng 3.7 So sánh for với foreach	89
Bảng 3.8 Ngoại lệ.....	98
Bảng 3.9 So sánh sự khác nhau giữa từ khóa throw và throws.....	104
Bảng 4.1 Thuộc tính kiểm soát truy nhập các thành phần của lớp.....	128
Bảng 5.1 Các phương thức quan trọng của lớp Object	148
Bảng 5.2 Một số phương thức tĩnh trong lớp Math.....	153
Bảng 6.1 Một số ví dụ về lớp cơ sở và lớp dẫn xuất.....	172
Bảng 9.1 Các phương thức quan trọng của ArrayList.....	231
Bảng 9.2 Các phương thức quan trọng của LinkedList.....	233
Bảng 9.3 Các phương thức quan trọng của HashMap.....	235
Bảng 9.4 Các phương thức chính của TreeSet	237
Bảng 9.5 Các phương thức chính của Stack.....	238
Bảng 9.6 Các phương thức chính của Queue	239
Bảng 9.7 Các phương thức quan trọng trong Collection Framework	241
Bảng 9.8 Các thẻ JavaDoc phổ biến.....	261

LỜI NÓI ĐẦU

Giáo trình “*Lập trình Java*” được biên soạn bởi tập thể giảng viên thuộc Khoa Công nghệ thông tin - Trường Đại học Công nghệ thông tin và Truyền thông, nhằm phục vụ việc giảng dạy của giảng viên và học tập của sinh viên trong Trường. Nội dung giáo trình bám sát nội dung đề cương chi tiết của học phần “*Lập trình hướng đối tượng*” đào tạo ở trình độ đại học. Ngoài ra, giáo trình cũng cung cấp thêm các kiến thức mở rộng và nâng cao liên quan khác.

Nội dung giáo trình này tập trung vào việc trang bị những kiến thức cơ bản về lập trình hướng đối tượng, áp dụng với ngôn ngữ lập trình Java. Người đọc sẽ được trang bị các nguyên tắc cốt lõi, phương pháp giải quyết bài toán theo cách tiếp cận hướng đối tượng, cùng các bài thực hành thực tế để củng cố kiến thức. Đồng thời, giáo trình còn hướng dẫn cách khai thác và sử dụng các công cụ, công nghệ, và dịch vụ cơ bản của các IDE trong lập trình, từ đó giúp người học dễ dàng làm quen và áp dụng hiệu quả phương pháp hướng đối tượng trong lập trình. Những kiến thức trong giáo trình này sẽ là nền tảng quan trọng, hỗ trợ người học trên con đường trở thành kỹ sư lập trình chuyên nghiệp sau khi tốt nghiệp.

Nhằm đáp ứng mục tiêu đào tạo kết hợp sử dụng cách tiếp cận học và đánh giá theo quy trình khoa học, nội dung của cuốn giáo trình này được tổ chức thành một chuỗi gồm 9 chương, bám sát mục tiêu và các chuẩn đầu ra của học phần. Mỗi chương được thiết kế dưới dạng một bài học với cấu trúc tương đồng nhau, bao gồm các nội dung chính như sau:

- 1) Mục tiêu bài học;
- 2) Nội dung kiến thức trang bị nhằm đáp ứng mục tiêu bài học;
- 3) Tổng kết nội dung bài học;
- 4) Các câu hỏi củng cố, case study, ví dụ áp dụng hoặc bài tập thực hành.

Dù đã nỗ lực dành nhiều thời gian và tâm huyết để biên soạn, giáo trình chắc chắn khó tránh khỏi những thiếu sót nhất định. Chúng tôi rất mong nhận được những ý kiến đóng góp quý báu từ Quý Thầy/Cô và bạn đọc để tài liệu này ngày càng hoàn thiện hơn. Mọi ý kiến đóng góp xin gửi về địa chỉ email: nvnui@ictu.edu.vn. Xin chân thành cảm ơn!

Thái Nguyên, năm 2024

Đại diện nhóm tác giả

TS. Nguyễn Văn Núi

Chương 1 TỔNG QUAN

Chương đầu giới thiệu:

- Những đặc trưng cơ bản của hai cách tiếp cận trong lập trình: lập trình hướng chức năng và lập trình hướng đối tượng;
- Những khái niệm, thành phần cơ bản của phương pháp hướng đối tượng;
- Ngôn ngữ lập trình hướng đối tượng.

1.1. GIỚI THIỆU PHƯƠNG PHÁP TIẾP CẬN TRONG LẬP TRÌNH

Phương pháp lập trình truyền thống chúng ta vẫn áp dụng đó là *lập trình có cấu trúc*. Phương pháp lập trình này thực hiện theo cách tiếp cận *hướng chức năng dựa chủ yếu vào việc phân tách các chức năng chính* của bài toán thành những chức năng đơn giản hơn và thực hiện làm mịn dần từ trên xuống (*top-down*). Theo cách tiếp cận hướng chức năng, chương trình được xem như là một tập các hàm chức năng, trong đó dữ liệu và các hàm là tách rời nhau. Đối với những hệ thống lớn, phức hợp, độ phức tạp của chương trình tăng lên, sự phụ thuộc của nó vào các kiểu dữ mà nó xử lý cũng tăng theo. Các kiểu dữ liệu được xử lý trong nhiều chức năng bên trong chương trình có cấu trúc, và khi có sự thay đổi về kiểu dữ liệu thì cũng phải thực hiện thay đổi ở mọi nơi mà dữ liệu đó được sử dụng. Một nhược điểm nữa của lập trình hướng chức năng là khi có nhiều người tham gia xây dựng chương trình, mỗi người được giao viết một số chức năng (hàm) riêng biệt nhưng lại phải sử dụng chung dữ liệu. Khi có nhu cầu cần thay đổi dữ liệu sẽ ảnh hưởng rất lớn đến công việc của nhiều người, v.v.

Phương pháp lập trình mà ngày nay chúng ta nghe nói tới nhiều đó là *lập trình hướng đối tượng*. Lập trình hướng đối tượng dựa trên nền tảng là các *đối tượng*. Đối tượng (thực thể) được xây dựng trên cơ sở gắn dữ liệu với các phép toán sẽ thể hiện được đúng cách mà chúng ta suy nghĩ, bao quát về chúng trong thế giới thực. Chẳng hạn, ô tô có bánh xe, di chuyển được và hướng của nó thay đổi được bằng cách thay đổi tay lái. Tương tự, cây là loại thực vật có thân gỗ và lá. Cây không phải là ô tô và những gì thực hiện được với ô tô sẽ không làm được với cây.

Lập trình hướng đối tượng cho phép chúng ta kết hợp những tri thức bao quát về các quá trình thực tế với những khái niệm trừu tượng được sử dụng trong máy tính. Chương trình hay hệ thống hướng đối tượng được xem như là tập các lớp đối tượng tương tác với nhau để thực hiện các yêu cầu của bài toán đặt ra.

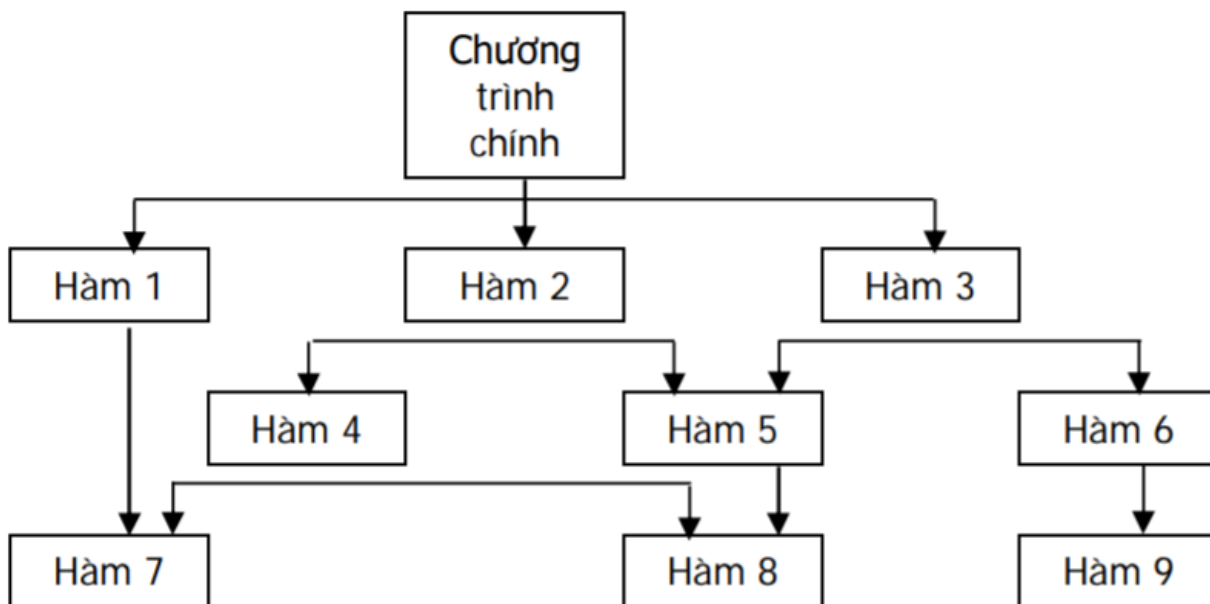
1.1.1. Lập trình tuyến tính

- Lập trình tuyến tính có 2 đặc trưng cơ bản sau:
 - + Đơn giản: chương trình được tiến hành đơn giản theo lối tuần tự, không phức tạp.
 - + Đơn luồng: Chỉ có một luồng công việc duy nhất, các công việc được thực hiện tuần tự trong các luồng đó.
- Ưu điểm: Chương trình đơn giản, dễ hiểu.
- Nhược điểm: Không thể dùng lập trình tuyến tính để giải quyết các bài toán phức tạp.

Ngày nay, lập trình tuyến tính chỉ tồn tại trong phạm vi các module nhỏ nhất của các phương pháp lập trình khác.

1.1.2. Lập trình hướng chức năng

Theo cách tiếp cận hướng chức năng thì một hệ thống phần mềm được xem như là tập các công việc (chức năng, nhiệm vụ) cần tổ chức thực thi; mỗi công việc đó sẽ được thực hiện bởi một số hàm nhất định. Như vậy *trọng tâm của cách tiếp cận này là các hàm chức năng*. Cấu trúc của chương trình được xây dựng theo cách tiếp cận hướng chức năng có dạng như **Hình 1.1** dưới đây.



Hình 1.1 Cấu trúc của chương trình theo hướng chức năng

Cách tiếp cận theo hướng thủ tục dựa vào chức năng, nhiệm vụ là chính và phân rã chức năng làm mịn dần theo cách từ trên xuống (Top/Down). Các đơn thể chức năng trao đổi với nhau bằng cách truyền tham số hay sử dụng dữ liệu chung.

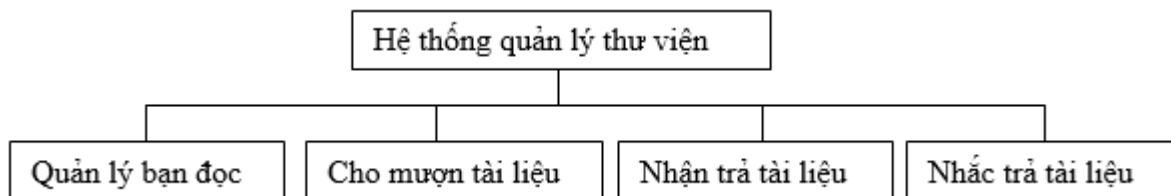
- **Dựa vào chức năng nhiệm vụ:** Khi khảo sát, phân tích một hệ thống chúng ta thường tập trung vào các nhiệm vụ mà nó cần thực hiện. Chúng ta tập trung trước hết nghiên cứu các yêu cầu của bài toán để xác định các chức năng chính của hệ thống.

Ví dụ khi cần xây dựng “hệ thống quản lý thư viện” thì trước hết chúng ta thường đi nghiên cứu, khảo sát trao đổi và phỏng vấn xem những người thủ thư, bạn đọc cần phải thực hiện những công việc gì để phục vụ được bạn đọc và quản lý tốt được các tài liệu. Qua nghiên cứu “hệ thống quản lý thư viện”, chúng ta xác định được các nhiệm vụ chính của hệ thống như: quản lý bạn đọc, cho mượn sách, nhận trả sách, thông báo nhắc trả sách, ...

- **Phân rã chức năng làm mịn từ trên xuống:** Khả năng của con người là có giới hạn khi khảo sát, nghiên cứu để hiểu và thực thi những gì mà hệ thống thực tế đòi hỏi. Để thống trị (quản lý được) độ phức tạp của những vấn đề phức tạp trong thực tế thường chúng ta phải sử dụng nguyên lý chia để trị (divide and conquer), nghĩa là phân tách nhỏ các chức năng chính thành các chức năng đơn giản hơn theo cách từ trên xuống. Quy trình này được lặp lại cho đến khi thu được những đơn thể chức năng tương đối đơn giản, hiểu được và thực hiện cài đặt chúng mà không làm tăng thêm độ phức tạp để liên kết chúng trong hệ thống.

Độ phức tạp liên kết các thành phần chức năng của hệ thống thường là tỉ lệ nghịch với độ phức tạp của các đơn thể. Vì thế một vấn đề đặt ra là có cách nào để biết khi nào quá trình phân tách các đơn thể chức năng hay còn gọi là quá trình làm mịn dần này kết thúc. Thông thường thì quá trình thực hiện phân rã các chức năng của hệ thống phụ thuộc nhiều vào độ phức hợp của bài toán ứng dụng và vào trình độ của những người tham gia phát triển phần mềm. Một hệ thống được phân tích dựa trên các chức năng hoặc quá trình sẽ được chia thành các hệ thống con và tạo ra cấu trúc phân cấp các chức năng.

Ví dụ 1: hệ thống quản lý thư viện có thể phân chia từ trên xuống như sau:

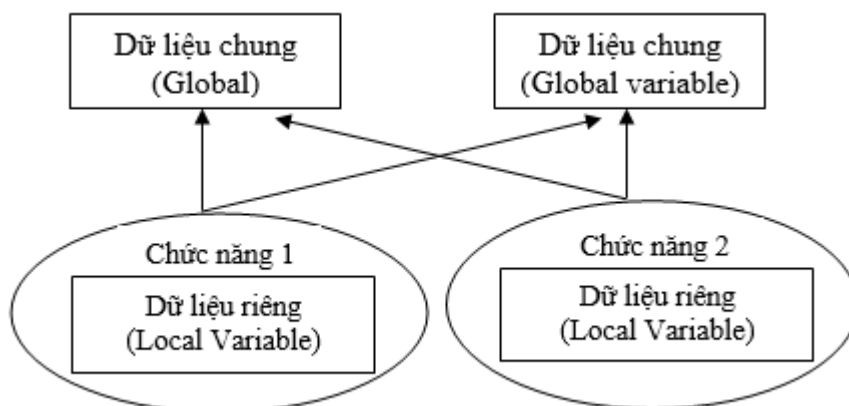


Hình 1.2 Sơ đồ chức năng của Hệ thống quản lý thư viện

Các chức năng của hầu hết các hệ thống thông tin quản lý đều có thể tổ chức thành sơ đồ chức năng theo cấu trúc phân cấp có thứ bậc.

- **Các đơn thể chức năng trao đổi với nhau bằng cách truyền tham số hay sử dụng dữ liệu chung:** Một hệ thống phần mềm bao giờ cũng phải được xem như là một thể thống nhất, do đó các đơn thể chức năng phải có quan hệ trao đổi thông tin, dữ liệu với nhau. Trong một chương trình gồm nhiều hàm (thực hiện nhiều chức năng khác nhau) muốn trao đổi dữ liệu được với nhau thì nhất thiết phải sử dụng dữ liệu chung hoặc liên kết với nhau bằng cách truyền tham biến. Mỗi đơn thể chức năng không những chỉ thao tác, xử lý trên những dữ liệu cục bộ (Local Variables) mà còn phải sử dụng các biến chung, thường đó là các biến toàn cục (Global Variable).

Mối quan hệ giữa dữ liệu và hàm trong chương trình hướng chức năng được mô tả trong dưới đây.



Hình 1.3 Quan hệ giữa dữ liệu và hàm trong lập trình hướng chức năng

Nhận xét:

- Hệ thống được xây dựng dựa vào chức năng là chính mà trong thực tế thì chức năng, nhiệm vụ của hệ thống lại hay thay đổi. Để đảm bảo cho hệ thống thực hiện được công việc theo yêu cầu, nhất là những yêu cầu về mặt chức năng đó lại bị thay đổi là công việc phức tạp và rất tốn kém.

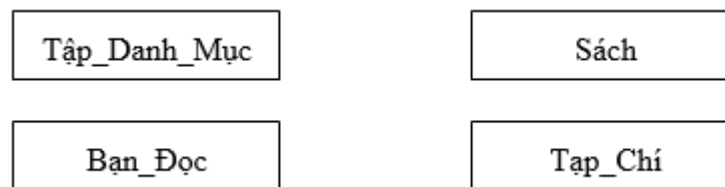
Ví dụ: giám đốc thư viện yêu cầu thay đổi cách quản lý bạn đọc hoặc hơn nữa, yêu cầu bổ sung chức năng theo dõi những tài liệu mới mà bạn đọc thường xuyên yêu cầu để đặt mua, v.v. Khi đó vấn đề bảo trì hệ thống phần mềm không phải là vấn đề dễ thực hiện. Nhiều khi có những yêu cầu thay đổi cơ bản mà việc sửa đổi không hiệu quả và vì thế đòi hỏi phải phân tích, thiết kế lại hệ thống thì hiệu quả hơn.

- Các bộ phận của hệ thống phải sử dụng biến toàn cục để trao đổi với nhau, do vậy khả năng thay đổi, mở rộng của chúng và của cả hệ thống là bị hạn chế. Như trên đã phân tích, những thay đổi liên quan đến các dữ liệu chung sẽ ảnh hưởng tới các bộ phận liên quan. Do đó, một thiết kế tốt phải rõ ràng, dễ hiểu và mọi sửa đổi chỉ có hiệu ứng cục bộ.
- Khả năng tái sử dụng (Reuse) bị hạn chế và không hỗ trợ cơ chế kế thừa (Inheritance). Để có độ thích nghi cao thì mỗi thành phần phải là tự chứa. Muốn là tự chứa hoàn toàn thì một thành phần không nên dùng các thành phần ngoại lai. Tuy nhiên, điều này lại mâu thuẫn với kinh nghiệm nói rằng các thành phần hiện có nên là dùng lại được. Vậy là cần có một sự cân bằng giữa tính ưu việt của sự dùng lại các thành phần (ở đây chủ yếu là các hàm) và sự mất mát tính thích ứng được của chúng. Các thành của hệ thống phải kết dính (Cohesion) nhưng phải tương đối lỏng để dễ thích nghi. Một trong cơ chế chính hỗ trợ để dễ có được tính thích nghi là kế thừa thì cách tiếp cận hướng chức năng lại không hỗ trợ. Đó là cơ chế biểu diễn tính tương tự của các thực thể, đơn giản hoá định nghĩa những khái niệm tương tự từ những sự vật đã được định nghĩa trước trên cơ sở bổ sung hay thay đổi một số các đặc trưng hay tính chất của chúng. Cơ chế này giúp chúng ta thực hiện được nguyên lý tổng quát hoá và chi tiết hoá các thành phần của hệ thống phần mềm.

1.1.3. Lập trình hướng đối tượng

Dựa trên nền tảng là các đối tượng: Đặt trọng tâm vào dữ liệu, xem hệ thống là tập các thực thể và tập các đối tượng.

- Đặt trọng tâm vào dữ liệu (thực thể). Khi khảo sát, phân tích một hệ thống chúng ta tìm hiểu xem nó gồm những thực thể nào. Thực thể hay còn gọi là đối tượng, là những gì như người, vật, sự kiện, v.v. mà chúng ta đang quan tâm, hay cần phải xử lý. Ví dụ, khi xây dựng “Hệ thống quản lý thư viện” thì trước hết chúng ta tìm hiểu xem nó gồm những lớp đối tượng hoặc những khái niệm nào.
- Xem hệ thống như là tập các thực thể, các đối tượng. Để hiểu rõ về hệ thống, chúng ta phân tách hệ thống thành các đơn thể đơn giản hơn. Quá trình này được lặp lại cho đến khi thu được những đơn thể tương đối đơn giản, dễ hiểu và thực hiện cài đặt chúng mà không làm tăng thêm độ phức tạp khi liên kết chúng trong hệ thống. Xét “Hệ thống quản lý thư viện”, chúng ta có các lớp đối tượng sau:



Hình 1.4 Tập các đối tượng của hệ thống quản lý thư viện

- Các lớp đối tượng trao đổi với nhau bằng các thông điệp (Message). Theo nghĩa thông thường thì lớp (Class) là nhóm một số người, vật có những đặc tính tương tự nhau hoặc có những hành vi ứng xử giống nhau. Trong mô hình đối tượng, khái niệm lớp là cấu trúc mô tả hợp nhất các thuộc tính (Attributes), hay dữ liệu thành phần (Data Member) thể hiện các đặc tính của mỗi đối tượng và các phương thức (Methods), hay hàm thành phần (Member Function) thao tác trên các dữ liệu riêng và là giao diện trao đổi với các đối tượng khác để xác định hành vi của chúng trong hệ thống. Khi có yêu cầu dữ liệu để thực hiện một nhiệm vụ nào đó, một đối tượng sẽ gửi một thông điệp (gọi một phương thức) cho đối tượng khác. Đối tượng nhận được thông điệp yêu cầu sẽ phải thực hiện một số công việc trên các dữ liệu mà nó sẵn có hoặc lại tiếp tục yêu cầu những đối tượng khác hỗ trợ để có những thông tin trả lời cho đối tượng yêu cầu. Với phương thức xử lý như thế thì một chương trình hướng đối tượng thực sự có thể không cần sử dụng biến toàn cục nữa.

1.2. CÁC KHÁI NIỆM CƠ BẢN CỦA LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

1.2.1. Đối tượng

- Đối tượng là thực thể của hệ thống được xác định thông qua định danh (tên gọi).
- Mỗi đối tượng bao gồm dữ liệu thành phần hay các thuộc tính mô tả các tính chất và phương thức thao tác trên dữ liệu để xác định hành vi của đối tượng.

Đối tượng = Thuộc tính (dữ liệu) + phương thức (hàm).

- Theo quan điểm của người lập trình đối tượng được xem như vùng nhớ được phân chia trong máy tính để lưu dữ liệu và tập các hàm tác động lên dữ liệu gắn với chúng.

Ví dụ 1: Quyển sách = thông tin về sách (khổ sách, độ dày, nội dung...) + Đọc (mở sách, lật trang, đọc 1 đoạn, đọc một trang, tìm mục lục...)

(đối tượng) = Thuộc tính + phương thức

Ta có thể đọc sách dưới ánh đèn.

Đèn = Thông tin trạng thái (sáng, tắt) + bật, tắt đèn.

Quyển sách chứa nội dung thông tin, nó cũng bao gồm các thông tin về trạng thái (sách đang mở, đóng) và có thể chứa các đối tượng nhỏ hơn (các trang sách) . Phương thức của sách cho phép truy cập đến trang và phương thức của trang truy cập đến thông tin trong trang đó.

- Đối tượng có thể là một thực thể trực quan (người, sự vật) hay là một khái niệm (phòng ban, bộ phận, đăng ký...).

Ta sẽ gọi tập hợp giá trị các thuộc tính của một đối tượng cụ thể tại một thời điểm nào đó là trạng thái (state) của nó.

- Một hành vi là cái mà đối tượng có thể thực hiện, và sẽ bằng cách nào đó tác động tới một hoặc nhiều thuộc tính của đối tượng (do đó ảnh hưởng đến trạng thái của đối tượng). *Xét một người có một thuộc tính tư thế - một trong các giá trị {đứng, ngồi, nằm, quỳ}” Một hành vi “đứng” có thể dẫn đến kết quả là người đó đứng dậy (nếu người đó đang không ở tư thế đứng)*
- Trao đổi thông điệp: Các đối tượng liên lạc với nhau bằng các thông điệp (message). Một thông điệp là một yêu cầu một đối tượng thực hiện một hành vi.

Ví dụ 2: nếu ta muốn một người nào đó đứng dậy, ta có thể gửi một thông điệp đến cho người đó để đề nghị người đó đứng dậy. Đối với con người, phần lớn việc “truyền thông điệp” (messagepassing) của ta được thực hiện qua đường tiếng nói. Ta sẽ thấy rằng cơ chế “truyền thông điệp” đó xảy ra trong lập trình hướng đối tượng qua các lời gọi hàm.

- Trong mô hình hướng đối tượng: các đối tượng tương tác với nhau để thực hiện nhiệm vụ.

Phân loại các đối tượng : Khả năng phân loại bẩm sinh của con người, khả năng trừu tượng hoá này cho phép ta đơn giản hoá và tổ chức cuộc sống của mình, và hiểu về một thế giới rất phức tạp. Có hai cách phân loại đối tượng mà ta hay dùng nhất: phân loại các đối tượng tương tự và phân loại các đối tượng theo thành phần.

- Phân loại các đối tượng tương tự: nhóm các đối tượng có cùng các thuộc tính và hành vi lại với nhau. Ví dụ, trong loại “ô tô”, ta có thể đưa vào đó các đối tượng có các thuộc tính màu sắc, nhãn hiệu, kiểu, và các hành vi lái, dừng, rẽ.

Lưu ý: phân loại theo "có hay không một thuộc tính", không phân loại theo giá trị của thuộc tính. Ở đây, ta không lập nhóm các xe ô tô có giá trị “đỏ” cho thuộc tính màu và giá trị “Honda” cho thuộc tính nhãn hiệu đối tượng chỉ cần có các thuộc tính đó là đủ để thuộc nhóm.

- Phân loại theo cấu tạo: Thuộc tính của một lớp nào đó có thể có giá trị là một đối tượng thuộc một lớp nào đó.

- Phân loại đối tượng theo cấu tạo là một trong các kiểu *quan hệ* có thể tồn tại giữa các đối tượng. Kiểu quan hệ này thường được gọi là quan hệ chứa “has-a”.

Mô hình hóa: Bước đầu tiên khi mô hình hoá một hệ thống gồm các lớp đối tượng, ta xác định xem:

- Có các lớp đối tượng nào?
- Chúng có các thuộc tính gì?
- Chúng có liên quan đến nhau như thế nào?
- Việc xác định hành vi của các lớp sẽ để sau

Ổn định tập thuộc tính cho mỗi lớp trước khi xác định các hành vi dùng để truy nhập các thuộc tính này.

Khi đã xác định được tất cả các lớp đối tượng cần thiết, các thuộc tính và quan hệ giữa chúng, bước tiếp theo là chỉ ra các hành vi của các lớp đối tượng đó.

- Có những hành vi nào?
- Các lớp đối tượng có thể cần những hành vi nào để có thể sử dụng các thuộc tính của mình?
- Các lớp đối tượng cần thực hiện những nhiệm vụ nào cho các đối tượng khác?

1.2.2. Lớp đối tượng

- Ta sẽ gọi một nhóm các đối tượng tương tự là một lớp đối tượng – object class. Một lớp là “một tập các đối tượng có cùng thuộc tính và hành vi”
- Mỗi đối tượng thuộc một lớp là một thể hiện (instance) của lớp đó. *Mỗi xe ô tô là một thể hiện của lớp ô tô.*
- Các thể hiện của một lớp phân biệt nhau bởi một số thuộc tính nhất định có giá trị duy nhất trong toàn lớp - định danh duy nhất “*định danh của sinh viên là mã sinh viên – không có hai sinh viên nào có mã giống nhau đôi khi không dễ tìm được cách phân biệt giữa các thể hiện*”
- Lớp định nghĩa kiểu dữ liệu và phương thức dùng để truy cập dữ liệu của đối tượng.

Ví dụ 3: *Lớp đơn xin việc = trường cần điền vào đơn + ghi/đọc đơn*

(đối tượng) = (dữ liệu) + (phương thức)

- Lớp mẫu đơn xin việc.

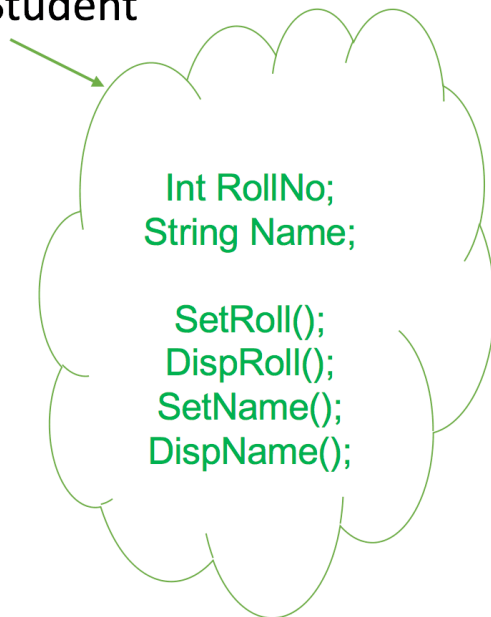
Những người đến xin việc họ phải được cung cấp một thực thể duy nhất của đơn, những thực thể tạo ra bằng cách lấy bản mẫu làm bản chính, sao chép tạo ra nhiều thực thể. Người xin việc phải điền các thông tin vào đơn bằng phương thức ghi đơn.

- Lớp được đặc tả bởi tên lớp: tập thuộc tính, tập các hàm.

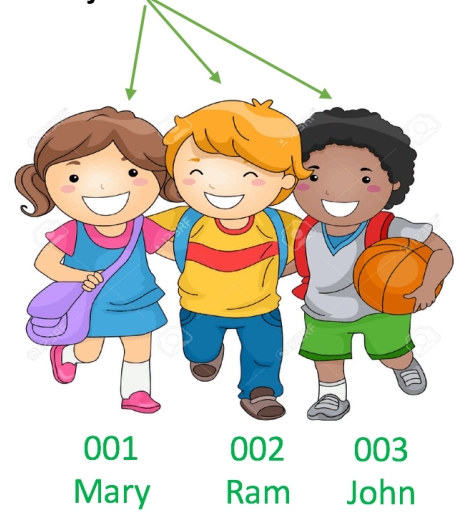
Ví dụ 4: *lớp Student có 2 thuộc tính, 4 hàm (phương thức)*

Đối tượng: *các đối tượng thuộc lớp Student nên nó mang đầy đủ thuộc tính và phương thức như lớp Student.*

Class Student



Object



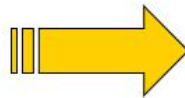
Hình 1.5 Lớp đối tượng

1.2.3. Trừu tượng hóa

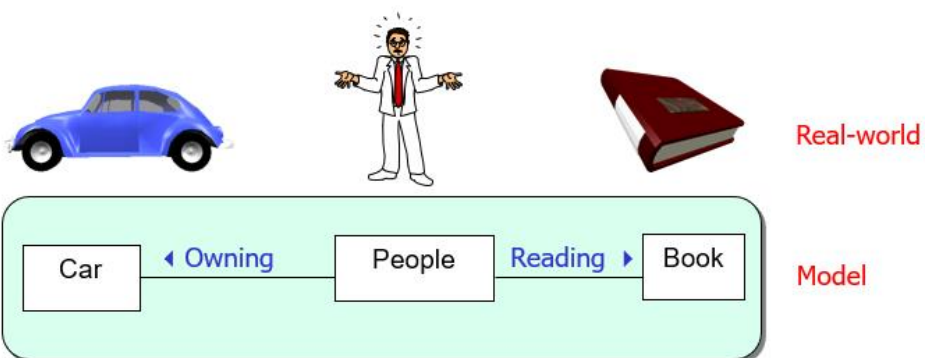
Trừu tượng hóa là cách biểu diễn đặc tính chính và bỏ qua những chi tiết.



Real-world

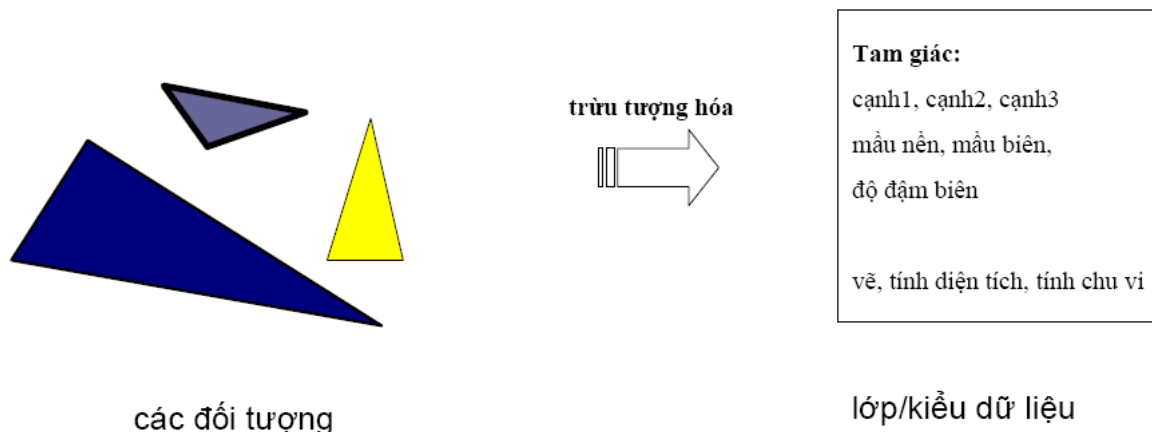


Model: globe for student/pupil



Hình 1.6 Trừu tượng hoá

Trừu tượng hóa là cách nhìn đơn giản hóa về một đối tượng mà trong đó chỉ bao gồm các đặc điểm được quan tâm và bỏ qua những chi tiết không cần thiết.



Hình 1.7 Sử dụng trừu tượng hoá để xây dựng lớp

- Trừu tượng hóa là sự mở rộng các khái niệm kiểu dữ liệu và cho phép định nghĩa những phép toán trừu tượng trên các kiểu dữ liệu trừu tượng.
- Để xác định lớp ta phải sử dụng khái niệm trừu tượng hóa.

1.2.4. Bao bọc và che giấu thông tin

- Đóng gói: Nhóm những gì có liên quan với nhau vào làm một, để sau này có thể dùng một cái tên để gọi đến: Các hàm/ thủ tục đóng gói các câu lệnh. Các đối tượng đóng gói dữ liệu của chúng và các thủ tục có liên quan.

Chi tiết về các thành viên của một đối tượng cần được đóng gói bên trong đối tượng, để cách duy nhất truy nhập đến chúng là qua các hành vi của đối tượng. Tương tự, chi tiết về cách thực hiện các hành vi của một đối tượng cũng nên được đóng gói để bên ngoài không biết chi tiết.

Tại sao đóng gói đối tượng? Đóng gói bắt chước thế giới thực: “*nếu tôi muốn anh đứng dậy (thay đổi thuộc tính “vị trí”) khả năng lớn là tôi sẽ đề nghị anh đứng dậy thay vì đến kéo anh dậy.*” điều này làm đơn giản hoá cuộc sống, ta thường không quan tâm đến chi tiết. Trong lập trình hướng đối tượng, lợi ích của việc đóng gói: khi che dấu chi tiết cài đặt, người dùng bên ngoài không phải bận tâm đến chuyện cái gì được làm như thế nào. Điều này đem lại lợi ích đáng kể trong việc bảo trì phần mềm - do người dùng không bao giờ biết chi tiết bên trong đối tượng, ta có thể thay đổi các chi tiết đó mà họ không cần biết (miễn là giao diện với bên ngoài không đổi)

- Xác định các vùng: riêng (private) công khai (public) hay bảo vệ (protected) nhằm điều khiển, hay hạn chế những truy nhập tùy tiện của những đối tượng khác.
- Việc đóng gói dữ liệu và hàm vào cùng một đơn vị cấu trúc (lớp) được xem là nguyên tắc bao bọc thông tin.
- Nguyên tắc bao bọc dữ liệu để cấm sự truy nhập trực tiếp gọi là sự che dấu thông tin.

Che dấu thông tin: đóng gói để che một số thông tin và chi tiết cài đặt nội bộ để bên ngoài không nhìn thấy. mục tiêu là để khách hàng của ta (thường là các lập trình viên khác) coi các đối tượng của ta là các hộp đen.

1.2.5. Kế thừa

- *Kế thừa*: Là nguyên lý kế thừa cho phép các đối tượng của lớp này được quyền sử dụng lại một số tính chất (thuộc tính và phương thức) của lớp khác.

Ví dụ 5: Nếu ta có một mô hình kế thừa giữa 2 lớp A và B; trong đó: A - lớp cha; B - lớp con.

→ Khi đó:

- + Lớp B sẽ kế thừa các thuộc tính và các hàm có khai báo public hoặc protected của lớp A.
- + Trong lớp B có thể bổ sung thêm một số tính chất, phương thức truy cập để thu hẹp phạm vi xác định đối tượng trong lớp mới.
- + *Những thuộc tính hay hàm được khai báo private thì không kế thừa lại được.*

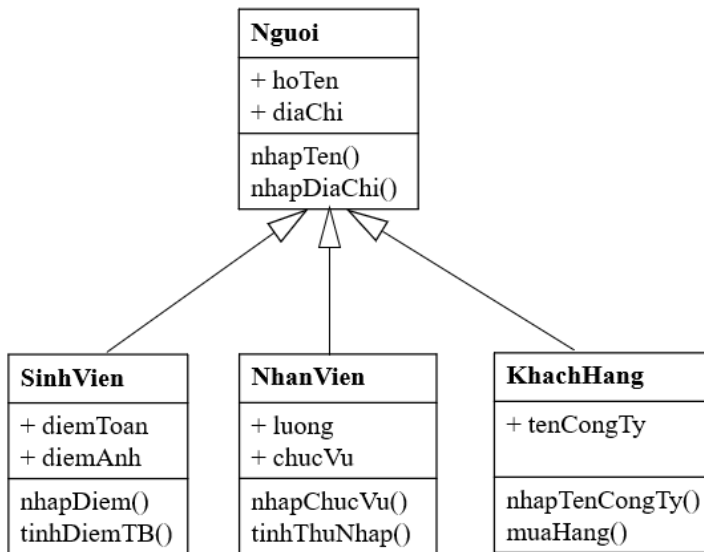
Lưu ý: Java chỉ hỗ trợ một cơ chế kế thừa (*giữa các lớp*) duy nhất, đó là kế thừa đơn, *không có kế thừa bội*. Để tận dụng lợi ích của kế thừa bội, Java xây dựng khái niệm và mở rộng kế thừa bội đối với các interface (*giao diện*).

Ví dụ 6: Trong một bài toán có 3 lớp cần quản lý: Sinh Viên, Nhân Viên, Khách Hàng. Rõ ràng rằng cả 3 lớp này đều có chung các thông tin của một người (họ tên, địa chỉ); vì vậy, để tối ưu hoá viết code và linh hoạt khi sử dụng sau này, chúng ta thường xây dựng mô hình kế thừa như sau:

- Lớp **Người** (*lớp cha*): Có các thuộc tính và phương thức/hàm như sau:
 - + Thuộc tính: *hoTen, diaChi*
 - + Phương thức: *nhapTen(), nhapDiaChi()*
- Lớp **SinhVien** (*lớp con*): Có các thuộc tính và phương thức/hàm như sau:
 - + Thuộc tính: *hoTen, diaChi, diemToan, diemAnh*
 - + Phương thức/hàm: *nhapTen(), nhapDiaChi(), nhapDiem(), tinhDiemTB()*
- Lớp **NhanVien** (*lớp con*): Có các thuộc tính và phương thức/hàm như sau:
 - + Thuộc tính: *hoTen, diaChi, luong, chucVu*
 - + Phương thức/hàm: *nhapTen(), nhapDiaChi(), nhapChucVu(), tinhThuNhap()*
- Lớp **KhachHang** (*lớp con*): Có các thuộc tính và phương thức/hàm như sau:
 - + Thuộc tính: *hoTen, diaChi, tenCongTy*
 - + Phương thức/hàm: *nhapTen(), nhapDiaChi(), nhapTenCongTy(), muaHang()*

→ Mô hình kế thừa và khung mã lệnh tương ứng sử dụng ngôn ngữ lập trình Java được thể hiện ở Hình 1.8

A) Mô hình kế thừa



B) Codes Java tương ứng

```

class Nguoi{
    private String hoTen;
    private String diaChi;

    public void nhapTen() {}
    public void nhapDiaChi() {}
}

class SinhVien extends Nguoi{
    private double diemToan;
    private double diemAnh;

    public void nhapDiem() {}
    public void tinhDiemTB() {}
}

class NhanVien extends Nguoi{
    private double luong;
    private String chucVu;

    public void nhapChucVu() {}
    public void tinhThuNhap() {}
}

class KhachHang extends Nguoi{
    private String tenCongTy;

    public void nhapTenCongTy() {}
    public void muaHang() {}
}
    
```

Hình 1.8 Ví dụ kế thừa

1.2.6. Đa hình

Đa hình: Là kỹ thuật sử dụng để mô tả khả năng gửi một thông điệp chung tới nhiều đối tượng mà mỗi đối tượng lại có cách xử lý riêng theo ngữ cảnh của mình.

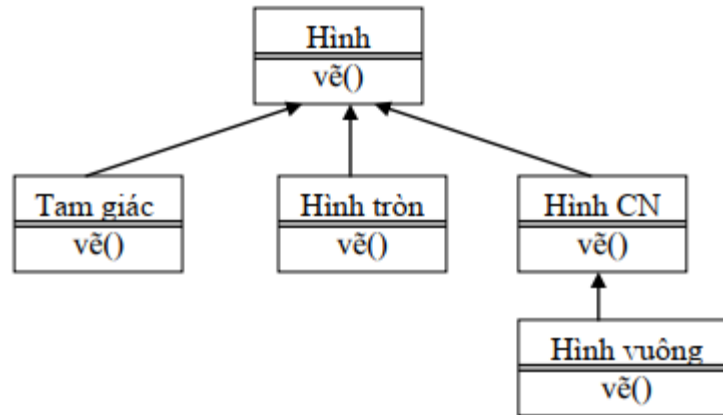
Khả năng của đối tượng có nhiều phương thức cùng tên nhưng nội dung thực hiện khác nhau. Một thông điệp (lời gọi hàm) được hiểu theo các cách khác nhau tùy theo danh sách tham số của thông điệp. Ví dụ: nhận được cùng một thông điệp “nhảy”, một con kangaroo và một con cóc nhảy theo hai kiểu khác nhau: chúng cùng có hành vi “nhảy” nhưng các hành vi này có nội dung khác nhau.



chú ý rằng kangaroo và cóc thuộc hai nhánh trong cây phả hệ động vật

Hình 1.9 Đa hình

Ví dụ 7: Hàm vẽ là hàm đa trị. Nó được xác định tùy ngữ cảnh khi sử dụng. Khi sử dụng làm vẽ với đối tượng tam giác thì thực hiện việc vẽ tam giác. Khi sử dụng hàm vẽ với đối tượng hình chữ nhật thì thực hiện việc vẽ hình chữ nhật...



Hình 1.10 Ví dụ đa hình

Nạp chồng: Là trường hợp đặc biệt của đa trị. Dùng một tên hàm cho nhiều định nghĩa hàm, nhưng khác nhau ở kiểu của tham số hoặc số lượng tham số thì khi đó nội dung thực trong mỗi hàm là khác nhau.

Ví dụ 8: hàm cộng()

`int cong(int, int)`

`float(float, float)`

`string cong(string, string)`

`int cong(int a, int b, int c)`

`int cong(int a[]) // danh sách đầu vào là một mảng`

1.3. CÁCH TIẾP CẬN VÀ GIẢI QUYẾT BÀI TOÁN THEO PHƯƠNG PHÁP HƯỚNG ĐỐI TƯỢNG

1.3.1. Mô tả bài toán

Mô tả bài toán là bước đầu tiên giúp chúng ta hiểu rõ mục tiêu của bài toán chúng ta cần giải quyết. Khi mô tả bài toán, chúng ta trả lời các câu hỏi cơ bản sau:

- Vấn đề là gì?: Xác định vấn đề cần giải quyết.
- Mục tiêu là gì?: Xác định mục tiêu cuối cùng mà hệ thống cần đạt được.
- Các thành phần chính của bài toán là gì?: Xác định các yếu tố hoặc thành phần chính của bài toán (có thể là người dùng, dữ liệu, các sự kiện chính, v.v.).
- Phạm vi bài toán: Xác định phạm vi bài toán, bao gồm những gì cần xây dựng và những gì nằm trong phạm vi của bài toán.

Bước mô tả bài toán đóng vai trò quan trọng giúp định hướng cho các bước tiếp theo. Nếu không mô tả rõ ràng, có thể dẫn đến việc phát triển hệ thống thiếu chính xác hoặc không đáp ứng được nhu cầu của người dùng.

Ví dụ 1: Ta có bài toán xây dựng một hệ thống quản lý thư viện. Mục tiêu là hỗ trợ nhân viên thư viện quản lý sách và người dùng, cùng với chức năng mượn và trả sách.

Các chức năng chính yêu cầu:

- Quản lý sách: Hệ thống cần có khả năng thêm, sửa, xóa thông tin về các cuốn sách.
- Quản lý người dùng: Cung cấp tính năng đăng ký và xóa tài khoản người dùng.
- Quản lý mượn trả: Ghi nhận và theo dõi tình trạng mượn trả sách, kiểm tra sách có sẵn hay không.

Lưu ý: Cần đặc biệt quan tâm đến các quy tắc nghiệp vụ. Ví dụ: một người dùng không được phép mượn nhiều hơn một số sách tối đa hoặc sách phải được trả trước khi mượn tiếp.

1.3.2. Đặc tả yêu cầu

Đặc tả yêu cầu là một bước phân tích chi tiết, giúp xác định rõ các yêu cầu của hệ thống. Các chức năng mà hệ thống cần phải thực hiện. Đây là những hành vi cụ thể mà hệ thống phải đáp ứng.

Ví dụ 2: Bảng đặc tả yêu cầu của bài toán quản lý thư viện

Bảng 1.1 Bảng đặc tả yêu cầu của bài toán quản lý thư viện

Mã yêu cầu	Yêu cầu	Loại yêu cầu	Mô tả
FR01	Thêm sách	Chức năng	Hệ thống cho phép quản trị viên thêm sách vào thư viện
FR02	Xóa sách	Chức năng	Hệ thống cho phép quản trị viên xóa sách khỏi thư viện
FR03	Sửa thông tin sách	Chức năng	Hệ thống cho phép quản trị viên cập nhật thông tin sách
FR04	Đăng ký người dùng	Chức năng	Hệ thống cho phép người dùng đăng ký tài khoản
FR05	Xóa người dùng	Chức năng	Hệ thống cho phép quản trị viên xóa tài khoản người dùng
FR06	Mượn sách	Chức năng	Người dùng có thể mượn sách nếu sách đang có sẵn
FR07	Trả sách	Chức năng	Người dùng có thể trả sách và cập nhật lại trạng thái sách

Việc đặc tả yêu cầu giúp hiểu rõ những gì hệ thống phải thực hiện, đảm bảo rằng mọi chức năng và tính chất mong muốn của hệ thống đều được liệt kê đầy đủ, từ đó giảm thiểu rủi ro sai sót trong quá trình phát triển.

1.3.3. Trích chọn đối tượng

Trích chọn đối tượng là quá trình xác định các đối tượng trong hệ thống. Các đối tượng là những thành phần có khả năng lưu trữ trạng thái và thực hiện các hành vi cụ thể. Trong một bài toán hướng đối tượng, mỗi đối tượng đại diện cho một khái niệm thực hoặc trừu tượng trong hệ thống, ví dụ như "Người dùng", "Sách", "Giỏ hàng".

Các tiêu chí trích chọn đối tượng:

- Đối tượng phải là các thực thể hoặc khái niệm có ý nghĩa độc lập trong bối cảnh bài toán.

- Đối tượng phải có trạng thái và hành vi rõ ràng: Ví dụ, trong hệ thống thư viện, đối tượng "Sách" có trạng thái (tiêu đề, tác giả, trạng thái mượn) và hành vi (cập nhật trạng thái, kiểm tra tính sẵn sàng).

Ví dụ 3: Các đối tượng chính trong hệ thống quản lý thư viện

- **Sách (Book):** Là thực thể đại diện cho một cuốn sách trong thư viện, với các thuộc tính như tiêu đề, tác giả, trạng thái (có sẵn, đang mượn).
- **Người dùng (User):** Đại diện cho người dùng của hệ thống (có thể là thành viên hoặc quản trị viên), với các thuộc tính như tên, số điện thoại.
- **Quản lý mượn trả (BorrowManager):** Đối tượng này đại diện cho chức năng quản lý các thao tác mượn và trả sách, giúp ghi nhận và kiểm tra tình trạng sách.

Bước trích chọn đối tượng giúp xác định các thành phần chính của hệ thống và cách chúng tương tác với nhau, từ đó tạo nền tảng cho các bước mô hình hóa và thiết kế lớp đối tượng sau này.

1.3.4. Mô hình hoá lớp đối tượng

Mô hình hóa lớp đối tượng là quá trình xác định các lớp (Classes) trong hệ thống và xây dựng các thuộc tính (Attributes) và phương thức (Methods) của chúng. Một lớp đại diện cho một nhóm đối tượng có cùng tính chất và hành vi.

- **Lớp (Class):** Là một mô hình hoặc bản thiết kế cho các đối tượng. Một lớp xác định các thuộc tính và phương thức mà các đối tượng của lớp đó sẽ có.
- **Thuộc tính (Attributes):** Là các đặc điểm, trạng thái của lớp hoặc đối tượng. Ví dụ, lớp "Sách" có các thuộc tính như "Tựa sách", "Tác giả".
- **Phương thức (Methods):** Là các hành vi hoặc hành động mà lớp hoặc đối tượng có thể thực hiện. Ví dụ, lớp "Sách" có thể có phương thức "kiểm tra tính sẵn sàng".

Ví dụ 4: Các lớp đối tượng trong hệ thống quản lý thư viện

Lớp Book (Sách)

- **Thuộc tính:**
 - + bookID: Mã định danh duy nhất của mỗi cuốn sách.
 - + title: Tiêu đề sách.
 - + author: Tác giả của sách.
 - + status: Trạng thái của sách (có sẵn, đang mượn).
- **Phương thức:**
 - + addBook(): Thêm sách mới vào thư viện.
 - + updateBookInfo(): Cập nhật thông tin sách.
 - + checkAvailability(): Kiểm tra xem sách có sẵn hay không.

Lớp User (Người dùng)

- **Thuộc tính:**
 - + userID: Mã định danh của người dùng.

- + name: Tên người dùng.
- + contactInfo: Thông tin liên hệ của người dùng.
- **Phương thức:**
 - + register(): Đăng ký tài khoản người dùng.
 - + deleteAccount(): Xóa tài khoản người dùng khỏi hệ thống.

Lớp **BorrowManager** (Quản lý mượn trả)

- **Thuộc tính:**
 - + borrowID: Mã định danh của phiên mượn.
 - + userID: Mã người dùng mượn sách.
 - + bookID: Mã sách đang được mượn.
 - + borrowDate: Ngày mượn sách.
 - + returnDate: Ngày trả sách.
- **Phương thức:**
 - + borrowBook(): Thao tác mượn sách.
 - + returnBook(): Thao tác trả sách.
 - + checkStatus(): Kiểm tra tình trạng mượn trả của sách.

Mô hình hóa lớp giúp hệ thống hóa các đối tượng và mối quan hệ giữa chúng. Bước này giúp dễ dàng hình dung cấu trúc của hệ thống, đồng thời tạo cơ sở cho việc triển khai mã nguồn.

1.3.5. Thiết kế tổng quát

Thiết kế tổng quát là bước xác định cách các đối tượng và lớp sẽ tương tác với nhau trong hệ thống. Thông qua thiết kế tổng quát, chúng ta mô tả cách các đối tượng phối hợp với nhau để thực hiện các chức năng chính của hệ thống.

Thiết kế tổng quát giúp hình dung cách mà hệ thống sẽ hoạt động trong thực tế, cung cấp bản vẽ chi tiết về dòng dữ liệu và logic xử lý. Đây là bước chuyển tiếp giữa lý thuyết và triển khai thực tế, đảm bảo các lớp và đối tượng có thể thực hiện các chức năng mong muốn khi tương tác.

Ví dụ 5: Luồng hoạt động chính cho chức năng mượn sách

1. Người dùng gửi yêu cầu mượn sách đến *BorrowManager*.
2. *BorrowManager* gọi phương thức *checkAvailability()* của lớp *Book* để kiểm tra tình trạng sách.
3. Nếu sách có sẵn, *BorrowManager* ghi nhận thông tin mượn và thay đổi trạng thái sách thành “đang mượn”.
4. Nếu sách không sẵn sàng, hệ thống sẽ thông báo cho người dùng.

1.3.6. Thiết kế chi tiết

Thiết kế chi tiết là quá trình xác định chi tiết các thuật toán, cách thức lưu trữ và truy xuất dữ liệu, và các mô hình tương tác bên trong của từng lớp. Ở bước này, các phương thức và thuộc tính được triển khai một cách cụ thể dựa trên các yêu cầu đã xác định.

Các thành phần của thiết kế chi tiết:

- Thuật toán: Mô tả các bước cụ thể để hoàn thành một tác vụ hoặc chức năng.
- Chi tiết triển khai phương thức: Mỗi phương thức trong từng lớp được mô tả chi tiết về logic xử lý, cách thức thực hiện các thao tác.
- Chi tiết cấu trúc dữ liệu: Lựa chọn các kiểu dữ liệu phù hợp để lưu trữ thông tin một cách tối ưu.

Thiết kế chi tiết là nền tảng để lập trình viên chuyển từ ý tưởng sang mã nguồn thực tế. Đây là bước quyết định để đảm bảo các đối tượng và lớp hoạt động chính xác và hiệu quả.

Ví dụ 6: Thiết kế chi tiết cho bài toán quản lý thư viện

1. Lớp *Book*

Lớp *Book* sẽ đại diện cho một cuốn sách và chứa các thuộc tính cùng phương thức liên quan.

```
public class Book {  
    private String bookID;  
    private String title;  
    private String author;  
    private boolean isAvailable; // true nếu có sẵn, false nếu đang mượn  
    public Book(String bookID, String title, String author) {  
        this.bookID = bookID;  
        this.title = title;  
        this.author = author;  
        this.isAvailable = true; // Mới thêm vào thư viện thì sách sẽ có sẵn  
    }  
    // Phương thức kiểm tra tính khả dụng của sách  
    public boolean isAvailable() {  
        return isAvailable;  
    }  
    // Phương thức cập nhật trạng thái sách  
    public void updateStatus(boolean status) {  
        this.isAvailable = status;  
    }  
    // Các phương thức getter  
    public String getBookID() {  
        return bookID;  
    }  
    public String getTitle() {
```

```

        return title;
    }
    public String getAuthor() {
        return author;
    }
}

```

2. Lớp *User*

Lớp *User* sẽ đại diện cho người dùng trong hệ thống.

```

public class User {
    private String userID;
    private String name;
    public User(String userID, String name) {
        this.userID = userID;
        this.name = name;
    }
    // Getter
    public String getUserID() {
        return userID;
    }
    public String getName() {
        return name;
    }
}

```

3. Lớp *BorrowManager*

Lớp *BorrowManager* sẽ quản lý các thao tác mượn và trả sách.

```

import java.util.HashMap;
import java.util.Map;
public class BorrowManager {
    private Map<String, Book> bookCollection; // Bảng chứa các cuốn sách
    private Map<String, String> borrowedBooks; // Bảng chứa các sách đã mượn
    (userID -> bookID)
    public BorrowManager() {
        bookCollection = new HashMap<>();
        borrowedBooks = new HashMap<>();
    }
}

```

```

    }
    // Phương thức thêm sách vào thư viện
    public void addBook(Book book) {
        bookCollection.put(book.getBookID(), book);
    }
    // Phương thức kiểm tra và mượn sách
    public String borrowBook(User user, String bookID) {
        // Kiểm tra xem sách có trong bộ sưu tập không
        Book book = bookCollection.get(bookID);
        if (book == null) {
            return "Sách không tồn tại trong thư viện.";
        }
        // Kiểm tra tính khả dụng của sách
        if (book.isAvailable()) {
            // Cập nhật trạng thái sách thành không có sẵn
            book.updateStatus(false);
            // Lưu bản ghi mượn sách
            borrowedBooks.put(user.getUserID(), bookID);
            return "Mượn sách thành công: " + book.getTitle();
        } else {
            return "Sách đã được mượn.";
        }
    }
    // Phương thức trả sách
    public String returnBook(User user, String bookID) {
        // Kiểm tra xem người dùng có bản ghi mượn sách không
        if (borrowedBooks.containsKey(user.getUserID()) &&
            borrowedBooks.get(user.getUserID()).equals(bookID)) {
            // Cập nhật trạng thái sách thành có sẵn
            Book book = bookCollection.get(bookID);
            if (book != null) {
                book.updateStatus(true);
            }
            // Xóa bản ghi mượn sách

```



```

        borrowedBooks.remove(user.getUserID());
        return "Trả sách thành công: " + book.getTitle();
    } else {
        return "Không tìm thấy bản ghi mượn cho sách này.";
    }
}
}

```

4. Sử dụng các lớp

Dưới đây là ví dụ về cách sử dụng các lớp đã được định nghĩa ở trên để thực hiện việc mượn và trả sách.

```

public class LibrarySystem {
    public static void main(String[] args) {
        // Tạo đối tượng quản lý mượn sách
        BorrowManager borrowManager = new BorrowManager();
        // Tạo và thêm sách vào thư viện
        Book book1 = new Book("B001", "Lập trình Java cơ bản", "Nguyễn Văn
A");
        Book book2 = new Book("B002", "Hệ thống thông tin", "Trần Thị B");
        borrowManager.addBook(book1);
        borrowManager.addBook(book2);
        // Tạo người dùng
        User user1 = new User("U001", "Nguyễn Văn C");
        // Mượn sách
        System.out.println(borrowManager.borrowBook(user1, "B001")); // Mượn
thành công
        System.out.println(borrowManager.borrowBook(user1, "B001")); // Sách đã
được mượn
        // Trả sách
        System.out.println(borrowManager.returnBook(user1, "B001")); // Trả thành
công
        System.out.println(borrowManager.returnBook(user1, "B001")); // Không
tìm thấy bản ghi mượn
    }
}

```

1.4. MỘT SỐ NGÔN NGỮ LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

Các ngôn ngữ lập trình thời kỳ đầu như Basic, Fortran... không có cấu trúc và cho phép viết những đoạn mã rối rắm (spaghetti code). Lập trình viên sử dụng các lệnh “goto” và “gosub” để nhảy đến mọi nơi trong chương trình. Đoạn trình trên khó theo dõi, khó hiểu, dễ gây lỗi, khó sửa đổi.

Đặc điểm của lập trình hướng chức hay lập trình thủ tục (Procedural Programming- PP) là:

- Sử dụng các cấu trúc vòng lặp: *for*, *while*, *repeat*, *do-while*
- Chương trình là một chuỗi các hàm/ thủ tục
- Mã chương trình tập trung thể hiện thuật toán: làm như thế nào

Hạn chế của lập trình thủ tục:

- Dữ liệu và phân xử lý tách biệt
- Dữ liệu thụ động, xử lý chủ động
- Không đảm bảo được tính nhất quán và các ràng buộc của dữ liệu
- Khó cam mã ứng dụng sửa dữ liệu của thư viện
- Khó bảo trì code
- Phân xử lý có thể nằm rải rác và phải hiểu rõ cấu trúc dữ liệu

Lập trình hướng đối tượng: *Lập trình hướng đối tượng* cho phép khắc phục các hạn chế nói trên

Các ngôn ngữ lập trình hướng đối tượng không mới: Simula (1967) là ngôn ngữ đầu tiên, có lớp, thừa kế, liên kết động (hay còn gọi là hàm ảo). Nhưng các ngôn ngữ hướng đối tượng chậm hơn các ngôn ngữ thời kỳ đầu nên chúng chỉ được dùng rộng rãi khi máy tính bắt đầu chạy nhanh (khoảng thời gian chiếc máy Pentium đầu tiên ra đời).

Dựa vào khả năng đáp ứng các khái niệm về hướng đối tượng, ta chia ra làm hai loại:

* Ngôn ngữ lập trình dựa trên đối tượng (object-based):

Lập trình dựa trên đối tượng là kiểu lập trình hỗ trợ chính cho việc bao bọc, che giấu thông tin và định danh các đối tượng. Lập trình dựa trên đối tượng có những đặc tính sau:

- Bao bọc dữ liệu,
- Cơ chế che giấu và hạn chế truy nhập dữ liệu,
- Tự động tạo lập và huỷ bỏ các đối tượng,
- Tính đa xạ.

Ngôn ngữ hỗ trợ cho kiểu lập trình trên được gọi là ngôn ngữ lập trình dựa trên đối tượng. Ngôn ngữ trong lớp này không hỗ trợ cho việc thực hiện kế thừa và liên kết động. Ada là ngôn ngữ lập trình dựa trên đối tượng.

* Ngôn ngữ lập trình hướng đối tượng (object-oriented)

Lập trình hướng đối tượng là kiểu lập trình dựa trên đối tượng và bổ sung thêm nhiều cấu trúc để cài đặt những quan hệ về kế thừa và liên kết động. Vì vậy đặc tính của LTHĐT có thể viết một cách ngắn gọn như sau:

Các đặc tính dựa trên đối tượng + kế thừa + liên kết động.

Ngôn ngữ hỗ trợ cho những đặc tính trên được gọi là ngôn ngữ lập trình hướng đối tượng, ví dụ như Java, C++, Smalltalk, Object Pascal hay Eiffel, v.v...

1.5. GIỚI THIỆU MỘT SỐ EDITOR BIÊN SOẠN CHƯƠNG TRÌNH JAVA

1.5.1. Eclipse

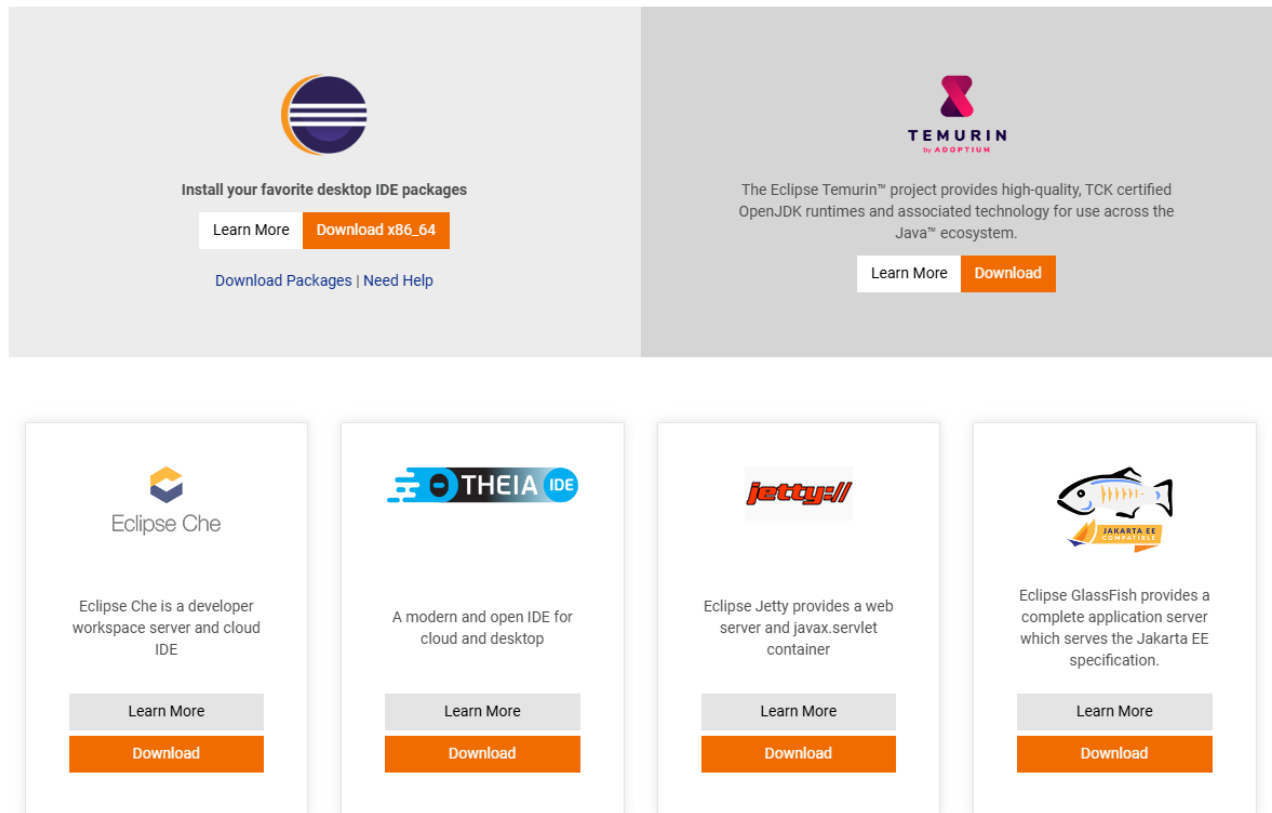
Eclipse là một công cụ lập trình mã nguồn mở mạnh mẽ, được thiết kế như một môi trường phát triển tích hợp chủ yếu cho Java, nhưng cũng hỗ trợ lập trình bằng nhiều ngôn ngữ lập trình khác như C/C++, PHP, Ruby,... Cùng với khả năng mở rộng mã nguồn thông qua việc tích hợp các plugin cho dự án, Eclipse là sự lựa chọn linh hoạt cho các nhà phát triển. Điều đặc biệt là Eclipse hoàn toàn miễn phí và đi kèm với nhiều plugin hữu ích.

Ưu điểm của Eclipse

- Sử dụng cho hai hệ điều hành là Windows và Linux.
- Hỗ trợ nhiều ngôn ngữ lập trình khác như Java, C/C++, PHP.
- Dễ dàng làm việc với cả GUI và non-GUI.
- Tốc độ tải nhanh với SWT/JFace.
- Đa dạng plugin hữu ích như Testing, Code formatter...
- Phù hợp cho các dự án lớn và triển khai trên máy tính cũng như đám mây lưu trữ.
- Mã nguồn mở giúp cá nhân hóa và tạo cấu trúc đầy đủ tính năng cho các công cụ tích hợp.

Tải và cài đặt Eclipse

- Truy cập trang tải Eclipse: <https://www.eclipse.org/downloads/>
- Chọn phiên bản Eclipse IDE phù hợp với nhu cầu lập trình của bạn và nhấn vào download.



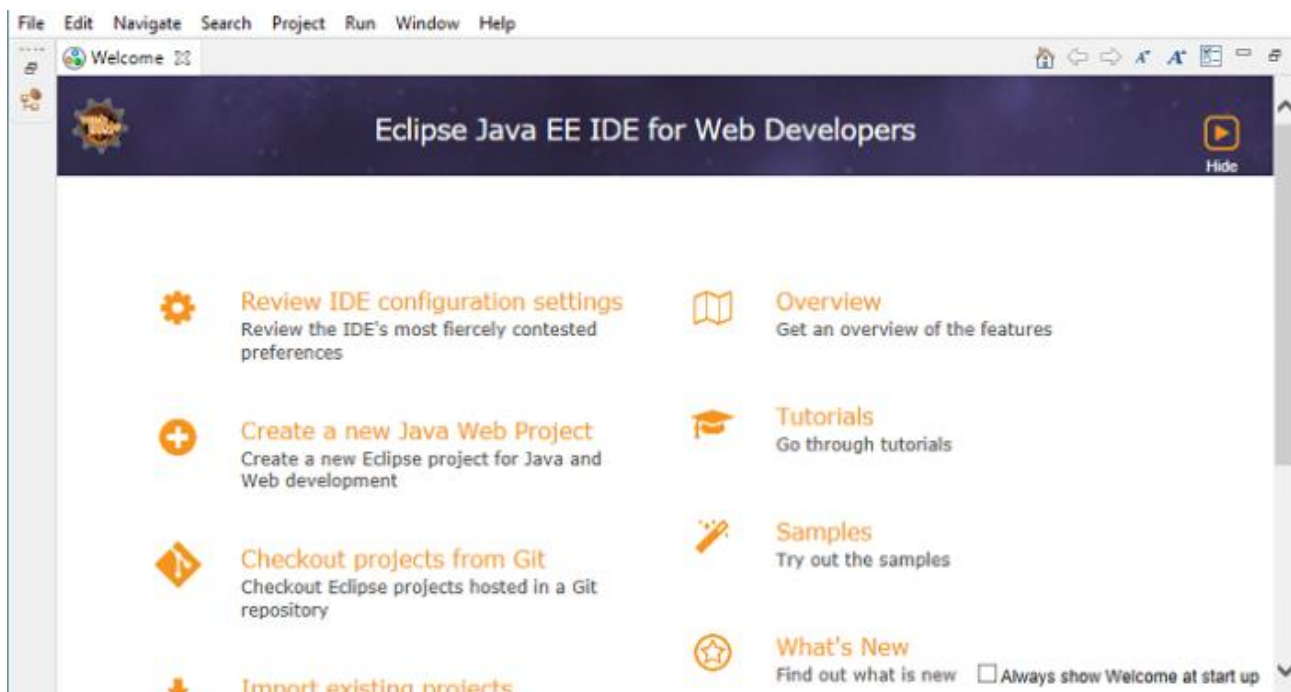
Hình 1.11 Tải Eclipse

- Tải về và mở file cài đặt Eclipse.
- Lựa chọn cài đặt Eclipse
 - + Tùy chọn 1: Cài đặt phiên bản Eclipse cho lập trình viên Java với các chức năng cơ bản.
 - + Tùy chọn 2: Eclipse cho Java EE, thích hợp cho các dự án web sử dụng Java.
 - + Tùy chọn 3: Cho phép lập trình C/C++ trên Eclipse.



Hình 1.12 Lựa chọn cài đặt Eclipse

- Sau khi cài đặt xong, nhấn "Launch" để mở Eclipse.
- Giao diện mở đầu của Eclipse sẽ hiện ra, sẵn sàng cho việc lập trình Java của bạn.

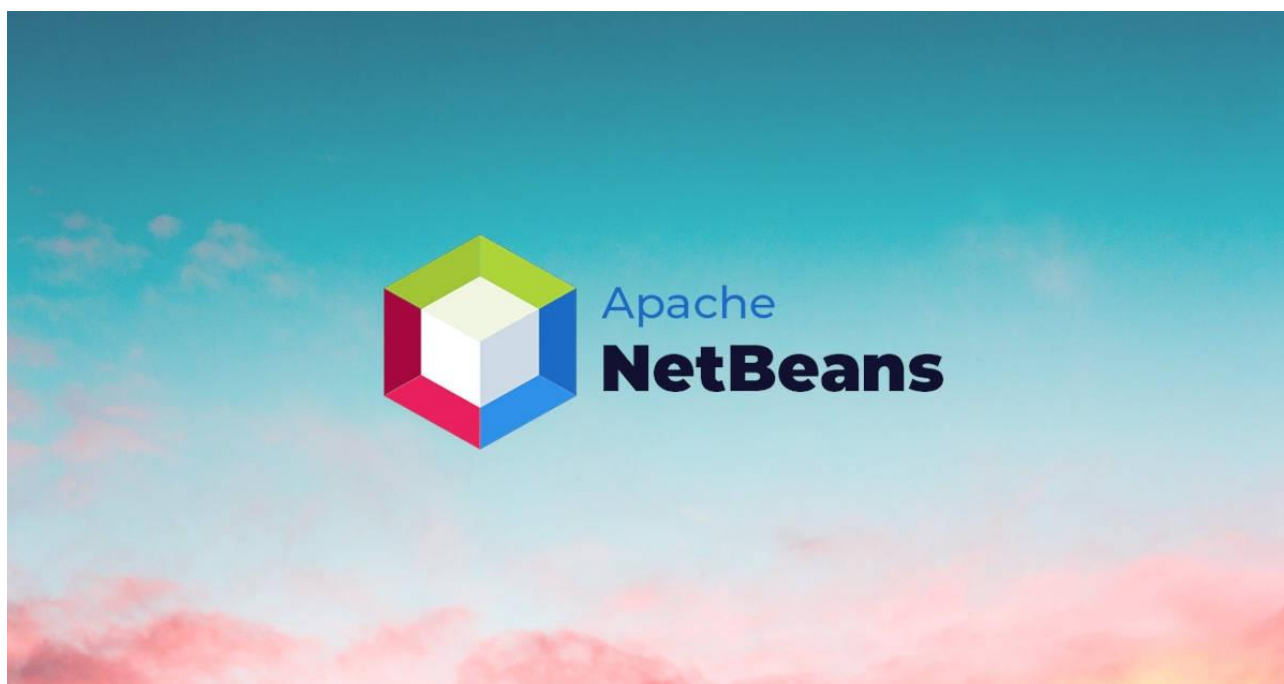


Hình 1.13 Giao diện mở đầu của Eclipse

1.5.2. NetBeans

NetBeans là một môi trường phát triển tích hợp (Integrated Development Environment - IDE) mã nguồn mở, ra đời từ năm 1996 dưới tên gọi Xelfi, ban đầu là dự án IDE Java được phát triển bởi sinh viên dưới sự hướng dẫn của Khoa Toán - Lý tại Đại học Charles, Prague. Sau đó, vào năm 1997, Roman Staněk đã thành lập một công ty quanh dự án này, và sau đó bán phiên bản thương mại của NetBeans cho đến khi Sun Microsystems mua lại vào năm 1999.

Sau khi mua lại, Sun Microsystems mở mã nguồn của NetBeans vào tháng 6 năm 2000, từ đó NetBeans trở thành một dự án mã nguồn mở phát triển dưới sự hỗ trợ của cộng đồng người dùng. Oracle Corporation mua lại Sun vào năm 2010, khiến NetBeans trở thành một phần của Oracle. Tuy nhiên, vào tháng 9/2016, Oracle đã đề xuất quyên tặng dự án NetBeans cho Apache Software Foundation, để mở ra mô hình quản trị mới.



Hình 1.14 NetBeans

NetBeans không chỉ hỗ trợ việc phát triển ứng dụng bằng nhiều ngôn ngữ lập trình như Java, PHP, C/C++, JavaScript và các ngôn ngữ khác, mà còn cung cấp một loạt các tính năng như gỡ lỗi, tự động hoàn thành mã, quản lý phiên bản và tích hợp với các công cụ hỗ trợ phát triển phổ biến. IDE này cũng cho phép người dùng mở rộng chức năng, thông qua việc cài đặt các plugin từ cộng đồng lập trình viên.

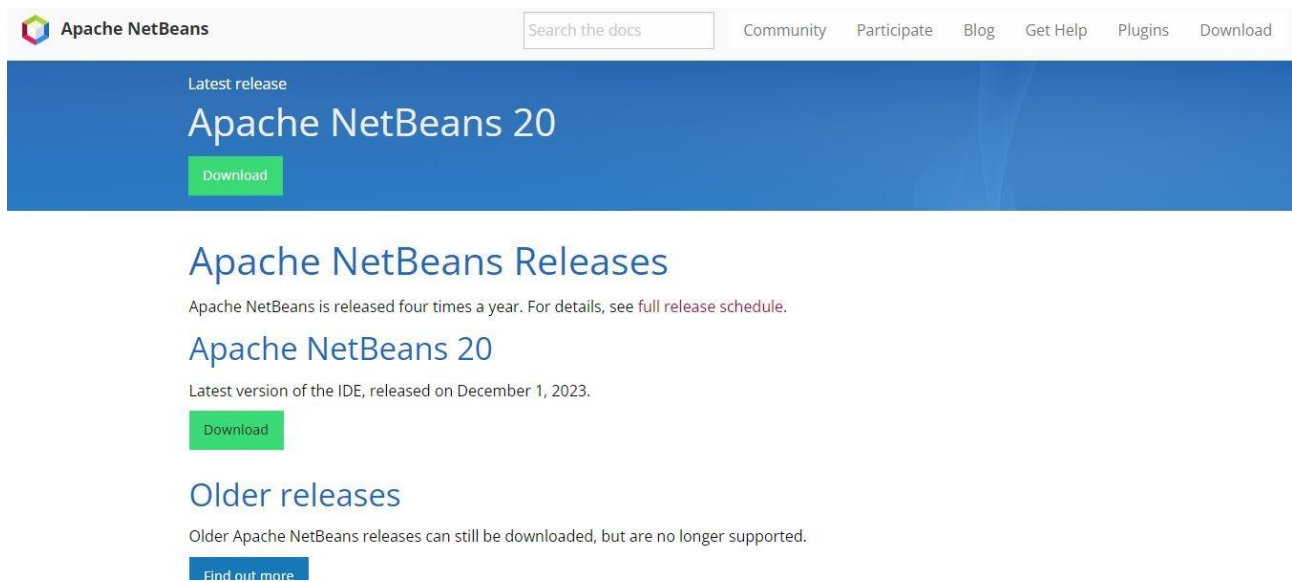
Nhờ khả năng hỗ trợ đa ngôn ngữ và giao diện người dùng thân thiện, NetBeans đã trở thành một công cụ lý tưởng cho việc phát triển ứng dụng đa nền tảng, và hỗ trợ nhu cầu của nhiều dự án khác nhau. Từ người mới bắt đầu đến những lập trình viên có kinh nghiệm, NetBeans đều cung cấp cho họ sự tiện ích, hỗ trợ rất nhiều trong quá trình phát triển phần mềm.

Các tính năng chính của NetBeans

NetBeans IDE đem đến sự thuận tiện và hiệu suất cao trong việc phát triển ứng dụng Java cũng như các ngôn ngữ lập trình khác, với nhiều tính năng nổi bật:

- Tích hợp với Apache: Được phát triển và hỗ trợ bởi Apache Software Foundation, NetBeans cho phép dễ dàng tích hợp các công nghệ và plugin từ Apache, tăng hiệu suất làm việc của người dùng.
- Hỗ trợ Git: Tích hợp sẵn hệ thống quản lý phiên bản Git, giúp các lập trình viên quản lý mã nguồn và làm việc nhóm một cách dễ dàng.
- Tích hợp Plugin: Hỗ trợ một loạt các plugin từ cộng đồng lập trình viên, mở rộng tính năng và tùy chỉnh IDE theo nhu cầu cụ thể của dự án.
- Hỗ trợ phát triển web: IDE hỗ trợ các framework phổ biến cho phát triển ứng dụng web như JavaServer Faces (JSF), Struts và Spring MVC.
- Tích hợp server ứng dụng: NetBeans có tích hợp các server ứng dụng phổ biến như Apache Tomcat, GlassFish và WildFly, giúp người dùng triển khai ứng dụng dễ dàng.
- Debug và Profiling: Cung cấp tính năng gỡ lỗi và đánh giá hiệu năng, giúp người dùng xác định, sửa lỗi trong mã nguồn nhanh chóng và hiệu quả.
- Tích hợp trình biên dịch: Tích hợp trình biên dịch cho nhiều ngôn ngữ, giúp kiểm tra và chạy ứng dụng mà không cần thoát khỏi môi trường IDE.
- Xác định mã nguồn thông minh: Cung cấp tính năng gợi ý mã, hoàn thành mã tự động và xác định lỗi cú pháp, tăng năng suất, giảm lỗi trong quá trình phát triển ứng dụng.
- Có thiết kế giao diện người dùng: NetBeans đi kèm với trình thiết kế giao diện Swing và JavaFX, để thiết kế giao diện người dùng một cách dễ dàng.
- Hỗ trợ dự án lớn: Có khả năng hỗ trợ các dự án lớn có cấu trúc phức tạp, quản lý mã nguồn hiệu quả.
- Hỗ trợ đa ngôn ngữ: Hỗ trợ phát triển ứng dụng trong nhiều ngôn ngữ lập trình như Java, PHP, C/C++, JavaScript, HTML, CSS và các ngôn ngữ khác.
- Giao diện người dùng thân thiện: Giao diện người dùng dễ sử dụng, phù hợp cho cả người mới bắt đầu và người có kinh nghiệm, giúp tăng hiệu suất làm việc.

Cài đặt NetBeans: Truy cập trang web chính thức của NetBeans IDE theo đường link: <https://netbeans.apache.org/front/main/download/index.html> và làm theo hướng dẫn



Hình 1.15 Cài đặt NetBeans

1.5.3. IntelliJ IDEA

IntelliJ IDEA là một môi trường phát triển tích hợp (IDE) mạnh mẽ được thiết kế đặc biệt cho lập trình Java, và cũng có khả năng hỗ trợ một số ngôn ngữ khác như Node.js, Python hay nhiều ngôn ngữ khác. Phiên bản đầu tiên của IntelliJ IDEA được phát hành vào tháng 1/2001, từ đó đến nay nó đã trở thành một trong những IDE hàng đầu trong cộng đồng phát triển phần mềm.



Hình 1.16 IntelliJ IDEA

Mặc dù ban đầu được biết đến chủ yếu là một IDE Java, IntelliJ IDEA đã mở rộng sự hỗ trợ của mình cho nhiều ngôn ngữ và framework khác nhau. Nó cung cấp một loạt các tính năng thông minh như trình soạn thảo mã thông minh, công cụ phân tích mã và bộ công cụ refactorings giúp tối ưu hóa quy trình phát triển phần mềm.

IntelliJ IDEA có hai phiên bản chính: phiên bản miễn phí (community) và phiên bản trả phí (ultimate). Phiên bản trả phí cung cấp nhiều tính năng mở rộng hơn như hỗ trợ JavaScript, TypeScript, các plugin GWT, Vaadin, kiểm tra đoạn mã trùng lặp cùng các tính năng nổi trội khác. Ngoài ra, IntelliJ IDEA cũng là nền tảng cho nhiều sản phẩm IDE khác của JetBrains như PyCharm, PhpStorm, RubyMine,... giúp đáp ứng nhu cầu phát triển phần mềm đa dạng của các nhà phát triển.

Lợi ích khi sử dụng IntelliJ IDEA:

- Tập trung quản lý tác vụ: IntelliJ IDEA cung cấp một môi trường tích hợp để các nhà phát triển có thể tập trung vào phát triển, quản lý tất cả các tác vụ hàng ngày của họ một cách hiệu quả.
- Phát triển và gỡ lỗi mã một cách linh hoạt: Nhờ vào các công cụ mạnh mẽ của IntelliJ IDEA, nhà phát triển có thể viết, gỡ lỗi, tái cấu trúc, kiểm tra và tìm hiểu mã của bạn một cách linh hoạt mà không bị ảnh hưởng.
- Hỗ trợ nhiều ngôn ngữ và công nghệ: IntelliJ IDEA xử lý một cơ sở mã hỗn hợp của nhiều ngôn ngữ lập trình như Java, Ruby, Groovy, Python và Scala, đồng thời hỗ trợ nhiều framework và công nghệ khác nhau.
- Duy trì chất lượng mã tự động: Công cụ tự động của IntelliJ IDEA giúp duy trì chất lượng mã thông qua việc theo dõi và sửa lỗi trên tất cả các cấp độ, từ câu lệnh đến kiến trúc tổng thể.
- Tạo mã sạch và nhanh chóng: IntelliJ IDEA được thiết kế để tạo ra mã sạch và có thể thực hiện mã trong thời gian ngắn nhất, giúp tăng hiệu suất làm việc của nhà phát triển.
- Đa quy mô: IntelliJ IDEA không chỉ phù hợp cho các nhà phát triển cá nhân mà còn đối với các doanh nghiệp với mọi quy mô, từ nhỏ đến lớn.
- Hỗ trợ hệ thống điều khiển phiên bản và CI/CD: IntelliJ IDEA tích hợp tốt với các hệ thống điều khiển phiên bản phổ biến và server tích hợp liên tục như TeamCity, giúp tạo ra một quy trình phát triển phần mềm mạnh mẽ và linh hoạt.

Cài đặt IntelliJ IDEA: Truy cập vào trang web chính thức của IntelliJ IDEA tại địa chỉ <https://www.jetbrains.com/idea/download> và làm theo hướng dẫn cài đặt.



Hình 1.17 Sử dụng IntelliJ IDEA

1.6. TỔNG KẾT CHƯƠNG

Chương 1 cung cấp một cái nhìn tổng quát về các phương pháp tiếp cận trong lập trình, từ nền tảng lý thuyết đến ứng dụng thực tế. Mục tiêu chính của chương là giúp người đọc nắm vững các khái niệm cơ bản trong lập trình và làm quen với lập trình hướng đối tượng, từ đó tạo nền tảng cho việc giải quyết các bài toán phức tạp hơn.

1. **Giới thiệu các phương pháp tiếp cận trong lập trình:** Ba phương pháp lập trình chính là lập trình tuyến tính, lập trình hướng chức năng, và lập trình hướng đối tượng được giới thiệu. Mỗi phương pháp có ưu điểm riêng và phù hợp với từng loại bài toán khác nhau, từ đó đặt nền tảng để hiểu rõ vai trò của lập trình hướng đối tượng.
2. **Các khái niệm cơ bản của lập trình hướng đối tượng:** Các khái niệm chính của lập trình hướng đối tượng bao gồm:
 - **Đối tượng:** Thành phần cơ bản trong lập trình hướng đối tượng, đại diện cho các thực thể có tính chất và hành vi riêng.
 - **Lớp đối tượng:** Định nghĩa khuôn mẫu của đối tượng, giúp tổ chức và tái sử dụng mã lệnh.

- **Trừu tượng hóa:** Quá trình đơn giản hóa đối tượng để tập trung vào các đặc điểm chính.
 - **Bao bọc và che giấu thông tin:** Giữ cho dữ liệu trong đối tượng an toàn và không bị can thiệp trực tiếp từ bên ngoài.
 - **Kế thừa:** Cho phép một lớp con kế thừa các thuộc tính và phương thức của lớp cha, giúp tái sử dụng mã và mở rộng chức năng.
 - **Đa hình:** Khả năng dùng cùng một phương thức để thực hiện các hành động khác nhau, tùy thuộc vào ngữ cảnh.
3. **Cách tiếp cận và giải quyết bài toán theo phương pháp hướng đối tượng:** Bao gồm các bước từ mô tả bài toán, đặc tả yêu cầu, trích chọn đối tượng, mô hình hoá lớp đối tượng, thiết kế tổng quát, và thiết kế chi tiết. Các bước này giúp cấu trúc và triển khai bài toán hiệu quả, đảm bảo rõ ràng và dễ bảo trì.
 4. **Ngôn ngữ lập trình hướng đối tượng phổ biến:** Một số ngôn ngữ như Java, C++, Python... được giới thiệu, nêu rõ ưu điểm và ứng dụng của từng ngôn ngữ trong lập trình hướng đối tượng.
 5. **Các Editor biên soạn chương trình phổ biến:** Giới thiệu về các công cụ hỗ trợ lập trình như Eclipse, NetBeans, và IntelliJ IDEA, giúp lập trình viên làm việc hiệu quả và tối ưu hoá quy trình phát triển.

1.7. CÂU HỎI TRẮC NGHIỆM

1. Lập trình tuyến tính tập trung vào cách:

- A. Sử dụng đối tượng và lớp để giải quyết vấn đề
- B. Tổ chức các lệnh theo trình tự
- C. Xây dựng các chức năng độc lập, không có trạng thái
- D. Mô hình hóa các thực thể trong thực tế

2. Lập trình hướng chức năng tập trung vào:

- A. Tính kế thừa và đa hình
- B. Tính bao bọc và che giấu thông tin
- C. Tổ chức chương trình thành các hàm và chức năng
- D. Sử dụng các lớp và đối tượng để mô hình hóa vấn đề

3. Lập trình hướng đối tượng chủ yếu sử dụng:

- A. Các cấu trúc tuyến tính
- B. Các hàm toán học và phương thức
- C. Các lớp và đối tượng
- D. Các mảng và biến số học

4. Trong lập trình hướng đối tượng, “lớp” là:

- A. Đối tượng đại diện cho một thực thể trong thế giới thực
- B. Cách tổ chức dữ liệu dưới dạng bảng

- C. Khuôn mẫu của đối tượng, chứa các thuộc tính và phương thức chung
- D. Một biến đơn giản dùng để lưu trữ dữ liệu

5. Kỹ thuật che giấu thông tin (Encapsulation) trong lập trình hướng đối tượng nhằm mục đích:

- A. Đảm bảo dữ liệu không bị truy cập trực tiếp từ bên ngoài đối tượng
- B. Cho phép đối tượng kế thừa các thuộc tính của đối tượng khác
- C. Tạo ra các bản sao khác nhau của cùng một đối tượng
- D. Thực hiện tính toán trên các hàm toán học

6. Khái niệm kế thừa (Inheritance) trong lập trình hướng đối tượng là:

- A. Một phương thức cho phép đối tượng tồn tại độc lập
- B. Cách đối tượng nhận thuộc tính và phương thức từ lớp cha
- C. Cách để chia nhỏ chương trình thành các phần chức năng độc lập
- D. Mô hình hóa các dữ liệu tuyến tính

7. Trong lập trình hướng đối tượng, “đa hình” (Polymorphism) có nghĩa là:

- A. Khả năng một đối tượng tồn tại dưới nhiều dạng khác nhau
- B. Khả năng chia nhỏ chương trình thành các mô-đun độc lập
- C. Khả năng các lớp chia sẻ cùng một phương thức
- D. Khả năng tổ chức dữ liệu thành các mảng và chuỗi

8. Quá trình đơn giản hóa đối tượng để chỉ tập trung vào các đặc điểm quan trọng gọi là:

- A. Bao bọc
- B. Đa hình
- C. Trừu tượng hóa
- D. Kế thừa

9. Một lớp trong lập trình hướng đối tượng có thể chứa các thành phần nào?

- A. Biến và hàm
- B. Biến và phương thức
- C. Đối tượng và phương thức
- D. Mảng và chuỗi

10. Lợi ích chính của lập trình hướng đối tượng là:

- A. Chia nhỏ chương trình thành các mô-đun độc lập
- B. Cho phép tạo ra các đối tượng với thuộc tính và hành vi cụ thể
- C. Tạo ra các lệnh thực thi theo trình tự
- D. Giúp chương trình chạy nhanh hơn

11. Bước đầu tiên trong quy trình giải quyết bài toán theo hướng đối tượng là:

- A. Trích chọn đối tượng
- B. Mô tả bài toán
- C. Thiết kế chi tiết
- D. Xác định lớp đối tượng

12. Phần mềm nào dưới đây là một IDE hỗ trợ lập trình hướng đối tượng?

- A. Microsoft Word
- B. Eclipse
- C. Adobe Photoshop
- D. VLC Media Player

13. Trong lập trình hướng đối tượng, đối tượng được định nghĩa là:

- A. Một thực thể đơn giản lưu trữ dữ liệu
- B. Một bản sao của lớp
- C. Một thuộc tính của lớp
- D. Một phương thức cho việc chia nhỏ chương trình

14. Ngôn ngữ lập trình nào dưới đây không phải là ngôn ngữ hướng đối tượng?

- A. Java
- B. C++
- C. Python
- D. C

15. Trong lập trình hướng đối tượng, phương thức là:

- A. Thuộc tính của lớp
- B. Hàm được định nghĩa trong lớp và dùng để thực hiện hành vi của đối tượng
- C. Một biến toàn cục dùng cho mọi đối tượng
- D. Phần mở rộng của đối tượng

Chương 2 GIỚI THIỆU NGÔN NGỮ LẬP TRÌNH JAVA

Chương 2 giới thiệu các vấn đề gồm:

- Giới thiệu tổng quan về ngôn ngữ lập trình Java
- Môi trường lập trình Java
- Cấu trúc chung của một chương trình Java
- Cơ chế biên dịch và thông dịch chương trình Java
- Các dạng chương trình ứng dụng Java

2.1. GIỚI THIỆU CHUNG

2.1.1. Lịch sử hình thành và phát triển của Java

Java là ngôn ngữ lập trình hướng đối tượng (tựa C++) do Sun Microsystem đưa ra vào giữa thập niên 90. Chương trình viết bằng ngôn ngữ lập trình java có thể chạy trên bất kỳ hệ thống nào có cài máy ảo java

Năm 1990: James Gosling và các công sự của Công ty Sun Microsystem tham gia dự án Green Team - xây dựng công nghệ mới cho ngành điện tử tiêu dung (cho các thiết bị điện dân dụng). Để giải quyết vấn đề này nhóm nghiên cứu phát triển đã xây dựng một ngôn ngữ lập trình mới đặt tên là Oak tương tự như C++ nhưng loại bỏ một số tính năng nguy hiểm của C++ và có khả năng chạy trên nhiều nền phần cứng khác nhau.

Năm 1993 world wide web bắt đầu phát triển. Năm 1994, Sun đưa ra trình duyệt web viết bằng ngôn ngữ oak là webrunner, sau đổi tên thành hostJava.

Sau đó, Sun đổi tên oak thành Java và được giới thiệu năm 1995 tại Sunworld 1995.

Java là tên gọi của một hòn đảo ở Indonexia, đây là nơi nhóm nghiên cứu phát triển đã chọn để đặt tên cho ngôn ngữ lập trình Java trong một chuyến đi tham quan và làm việc trên hòn đảo này. Hòn đảo Java này là nơi rất nổi tiếng với nhiều khu vườn trồng cafe, đó chính là lý do chúng ta thường thấy biểu tượng ly cafe trong nhiều sản phẩm phần mềm, công cụ lập trình Java của Sun cũng như một số hãng phần mềm khác đưa ra.

2.1.2. Ngữ cảnh các ứng dụng của ngôn ngữ Java

Ngôn ngữ Java có những đặc trưng cơ bản sau:

Đơn giản: phát triển trên một ngôn ngữ dễ học và quen thuộc với đa số người lập trình là C và Java loại bỏ các phức tạp của C và C++ như: con trỏ, đa kế thừa, định nghĩa chồng toán tử, không sử dụng lệnh “goto” cũng như file header (.h), loại bỏ cấu trúc “struct” và “union”.

Hướng đối tượng Java là ngôn ngữ lập trình hoàn toàn hướng đối tượng:

- Mọi thực thể trong hệ thống đều được coi là một đối tượng, tức là một thể hiện cụ thể của một lớp xác định.
- Tất cả các chương trình đều phải nằm trong một class nhất định.

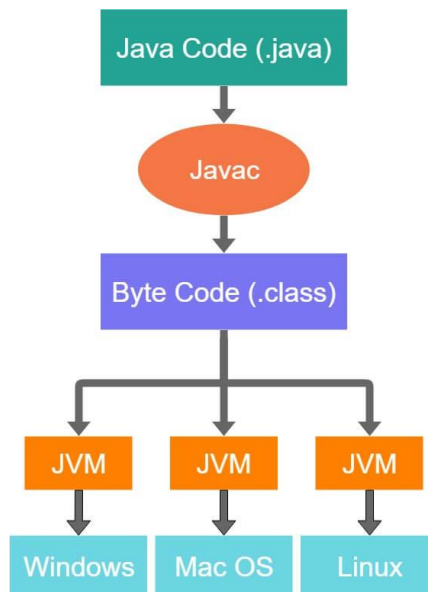
- Không thể dùng Java để viết một chức năng mà không thuộc vào bất kì một lớp nào. Tức là Java không cho phép định nghĩa dữ liệu và hàm tự do trong chương trình.

Độc lập phần cứng và hệ điều hành

Đối với các ngôn ngữ lập trình truyền thống như C/C++, phương pháp biên dịch được thực hiện như sau:

Với các nền phần cứng khác nhau, có một trình biên dịch khác nhau để biên dịch mã nguồn chương trình cho phù hợp với nền phần cứng ấy. Do vậy, khi chạy trên một nền phần cứng khác, bắt buộc phải biên dịch lại mã nguồn.

Đối các chương trình viết bằng Java, chỉ cần cài máy ảo Java (JVM) là có thể chạy được.



Hình 2.1 Chương trình java chạy trên mọi hệ điều hành

Máy ảo Java (Java Virtual Machine).

- Là một phần mềm dựa trên cơ sở máy tính ảo
- Là tập hợp các lệnh logic để xác định hoạt động của máy tính
- Được xem như là một hệ điều hành thu nhỏ
- Nó thiết lập lớp trừu tượng cho:
 - + Phần cứng bên dưới
 - + Hệ điều hành
 - + Mã đã biên dịch

Mạnh mẽ Java là ngôn ngữ yêu cầu chặt chẽ về kiểu dữ liệu:

- Kiểu dữ liệu phải được khai báo tường minh.
- Java không sử dụng con trỏ và các phép toán con trỏ.
- Java kiểm tra việc truy nhập đến mảng, chuỗi khi thực thi để đảm bảo rằng các truy nhập đó không ra ngoài giới hạn kích thước mảng.

- Quá trình cấp phát, giải phóng bộ nhớ cho biến được thực hiện tự động, nhờ dịch vụ thu nhặt những đối tượng không còn sử dụng nữa (garbage collection).
- Cơ chế bắt lỗi của Java giúp đơn giản hóa quá trình xử lý lỗi và khôi phục sau lỗi.

Bảo mật Java cung cấp một môi trường quản lý thực thi chương trình với nhiều mức để kiểm soát tính an toàn:

- Ở mức thứ nhất, dữ liệu và các phương thức được đóng gói bên trong lớp. Chúng chỉ được truy xuất thông qua các giao diện mà lớp cung cấp.
- Ở mức thứ hai, trình biên dịch kiểm soát để đảm bảo mã là an toàn, và tuân theo các nguyên tắc của Java.
- Mức thứ ba được đảm bảo bởi trình thông dịch. Chúng kiểm tra xem bytecode có đảm bảo các qui tắc an toàn trước khi thực thi.
- Mức thứ tư kiểm soát việc nạp các lớp vào bộ nhớ để giám sát việc vi phạm giới hạn truy xuất trước khi nạp vào hệ thống.

Phân tán Java được thiết kế để hỗ trợ các ứng dụng chạy trên mạng bằng các lớp Mạng (java.net). Hơn nữa, Java hỗ trợ nhiều nền chạy khác nhau.

Đa luồng Java cung cấp giải pháp đa luồng (Multithreading) để thực thi các công việc cùng đồng thời và đồng bộ giữa các luồng.

Linh động Java được thiết kế như một ngôn ngữ động để đáp ứng cho những môi trường mở. Các chương trình Java chứa rất nhiều thông tin thực thi nhằm kiểm soát và truy nhập đối tượng lúc chạy.

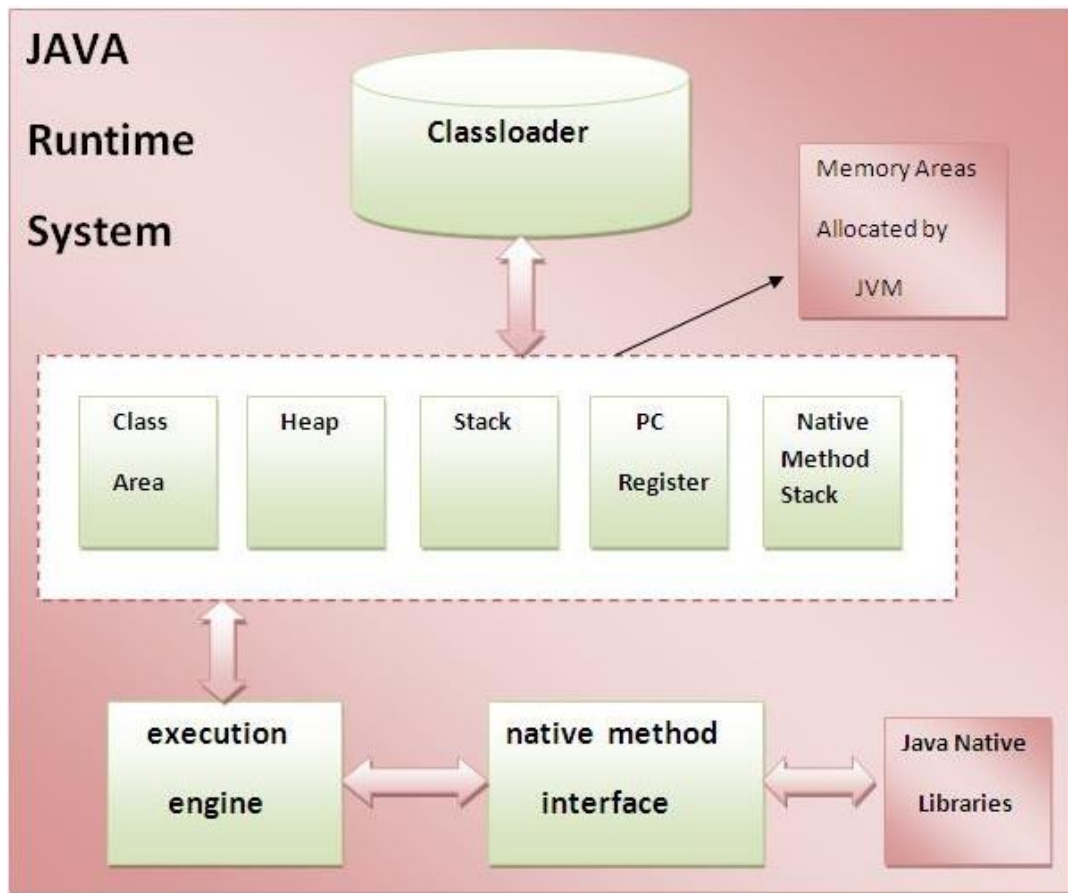
2.2. MÔI TRƯỜNG LẬP TRÌNH JAVA

2.2.1. Máy ảo Java (JVM - Java Virtual Machine)

Tất cả các chương trình muốn thực thi được thì phải được biên dịch ra mã máy. Mã máy của từng kiến trúc CPU của mỗi máy tính là khác nhau (tập lệnh mã máy của CPU Intel, CPU Solarix, CPU Macintosh ... là khác nhau), vì vậy trước đây một chương trình sau khi được biên dịch xong chỉ có thể chạy được trên một kiến trúc CPU cụ thể nào đó. Đối với CPU Intel chúng ta có thể chạy các hệ điều hành như Microsoft Windows, Unix, Linux, OS/2, ...

Chương trình thực thi được trên Windows được biên dịch dưới dạng file có đuôi .EXE còn trên Linux thì được biên dịch dưới dạng file có đuôi .ELF, vì vậy trước đây một chương trình chạy được trên Windows muốn chạy được trên hệ điều hành khác như Linux chẳng hạn thì phải chỉnh sửa và biên dịch lại.

Ngôn ngữ lập trình Java ra đời, nhờ vào máy ảo Java mà khó khăn nêu trên đã được khắc phục. Một chương trình viết bằng ngôn ngữ lập trình Java sẽ được biên dịch ra mã của máy ảo java (mã java bytecode). Sau đó máy ảo Java chịu trách nhiệm chuyển mã java bytecode thành mã máy tương ứng. Sun Microsystem chịu trách nhiệm phát triển các máy ảo Java chạy trên các hệ điều hành trên các kiến trúc CPU khác nhau.



Hình 2.2 Kiến trúc máy ảo Java

Trong đó:

- **Classloader:** Là một hệ thống con của JVM được sử dụng để tải class file.
- **Class (method) Area:** Lưu trữ cấu trúc mỗi lớp, chẳng hạn như hằng, trường, dữ liệu phương thức, code của phương thức, ...
- **Heap:** Nó là khu vực dữ liệu runtime mà trong đó đối tượng được cấp phát.
- **Stack:** Stack trong Java lưu giữ các Frame. Nó giữ các biến cục bộ và các kết quả cục bộ, và thực hiện một phần nhiệm vụ trong phần triệu hồi và trả về phương thức. Mỗi Thread có một Stack riêng, được tạo tại cùng thời điểm với Thread. Một Frame mới được tạo mỗi khi một phương thức được triệu hồi và bị hủy khi lời triệu hồi phương thức là kết thúc.
- **Program Counter Register:** Nó chứa địa chỉ của chỉ lệnh JVM hiện tại đang được thực thi.
- **Native Method Stack:** Bao gồm tất cả các phương thức tự nhiên được sử dụng trong ứng dụng.
- **Execution Engine:** Phần này bao gồm: Một bộ xử lý ảo Virtual Processor Một trình thông dịch Interpreter. Đọc Bytecode Stream sau đó thực thi các chỉ thị.

- **Just-In-Time (JIT) Compiler:** được sử dụng để cải thiện hiệu suất. JIT biên dịch các phần của Bytecode mà có cùng tính năng tại cùng một thời điểm, và vì thế giảm lượng thời gian cần thiết để biên dịch. Ở đây khái niệm Compiler là một bộ biên dịch tập chỉ thị của JVM thành tập chỉ thị của một CPU cụ thể.

2.2.2. Bộ công cụ phát triển JDK

JDK – Java Develop Kit – được dịch ra tiếng Việt là bộ công cụ phát triển ngôn ngữ lập trình bằng ứng dụng Java. JDK chính là một công nghệ và trong đó là tập hợp một loạt các công cụ nhằm hỗ trợ cho người dùng trong việc viết lên các chương trình ứng dụng dưới dạng ngôn ngữ lập trình. JDK dùng để viết ứng dụng mà trong đó gồm có JRE – dùng để chạy những ứng dụng mà JDK cần đến, để góp thêm phần làm ra những ứng dụng trong một hệ ngôn từ lập trình mà trong đó JDK có được sự tương hỗ từ những chương trình có sẵn trong JRE.

Từ khi ngôn từ lập trình Java sinh ra cũng đồng nghĩa tương quan với sự hình thành và tăng trưởng của bộ công cụ tương hỗ này ngay sau đó. Đây là một trong những bộ công cụ thông dụng và được sử dụng thoáng rộng và thông dụng bậc nhất đi cùng với sự tăng trưởng của ngôn từ lập trình JAVA. JDK cũng là một công nghệ tiên tiến được tăng trưởng bởi Sun Microsystem và được công bố cấp phép bởi GNU vào ngày 11/7/2006 – giấy phép về hội đồng GNU – một dạng giấy phép về ứng dụng tự do được cho phép người dùng hoàn toàn có thể san sẻ, sửa đổi và điều tra và nghiên cứu về những loại ứng dụng và được sử dụng rất thông dụng. JDK đã được cung ứng nguồn mở và trở thành một ứng dụng tự do. Nguồn mở không lấy phí – Openjdk được phát hành vào năm 2007 ngay sau đó.

Bộ dụng cụ trong JDK gồm:

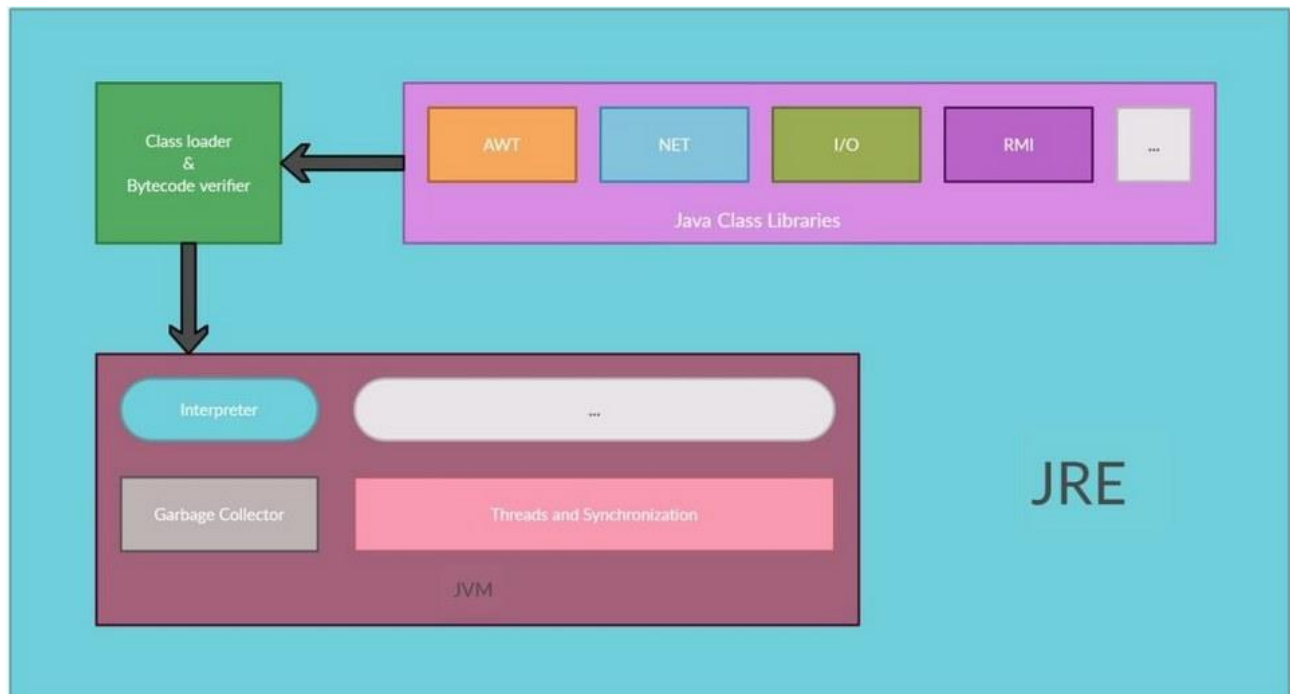
- Trình tương hỗ thực thi những chương trình Java hay hiểu cách khác đây chính là môi trường tự nhiên để cho Java chạy và tăng trưởng. Nó chính là thiên nhiên và môi trường được tạo dựng nên để đem lại sự tăng trưởng cho những chương trình khác. Với tên gọi JRE (Java Runtime Environment).
- Trình biên dịch tương hỗ trong việc biên dịch ngôn từ lập trình từ ngôn từ nguồn (mã nguồn) sang ngôn từ đích. Thông thường những ngôn từ nguồn thường sẽ đơn thuần hơn những ngôn từ được biên dịch ra ngôn từ đích. (Java Compile).
- Trình tạo bản phân phối được cho phép toàn bộ những tệp thành một bản duy nhất việc làm này có tính năng rất lớn trong việc tiết kiệm chi phí dung tích trong quy trình sử dụng những ứng dụng và việc tàng trữ tài liệu hay gửi đi một nơi khác trong thực tiễn trở nên nhỏ gọn và thuận tiện hơn. hay người ta gọi trình tạo bản phân phối này là Java Achieved – nén những file lại thành một bản duy nhất với những đuôi Jav một cách đồng nhất hóa.
- Trình sửa lỗi – Debug – chuyên đi nghiên cứu và phân tích và gỡ lỗi cho những chương trình Java.
- Trình tạo tài liệu – Java Doc
- Thư viện lập trình trong thực tiễn JDK = JRE + những công cụ tăng trưởng ứng dụng bên trong JRE như: Trình thông dịch, biên dịch, tàng trữ, trình sửa lỗi, ...

JRE - Java Runtime Environment – là môi trường để các ứng dụng thực thi trên đó trong khoảng thời gian lập trình. JRE có chứa các thư viện lớp Java, trình tải lớp và máy ảo Java.

Máy ảo Java, tiếng Anh là Java virtual machine (viết tắt: JVM) là một máy ảo cho phép chạy các chương trình Java cũng như các chương trình khác được viết bằng các ngôn ngữ khác mà được biên dịch sang mã byte Java. JVM được mô tả chi tiết bởi một đặc tả mô tả chính thức những gì cần cho một hiện thực JVM. Đặc tả đó để bảo đảm khả năng tương tác của các chương trình Java trên những hiện thực khác nhau để tác giả các chương trình đó có thể sử dụng bộ công cụ phát triển Java (JDK) mà không cần phải lo lắng về đặc tính của nền tảng phần cứng bên dưới.

2.2.3. Java Runtime Environment (JRE)

Java Runtime Environment (JRE) là môi trường thực thi tập lệnh chạy ứng dụng Java. Công nghệ bao gồm các thư viện class của Java, trình quản lý bộ nhớ, trình điều khiển cơ sở dữ liệu và các file hỗ trợ cần thiết khác. JRE cung cấp môi trường chạy ổn định cho các ứng dụng viết bằng ngôn ngữ lập trình Java. Java Runtime Environment cho phép các ứng dụng Java chạy trên nhiều nền tảng máy tính khác nhau mà không cần sửa đổi mã nguồn.



Hình 2.3 Java Runtime Environment

Chương trình hoạt động của JRE có Java Virtual Machine (JVM) là phần quan trọng chịu trách nhiệm thực thi mã bytecode của Java. Công nghệ cung cấp khả năng quản lý bộ nhớ và đa luồng. Ngoài ra, Java Runtime Environment cũng có các thư viện và file hỗ trợ cần thiết để chạy ứng dụng Java. Mặc dù có thể cài đặt JRE độc lập nhưng JRE thường đi kèm với các ứng dụng hoặc trình duyệt web có yêu cầu liên quan. Từ đó mà các ứng dụng hoặc trình duyệt này có thể chạy các ứng dụng và Applet Java trên máy tính của người dùng.

Các thành phần chính của JRE:

- **Java Virtual Machine (JVM):** JVM là thành phần chính của JRE, chịu trách nhiệm thực thi mã bytecode và quản lý bộ nhớ trong quá trình chạy ứng dụng Java. Công nghệ cung cấp môi trường thực thi đa nền tảng cho các ứng dụng Java.
- **Thư viện chuẩn (Standard Library):** JRE đi kèm với một bộ thư viện chuẩn, bao gồm các gói như java.lang, java.util, java.io, java.net. Các thư viện này cung cấp các lớp và phương thức để thực hiện nhiều tác vụ khác nhau như xử lý chuỗi, thao tác với các cơ sở dữ liệu, đọc ghi tập tin, gửi nhận thông tin qua mạng và nhiều chức năng khác.
- **Trình điều khiển JDBC (JDBC Driver):** JRE cũng bao gồm trình điều khiển JDBC, cho phép ứng dụng Java tương tác với cơ sở dữ liệu thông qua ngôn ngữ lập trình Java.
- **Java Launcher:** Đây là một phần của JRE giúp chạy các ứng dụng Java. Công nghệ cung cấp giao diện người dùng hoặc dòng lệnh để khởi động và chạy các ứng dụng Java.
- **Các file hỗ trợ:** JRE cũng đi kèm với các file hỗ trợ cần thiết, bao gồm file cấu hình, file thư viện, file hệ thống và các tài nguyên khác cần thiết cho quá trình chạy ứng dụng Java.

JRE được đánh giá cao về mức độ ứng dụng quan trọng bởi khả năng cung cấp môi trường chạy ổn định cho các ứng dụng được viết bằng ngôn ngữ lập trình Java. JRE là một yếu tố cần thiết của môi trường phát triển và triển khai Java:

- **Đa nền tảng:** JRE cho phép các ứng dụng Java chạy trên nhiều nền tảng máy tính khác nhau mà không cần sửa đổi mã nguồn. Điều này có tác dụng tăng tính linh hoạt và tầm nhìn của các ứng dụng Java.
- **Quản lý bộ nhớ và đa luồng:** JRE bao gồm Java Virtual Machine (JVM), có trách nhiệm thực thi mã bytecode của Java, quản lý bộ nhớ và đa luồng. Công nghệ đảm bảo ứng dụng chạy mượt mà và ổn định.
- **Hỗ trợ cho ứng dụng web:** JRE hỗ trợ chạy các ứng dụng và applet Java trong trình duyệt web, cung cấp một cách tiện lợi để triển khai ứng dụng trên Internet.
- **Công cụ phát triển:** Một số công cụ phát triển và IDE (Integrated Development Environment) yêu cầu JRE để chạy và thực thi mã Java.

Java Runtime Environment có vai trò quan trọng và phức tạp trong ngôn ngữ lập trình Java:

- **Thực thi mã bytecode:** JRE bao gồm Java Virtual Machine (JVM), môi trường thực thi cho mã bytecode được tạo ra từ mã nguồn Java. JVM làm cho mã nguồn Java có thể chạy trên bất kỳ hệ thống nào có JRE mà không cần biên dịch lại mã nguồn sang mã máy tính cụ thể cho từng nền tảng.
- **Quản lý bộ nhớ:** JVM trong Java Runtime Environment quản lý việc cấp phát bộ nhớ cho các đối tượng và thực thi thu gom rác để giải phóng bộ nhớ không còn sử dụng. Công nghệ giúp ứng dụng Java tránh lỗi tràn bộ nhớ và tối ưu hiệu suất.
- **Hỗ trợ đa luồng:** JRE đi kèm với một hệ thống quản lý luồng mạnh mẽ, cho phép ứng dụng Java thực thi các tác vụ song song một cách hiệu quả. Từ đó tối ưu hóa việc sử dụng tài nguyên hệ thống.

- **Thư viện chuẩn:** Java Runtime Environment cung cấp bộ thư viện chuẩn như java.lang, java.util, java.io, java.net để phục vụ cho các nhu cầu lập trình thông thường. Chẳng hạn như xử lý chuỗi, thao tác I/O, quản lý tập hợp và giao tiếp mạng.
- **Công cụ phát triển và triển khai:** JRE cung cấp các công cụ để phát triển và triển khai ứng dụng Java, từ việc thực thi mã cho đến quản lý gói ứng dụng và đóng gói ứng dụng thành các file JAR hoặc WAR.

2.2.4. Java Integrated Development Environment (IDE)

IDE (Integrated Development Environment) là một môi trường phát triển tích hợp, cung cấp các công cụ, giao diện và chức năng giúp lập trình viên thực hiện việc phát triển phần mềm một cách hiệu quả. IDE thường hỗ trợ cho nhiều ngôn ngữ lập trình khác nhau và cung cấp các chức năng giúp giảm thiểu công việc lặp lại và tăng cường năng suất.



Hình 2.4 Integrated Development Environment

Môi trường IDE bao gồm:

- Trình soạn thảo mã nguồn (source code editor): dùng để viết mã nguồn.
- Trình biên dịch (compiler) và/hoặc trình thông dịch (interpreter).
- Công cụ xây dựng tự động: khi sử dụng sẽ biên dịch (hoặc thông dịch) mã nguồn, thực hiện liên kết (linking), và có thể chạy chương trình một cách tự động.
- Trình gỡ lỗi (debugger): hỗ trợ tìm lỗi.
- Ngoài ra, còn có thể bao gồm hệ thống quản lý phiên bản và các công cụ nhằm đơn giản hóa công việc xây dựng giao diện người dùng đồ họa (GUI).
- Trình duyệt lớp (class browser), trình quản lý đối tượng (object inspector), lược đồ phân cấp lớp (class hierarchy diagram),... để sử dụng trong việc phát triển phần mềm theo hướng đối tượng.

Lợi ích của việc sử dụng IDE trong lập trình:

- IDE giúp tối ưu hóa quy trình phát triển phần mềm bằng cách cung cấp một nền tảng tích hợp để thực hiện nhiều tác vụ. Thay vì phải sử dụng nhiều công cụ riêng lẻ, lập trình viên có thể sử dụng môi trường phát triển đa chức năng trong IDE để thực hiện các công việc như viết code, xây dựng và biên dịch dự án, quản lý phiên bản và triển khai phần mềm.
- IDE cung cấp các tính năng như tự động hoàn thiện code, kiểm tra lỗi tự động, định dạng code và đề xuất các phương thức sửa lỗi. Điều này giúp lập trình viên tiết kiệm thời gian và giảm những sai sót do nhập nhầm hoặc thiếu sót trong việc viết code.
- IDE cũng cung cấp môi trường phát triển dễ sử dụng và trực quan. Giao diện trực quan và có thể tùy chỉnh của IDE giúp lập trình viên dễ dàng tìm hiểu và sử dụng các tính năng của nó. Việc tùy chỉnh giao diện và các phím tắt theo sở thích riêng cũng giúp tăng hiệu suất làm việc và thuận tiện cho lập trình viên.
- IDE cho phép lập trình viên kết nối và tích hợp các công cụ như trình biên dịch, trình kiểm tra mã, trình quản lý phiên bản và trình triển khai phần mềm. Điều này giúp lập trình viên nhanh chóng truy cập và sử dụng các công cụ cần thiết để phát triển và quản lý phần mềm của mình.

Các IDE phổ biến hiện nay:

- **Eclipse:** Eclipse được biết là một IDE mã nguồn mở được phát triển bởi Eclipse Foundation. Nó hỗ trợ nhiều ngôn ngữ lập trình như Java, C++, Python và PHP. Eclipse có các tính năng như gỡ lỗi, phân tích mã nguồn và quản lý phiên bản.
- **Visual Studio:** Được phát triển bởi Microsoft, Visual Studio là IDE dành riêng cho các ngôn ngữ lập trình chủ đạo của Microsoft như C#, Visual Basic và F#. IDE này cung cấp giao diện dễ sử dụng, trình gỡ lỗi mạnh mẽ và tích hợp với các công cụ phát triển ứng dụng Microsoft.
- **IntelliJ IDEA:** Đây là một IDE mạnh mẽ dành cho phát triển ứng dụng Java. Với các tính năng thông minh như dò lỗi tự động, gợi ý mã và phân tích mã nguồn, IntelliJ IDEA giúp tăng năng suất và chất lượng của quá trình phát triển Java.
- **Xcode:** Xcode là IDE chính thức cho phát triển ứng dụng trên nền tảng iOS và macOS của Apple. IDE này cung cấp môi trường phát triển đầy đủ cho việc phát triển ứng dụng di động và ứng dụng máy tính.
- **NetBeans:** NetBeans là một IDE mã nguồn mở hỗ trợ nhiều ngôn ngữ lập trình như Java, C/C++ và PHP. IDE này cung cấp các tính năng tương tự như gỡ lỗi, gợi ý mã và quản lý phiên bản.

2.2.5. Java Enterprise Edition (Java EE)

Java EE (Java Enterprise Edition) là một nền tảng phát triển phần mềm của Oracle dành cho các ứng dụng doanh nghiệp được triển khai trên các máy chủ ứng dụng. Java EE cung cấp một tập hợp các API (giao diện lập trình ứng dụng), thư viện và công cụ để phát triển và triển khai các ứng dụng doanh nghiệp có thể mở rộng, đáng tin cậy, bảo mật và hiệu quả.

Java EE cung cấp một loạt các tính năng giúp phát triển và triển khai các ứng dụng doanh nghiệp dễ dàng hơn, bao gồm:

- **Tự động hóa triển khai ứng dụng:** Java EE hỗ trợ tính năng tự động triển khai ứng dụng, cho phép triển khai ứng dụng một cách nhanh chóng và đáng tin cậy. Điều này giúp giảm bớt công sức và thời gian của nhà phát triển trong việc triển khai ứng dụng lên các máy chủ ứng dụng. Ngoài ra, tính năng này cũng đảm bảo tính nhất quán và đáng tin cậy của ứng dụng khi được triển khai trên môi trường sản xuất.
- **Quản lý giao dịch:** Java EE cung cấp cơ chế quản lý giao dịch để đảm bảo tính toàn vẹn dữ liệu và phục hồi sau lỗi trong các hệ thống phân tán. Điều này cho phép các giao dịch diễn ra trơn tru và đa luồng trong các ứng dụng doanh nghiệp phức tạp. Việc quản lý giao dịch cũng giúp đảm bảo tính nhất quán của dữ liệu trong các hệ thống phân tán và ngăn chặn các vấn đề như mất dữ liệu hoặc trùng lặp dữ liệu.
- **Truy cập dữ liệu quan hệ và NoSQL:** Java EE cung cấp các API để đơn giản hóa việc truy cập dữ liệu quan hệ thông qua JPA. Điều này cho phép nhà phát triển tương tác với cơ sở dữ liệu quan hệ một cách dễ dàng và hiệu quả hơn. Ngoài ra, Java EE cũng hỗ trợ truy cập dữ liệu NoSQL thông qua các API như JDO, giúp đơn giản hóa việc lưu trữ và truy xuất dữ liệu phi cấu trúc trong các ứng dụng doanh nghiệp.
- **Xử lý sự kiện:** Java EE hỗ trợ xử lý sự kiện Java (JEP), là một khuôn khổ cho phép các ứng dụng phản ứng với các sự kiện bên ngoài một cách không đồng bộ. Điều này cho phép các ứng dụng có thể xử lý các sự kiện đa dạng từ nhiều nguồn khác nhau, đảm bảo tính linh hoạt và đa dạng trong việc thực thi các chức năng của ứng dụng.

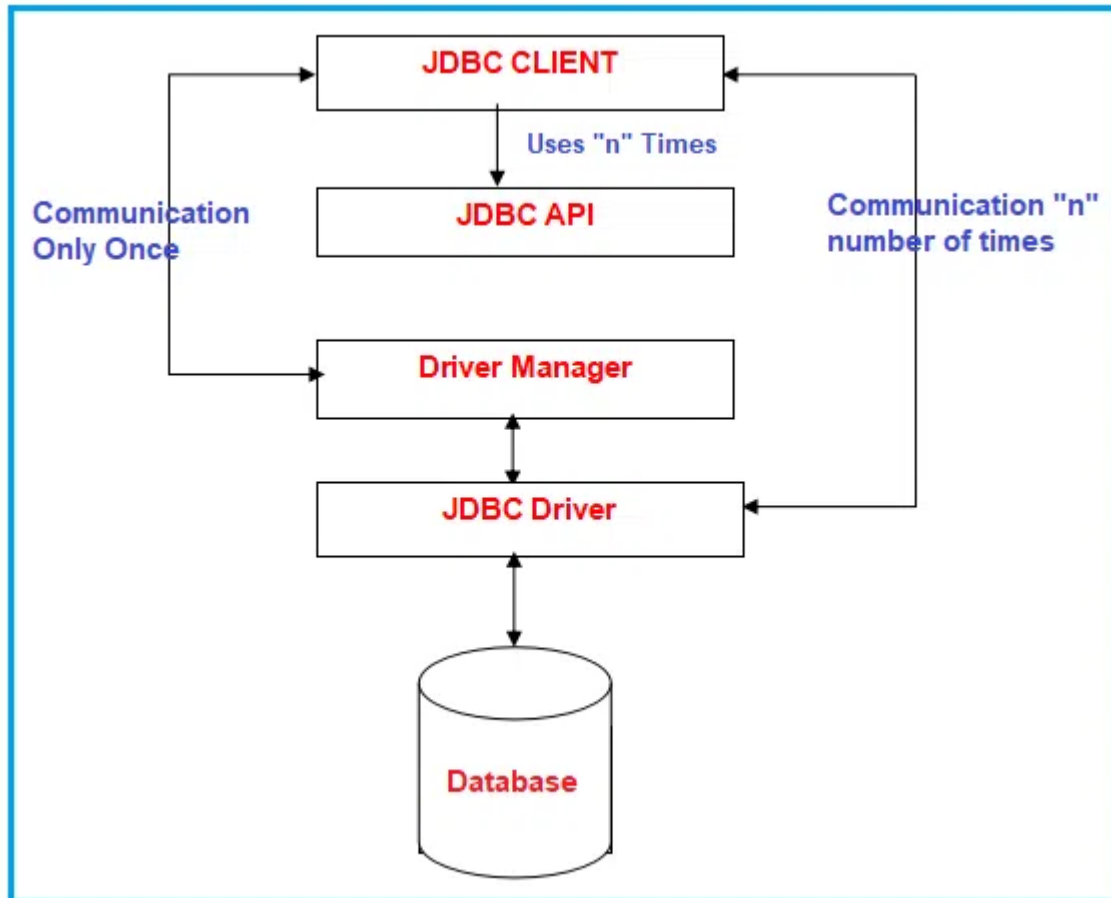
Các framework phổ biến sử dụng Java EE:

- **Spring Framework:** Spring là một trong những framework phổ biến nhất trong việc phát triển ứng dụng Java EE. Nó cung cấp các module cho việc xử lý giao dịch, quản lý bean, tương tác với cơ sở dữ liệu và các công nghệ web khác. Spring cũng hỗ trợ tích hợp với các framework khác như Hibernate và Struts.
- **Struts Framework:** Struts là một framework được phát triển bởi Apache để giúp tổ chức các thành phần của một ứng dụng Java EE. Nó cung cấp một tiêu chuẩn trong việc phát triển các ứng dụng web và đơn giản hóa việc xử lý yêu cầu/đáp ứng.
- **Hibernate Framework:** Hibernate là một framework ORM (Object-Relational Mapping) cho phép ánh xạ giữa các đối tượng trong mã Java và các bảng trong cơ sở dữ liệu. Điều này giúp đơn giản hóa việc truy cập và quản lý dữ liệu trong các ứng dụng doanh nghiệp.

Từ khi ra đời vào năm 1999, Java EE đã trở thành một trong những nền tảng phát triển ứng dụng doanh nghiệp hàng đầu trên thế giới. Với một loạt các tính năng và thành phần hỗ trợ, Java EE giúp đơn giản hóa quá trình phát triển và triển khai các ứng dụng doanh nghiệp đa nền tảng, đáng tin cậy và hiệu quả. Các framework phát triển trên nền tảng Java EE cũng giúp làm tăng tính linh hoạt và khả năng mở rộng của các ứng dụng. Vì vậy, việc học và sử dụng Java EE được coi là một lợi thế lớn cho các nhà phát triển phần mềm trong thời đại công nghệ hiện nay.

2.2.6. Java Database Connectivity (JDBC)

JDBC - Java Database Connectivity, là một API chuẩn để kết nối giữa ngôn ngữ lập trình Java và các database. Nó là một phần của JavaSE (Java Standard Edition). API JDBC sử dụng JDBC drivers để kết nối với cơ sở dữ liệu.



Hình 2.5 Kiến trúc JDBC

- JDBC API: Cho phép chương trình Java thực thi các câu lệnh SQL và truy xuất kết quả. Một số lớp và interface quan trọng được định nghĩa trong JDBC API như sau:
- DriverManager: Đóng vai trò quan trọng trong kiến trúc JDBC. Nó sử dụng một số quy trình điều khiển dành riêng cho cơ sở dữ liệu để kết nối hiệu quả các ứng dụng doanh nghiệp với database
- JDBC Driver: Để giao tiếp với nguồn dữ liệu thông qua JDBC, bạn cần JDBC Driver để giao tiếp với nguồn dữ liệu tương ứng. Có 4 loại JDBC drivers:
 - + JDBC - ODBC Bridge Driver
 - + Native Drive
 - + Network Protocol Driver
 - + Thin driver

Các thành phần chính trong JDBC:

- DriverManager: Là một class dùng để quản lý danh sách các Driver
- Driver: Là một interface, chịu trách nhiệm xử lý các hoạt động giao tiếp giữa ứng dụng và database
- Connection: Là một interface, cung cấp các phương thức cho việc thao tác với database
- Statement: Là một interface, để thực thi các câu lệnh SQL xuống database
- ResultSet: Là một bảng dữ liệu mà biểu diễn tập kết quả từ cơ sở dữ liệu trả về bởi các lệnh SQL
- SQLException: Xử lý ngoại lệ xảy ra trong database

Các bước kết nối database với Java:

- Load driver
- Tạo kết nối (Create Connection)
- Tạo câu lệnh truy vấn SQL (Statement)
- Thực thi câu lệnh truy vấn SQL (Excute query)
- Đóng kết nối (Close Connection)

2.3. CẤU TRÚC CỦA MỘT CHƯƠNG TRÌNH JAVA

Cấu trúc chung:

```
package <package_name>;
import <other_package>;
public class ClassName {
    <Variables (also known as fields)>;
    <constructor method(s)>;
    <other methods>;
}
```

Trong đó:

- **package**: Một package (gói) mô tả không gian tên có chứa các lớp của Java, sử dụng ký tự thường và dấu chấm để định nghĩa tên, chúng ta có thể xem package như là một thư mục, còn class chính là các file trực thuộc thư mục.
- **import**: Từ khóa được sử dụng trong Java nhằm để xác định các class hoặc các package được sử dụng trong lớp này.
- **class**: Từ khóa nhằm để định nghĩa lớp của Java. Nó đứng trước khai báo tên lớp của Java. Ngoài ra còn có từ khóa **public**, từ khóa này xác định phạm vi truy cập của lớp. Đặc tính này chính là tính đóng gói trong lập trình hướng đối tượng. (chúng ta sẽ tìm hiểu phần này ở các bài sau)
- **variables**: Biến hay còn gọi là trường, cũng có một số tài liệu gọi là thuộc tính trực thuộc lớp. Nó chứa thông tin cụ thể liên quan tới các đối tượng là thể hiện của lớp.
- **methods**: Phương thức hay còn gọi là hàm chứa các hành động thực thi của đối tượng. Đương nhiên nội dung của phương thức chính là các đoạn mã thực thi của chính phương thức này.

- **constructors** : Phương thức khởi tạo (Hay hàm khởi tạo) của đối tượng. Hình dạng của đối tượng được thể hiện ra sao sẽ phụ thuộc vào phương thức này.

Ví dụ 1:

```
package Hello;
import java.io.*;
public class HelloWorldApp{
    public static void main(String[] args){
        System.out.println("HelloWorld");
    }
}
```

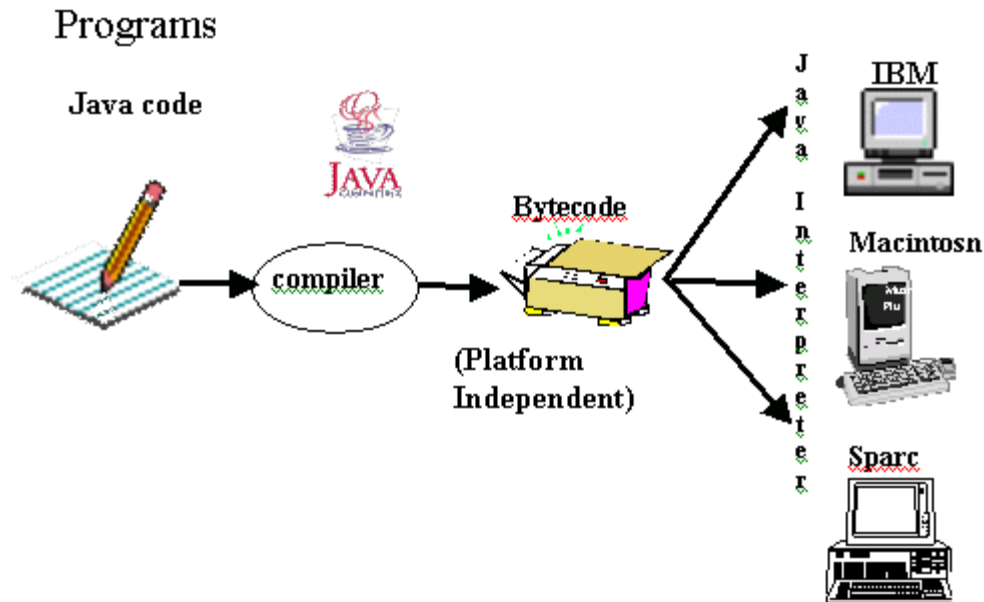
2.4. BIÊN DỊCH VÀ THÔNG DỊCH CHƯƠNG TRÌNH JAVA

Ngôn ngữ lập trình thường được chia ra làm 2 loại (tùy theo các hiện thực hóa ngôn ngữ đó) là ngôn ngữ thông dịch (Interpreted Language) và ngôn ngữ biên dịch (Compiled Language).

- Thông dịch (Interpreter) : Nó dịch từng lệnh rồi chạy từng lệnh, lần sau muốn chạy lại thì phải dịch lại.
- Biên dịch (Compiler): Code sau khi được biên dịch sẽ tạo ra 1 file thường là .exe, và file .exe này có thể đem sử dụng lại không cần biên dịch nữa.

Ngôn ngữ lập trình Java thuộc loại ngôn ngữ thông dịch. Chính xác hơn, Java là loại ngôn ngữ vừa biên dịch (Interpreted Language) vừa thông dịch. Khi viết mã, hệ thống tạo ra một tệp .java. Khi biên dịch mã nguồn của chương trình sẽ được biên dịch ra mã byte code. Máy ảo Java (Java Virtual Machine) sẽ thông dịch mã byte code này thành machine code (hay native code) khi nhận được yêu cầu chạy chương trình. Quá trình dịch chương trình Java:

- Trình biên dịch chuyển mã nguồn thành tập các lệnh không phụ thuộc vào phần cứng cụ thể.
- Trình thông dịch trên mỗi máy chuyển tập lệnh này thành chương trình thực thi
- Máy ảo tạo ra một môi trường để thực thi các lệnh bằng cách:
 - + Nạp các file .class
 - + Quản lý bộ nhớ
 - + Dọn “rác



Hình 2.6 Chạy và biên dịch chương trình java

2.5. CÁC DẠNG CHƯƠNG TRÌNH ỨNG DỤNG CỦA JAVA

2.5.1. Phân loại theo cách vận hành, dịch và chạy chương trình

2.5.1.1. Ứng dụng độc lập

Các ứng dụng Java độc lập là các chương trình được viết bằng Java để thực hiện các tác vụ nhất định. Các ứng dụng này chạy trực tiếp bởi Java. Một ứng dụng độc lập được cài đặt trên mỗi máy tính trước khi chạy. Nó có thể có giao diện dòng lệnh (CLI) hoặc Giao diện người dùng (GUI). Ví dụ như trình soạn thảo văn bản, trình xử lý văn bản và ứng dụng trò chơi.

Ví dụ 1: gõ đoạn mã sau:

```

package Hello;
import java.io.*;
public class HelloWorldApp{
    public static void main(String[] args){
        System.out.println("HelloWorld");
    }
}
  
```

Lưu lại với tên HelloWorldApp.java

Để biên dịch mã nguồn, ta sử dụng trình biên dịch javac.

- Mở cửa sổ Command Prompt.
- Chuyển đến thư mục chứa tập tin nguồn vừa tạo ra.
- Thực hiện câu lệnh: **javac HelloWorldApp.java**

Trình dịch javac tạo ra file **HelloWordApp.class** chứa các mã “bytecodes”. Để chương trình thực thi được ta cần dùng trình thông dịch:

```
java HelloWordApp
```

ta sẽ thấy dòng chữ HelloWord trên màn hình Console.

2.5.1.2. Ứng dụng nhúng applet

Applet là một chương trình Java có thể chạy trong các trình duyệt web có hỗ trợ Java. Tất cả các applet đều là các lớp con của lớp Applet. Để tạo applet, ta cần import hai gói sau:

```
import java.applet.*;
```

```
import java.awt.*;
```

Cấu trúc của một Applet:

```
public class <Tên lớp applet> extends Applet{  
    ... // Các thuộc tính  
    public void init(){...}  
    public void start(){...}  
    public void stop(){...}  
    public void destroy(){...}  
    ... // Các phương thức khác  
}
```

Các phương thức cơ bản của một applet:

- init(): Khởi tạo các tham số, nếu có, của applet.
- start(): Applet bắt đầu hoạt động
- stop(): Chấm dứt hoạt động của applet.
- destroy(): Thực hiện các thao tác dọn dẹp trước khi thoát khỏi applet.

Lưu ý:

- Không phải tất cả các applet đều phải cài đặt đầy đủ 4 phương thức cơ bản trên. Applet còn có thể cài đặt một số phương thức tùy chọn (không bắt buộc) sau:
- paint(Graphics): Phương thức vẽ các đối tượng giao diện bên trong applet. Các thao tác vẽ này được thực hiện bởi đối tượng đồ họa Graphics (là tham số đầu vào).
- repaint(): Dùng để vẽ lại các đối tượng trong applet. Phương thức này sẽ tự động gọi phương thức update().
- update(Graphics): Phương thức này được gọi sau khi thực hiện phương thức paint nhằm tăng hiệu quả vẽ. Phương này sẽ tự động gọi phương thức paint().

Ví dụ 2: cài đặt một applet đơn giản vẽ ra chuỗi “hello word!”

```
import java.awt.*;
```

```
import java.applet.*;
```

```
public class SimpleApplet extends Applet{
```

```

    public void paint(Graphics g){
        g.drawString( " Hello world!",50,50);
    }
}

```

Applet không thể chạy như một ứng dụng Java độc lập (nó không có hàm main), mà nó chỉ chạy được khi được nhúng trong một trang HTML (đuôi .htm, .html) và chạy bằng một trình duyệt web thông thường.

Các bước xây dựng và sử dụng một applet bao gồm:

- Biên dịch mã nguồn thành lớp .class
- Nhúng mã .class của applet vào trang html.

Để nhúng một applet vào một trang html, ta dùng thẻ (tag) <Applet>.

Trong trang myHtml.htm có chứa nội dung như sau:

```

<HTML>
  <HEAD>
    <TITLE> A simple applet </TITLE>
  </HEAD>
  <BODY> This is the output of applet:
    <APPLET CODE="SimpleApplet.class" WIDTH=200 HEIGHT=20>
  </APPLET>
</BODY>
</HTML>

```

Mở trang myHtml trên các trình duyệt thông thường hoặc dùng lệnh appletviewer để chạy

2.5.2. Phân loại theo dạng chương trình ứng dụng, mục đích sử dụng

2.5.2.1. Desktop applications – Java SE (Standard Edition)

Các ứng dụng máy tính để bàn (desktop applications) như Microsoft Word hoặc Adobe Photoshop được cài đặt trên máy tính. các lập trình viên thường ưa chuộng Java vì tính di động, linh hoạt và lý tưởng cho các ứng dụng đa nền tảng. Java SE (Standard Edition) là phiên bản tiêu chuẩn của Java, thường được sử dụng để phát triển các ứng dụng chạy trên desktop, tức là các chương trình hoạt động trực tiếp trên máy tính mà không cần truy cập internet. Dưới đây là một số dạng ứng dụng trên desktop phổ biến được phát triển bằng Java SE:

- **Ứng dụng quản lý tài liệu và cơ sở dữ liệu:** Những ứng dụng này giúp người dùng quản lý dữ liệu như thông tin khách hàng, sản phẩm, hoặc thông tin cá nhân. Các ứng dụng quản lý thư viện, quản lý bệnh viện, và quản lý khách sạn là ví dụ điển hình.
- **Ứng dụng văn phòng:** Java SE cũng có thể được dùng để phát triển các ứng dụng văn phòng cơ bản như trình soạn thảo văn bản, bảng tính, hay các phần mềm hỗ trợ quản lý công việc cá nhân.

- **Ứng dụng đa phương tiện:** Java SE có thể phát triển các ứng dụng xử lý hình ảnh, âm thanh và video trên desktop như trình nghe nhạc, xem video hoặc phần mềm chỉnh sửa ảnh đơn giản.
- **Ứng dụng tài chính:** Các công cụ tính toán tài chính, phần mềm quản lý chi tiêu cá nhân, và các ứng dụng kế toán cũng có thể được viết bằng Java SE.
- **Công cụ lập trình và phát triển:** Nhiều IDE (Môi trường phát triển tích hợp) cũng được phát triển bằng Java, như Eclipse và NetBeans, là những ví dụ điển hình của ứng dụng desktop dành cho các lập trình viên.
- **Trò chơi (Game):** Một số trò chơi 2D và các trò chơi đơn giản được xây dựng bằng Java SE, đặc biệt là các trò chơi độc lập hoặc mini-game.

2.5.2.2. Server applications – Java EE (Enterprise Edition)

Java EE (Enterprise Edition) là nền tảng Java được thiết kế đặc biệt cho các ứng dụng server, thường được triển khai trong các môi trường doanh nghiệp lớn, như các hệ thống ngân hàng, quản lý chuỗi cung ứng, và các trang web thương mại điện tử. Với Java EE, các ứng dụng server có thể dễ dàng mở rộng, bảo mật, và quản lý tốt hơn. Dưới đây là một số dạng ứng dụng server phổ biến được phát triển bằng Java EE:

- **Ứng dụng web doanh nghiệp:** Java EE hỗ trợ xây dựng các ứng dụng web lớn, có khả năng phục vụ hàng nghìn người dùng cùng lúc. Các ứng dụng này thường có giao diện cho người dùng (frontend) và tương tác với cơ sở dữ liệu để xử lý thông tin người dùng (backend). Ví dụ điển hình là các trang thương mại điện tử, hệ thống đặt hàng trực tuyến, và các ứng dụng quản lý dịch vụ khách hàng.
- **Dịch vụ web (Web services):** Java EE hỗ trợ xây dựng các dịch vụ web thông qua các API như RESTful Web Services (JAX-RS) và SOAP Web Services (JAX-WS). Các dịch vụ này cho phép ứng dụng giao tiếp và trao đổi dữ liệu với các hệ thống khác, bất kể nền tảng nào.
- **Ứng dụng quản lý doanh nghiệp (Enterprise Applications):** Các hệ thống ERP (Enterprise Resource Planning), CRM (Customer Relationship Management), và SCM (Supply Chain Management) là các ví dụ phổ biến của ứng dụng Java EE trong doanh nghiệp. Những ứng dụng này giúp doanh nghiệp quản lý tài chính, quan hệ khách hàng, và chuỗi cung ứng hiệu quả.
- **Ứng dụng giao dịch tài chính:** Java EE cũng được dùng để xây dựng các hệ thống ngân hàng và tài chính, có khả năng xử lý các giao dịch phức tạp, yêu cầu tính bảo mật và tính ổn định cao. Ví dụ: hệ thống ngân hàng trực tuyến, hệ thống thanh toán điện tử.
- **Hệ thống xử lý nghiệp vụ (Business Process Management):** Java EE hỗ trợ triển khai các hệ thống BPM giúp doanh nghiệp tự động hóa quy trình nghiệp vụ, theo dõi tiến độ và cải thiện hiệu suất làm việc.
- **Hệ thống quản lý nội dung (CMS):** Nhiều hệ thống quản lý nội dung và quản lý tài liệu doanh nghiệp lớn được xây dựng bằng Java EE, cho phép quản lý và chia sẻ tài liệu, bài viết, hình ảnh hiệu quả.

2.5.2.3. Mobile (embedded) applications – Java ME (Micro Edition)

Ngày nay, một trong những trường hợp sử dụng phổ biến nhất của Java là phát triển các ứng dụng Android. Nhưng đó chủ yếu là do sự phát triển di động đã bắt đầu phát triển nhanh các ứng dụng khác. Do đó, nhiều nhà phát triển phát triển Java đã chuyển sang phát triển di động. Java ME (Micro Edition) là phiên bản Java dành cho các ứng dụng di động và các thiết bị nhúng, được thiết kế để chạy trên các thiết bị có tài nguyên hạn chế như điện thoại di động, thiết bị nhúng, và hệ thống IoT. Đây là nền tảng phổ biến cho các ứng dụng trên các thiết bị di động đời cũ, các thiết bị cầm tay và các hệ thống cần khả năng xử lý nhẹ nhàng và hiệu quả. Dưới đây là một số dạng ứng dụng phổ biến của Java ME:

- **Ứng dụng di động cơ bản:** Trước khi các hệ điều hành như Android và iOS trở nên phổ biến, Java ME từng được sử dụng rộng rãi để phát triển các ứng dụng di động cơ bản như trình duyệt, công cụ quản lý tin nhắn, lịch làm việc và danh bạ điện thoại.
- **Trò chơi di động:** Java ME được sử dụng để phát triển các trò chơi cho điện thoại di động cơ bản, đặc biệt là các trò chơi 2D như Snake, Tetris, và các game arcade. Với bộ công cụ nhẹ nhàng, Java ME cung cấp thư viện đồ họa và âm thanh đủ để xây dựng các game có lối chơi đơn giản và độ phức tạp vừa phải.
- **Ứng dụng nhắn tin và liên lạc:** Các ứng dụng nhắn tin và quản lý liên lạc đơn giản có thể được xây dựng bằng Java ME. Chúng bao gồm các ứng dụng gửi tin nhắn SMS, trò chuyện theo nhóm, và quản lý danh bạ.
- **Ứng dụng IoT (Internet of Things):** Java ME hiện nay được ứng dụng rộng rãi trong các thiết bị IoT và các thiết bị nhúng như cảm biến, thiết bị thông minh và thiết bị điều khiển từ xa. Các thiết bị như máy điều hòa thông minh, hệ thống chiếu sáng tự động, và đồng hồ đo thông minh thường sử dụng Java ME để quản lý và thu thập dữ liệu, cũng như để thực hiện các thao tác điều khiển.
- **Ứng dụng ngân hàng và thanh toán di động:** Một số ngân hàng và tổ chức tài chính đã sử dụng Java ME để phát triển các ứng dụng ngân hàng và thanh toán trên các thiết bị di động cũ. Các ứng dụng này thường bao gồm các tính năng cơ bản như kiểm tra số dư, chuyển tiền, và quản lý tài khoản.
- **Ứng dụng quản lý thiết bị di động (Device Management):** Java ME cho phép quản lý các tính năng của thiết bị di động, bao gồm quản lý cấu hình thiết bị, kết nối mạng, và theo dõi hiệu suất.

2.8. TỔNG KẾT CHƯƠNG

Chương 2 đã cung cấp một cái nhìn tổng quan và chi tiết về ngôn ngữ lập trình Java, từ lịch sử phát triển đến các công cụ, môi trường và cấu trúc cần thiết để phát triển phần mềm.

- **Lịch sử phát triển Java:** Java đã phát triển qua nhiều giai đoạn từ khi được tạo ra, không ngừng cải tiến và trở thành một ngôn ngữ linh hoạt, đa nền tảng, được ưa chuộng trong nhiều lĩnh vực, đặc biệt là trong các hệ thống phân tán và ứng dụng doanh nghiệp.

- **Môi trường lập trình Java:** Các công cụ như JVM, JDK, JRE và các IDE hỗ trợ lập trình viên phát triển ứng dụng Java một cách hiệu quả. JVM cho phép chương trình Java chạy trên bất kỳ nền tảng nào mà không cần thay đổi mã nguồn, JDK cung cấp bộ công cụ phát triển toàn diện, trong khi JRE cung cấp môi trường runtime cần thiết để thực thi các chương trình Java. IDE là công cụ không thể thiếu giúp lập trình viên viết mã, gỡ lỗi và kiểm tra ứng dụng nhanh chóng.
- **Cấu trúc chương trình Java:** Cấu trúc của một chương trình Java bao gồm các lớp và đối tượng là những thành phần cơ bản. Mỗi chương trình Java sẽ bắt đầu từ một lớp chính (main class) chứa phương thức main(), nơi chương trình bắt đầu thực thi.
- **Biên dịch và thông dịch Java:** Java sử dụng mô hình biên dịch kết hợp với thông dịch, qua đó mã nguồn Java được biên dịch thành bytecode và thực thi bởi JVM. Điều này giúp chương trình Java có thể chạy trên bất kỳ nền tảng nào mà không cần chỉnh sửa.
- **Các dạng ứng dụng Java:** Java có thể được sử dụng để phát triển nhiều loại ứng dụng khác nhau, từ ứng dụng độc lập, ứng dụng web, cho đến các ứng dụng phân tán, doanh nghiệp và các hệ thống tích hợp cơ sở dữ liệu.

2.9. CÂU HỎI TRẮC NGHIỆM

1. Java được phát triển bởi công ty nào?

- A. Microsoft
- B. Sun Microsystems
- C. Google
- D. Oracle

2. Ai là người sáng lập ngôn ngữ Java?

- A. James Gosling
- B. Dennis Ritchie
- C. Linus Torvalds
- D. Bjarne Stroustrup

3. Java có thể chạy trên nền tảng nào?

- A. Chỉ Windows
- B. Chỉ Linux
- C. Tất cả các nền tảng hỗ trợ JVM
- D. Chỉ macOS

4. Tính năng nổi bật của Java là gì?

- A. Tính đa nền tảng
- B. Tính đơn giản
- C. Tính bảo mật cao
- D. Tính nhanh chóng

- 5. Máy ảo Java (JVM) có vai trò gì trong việc thực thi chương trình Java?**
- A. Biên dịch mã nguồn Java thành bytecode
 - B. Biên dịch bytecode thành mã máy
 - C. Cung cấp các thư viện cho chương trình Java
 - D. Quản lý bộ nhớ trong ứng dụng
- 6. Java Development Kit (JDK) bao gồm các thành phần nào?**
- A. JVM
 - B. JRE
 - C. Các công cụ phát triển như biên dịch, gỡ lỗi
 - D. Tất cả các đáp án trên
- 7. Java Runtime Environment (JRE) bao gồm thành phần nào?**
- A. JVM
 - B. Công cụ phát triển như biên dịch, gỡ lỗi
 - C. Các thư viện hỗ trợ
 - D. Cả A và C
- 8. JDK là viết tắt của gì?**
- A. Java Deployment Kit
 - B. Java Development Kit
 - C. Java Domain Kit
 - D. Java Document Kit
- 9. Java Integrated Development Environment (IDE) giúp làm gì?**
- A. Phân tích và tối ưu hóa mã
 - B. Tạo và quản lý cơ sở dữ liệu
 - C. Hỗ trợ lập trình viên viết mã, gỡ lỗi và biên dịch
 - D. Tất cả các đáp án trên
- 10. Máy ảo Java (JVM) có thể chạy trên hệ điều hành nào?**
- A. Chỉ Linux
 - B. Chỉ Windows
 - C. Chỉ macOS
 - D. Tất cả các hệ điều hành hỗ trợ JVM
- 11. Khi chương trình Java được biên dịch, nó sẽ tạo ra loại tệp gì?**
- A. Tệp .exe
 - B. Tệp .java
 - C. Tệp .class

D. Tập .xml

12. Phương thức main() trong chương trình Java có vai trò gì?

- A. Là điểm bắt đầu của chương trình
- B. Là nơi thực thi các phép toán
- C. Là nơi khai báo các đối tượng
- D. Là nơi lưu trữ các giá trị

13. Phần mềm nào là công cụ hỗ trợ lập trình Java?

- A. Microsoft Visual Studio
- B. NetBeans
- C. Adobe Dreamweaver
- D. Unity

14. Java có phải là ngôn ngữ biên dịch hay thông dịch?

- A. Chỉ biên dịch
- B. Chỉ thông dịch
- C. Biên dịch kết hợp thông dịch
- D. Không biên dịch và không thông dịch

15. Quá trình biên dịch của Java là gì?

- A. Chuyển đổi mã nguồn Java thành bytecode
- B. Chạy chương trình Java trên JVM
- C. Chuyển bytecode thành mã máy
- D. Cả A và B

16. Một chương trình Java có thể gọi các phương thức của các lớp khác như thế nào?

- A. Qua lớp con
- B. Qua đối tượng
- C. Qua thư viện
- D. Qua lớp chính

17. Java có thể được sử dụng để phát triển loại ứng dụng nào?

- A. Ứng dụng di động
- B. Ứng dụng web
- C. Ứng dụng desktop
- D. Tất cả các loại ứng dụng trên

18. Java Database Connectivity (JDBC) dùng để làm gì?

- A. Kết nối với cơ sở dữ liệu
- B. Quản lý mã nguồn

- C. Gỡ lỗi chương trình
 - D. Xử lý sự kiện trong ứng dụng
- 19. Java có hỗ trợ lập trình hướng đối tượng không?**
- A. Có
 - B. Không
- 20. Trong một chương trình Java, class dùng để làm gì?**
- A. Định nghĩa đối tượng
 - B. Định nghĩa phương thức
 - C. Định nghĩa cấu trúc dữ liệu
 - D. Định nghĩa một kiểu dữ liệu mới
- 21. Java có phải là ngôn ngữ tĩnh hay động?**
- A. Tĩnh
 - B. Động
- 22. Khi một chương trình Java được biên dịch thành bytecode, chúng sẽ chạy trên nền tảng nào?**
- A. Chỉ trên Windows
 - B. Chỉ trên Linux
 - C. Trên bất kỳ nền tảng nào hỗ trợ JVM
 - D. Chỉ trên máy tính cá nhân
- 23. Java Enterprise Edition (Java EE) chủ yếu được sử dụng để phát triển ứng dụng nào?**
- A. Ứng dụng di động
 - B. Ứng dụng doanh nghiệp lớn
 - C. Ứng dụng đồ họa
 - D. Ứng dụng web tĩnh
- 24. Cấu trúc cơ bản của một chương trình Java bao gồm những gì?**
- A. Lớp và phương thức
 - B. Mảng và vòng lặp
 - C. Câu lệnh điều kiện
 - D. Tất cả các đáp án trên
- 25. Java có phải là ngôn ngữ biên dịch trực tiếp thành mã máy không?**
- A. Có
 - B. Không
- 26. Java hỗ trợ mô hình lập trình nào?**
- A. Hướng sự kiện

- B. Hướng đối tượng
- C. Hướng hàm
- D. Hướng dữ liệu

27. Chương trình Java có thể chạy trên hệ điều hành nào mà không thay đổi mã nguồn nhờ vào tính năng gì?

- A. Tính kế thừa
- B. Tính đa nền tảng
- C. Tính bao quát
- D. Tính đồng nhất

28. Java có thể sử dụng các thư viện bên ngoài thông qua công cụ nào?

- A. Maven
- B. Eclipse
- C. IntelliJ IDEA
- D. JDK

29. Để tạo một đối tượng trong Java, bạn cần sử dụng từ khóa gì?

- A. create
- B. new
- C. class
- D. object

30. Điều gì sẽ xảy ra nếu không có phương thức main() trong chương trình Java?

- A. Chương trình sẽ không chạy được
- B. Chương trình vẫn chạy nhưng không có đầu ra
- C. Chương trình sẽ chạy nhưng bị lỗi
- D. Chương trình sẽ tự động tạo phương thức main()

2.10. BÀI TẬP THỰC HÀNH

Bài tập 1 Thực hiện các công việc:

1. Cài đặt JDK và cài đặt NetBean (hoặc eclipse), làm quen với các chức năng của NetBean IDE (hoặc Eclipse IDE), phân tích ưu nhược điểm của hai môi trường phát triển này.
2. Thực hiện cài đặt lại các bài giải mẫu
3. Làm quen, hiểu với một số kiểu lỗi cú pháp đơn giản và cách khắc phục

Bài tập 2 Viết chương trình:

1. Khai báo (import) lớp Scanner trong gói java.util
2. Sử dụng phương thức *nextLine ()* của lớp Scanner để nhập vào một xâu và in xâu vừa nhập ra màn hình.

Bài tập 3 Viết chương trình:

1. Sử dụng lớp Scanner trong gói java.util
2. Sử dụng phương thức nextInt() để nhập vào ba số a, b, c tương ứng với độ dài của ba cạnh của một tam giác. in ra thông tin chu vi ($p=a+b+c$) của tam giác tương ứng.

Chương 3 **CÁC THÀNH PHẦN CƠ SỞ CỦA JAVA**

Chương 3 giới thiệu chi tiết các thành phần cơ sở của Java, bao gồm:

- Các phân tử cơ sở của Java
- Các kiểu dữ liệu trong Java
- Biến, hằng, biểu thức và các phép toán cơ bản
- Các cấu trúc điều khiển chương trình trong Java
- Cấu trúc mảng, xâu ký tự
- Ngoại lệ và xử lý ngoại lệ trong Java

3.1. CÁC PHẦN TỬ CƠ SỞ CỦA JAVA

Định danh (Tên gọi)

Trong Java định danh là một dãy các ký tự gồm các chữ cái, chữ số và một số các ký hiệu như: ký hiệu gạch dưới nối câu "_", các ký hiệu tiền tệ \$, ©, Ê, Â, và không được bắt đầu bằng chữ số.

Java phân biệt chữ thường và chữ hoa. Độ dài (số ký tự) của định danh trong Java về lý thuyết là không bị giới hạn.

Quy ước (lời khuyên):

- Định danh được đặt tên cho các lớp, các đối tượng: chữ cái đầu của mỗi từ viết hoa.
- Định danh cho các biến, phương thức, tên phương thức thường bằng động từ (viết in thường, và các danh từ phía sau thì chữ cái đầu viết chữ in hoa) và , đối tượng: chữ cái đầu của mỗi từ trong định danh đều viết hoa trừ từ đầu tiên (“náo”).

Chú thích (Comment)

```
// Chú thích trên một dòng
/* Chú thích trên nhiều dòng
... */
/** Chú thích trong tư liệu (javadoc)
... */
```

Các từ khóa của java

Bảng 3.1 Các từ khóa của Java

abstract	boolean	break	byte
case	catch	char	class
const	continue	default	do
double	else	extends	final
finally	float	for	goto
if	implements	import	instanceof
int	interface	long	native
new	package	private	protected
public	return	short	static
super	switch	synchronized	this
throw	throws	transient	try
void	volatile	while	

Các kí tự định dạng xuất dữ liệu (Escape Sequences)

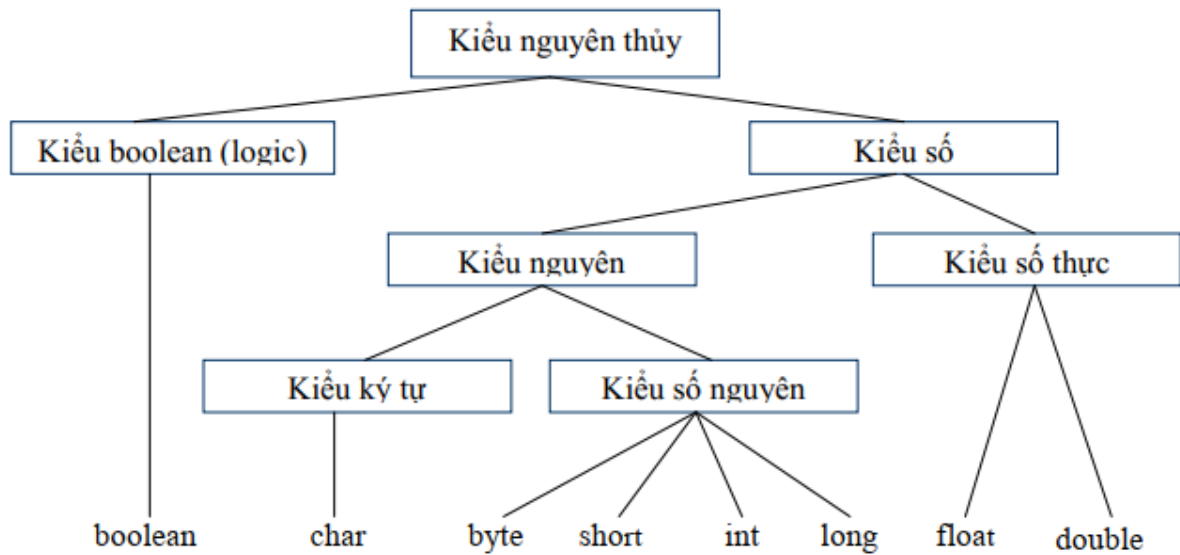
Bảng 3.2 Các kí tự định dạng xuất dữ liệu

Escape Sequence	Mô tả
<code>\n</code>	Xuống dòng mới
<code>\r</code>	Chuyển con trỏ đến đầu dòng hiện hành
<code>\t</code>	Chuyển con trỏ đến vị trí dừng Tab kế tiếp (ký tự Tab)
<code>\\</code>	In dấu \
<code>\'</code>	In dấu nháy đơn (')
<code>\"</code>	In dấu nháy kép (")

3.2. CÁC KIỂU DỮ LIỆU TRONG JAVA

Java có 2 dạng dữ liệu: Dữ liệu nguyên thủy và dữ liệu tham chiếu đối tượng.

Các kiểu dữ liệu nguyên thủy là các kiểu dữ liệu có sẵn và định nghĩa trước của java:



Hình 3.1 Các kiểu dữ liệu nguyên thủy trong Java

Mỗi kiểu nguyên thủy có một lớp bao bọc (Wrapper hay còn gọi lớp nguyên thủy) tương ứng để sử dụng các giá trị nguyên thủy như là các đối tượng. Ví dụ ứng với kiểu int có lớp Integer, ứng với char là Char vv.. các lớp này sẽ cung cấp một số phương thức hỗ trợ trên kiểu tương ứng.

3.3. BIẾN, HẰNG, BIỂU THỨC VÀ CÁC PHÉP TOÁN CƠ BẢN

3.3.1. Biến (variable)

Biến là một tên được đặt để đại diện cho một giá trị dữ liệu được lưu trữ trong bộ nhớ. Các biến trong Java là thành phần cơ bản để lưu trữ và quản lý dữ liệu trong chương trình. Mỗi biến có một kiểu dữ liệu nhất định, như kiểu số nguyên, số thực, ký tự, chuỗi, v.v.

Khai báo biến:

[phạm vi] <Kiểu dữ liệu> <tên biến>;

hoặc khai báo đồng thời khởi tạo giá trị ban đầu luôn:

[phạm vi] <Kiểu dữ liệu> <tên biến> = <giá trị>;

Lưu ý: chỉ khai báo biến thành phần của lớp mới có phạm vi truy cập.

<i>Kiểu dữ liệu</i>	<i>Giá trị mặc định</i>
boolean	false
char	'\u0000'
byte, short, int, long	0
float, double	+0.0F, +0.0D
Tham chiếu đối tượng	null

Hình 3.2 Giá trị mặc định của các kiểu dữ liệu nguyên thủy

Khi khai báo: Nếu các biến không được khởi tạo giá trị tường minh thì:

- Các biến tĩnh luôn được khởi tạo với các giá trị mặc định.
- Các biến thành phần được khởi tạo mặc định mỗi khi đối tượng của lớp có thành phần đó được khởi tạo.
- Biến tham chiếu được gán mặc định là null nếu không tạo lập tường minh theo toán tử new và toán tử tạo lập (constructor).

Phạm vi hoạt động của biến

Một biến có phạm vi hoạt động là sau khi khai báo và trong toàn bộ khối lệnh mà nó được khai báo. Một khối lệnh bắt đầu bằng dấu “{” và kết thúc bằng dấu “}”:

- Nếu biến được khai báo trong một cấu trúc lệnh điều khiển, biến đó có phạm vi hoạt động trong khối lệnh tương ứng.
- Nếu biến được khai báo trong một phương thức (Không nằm trong khối lệnh nào), biến đó có phạm vi hoạt động trong phương thức tương ứng: có thể được sử dụng trong tất cả các khối lệnh của phương thức.

Nếu biến được khai báo trong một lớp (Không nằm trong một phương thức nào), biến đó có phạm vi hoạt động trong toàn bộ lớp tương ứng: có thể được sử dụng trong tất cả các phương thức của lớp.

Dựa vào phạm vi sử dụng, Java có ba loại biến chính:

- **Biến cục bộ (Local Variable):** Là biến được khai báo bên trong một phương thức (method), constructor hoặc khối mã (block). Phạm vi của biến này chỉ nằm trong phương thức hoặc khối mã chứa nó. Khi phương thức kết thúc, biến cục bộ sẽ bị hủy.

Ví dụ 1:

```
public void myMethod() {  
    int localVar = 5; // Biến cục bộ  
    System.out.println(localVar);  
}
```

- **Biến instance (Instance Variable):** Là biến được khai báo trong một lớp (class) nhưng bên ngoài mọi phương thức. Biến instance được tạo ra khi một đối tượng của lớp được tạo ra, và nó thuộc về đối tượng đó. Các biến instance có thể có các giá trị khác nhau đối với các đối tượng khác nhau.

Ví dụ 2:

```
public class MyClass {  
    int instanceVar = 10; // Biến instance  
}
```

- **Biến tĩnh (Static Variable):** Còn được gọi là biến lớp (class variable), biến tĩnh được khai báo với từ khóa static. Biến tĩnh thuộc về lớp chứ không phải một đối tượng cụ thể nào.

Nó có thể được truy cập trực tiếp thông qua tên lớp mà không cần tạo đối tượng. Tất cả các đối tượng của lớp sẽ chia sẻ cùng một biến tĩnh.

Ví dụ 3:

```
public class MyClass {  
    static int staticVar = 20; // Biến tĩnh  
}
```

3.3.2. Hằng (constant)

Hằng là một giá trị không thể thay đổi sau khi được gán lần đầu tiên. Hằng thường được sử dụng để lưu trữ các giá trị cố định trong chương trình, giúp mã nguồn trở nên rõ ràng và dễ bảo trì hơn. Để khai báo một hằng trong Java, ta sử dụng từ khóa **final**.

Ví dụ 4: **final int** MAX_VALUE = 100;

Trong đó:

- **final** là từ khóa dùng để tạo ra một hằng.
- MAX_VALUE là tên của hằng số. Theo quy ước, tên hằng số thường được viết bằng chữ in hoa để dễ nhận biết.
- 100 là giá trị cố định của hằng số MAX_VALUE và không thể thay đổi được.

Hằng số trong lớp (Class Constants): Hằng số thường được khai báo là **static final** trong các lớp để có thể truy cập mà không cần tạo đối tượng. Khi đó, hằng số sẽ là thuộc tính của lớp (class attribute) và không thuộc về một đối tượng cụ thể nào.

3.3.3. Các phép toán cơ bản trong Java

Toán tử số học

Bảng 3.3 Toán tử số học

Toán tử	Ý nghĩa
+	Cộng
-	Trừ
*	Nhân
/	Chia
%	Chia dư
++	Tăng 1
--	Giảm 1

- Phép chia nguyên (2 toán hạng đều là kiểu nguyên) đòi hỏi số chia phải khác 0.
- Phép chia số thực (ít nhất một toán hạng kiểu số thực) cho phép chia cho 0 và kết quả phép chia cho 0.0 là INF (số lớn vô cùng) hoặc -INF (số âm vô cùng), hai hằng đặc biệt trong Java.

- Trong Java, phép chia lấy số dư % thực hiện được cả đối với số thực. (float m = 11.5 % 2.5; // Cho m là 1.5)
- Khi sử dụng các phép toán đơn nguyên đối với các đối số kiểu byte, short, hoặc char thì trước tiên toán hạng phải được tính rồi chuyển về kiểu int và kết quả là kiểu int.

Ví dụ 5:

short h = 30; // OK: 30 kiểu int chuyển về short (mặc định đối với hằng nguyên)

h = h + 4; // Lỗi vì h + 4 cho kết quả kiểu int không thể gán cho h kiểu short phải ép kiểu: h = (short) (h+4);

Toán tử quan hệ & logic

Bảng 3.4 Toán tử quan hệ & logic

Toán tử	Ý nghĩa
==	So sánh bằng
!=	So sánh khác
>	So sánh lớn hơn
<	So sánh nhỏ hơn
>=	So sánh lớn hơn hay bằng
<=	So sánh nhỏ hơn hay bằng
	OR (biểu thức logic)
&&	AND (biểu thức logic)
!	NOT (biểu thức logic)

Toán tử gán (assignment)

Bảng 3.5 Toán tử gán

Toán tử	Ví dụ	Giải thích
=	int c=3, d=5; c=4;	
+=	c+=7	c=c+7
-=	d-=4	d=d-4
=	e=5	e=e*5
/=	f/=3	f=f/3

Toán tử điều kiện

<điều kiện> ? <biểu thức 1> : <biểu thức 2>

Ví dụ:

```
int x = 10;
int y = 20;
int Z = (x < y) ? 30 : 40;
```

Kết quả: 30

Phép dịch trái: <<

$a \ll n$ Dịch tất cả các bit của a sang trái n lần, điền số 0 vào bên phải. ($a \ll n$ tương đương với $a * 2^n$).

Phép dịch phải và điền bit dấu: >>

$a \gg n$ Dịch tất cả các bit của a sang phải n lần, điền bit dấu vào bên trái.

Phép dịch phải và điền bit 0: >>>

$a \ggg n$ Dịch tất cả các bit của a sang phải n lần, điền 0 vào bên trái. $A \ggg n$ tương đương với $a * n / 2$

Giá trị của đối số bên phải (ở trên là n) luôn là số nguyên (dương) do vậy đối số bên trái (ở trên là a) nếu là *byte* hoặc *short* thì phải đổi sang kiểu *int*. Kết quả của các phép chuyển dịch vì vậy sẽ luôn là *int* (hoặc là kiểu *long* nếu đối số thứ nhất là *long*).

Chuyển đổi kiểu:

– Ép kiểu

Qui tắc ép kiểu có dạng: (**<type>**) **<exp>**

Chuyển kết quả tính toán của biểu thức **<exp>** sang kiểu được ép là **<type>**.

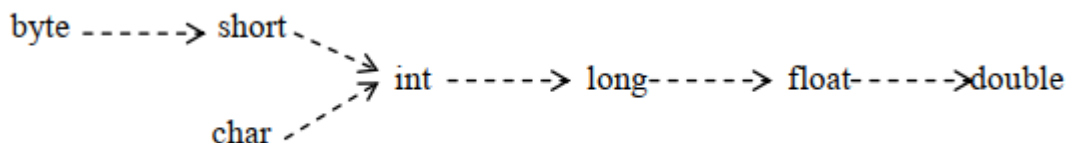
Ví dụ: float f = (float) 100.15D; // Chuyển số 100.15 dạng kiểu *double* sang *float*

Lưu ý:

- Không cho phép chuyển đổi giữa các kiểu nguyên thủy với kiểu tham chiếu, ví dụ kiểu *double* không thể ép sang các kiểu lớp như *HocSinh* được.
- Kiểu giá trị *boolean* (logic) không thể chuyển sang các kiểu dữ liệu số và ngược lại.

– Mở rộng và thu hẹp kiểu

Giá trị của kiểu hẹp hơn (chiếm số byte ít hơn) có thể được chuyển sang những kiểu rộng hơn (chiếm số byte nhiều hơn) mà không tổn thất thông tin. Cách chuyển kiểu đó được gọi là mở rộng kiểu



Hình 3.3 Mở rộng kiểu

Ví dụ: **char** c = "A";

int k = c; // mở rộng kiểu char sang kiểu int (mặc định)

Chuyển đổi kiểu theo chiều ngược lại, từ kiểu rộng về kiểu hẹp hơn được gọi là *thu hẹp kiểu*.
Lưu ý là *thu hẹp kiểu có thể dẫn tới mất thông tin*.

3.4. CÁC CÂU LỆNH NHẬP XUẤT DỮ LIỆU CƠ BẢN QUA CÁC THIẾT BỊ VÀO RA CHUẨN

3.4.1. Nhập dữ liệu từ bàn phím

Java là một ngôn ngữ lập trình thuần hướng đối tượng, nên cách nhập dữ liệu từ bàn phím cũng phải sử dụng theo hướng đối tượng. Tức là ta phải khai báo một đối tượng, sau đó gọi hàm nhập, rồi gán giá trị cho biến.

Scanner là một lớp trong thư viện Java.util giúp nhập và lưu dữ liệu từ bàn phím. Ngoài scanner, còn có nhiều lớp khác nữa, nhưng đây là lớp được sử dụng nhiều nhất.

Các phương thức thường dùng trong lớp **Scanner**:

Tên phương thức	Tác dụng
nextBoolean	Nhập vào kiểu Boolean (true – false) từ bàn phím
nextByte	Nhập vào kiểu dữ liệu Byte
nextShort	Nhập vào kiểu Short (số nguyên từ -32768 đến 32767)
nextInt	Nhập vào kiểu số nguyên từ bàn phím
nextFloat	Nhập vào kiểu số thực
nextDouble	Nhập vào kiểu Double
nextLine	Nhập vào kiểu String
nextLong	Nhập vào số nguyên lớn

Cú pháp: Scanner console = **new** Scanner(System.in);

console.nextInt(); // Gọi hàm nhập số nguyên. Các hàm khác cú pháp tương tự

3.4.2. Xuất dữ liệu ra màn hình

Cú pháp

System.out.println("Nội dung cần in" + biểu thức);

System.out.print("Nội dung cần in" + biểu thức);

System.out.printf("Nội dung cần in" + biểu thức);

- Với *print*: Xuất kết quả ra màn hình nhưng con trỏ chuột không xuống dòng.
- Với *println*: Xuất kết quả ra màn hình đồng thời con trỏ chuột nhảy xuống dòng tiếp theo.
- Với *printf*: Xuất ra màn hình kết quả đồng thời có thể định dạng được kết quả đó nhờ vào các đối số thích hợp.

System.out.printf(local, format, arguments1, arguments2,..., argumentsn);

Trong đó:

- *local*: Nếu khác null sẽ được tự động định dạng theo khu vực.
- *format*: Quy định chuẩn định dạng đầu ra cho các đối số
- Các *argument*: Đối số cần định dạng.

Các bộ định dạng có sẵn trong printf:

%c: Ký tự
 %d: Số thập phân (số nguyên) (cơ số 10)
 %e: Dấu phẩy động theo cấp số nhân
 %f: Dấu phẩy động
 %i: Số nguyên (cơ số 10)
 %o: Số bát phân (cơ số 8)
 %s: Chuỗi
 %u: Số thập phân (số nguyên) không dấu
 %x: Số trong hệ thập lục phân (cơ số 16)
 %t: Định dạng ngày / giờ
 %%: Dấu phần trăm
 \%%: Dấu phần trăm

Ví dụ 1: Chương trình tính tổng hai số nguyên được nhập từ bàn phím và hiển thị tổng đó ra màn hình:

```
import java.util.Scanner;

public class SumTwoIntegers {
    public static void main(String[] args) {
        // Tạo đối tượng Scanner để nhận đầu vào từ bàn phím
        Scanner scanner = new Scanner(System.in);
        // Yêu cầu người dùng nhập số nguyên thứ nhất
        System.out.print("Nhập số nguyên thứ nhất: ");
        int number1 = scanner.nextInt();
        // Yêu cầu người dùng nhập số nguyên thứ hai
        System.out.print("Nhập số nguyên thứ hai: ");
        int number2 = scanner.nextInt();
        // Tính tổng hai số
        int sum = number1 + number2;
        // Hiển thị kết quả
        System.out.println("Tổng hai số nguyên là: " + sum);
        // Đóng Scanner để tránh rò rỉ tài nguyên
```

```

        scanner.close();
    }
}

```

3.5. CÁC CẤU TRÚC ĐIỀU KHIỂN CHƯƠNG TRÌNH TRONG JAVA

3.5.1. Câu lệnh rẽ nhánh

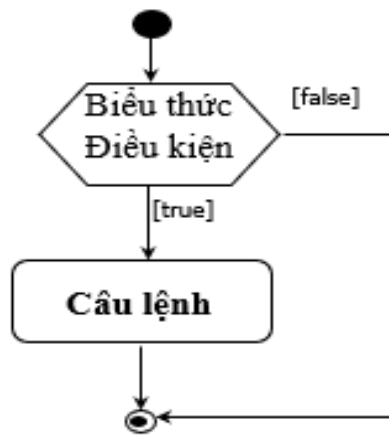
Câu lệnh if đơn giản

```

if ( <Biểu thức điều kiện>
    <Câu lệnh>;

```

Nếu biểu thức điều kiện đúng thì thực hiện câu lệnh



Hình 3.4 Câu lệnh if đơn giản

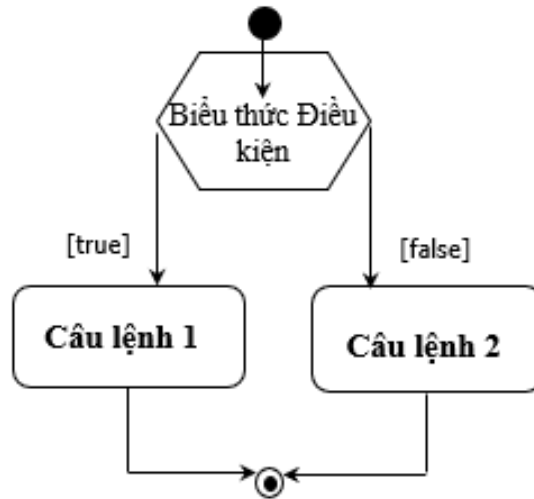
Câu lệnh if ... else

```

if ( <Biểu thức điều kiện>
    <Câu lệnh 1>;
else
    <Câu lệnh 2>;

```

Nếu biểu thức điều kiện đúng thì thực hiện câu lệnh 1 nếu không thì thực hiện câu lệnh 2 (nếu có **else**)



Hình 3.5 Câu lệnh if ... else

Ví dụ 1:

```

public class Test {
    public static void main(String[] args) {
        int number = 13;
        if (number % 2 == 0) {
            System.out.println("Số " + number + " là số chẵn.");
        } else {
            System.out.println("Số " + number + " là số lẻ.");
        }
    }
}

```

Kết quả:

Số 13 là số lẻ.

Câu lệnh switch

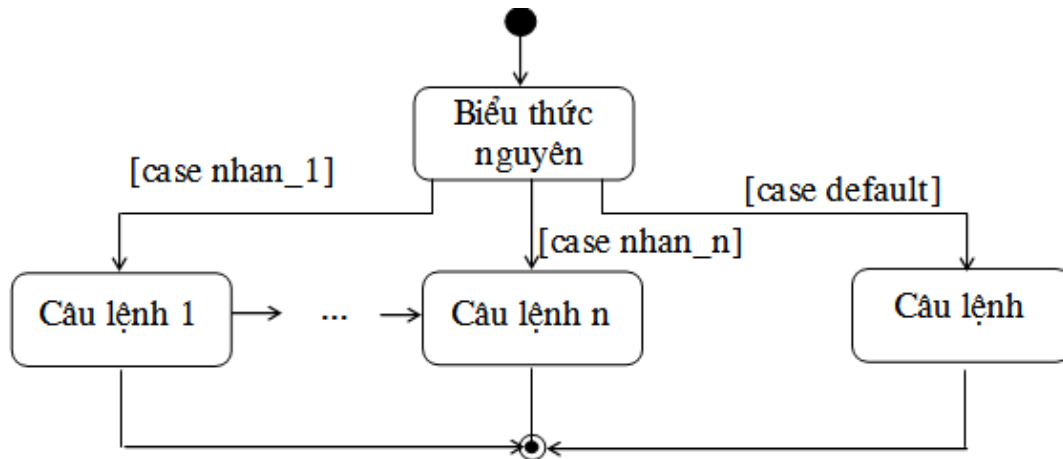
```

switch (<Biểu thức nguyên>) {
    case nhan_1: <Câu lệnh 1>;
    case nhan_2: <Câu lệnh 2>;
    ...
    case nhan_n: <Câu lệnh n>;
    [default: <Câu lệnh >];
}

```


Câu lệnh *switch* sẽ kiểm tra giá trị của *Biểu thức nguyên* và so sánh với từng giá trị *nhân1*, *nhân2*, ..., lần lượt từ trên xuống dưới, mỗi giá trị cần so sánh gọi là một *case*. Khi một trường hợp đúng (true), khối lệnh ở trong *case* đó sẽ được thực thi.

Nếu tất cả các trường hợp đều sai (tức là các *nhân* ở *case* không bằng với *Biểu thức nguyên*), thì câu lệnh ở trong *default* sẽ được thực thi.



Hình 3.6 Câu lệnh switch

Ví dụ 2:

```

public class SwitchExample {
    public static void main(String[] args) {
        int number = 20;
        switch (number) {
            case 10:
                System.out.println("10");
                break;
            case 20:
                System.out.println("20");
                break;
            case 30:
                System.out.println("30");
                break;
            default:
                System.out.println("Not in 10, 20 or 30");
        }
    }
}

```

```
}
```

Kết quả:

20

Lưu ý:

- Khối *default* là không bắt buộc có ở cấu trúc *switch case* trong Java
- Trong một *switch* có thể có nhiều *case*
- Nếu không gặp lệnh *break* trong khối lệnh này, thì chương trình sẽ thực hiện tiếp các *case* bên dưới cho tới khi nó gặp lệnh *break* thì nó sẽ thoát ra khỏi *switch*, hoặc cho đến khi thực hiện hết câu lệnh.

Ví dụ 3: mệnh đề *switch-case* khi không sử dụng '*break*'

```
public class SwitchExample2 {  
    public static void main(String[] args) {  
        int number = 20;  
        switch (number) {  
            case 10:  
                System.out.println("10");  
            case 20:  
                System.out.println("20");  
            case 30:  
                System.out.println("30");  
            default:  
                System.out.println("Not in 10, 20 or 30");  
        }  
    }  
}
```

Kết quả:

20

30

Not in 10, 20 or 30

3.5.2. Các câu lệnh lặp

Vòng lặp for

for (<Biểu thức bắt đầu>; <Điều kiện lặp>; <Biểu thức gia tăng>)
<Khối lệnh>

Trong đó:

- *Biểu thức bắt đầu*: được thực thi đầu tiên, và chỉ một lần. Bước này cho phép khai báo và khởi tạo bất kỳ biến điều khiển vòng lặp. Tất cả các biến được khai báo trong <Biểu thức bắt đầu> đều là cục bộ trong khối thân của chu trình **for**.
- *Điều kiện lặp*: Nếu nó là **true**, phần *Khối lệnh* được thực thi. Nếu nó là **false**, phần *Khối lệnh* không thực thi và luồng điều khiển nhảy tới lệnh tiếp theo sau vòng lặp **for**.
- *Biểu thức gia tăng*: Thay đổi (tăng hoặc giảm) giá trị của biến điều khiển sau mỗi lần lặp

Ví dụ 4:

```
public class Main {
    public static void main(String args[]) {
        for (int i = 0; i < 5; i++) {
            System.out.println("Vị trí thứ: " + i);
        }
    }
}
```

Kết quả:

```
Vị trí thứ 0
Vị trí thứ 1
Vị trí thứ 2
Vị trí thứ 3
Vị trí thứ 4
```

Lưu ý:

- Các thành phần của chu trình **for** là tùy chọn. Một trong <Biểu thức bắt đầu>, <Biểu thức gia tăng>, <Điều kiện kết thúc> có thể trống. Trường hợp <Điều kiện kết thúc> là trống thì điều kiện lặp của chu trình được xem là **true**.
- **for** (; ;) được sử dụng để xây dựng chu trình lặp vô điều kiện.

Vòng lặp foreach

```
for (<kiểu dữ liệu> <tên biến chạy> : <tên mảng>) {
    <Khối lệnh>
}
```

Vòng lặp *for-each* được sử dụng để duyệt qua từng phần tử trong một mảng hoặc một tập hợp. trong đó:

- <kiểu dữ liệu>: Kiểu dữ liệu của từng phần tử trong mảng hoặc tập hợp mà bạn muốn duyệt qua. Kiểu này phải trùng với kiểu của các phần tử trong mảng.
- <tên biến chạy>: Tên của biến sẽ đại diện cho từng phần tử trong mảng khi vòng lặp chạy qua. Biến này sẽ lần lượt nhận giá trị của từng phần tử trong mảng.

- <tên mảng>: Tên của mảng hoặc tập hợp chứa các phần tử mà bạn muốn duyệt qua.
- <Khối lệnh>: Là khối lệnh được thực thi cho mỗi phần tử trong mảng. Trong khối này, có thể sử dụng <tên biến chạy> để truy cập giá trị của phần tử hiện tại.

Ví dụ 5:

```
public class Main {
    public static void main(String args[]) {
        int[] mang = {4,3,2,1,0};
        for (int i : mang) {
            System.out.println("Giá trị thứ : " + i);
            i++;
        }
    }
}
```

Kết quả:

```
Giá trị thứ : 4
Giá trị thứ : 3
Giá trị thứ : 2
Giá trị thứ : 1
Giá trị thứ : 0
```

Lưu ý: Vòng lặp for-each chỉ cho phép duyệt qua các phần tử của mảng hoặc tập hợp; không thể thay đổi trực tiếp các phần tử trong mảng.

Vòng lặp while

```
while (<điều kiện>)
    <khối lệnh>
```

Trong đó:

- *điều kiện*: Biểu thức **boolean**, nó trả về giá trị **true** hoặc **false**. Vòng lặp sẽ tiếp tục cho đến khi nào giá trị **true** được trả về
- *khối lệnh*: Các câu lệnh được thực hiện nếu *điều kiện* nhận giá trị True

Ví dụ 6:

```
public class Main {
    public static void main(String args[]) {
        int i=0;
        while(i<5){
            System.out.println("Vị trí thứ "+i);
            i++;
        }
    }
}
```

```
}  
}
```

Vòng lặp **while** kiểm tra điều kiện nếu $i < 5$ thì in ra vị trí i . Mỗi lần lặp, chúng ta tăng i thêm 1 đơn vị, do đó, sau 5 lần lặp, i có giá trị là 5 và vòng lặp sẽ dừng lại.

Kết quả:

Vị trí thứ 0
Vị trí thứ 1
Vị trí thứ 2
Vị trí thứ 3
Vị trí thứ 4

Vòng lặp do...while

Vòng lặp do ... while là tương tự như Vòng lặp while, ngoại trừ rằng phần thân của vòng lặp do...while được bảo đảm thực thi ít nhất một lần. Nói cách khác, vòng lặp do ... while thực hiện phần thân vòng lặp trước khi kiểm tra điều kiện.

```
do{  
    <khối lệnh>  
}while (<điều kiện>)
```

Trong đó:

- *điều kiện*: Biểu thức **boolean**, nó trả về giá trị **true** hoặc **false**. Vòng lặp sẽ tiếp tục cho đến khi nào giá trị **true** được trả về
- *khối lệnh*: Các câu lệnh được thực hiện 1 lần sau đó kiểm tra điều kiện, nếu đúng thì thực hiện tiếp và ngược lại.

Ví dụ 7:

```
public class Main {  
    public static void main(String args[]) {  
        int i = 0;  
        do {  
            System.out.println("Vị trí thứ : " + i);  
            i++;  
        } while (i < 5);  
    }  
}
```

Kết quả:

Vị trí thứ 0
Vị trí thứ 1

Vị trí thứ 2

Vị trí thứ 3

Vị trí thứ 4

3.5.3. Các lệnh chuyển vị

Câu lệnh break:

Câu lệnh **break** được sử dụng trong các khối được gắn nhãn, trong các chu trình lặp (**for**, **while**, **do-while**) và câu lệnh **switch** để chuyển điều khiển thực hiện chương trình ra khỏi khối trong cùng chứa nó.

- Trong gắn nhãn: phần còn lại của khối bị bỏ qua
- Trong chu trình lặp: khi gặp lệnh break thì phần còn lại của thân chu trình được bỏ qua và kết thúc chu trình đó,
- Trong lệnh **switch**: phần còn lại của lệnh **switch** bị bỏ qua và tiếp tục thực hiện lệnh đứng sau lệnh **switch** đó.

Ví dụ 8:

```
public class BreakExample {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 10; i++) {  
            if (i == 5) {  
                break;  
            }  
            System.out.println(i);  
        }  
    }  
}
```

Kết quả:

1
2
3
4

Ví dụ 9: sử dụng break với vòng lặp bên trong vòng lặp for khác:

```
public class BreakExample2 {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 3; i++) {  
            for (int j = 1; j <= 3; j++) {  
                if (i == 2 && j == 2) {
```

```

        break;
    }
    System.out.println(i + " " + j);
}
}
}
}

```

Kết quả:

```

1 1
1 2
1 3
2 1
3 1
3 2
3 3

```

Câu lệnh continue

Câu lệnh **continue** được sử dụng trong các chu trình lặp **for**, **while**, **do-while** để dừng sự thực hiện của lần lặp hiện thời và bắt đầu lặp lại lần tiếp theo nếu điều kiện lặp còn thoả mãn (*true*).

Ví dụ 10: sử dụng **continue** trong java với vòng lặp **for**:

```

public class ContinueExample {
    public static void main(String[] args) {
        for (int i = 1; i <= 10; i++) {
            if (i == 5) {
                continue;
            }
            // Khi i == 5 thì không in i = 5 ra màn hình
            System.out.println(i);
        }
    }
}

```

Kết quả:

```

1
2
3

```

4
6
7
8
9
10

Ví dụ 11: sử dụng **continue** với vòng lặp bên trong vòng lặp **for** khác:

```
public class ContinueExample2 {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 3; i++) {  
            for (int j = 1; j <= 3; j++) {  
                if (i == 2 && j == 2) {  
                    continue;  
                }  
                // Không in trường hợp i=2 và j=2 ra màn hình  
                System.out.println(i + " " + j);  
            }  
        }  
    }  
}
```

Kết quả:

1 1
1 2
1 3
2 1
2 3
3 1
3 2
3 3

Câu lệnh return

Câu lệnh **return** được sử dụng để kết thúc thực hiện của hàm hiện thời và chuyển điều khiển chương trình về lại sau lời gọi hàm đó.

Bảng 3.6 Dạng câu lệnh return

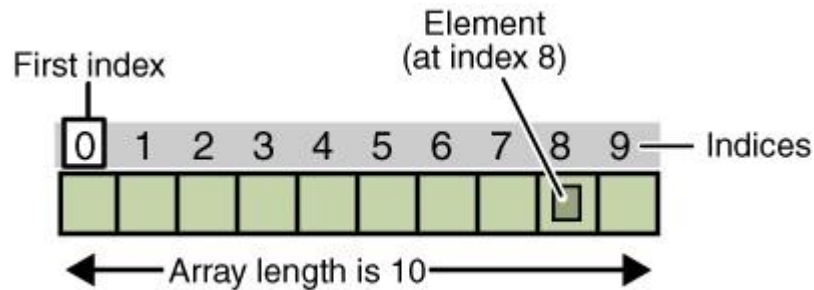
Dạng câu lệnh return	Hàm khai báo void	Hàm có kiểu trả lại khác void
----------------------	-------------------	-------------------------------

return	Tùy chọn	Không cho phép
return <Biểu thức>	Không cho phép	Bắt buộc

3.6. CẤU TRÚC MẢNG, XÂU KÝ TỰ TRONG JAVA

3.6.1. Cấu trúc mảng

Mảng (array) là tập hợp các phần tử có cùng kiểu dữ liệu và có số phần tử là cố định (không thay đổi được số phần tử sau khi tạo lập). Trong java mảng là kiểu tham chiếu do vậy ta phải khởi tạo mảng trước khi sử dụng.



Hình 3.7 Mảng

Khai báo

<Kiểu các phần tử> [] <Tên mảng>;

<Kiểu các phần tử> <Tên mảng>[];

Trong đó <Kiểu các phần tử> có thể là kiểu nguyên thủy hoặc là kiểu lớp (tên lớp).

Kích thước của mảng chưa xác định khi khai báo

Tạo lập mảng

Để tạo lập đối tượng mảng thì phải xác định số phần tử của mảng đó.

<Tên mảng> = new <Kiểu các phần tử> [<Số phần tử>];

Trong đó:

<Kiểu các phần tử> là kiểu tương thích với kiểu mà mảng đã khai báo. Giá trị cực tiểu của <Số phần tử> là 0.

Kết hợp cả khai báo với tạo lập mảng như sau:

<Kiểu các phần tử 1> <Tên mảng>[] = new <Kiểu các phần tử 2> [<Số phần tử>];

Khai báo và tạo lập mảng <Tên mảng> kiểu <Kiểu các phần tử 1> để chứa <Số phần tử> các phần tử có <Kiểu các phần tử 2>.

<Kiểu các phần tử 1> và <Kiểu các phần tử 2> là hai kiểu tương thích với nhau. Nếu mảng đó có kiểu nguyên thủy thì hai kiểu đó phải trùng nhau. Nếu là kiểu lớp thì <Kiểu các phần tử 2> là lớp con của <Kiểu các phần tử 1> (một lớp cũng được xem là lớp con của chính nó).

Khi một mảng được tạo lập thì tất cả các phần tử của nó được khởi tạo giá trị mặc định (0 hoặc 0.0 đối với kiểu số, false đối với kiểu boolean, null cho kiểu lớp).

Ví dụ 1:

```
float mangFloat[] = new float[20]; // Các phần tử được gán trị mặc định
```

```
Object[] dayDen = new BongDen[5]; // Các phần tử được gán mặc định null khởi tạo các mảng.
```

Khai báo, tạo lập và gán ngay các giá trị ban đầu:

```
<Kiểu các phần tử>[] <Tên mảng> = {<Các giá trị ban đầu>};
```

Ví dụ 2: **int**[] mangInt = {1, 3, 5, 7, 9};

Tạo ra *mangInt* có 5 phần tử với phần tử đầu có giá trị là 1 (*mangInt*[0] = 1), phần tử thứ hai là 3 (*mangInt*[1] = 3), v.v.

```
Object[] dayDT = {new BongDen(), new BongDen(), null};
```

```
char[] charArr = {"a", "b", "a"}; // mảng 3 ký tự và hoàn toàn khác với "aba"
```

Truy cập các phần tử của mảng

Truy cập một phần tử mảng bằng cách sử dụng số chỉ số (index) trong đó $0 \leq index < (\text{số phần tử mảng})$. Phần tử thứ *i* của mảng là phần tử có chỉ số *i* - 1 (*<Tên mảng>*[*i* - 1]). Nếu truy cập vượt quá phạm vi xác định của mảng sẽ gặp ngoại lệ *ArrayIndexOutOfBoundsException*

Truy cập vào phần tử của mảng: *<tên mảng>*[*i*]

Ví dụ 3:

```
public class TruyCapArray1 {  
    public static void main(String[] args) {  
        String[] cars = { "Honda", "BMW", "Ford", "Mazda" };  
        System.out.println(cars[0]);  
    }  
}
```

Kết quả:

Honda

Để thay đổi giá trị của một phần tử cụ thể trong mảng, ta cũng dùng chỉ số:

```
public class TruyCapArray2 {  
    public static void main(String[] args) {  
        String[] cars = { "Honda", "BMW", "Ford", "Mazda" };  
        // thay đổi phần tử đầu tiên của mảng cars  
        cars[0] = "Morning";  
        // hiển thị phần tử đầu tiên của mảng cars  
        System.out.println("Phần tử đầu tiên: " + cars[0]);  
    }  
}
```

Kết quả:

Phần tử đầu tiên: Morning

Để biết có bao nhiêu phần tử trong một mảng, ta sử dụng thuộc tính **length**:

```
public class DodaiArray1 {  
    public static void main(String[] args) {  
        String[] cars = { "Honda", "BMW", "Ford", "Mazda" };  
        System.out.println("Độ dài của mảng cars là: " + cars.length);  
    }  
}
```

Kết quả:

4

Duyệt các phần tử của mảng trong Java Có 2 cách để duyệt các phần tử của mảng:

- Sử dụng vòng lặp for
- Sử dụng foreach

Sử dụng vòng lặp for

Ta có thể lặp qua các phần tử mảng bằng vòng lặp for và sử dụng thuộc tính length để chỉ định số lần vòng lặp sẽ chạy.

Ví dụ 4:

```
public class DuyệtArray1 {  
    public static void main(String[] args) {  
        String[] cars = { "Honda", "BMW", "Ford", "Mazda" };  
        for (int i = 0; i < cars.length; i++) {  
            System.out.println(cars[i]);  
        }  
    }  
}
```

Kết quả:

Honda

BMW

Ford

Mazda

Sử dụng foreach

```
public class DuyệtArray2 {  
    public static void main(String[] args) {
```

```
String[] cars = { "Honda", "BMW", "Ford", "Mazda" };
for (String car : cars) {
    System.out.println(car);
}
}
```

Kết quả:

```
Honda
BMW
Ford
Mazda
```

So sánh for với foreach

Bảng 3.7 So sánh for với foreach

Vòng lặp for là một cấu trúc điều khiển để chỉ định phép lặp cho phép lệnh được thực thi nhiều lần.	Vòng lặp foreach là một cấu trúc điều khiển để duyệt các mục trong một mảng hoặc một tập hợp.
Vòng lặp for có thể được sử dụng để truy xuất một tập hợp các phần tử cụ thể.	Vòng lặp foreach không thể được sử dụng để truy xuất một tập hợp các phần tử cụ thể.
Vòng lặp for khó đọc và ghi hơn vòng lặp foreach.	Vòng lặp foreach dễ đọc hơn vòng lặp for.
Vòng lặp for được sử dụng như một vòng lặp mục đích chung.	Vòng lặp foreach được sử dụng cho mảng và tập hợp.

Mảng nhiều chiều

Định nghĩa:

<Kiểu các phần tử> [][]...[] <Tên mảng>;

hoặc

<Kiểu các phần tử> <Tên mảng> [][]...[];

Ví dụ 5:

int[][] mang1; // Mảng hai chiều

tương đương với *int mang1[][];*

tương đương với *int[] mang1[];*

Khai báo với thiết lập mảng nhiều chiều tương tự mảng đơn.

int[][] mangA = new int[4][5]; // Ma trận có 4 hàng, 5 cột

Truy nhập tới từng phần tử : *mangA[i][j]*.

Kích thước của *mangA* là *mangA.length = 4* và mỗi phần tử của nó lại là mảng có kích thước là *mangA[i].length = 5*, *i = 1, 2, ..., 4*.

double[][] maTran = {

```

{1, 2, 3, 4}, // hàng 1
{0, 2, 0, 0}, // hàng 2
{0, 0, 3, 0}, // hàng 3
{0, 0, 0, 4}, // hàng 4

```

};

Lưu ý: Các mảng trong mảng nhiều chiều không nhất thiết phải có số phần tử giống nhau.

3.6.2. Xâu ký tự

Lớp String Lớp String dùng cho những xâu ký tự bất biến (xâu chỉ đọc), khi được khởi tạo giá trị đầu, giá trị đó không thay đổi được nữa.

Phương thức khởi tạo (Constructor):

```

String(String s)
String(<Kieu>[] mang)
String(StringBuffer buf)

```

Ví dụ 6:

```

byte[] bytes = {97, 98, 98, 97};
char[] characters = {"a", "b", "b", "a"};
StringBuffer buff = new StringBuffer("abba");
String byteStr = new String(bytes); // Chuyển mảng bytes về xâu
String charStr = new String(characters); // charStr = "abba"
String buffStr = new String(buff);
String str = new String("abba");

```

Có thể sử dụng: String str = "Dung dong den toi";

Ví dụ 7: Khởi tạo chuỗi trong Java

```

public class StringInitialization {
    public static void main(String a[]) {
        String objStr_01 = "This is a string object";
        String objStr_02 = new String("This is also string object");
        char[] arrChar = {'V', 'N', 'L', 'T', 'V', 'E', 'S'};
        String objStr_03 = new String(arrChar);
        String objStr_04 = objStr_03 + " This is another String object";
        System.out.println("objStr_01: " + objStr_01);
        System.out.println("objStr_02: " + objStr_02);
        System.out.println("objStr_03: " + objStr_03);
        System.out.println("objStr_04: " + objStr_04);
    }
}

```

```
}  
}
```

Kết quả chạy chương trình:

```
objStr_01: This is a string object  
objStr_02: This is also string object  
objStr_03: VNLIVES  
objStr_04: VNLIVES This is another String object
```

Đọc từng ký tự

+ *int length()*: kích thước chuỗi.

+ *char charAt(int index)*: cho lại ký tự thứ *index* (bắt đầu từ 0) của chuỗi hiện thời ($0 \leq \text{index} < \text{length}()$). (ngoại lệ *StringIndexOutOfBoundsException*).

So sánh các chuỗi

boolean equals(Object obj): kiểm tra xem đối tượng lớp *String* và đối tượng ở tham số có cùng tập ký hiệu hay không

boolean equalsIgnoreCase(String str2) kiểm tra xem đối tượng lớp *String* và đối tượng ở tham số có cùng tập ký hiệu hay không (phân biệt chữ hoa và chữ thường)

boolean startsWith(String str)

boolean endsWith(String str)

Hai hàm này cho kết quả *true* nếu chuỗi bắt đầu (kết thúc) bằng đối số *str*.

int compareTo(String str2)

int compareTo(Object obj)

+ 0 nếu chuỗi hiện thời bằng chuỗi đối số,

+ nhỏ hơn 0, nếu chuỗi đó nhỏ hơn chuỗi đối số theo thứ tự từ điển,

+ lớn hơn 0, nếu chuỗi đó lớn hơn chuỗi đối số theo thứ tự từ điển,

Hàm *compareTo()* thứ hai thực hiện tương tự như hàm đầu nếu đối số là chuyển về được đối tượng chuỗi, ngược lại sẽ cho qua để xử lý ngoại lệ *ClassCastException*.

Chuyển đổi chuỗi:

String toUpperCase()

String toLowerCase()

Ghép chuỗi

String concat(String str)

Tìm các ký tự và các chuỗi con Nếu không tìm được các hàm này sẽ cho kết quả là -1:

int indexOf(int ch) Tìm chỉ số của lần xuất hiện đầu tiên của *ch* trong chuỗi.

int indexOf(int ch, int fromIndex) Tìm chỉ số của lần xuất hiện đầu tiên của *ch* trong chuỗi bắt đầu từ *fromIndex*.

int indexOf(String str) Tìm chỉ số của lần xuất hiện đầu tiên của xâu con *str* trong xâu hiện thời.

int indexOf(String str, int fromIndex) Tìm chỉ số của lần xuất hiện đầu tiên của xâu con *str* bắt đầu từ *fromIndex* trong xâu hiện thời.

int lastIndexOf(int ch) Tìm chỉ số của lần xuất hiện cuối cùng của ký tự *ch* trong xâu.

int lastIndexOf(int ch, int fromIndex) Tìm chỉ số của lần xuất hiện cuối cùng của *ch* bắt đầu từ *fromIndex* trong xâu.

int lastIndexOf(String str) Tìm chỉ số của lần xuất hiện cuối cùng của xâu con *str* trong xâu hiện thời.

int lastIndexOf(String str, int fromIndex) Tìm chỉ số của lần xuất hiện cuối cùng của xâu con *str* bắt đầu từ *fromIndex* trong xâu hiện thời.

String replace(char cu, char moi) Thay tất cả các lần xuất hiện của ký tự *cu* bằng ký tự *moi* trong xâu.

Trích ra các xâu con

String trim() tạo một xâu mới, các ký tự “trắng” (những ký tự có giá trị nhỏ hơn giá trị dấu cách „ „) ở trong xâu đều bị loại bỏ.

String substring(int startIndex) Cho kết quả là một xâu con được triết ra từ vị trí *startIndex* đến cuối của xâu.

String substring(int startIndex, int endIndex) Cho kết quả là một xâu con được triết ra từ vị trí *startIndex* đến vị trí *endIndex* của xâu. (ngoại lệ *StringIndexOutOfBoundsException*).

Chuyển các đối tượng của Object về String

static String valueOf(Object obj)

static String valueOf(char[] characters)

Hai hàm này được nạp chồng để chuyển các đối tượng *obj* và mảng các ký tự *characters* về xâu ký tự.

static String valueOf(boolean b)

static String valueOf(char c)

Hàm được nạp chồng để chuyển hai giá trị boolean: *true*, *false* về xâu ký tự “*true*”, “*false*” tương ứng.

static String valueOf(int i)

static String valueOf(long l)

static String valueOf(float f)

static String valueOf(double d)

Các hàm này được nạp chồng để chuyển các giá trị số về dạng xâu ký tự.

Ví dụ 8: Sử dụng các hàm cơ bản của chuỗi.

public class ChuoiCoBan {

```

public static void main(String[] args) {
    String str = "Android.Vn Android.Vn";
    System.out.println("str = " + str);
    // Lấy từ vị trí số 8 tới cuối cùng của chuỗi
    System.out.println("str.substring(8) = " + str.substring(8));
    // Lấy từ vị trí số 3 tới vị trí số 9
    System.out.println("str.substring(3,9) = " + str.substring(3, 9));
    // Độ dài chuỗi:
    System.out.println("Độ dài chuỗi: str.length() = " + str.length());
    // Lấy ra ký tự trong chuỗi theo chỉ số
    char ch;
    ch = str.charAt(4);
    System.out.println("str.charAt(4) = " + ch);
    // Thay 1 ký tự bằng ký tự khác trong chuỗi:
    System.out.println("Thay tất cả ký tự 'n' bằng ký tự 'x' = " + str.replace('n', 'x'));
    // Tìm chuỗi "And" là chuỗi con của chuỗi str, thay kết quả đầu tiên bằng chuỗi
    "xxx"
    System.out.println("Thay And đầu tiên bằng xxx = " + str.replaceFirst("And",
    "xxx"));
    // Thay toàn bộ chuỗi "And" của chuỗi str bằng chuỗi "xxx"
    System.out.println("Thay tất cả And bằng xxx = " + str.replaceAll("And",
    "xxx"));
    // Chuyển thành chữ thường:
    System.out.println("str chuyển về viết thường: " + str.toLowerCase());
    // Chuyển thành chữ hoa:
    System.out.println("str chuyển về viết hoa: " + str.toUpperCase());
    // Loại bỏ khoảng trống 2 bên chuỗi
    String str1 = " " + str + " ";
    System.out.println(" Android.Vn Android.Vn --> " + str1.trim());
}
}

```

Lớp StringBuffer Lớp StringBuffer được sử dụng đối với những xâu ký tự động, có thể thay đổi nội dung.

Tạo lập đối tượng StringBuffer

StringBuffer(String str) Tạo ra đối tượng mới nội dung giống như *xâu* đối số *str* kích thước bằng kích thước của *xâu str* + 16.

StringBuffer(int length) Tạo ra đối tượng mới chưa có nội dung và đặt kích thước bằng đối số *length*, nếu đối số lớn hơn 0.

StringBuffer() Tạo ra đối tượng mới chưa có nội dung và đặt kích thước của *xâu buffer* là 16.

Đọc và thay đổi các ký tự trong StringBuffer

int length() Cho số ký tự trong *xâu buffer*.

char charAt(int index)

void setCharAt(int index, char ch)

Hàm đầu đọc một ký tự và hàm thứ hai thay đổi ký tự ở vị trí *index* trong *xâu buffer* thành *ch*. Hệ thống sẽ cho qua ngoại lệ *StringIndexOutOfBoundsException* nếu các chỉ số không tương thích.

Chuyển StringBuffer sang String

Lớp *StringBuffer* nạp chồng hàm *toString()* để chuyển *xâu buffer* về *xâu* của lớp cố định *String*.

Hàm *append()* bổ sung các ký tự vào cuối của *xâu*.

StringBuffer append(Object obj)

Đối tượng *obj* được chuyển về *xâu* (bằng hàm *String.valueOf()*) và sau đó bổ sung vào *xâu buffer*.

StringBuffer append(String str)

StringBuffer append(char[] str)

StringBuffer append(char[] str, int offset, int len)

StringBuffer append(char c)

Các mảng ký tự *str* và ký tự *c* được chuyển về *xâu* và sau đó bổ sung vào *xâu buffer*.

StringBuffer append(boolean b)

StringBuffer append(int i)

StringBuffer append(long l)

StringBuffer append(float f)

StringBuffer append(double d)

Các giá trị nguyên thủy được chuyển về *xâu* và sau đó bổ sung vào *xâu buffer*.

StringBuffer insert(int offset, Object obj)

StringBuffer insert(int offset, String str)

StringBuffer insert(int offset, char[] str)

StringBuffer insert(int offset, char c)

StringBuffer insert(int offset, boolean b)

StringBuffer insert(int offset, int i)

StringBuffer insert(int offset, long l)

StringBuffer insert(int offset, float f)

StringBuffer insert(int offset, double d)

chèn đối số thứ hai (sau khi được chuyển về dạng chuỗi bằng cách sử dụng hàm *String.valueOf()*) vào từ vị trí *offset* trong chuỗi *buffer*.

StringBuffer delete(int index)StringBuffer delete(int start, int end)

Hàm đầu xóa đi ký tự ở vị trí *index*, hàm sau xóa đi một chuỗi con kể từ vị trí *start* đến *end*.

StringBuffer reverse() đảo ngược chuỗi

Kiểm soát kích thước của StringBuffer

int capacity(): Cho lại kích thước (khả năng chứa) của chuỗi *buffer*.

void ensureCapacity(int minCap): Đảm bảo đủ chỗ cho ít nhất *minCap* ký tự của chuỗi *buffer*.

void setLength(int newLen): Đặt độ dài của chuỗi *buffer* là *newLen*.

3.7. NGOẠI LỆ VÀ XỬ LÝ NGOẠI LỆ TRONG JAVA

Mọi đoạn chương trình đều tiềm ẩn khả năng sinh lỗi. Lỗi chủ quan: do lập trình sai, lỗi khách quan: do dữ liệu, do trạng thái của hệ thống.

Ngoại lệ: các trường hợp hoạt động không bình thường của chương trình ví dụ: đọc 1 file không tồn tại, lỗi chia cho 0 với số nguyên, truy cập vượt quá phạm vi xác định của mảng v.v... Xử lý ngoại lệ như thế nào? Làm thế nào để có thể tiếp tục thực hiện?

Cách xử lý lỗi truyền thống: Cài đặt mã xử lý tại nơi phát sinh ra lỗi, làm cho chương trình trở nên khó hiểu, không phải lúc nào cũng đầy đủ thông tin để xử lý, không nhất thiết phải xử lý, truyền trạng thái lên mức trên, thông qua tham số, giá trị trả lại hoặc biến tổng thể (flag). Dễ nhầm, khó kiểm soát được hết các trường hợp, lỗi số học, lỗi bộ nhớ,... Lập trình viên thường quên không xử lý lỗi do bản chất con người, thiếu kinh nghiệm, cố tình bỏ qua.

Ví dụ, việc chia cho 0 sẽ tạo một lỗi trong chương trình hoặc khi xét thao tác nhập xuất (I/O) trong một tập tin. Nếu việc chuyển đổi kiểu dữ liệu không thực hiện đúng, một ngoại lệ sẽ xảy ra và chương trình bị hủy mà không đóng lại tập tin. Lúc đó tập tin dễ bị hư hại và các nguồn tài nguyên được cấp phát cho tập tin không được thu hồi lại cho hệ thống.

Ví dụ 1: chương trình chia 2 số. Nếu ta cho mẫu số = 0 thì phát sinh lỗi và đó được coi là 1 ngoại lệ.

```
public class ExceptionExample1 {  
    public static void main(String[] args) {  
        int zero = 0;  
        int average = 10 / zero;  
        System.out.println("Average = " + average);  
    }  
}
```

Khi thực thi chương trên, sẽ nhận được thông báo lỗi như sau:

Exception in thread "main" java.lang.ArithmeticException: / by zero

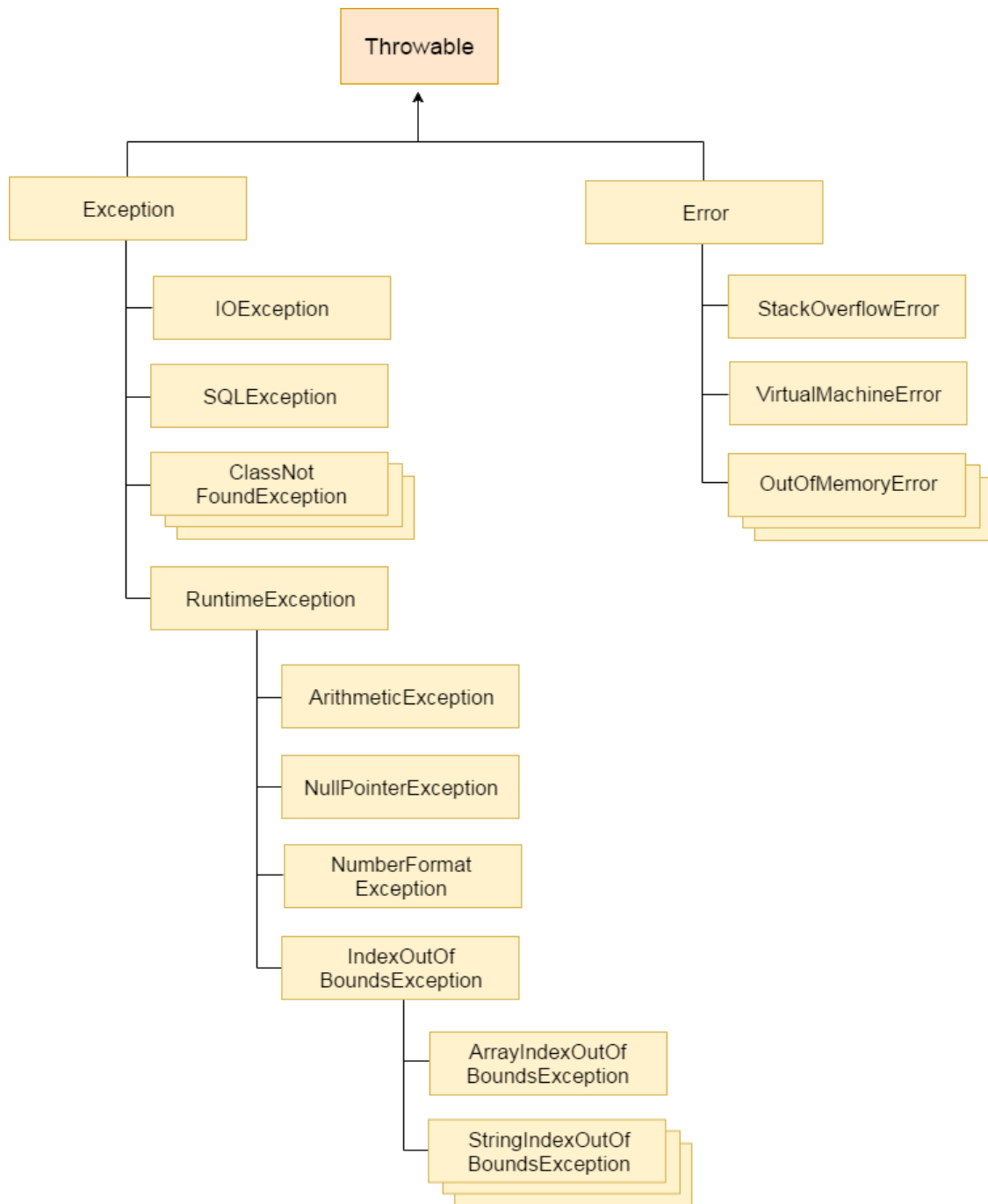
Ngôn ngữ Java cung cơ chế tự động dùng cho bắt và xử lý ngoại lệ. Có hai cách để xử lý ngoại lệ: Bắt, xử lý ngoại lệ và *ném* ngoại lệ (chuyển hướng hay ủy quyền xử lý ngoại lệ đến một vị trí khác của chương trình).

Ưu điểm của bắt ngoại lệ

- Dễ sử dụng
- Dễ dàng chuyển điều khiển đến nơi có khả năng xử lý ngoại lệ
- Có thể ném nhiều loại ngoại lệ
- Tách xử lý ngoại lệ khỏi thuật toán
- Tách mã xử lý
- Sử dụng cú pháp khác
- Không bỏ sót ngoại lệ (ném tự động)
- Làm chương trình dễ đọc hơn, an toàn hơn

3.7.1. Cấu trúc phân cấp của các lớp xử lý ngoại lệ

Trong Java, cấu trúc phân cấp của các lớp xử lý ngoại lệ được xây dựng trên gốc là lớp Throwable, chia thành hai nhánh chính: Error và Exception. Cấu trúc phân cấp chi tiết được mô tả như hình 3.8 dưới đây:



Hình 3.8 Ngoại lệ

Throwable: Là lớp gốc trong hệ thống xử lý ngoại lệ của Java. Chỉ các đối tượng thuộc lớp Throwable (hoặc các lớp con của nó) mới có thể được ném ra bằng câu lệnh throw và bắt bằng câu lệnh catch.

Error: Đại diện cho các lỗi nghiêm trọng liên quan đến môi trường thực thi (runtime environment), không thể hoặc không nên xử lý bằng chương trình. Lớp này chủ yếu dành cho các lỗi mà chương trình không nên xử lý, vì chúng thường yêu cầu sửa chữa môi trường hoặc cấu hình.

Bảng 3.8 Ngoại lệ

Ngoại lệ	Lớp cha của thứ tự phân cấp ngoại lệ
RuntimeException	Lớp cơ sở cho nhiều ngoại lệ java.lang
ArithmeticException	Trạng thái lỗi về số, ví dụ như “chia cho 0”
IllegalAccessException	Lớp không thể truy cập
IllegalArgumentException	Phương thức nhận một đối số không hợp lệ
ArrayIndexOutOfBoundsException	Kích thước của mảng lớn hơn 0 hay lớn hơn kích thước thật sự của mảng
NullPointerException	Khi muốn truy cập đối tượng null
SecurityException	Việc thiết lập cơ chế bảo mật không được hoạt động
ClassNotFoundException	Không thể nạp lớp yêu cầu
NumberFormatException	Việc chuyển đổi không thành công từ chuỗi sang số thực
AWTException	Ngoại lệ về AWT
IOException	Lớp cha của các ngoại lệ I/O
FileNotFoundException	Không thể định vị tập tin
EOFException	Kết thúc một tập tin
NoSuchMethodException	Phương thức yêu cầu không tồn tại
InterruptedException	Khi một luồng bị ngắt

Trong Java có 2 loại exception: checked và unchecked. Tất cả các checked exception được kế thừa từ lớp Exception ngoại trừ lớp RuntimeException. RuntimeException là lớp cơ sở của tất cả các lớp unchecked exception. Đó cũng là dấu hiệu để nhận biết đâu là checked exception và đâu là unchecked exception.

Điểm khác biệt giữa các lớp checked và unchecked exception chính là thời điểm xác định được exception có thể xảy ra.

Checked exceptions

Là loại exception xảy ra trong lúc compile time, nó cũng có thể được gọi là **compile time exceptions**. Loại exception này không thể bỏ qua được trong quá trình compile, bắt buộc ta phải handle nó. Các lớp extends từ lớp Throwable ngoại trừ RuntimeException và Error được gọi là checked exception.

Ví dụ: IOException, FileNotFoundException, NoSuchFieldException,

Unchecked exceptions

Là loại exception xảy ra tại thời điểm thực thi chương trình, nó cũng có thể gọi là **runtime exceptions** đó là programming bugs, lỗi logic của chương trình... Loại exception này được bỏ qua trong quá trình compile, không bắt buộc ta phải handle nó.

Các lớp extends từ RuntimeException được gọi là unchecked exception.

Ví dụ: NullPointerException, NumberFormatException, ArrayIndexOutOfBoundsException, DivideByZeroException, ...

Error

Error là những vấn đề nghiêm trọng liên quan đến môi trường thực thi của ứng dụng hoặc hệ thống mà lập trình viên không thể kiểm soát. Nó thường làm chết chương trình.

Lớp Error định nghĩa các ngoại lệ mà không thể bắt (catch) từ chương trình.

Ví dụ: OutOfMemoryError, VirtualMachineError, and StackOverflowError, ...

Xử lý ngoại lệ (Exception Handling) trong java

Xử lý ngoại lệ (Exception Handling) trong java là một cơ chế xử lý các lỗi runtime để có thể duy trì luồng bình thường của ứng dụng.

Quá trình xử lý exception được gọi là catch exception, nếu Runtime System không xử lý được ngoại lệ thì chương trình sẽ kết thúc.



Hình 3.9 Quá trình xử lý ngoại lệ

Khi một lỗi xảy ra trên một method, method đó sẽ tạo ra một object và đưa nó vào **Runtime System**. Object đó được gọi là Exception Object, nó chứa tất cả các thông tin về lỗi và trạng thái của chương trình khi xảy ra lỗi.

Sau đó, Runtime System sẽ xử lý sẽ tìm cách xử lý ngoại lệ phù hợp được sử dụng tại method ấy. Nếu không có thì JVM tiếp tục tìm xử lý ngoại lệ phù hợp ở các method trên (là method gọi lớp hiện tại). Nếu không có method nào có xử lý ngoại lệ phù hợp thì Thread mà đang thực hiện chuỗi method xảy ra ngoại lệ bị ngắt. Nếu thread ấy là thread main thì chương trình sẽ bị lỗi.

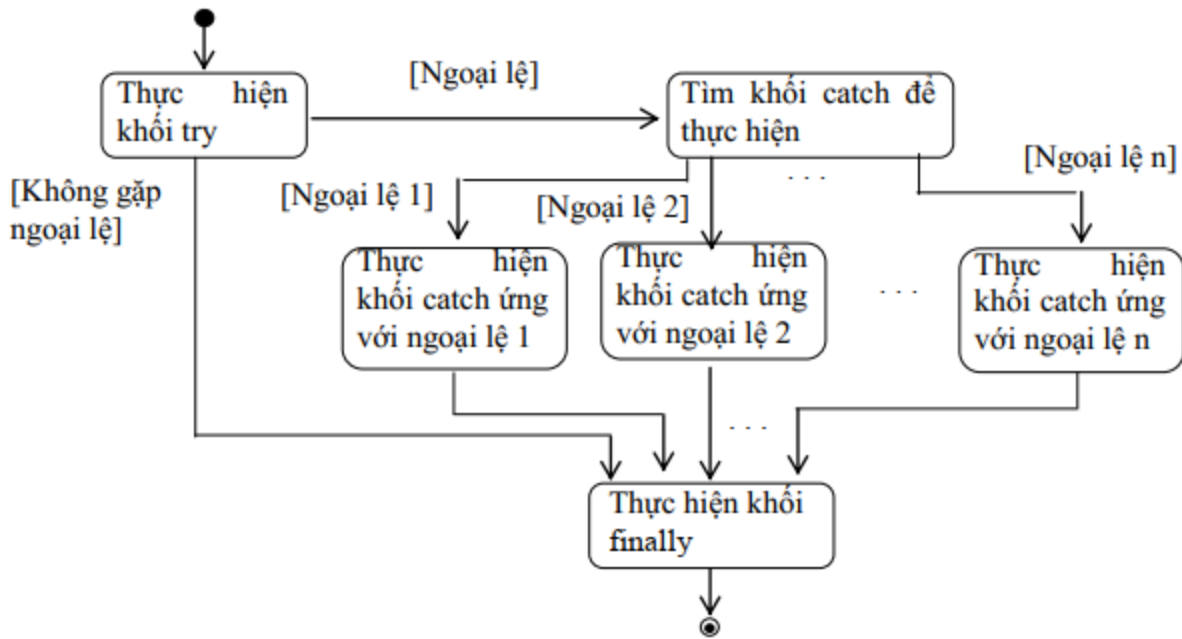
3.7.2. Khối lệnh try, catch và finally

try {	// Khối try
<Các câu lệnh có khả năng gây ra ngoại lệ>	
}	
catch (<Kiểu ngoại lệ 1> <Tham biến 1>) {	// Khối catch
<Các câu lệnh xử lý khi xuất hiện kiểu ngoại lệ 1>	
}	
...	
catch (<Kiểu ngoại lệ n> <Tham biến n>) {	// Khối catch
<Các câu lệnh xử lý khi xuất hiện kiểu ngoại lệ n>	
}	
finally {	// Khối finally
< Các câu lệnh luôn được thực hiện >	
}	

Hình 3.10 Khối Try catch

Các ngoại lệ sẽ được cho qua trong quá trình thực hiện khối *try* và sẽ bị tóm lại để xử lý ở các khối *catch* tương ứng nếu có. Khối *finally* phải thực hiện đến cùng, bất luận có gặp phải ngoại lệ hay không.

Hoạt động của các khối trên được minh họa như sau:



Hình 3.11 Hoạt động của khối Try catch

Khối try

Một chương trình cũng có thể chứa các khối *try* lồng nhau. Khi sử dụng các “*try*” lồng nhau, khối *try* bên trong được thi hành đầu tiên. Bất kỳ ngoại lệ nào bị chặn trong khối *try* sẽ bị bắt giữ trong các khối *catch* theo sau. Nếu khối *catch* thích hợp không được tìm thấy thì các khối *catch* của các khối *try* bên ngoài sẽ được xem xét. Nếu không, Java Runtime Environment xử lý các ngoại lệ.

Khối catch

Khối này chỉ được sử dụng để xử lý ngoại lệ nếu có. Khi một khối *catch* được thực hiện thì các khối *catch* còn lại sẽ bị bỏ qua.

Có thể bắt nhiều loại ngoại lệ khác nhau bằng cách sử dụng nhiều khối lệnh *catch* đặt kế tiếp

Khối lệnh *catch* sau không thể bắt ngoại lệ là lớp dẫn xuất của ngoại lệ được bắt trong khối lệnh *catch* trước.

Khối finally

Khi một ngoại lệ xuất hiện, phương thức đang được thực thi có thể bị dừng mà không được thi hành toàn vẹn. Nếu điều này xảy ra, thì các đoạn mã sẽ không bao giờ được gọi. Khối *finally* thực hiện tất cả các việc thu dọn khi một ngoại lệ xảy ra. Khối *finally* bảo đảm lúc nào cũng được thực thi, bất chấp có ngoại lệ xảy ra hay không. Khối *finally* là tùy ý, không bắt buộc.

Khối *finally* có thể dùng để:

- Đóng tập tin.
- Đóng lại bộ kết quả (sử dụng trong chương trình cơ sở dữ liệu).
- Đóng lại các kết nối được tạo trong cơ sở dữ liệu.

Ví dụ 2: minh họa cách sử dụng các khối *try* và *catch*.


```

class TryClass{
    public static void main(String args[])
    {
        int demo=0;
        try { System.out.println(20/demo); }
        catch(ArithmeticException a) {
            System.out.println("Cannot Divide by zero");
        }
    }
}

```

Kết quả:

Cannot Divide by zero

a được sử dụng như một đối tượng của `ArithmeticException` để in các chi tiết về các toán tử ngoại lệ mà hệ thống cung cấp. Nếu bạn thay thế lệnh `System.out.println` của khối `catch` bằng lệnh `System.out.println(a.getMessage())` thì kết quả của chương trình sẽ như sau:

/ by zero

3.7.3. Ném ngoại lệ bằng lệnh ‘throw’

Để tạm thời bỏ qua ngoại lệ chúng ta có thể sử dụng câu lệnh `throw`. Đoạn lệnh sau chỉ ra cách sử dụng của lệnh `throw`:

```

try{ if (flag<0)
    {throw new MyException();} // user-defined
}

```

Ví dụ 3:

```

if (0==denominator) {
    throw new Exception();
} else res = nominator / denominator;

```

Giả sử phương thức `x()` gọi phương thức `y()`. Phương thức `y()` chặn một ngoại lệ không được xử lý. Phương thức gọi `x()` nên khai báo việc chặn cùng một ngoại lệ với phương thức được gọi `y()`. Ta nên khai báo khối `try catch` trong phương thức `x()` để đảm bảo rằng ngoại lệ không được truyền cho các phương thức mà gọi phương thức này.

Phương thức được định nghĩa lại tại lớp dẫn xuất có thể không ném ngoại lệ. Nếu ném ngoại lệ, chỉ có thể ném ngoại lệ giống tại phương thức của lớp cơ sở hoặc ngoại lệ là lớp dẫn xuất của ngoại lệ được ném tại phương thức của lớp cơ sở.

Ví dụ 4:

```

class A {public void methodA() throws RuntimeException {}}

```

```

    }
    class B extends A {
        public void methodA() throws ArithmeticException {}
    }
    class C extends A {
        public void methodA() throws Exception { }
    }
    class D extends A {
        public void methodA() { }
    }
    A a = new B();
    try { a.methodA();}
    catch (RuntimeException e) { ...}

```

Ném lại ngoại lệ: Sau khi bắt ngoại lệ, nếu thấy cần thiết chúng ta có thể ném lại chính ngoại lệ vừa bắt được để cho chương trình mức trên tiếp tục xử lý.

```

    try {...}
    catch (Exception e) {
        System.out.println(e.getMessage());
        throw e;}

```

3.7.4. Mệnh đề throws

Khi thiết kế các hàm thành phần, chúng ta có thể sử dụng mệnh đề throws để tạm thời cho qua ngoại lệ mà thực hiện một số công việc cần thiết khác. Chúng ta ném ngoại lệ khỏi phương thức khi:

- Không nhất thiết phải xử lý ngoại lệ trong phương thức
- Không đủ thông tin để xử lý
- Không đủ thẩm quyền

Định nghĩa hàm với mệnh đề *throws* có dạng:

<Thuộc tính của hàm> <tên hàm>(<Danh sách các tham biến>) throws <Danh sách các kiểu ngoại lệ> { /* ... */}

Trong đó <Danh sách các kiểu ngoại lệ> là các lớp xử lý ngoại lệ được kế thừa từ lớp Exception và được phân tách bởi dấu ‘,’.

Ném ngoại lệ từ main()

- Nếu không có phương thức nào bắt ngoại lệ, ngoại lệ sẽ được truyền lên phương thức main() và được cần được xử lý tại đây.
- Nếu vẫn không muốn xử lý ngoại lệ, chúng ta có thể để ngoại lệ truyền lên mức điều khiển của máy ảo bằng cách khai báo main() ném ngoại lệ

- chương trình sẽ bị dừng và hệ thống sẽ in thông tin về ngoại lệ trên Console (printStackTrace())

Ví dụ 5:

```
public class VD_NgoaiLe2 {
    public static void main(String[] args) {
        try {
            divide(10, 0);
        } catch (ArithmeticException e) {
            System.out.println("Không thể chia cho số 0!");
        }
    }

    public static void divide(int a, int b) throws ArithmeticException {
        int result = a / b;
        System.out.println("Kết quả là: " + result);
    }
}
```

So sánh sự khác nhau giữa từ khóa throw và throws trong java được mô tả trong bảng dưới đây:

Bảng 3.9 So sánh sự khác nhau giữa từ khóa throw và throws

	throw	throws
1	Từ khóa throw trong java được sử dụng để ném ra một ngoại lệ rõ ràng.	Từ khóa throws trong java được sử dụng để khai báo một ngoại lệ.
2	Ngoại lệ checked không được truyền ra nếu chỉ sử dụng từ khóa throw.	Ngoại lệ checked được truyền ra ngay cả khi chỉ sử dụng từ khóa throws.
3	Sau throw là một instance.	Sau throws là một hoặc nhiều class.
4	throw được sử dụng trong phương thức.	throws được khai báo ngay sau dấu đóng ngoặc đơn của phương thức.
5	không thể throw nhiều exceptions.	có thể khai báo nhiều exceptions, Ví dụ: public void method()throws IOException,SQLException.

3.7.5. Tự định nghĩa ngoại lệ

Chúng ta có thể tạo lớp ngoại lệ để phục vụ các mục đích riêng

- Lớp ngoại lệ mới phải kế thừa từ lớp Exception hoặc lớp dẫn xuất của lớp này.
- Có thể cung cấp hai constructor

- constructor nhận một tham số String và truyền tham số này cho phương thức khởi tạo của lớp cơ sở.

Ví dụ 6:

```
class SimpleException extends Exception { }
class MyException extends Exception {
    public MyException() {}
    public MyException(String msg) {
        super(msg);}
}
```

Minh họa ngoại lệ được định nghĩa bởi người dùng *ArraySizeException*:

```
class ArraySizeException extends NegativeArraySizeException
{
    ArraySizeException() // constructor
    {
        super("You have passed an illegal array size");
    }
}
class ThrowDemo
{
    int size, array[];
    ThrowDemo(int s)
    {
        size=s;
        try {
            checkSize();
        }
        catch(ArraySizeException e){
            System.out.println(e);
        }
    }
    void checkSize() throws ArraySizeException
    {
        if (size < 0)
            throw new ArraySizeException();
    }
}
```

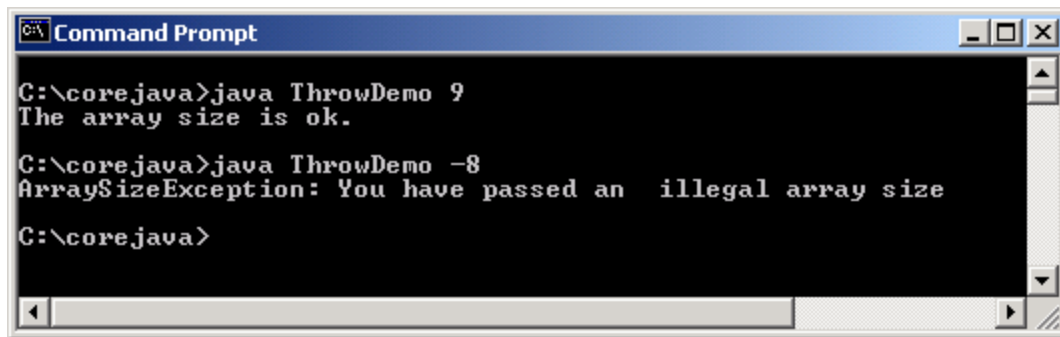
```

        else
            System.out.println("The array size is ok.");
        array = new int[3];
        for (int i=0; i<3; i++)
            array[i] = i+1;
    }
    public static void main(String arg[])
    {
        new ThrowDemo(Integer.parseInt(arg[0]));
    }
}

```

Lớp được định nghĩa bởi người dùng *ArraySizeException* là lớp con của lớp *NegativeArraySizeException*. Khi một đối tượng được tạo từ lớp này, thông báo về ngoại lệ được in ra. Phương thức *checkSize()* được gọi để chặn ngoại lệ *ArraySizeException* mà được chỉ ra bởi mệnh đề *throws*. Kích thước của mảng được kiểm tra trong cấu trúc *if*. Nếu kích thước là số âm thì đối tượng của lớp *ArraySizeException* được tạo. Phương thức *call()* được bao quanh trong khối *trycatch*, là nơi mà giá trị của đối tượng được in ra. Phương thức *call()* cần được bao trong khối *try*, để cho khối *catch* tương ứng có thể in ra giá trị

Kết quả của chương trình:



```

C:\core.java>java ThrowDemo 9
The array size is ok.

C:\core.java>java ThrowDemo -8
ArraySizeException: You have passed an illegal array size

C:\core.java>

```

3.7.6. Làn vết ngoại lệ StackTrace

Có thể sử dụng phương thức *printStackTrace()* để làn vết vị trí phát sinh ngoại lệ debug chương trình.

Ví dụ 7:

```

public class Test4 {
    void methodA() throws Exception {
        methodB();
        throw new Exception();
    }
}

```

```

    void methodB() throws Exception {
        methodC();
        throw new Exception();}
    void methodC() throws Exception {
        throw new Exception();
    }
    public static void main(String[] args) {
        Test4 t = new Test4();
        try {
            t.methodA();
        }
        catch(Exception e) {
            e.printStackTrace();}
    }
}

```

3.8. CASE STUDY

Case Study 1: Quản lý Điểm thi Sinh viên

Mục tiêu: Sinh viên sử dụng các biến, hằng, phép toán cơ bản, và câu lệnh nhập/xuất dữ liệu.

Yêu cầu chi tiết:

- **Nhập thông tin từ bàn phím:**
 - + Tên sinh viên (String).
 - + Số lượng môn học (int).
 - + Điểm số từng môn (float).
 - + Số tín chỉ của từng môn (int).
- **Tính toán:**
 - + Tổng số tín chỉ: Tổng tín chỉ = $\sum$ tín chỉ từng môn
 - + ĐTB: $\text{ĐTB} = \frac{\sum(\text{điểm từng môn} \times \text{tín chỉ})}{\text{Tổng tín chỉ}}$
- **Phân loại học lực:**
 - + $\text{ĐTB} \geq 3.5$: Xuất sắc.
 - + ĐTB từ 3.0 - 3.5: Giỏi.
 - + ĐTB từ 2.0 - 3.0: Khá.
 - + $\text{ĐTB} < 2.0$: Trung bình.
- **Xuất kết quả:**
 - + Tên sinh viên, ĐTB, và học lực.

Gợi ý triển khai: Sử dụng mảng để lưu điểm số và tín chỉ từng môn. Dùng vòng lặp *for* để tính tổng tín chỉ và điểm trung bình.

Case Study 2: Máy Tính Đơn Giản

Mục tiêu: Áp dụng các phép toán cơ bản, cấu trúc rẽ nhánh, và xử lý ngoại lệ.

Yêu cầu chi tiết:

- **Nhập hai số nguyên từ bàn phím:**
 - + Sử dụng Scanner.
- **Hiển thị menu với các lựa chọn:**
 - + Phép cộng: +.
 - + Phép trừ: -.
 - + Phép nhân: *.
 - + Phép chia: /.
- **Xử lý lựa chọn người dùng:**
 - + Nếu chia, kiểm tra số chia có bằng 0 hay không. Nếu có, ném ngoại lệ và xử lý bằng try-catch.
 - + Xuất kết quả sau mỗi phép tính.
- **Tiếp tục hay kết thúc:**
 - + Sau mỗi phép tính, hỏi người dùng có muốn tiếp tục không (Yes/No).
 - + Nếu chọn Yes, quay lại menu. Nếu chọn No, thoát chương trình.

Gợi ý triển khai: Sử dụng vòng lặp do-while để đảm bảo chương trình chạy ít nhất một lần và cho phép lặp lại khi cần.

Case Study 3: Quản lý Danh sách Sản phẩm

Mục tiêu: Làm việc với mảng, cấu trúc dữ liệu, và cấu trúc điều khiển.

Yêu cầu chi tiết:

- **Dữ liệu sản phẩm:**
 - + Tên sản phẩm (String).
 - + Giá sản phẩm (double).
 - + Số lượng trong kho (int).
- **Chức năng chương trình:**
 - + **Thêm sản phẩm mới:** Nhập thông tin sản phẩm và lưu trữ vào danh sách.
 - + **Tìm kiếm sản phẩm theo tên:** Dùng vòng lặp để duyệt danh sách sản phẩm. Nếu tên khớp, in ra thông tin sản phẩm.
 - + **Hiển thị danh sách sản phẩm:** Xuất danh sách gồm tên, giá, số lượng, và giá trị tổng (giá $\times$ số lượng).

Gợi ý triển khai: Sử dụng mảng để lưu danh sách sản phẩm. Triển khai chức năng tìm kiếm không phân biệt chữ hoa/chữ thường với equalsIgnoreCase.

Case Study 4: Giải Phương Trình Bậc Hai

Mục tiêu: Rèn luyện kỹ năng xử lý ngoại lệ, cấu trúc điều kiện, và luồng logic.

Yêu cầu chi tiết:

- **Nhập ba hệ số a,b,c, từ bàn phím.**
- **Kiểm tra điều kiện:**
 - + Nếu a=0: Ném ngoại lệ tùy chỉnh (IllegalArgumentException) với thông báo "Hệ số a không hợp lệ".
 - + **Tính nghiệm của phương trình:** Dùng định thức $\Delta = b^2 - 4ac$
 - Nếu $\Delta < 0$: Phương trình vô nghiệm.
 - Nếu $\Delta = 0$: Phương trình có nghiệm kép.
 - Nếu $\Delta > 0$: Phương trình có hai nghiệm phân biệt.
- **Hiển thị kết quả và StackTrace (nếu có lỗi).**

Gợi ý triển khai: Sử dụng ngoại lệ tùy chỉnh (CustomException) và in StackTrace bằng printStackTrace() trong khối catch.

Case Study 5: Thống Kê Dữ Liệu

Mục tiêu: Luyện tập với mảng, vòng lặp, và cấu trúc điều khiển.

Yêu cầu chi tiết:

- **Nhập danh sách số nguyên:**
 - + Người dùng nhập số lượng phần tử của mảng.
 - + Sau đó, nhập từng giá trị của mảng.
- **Thực hiện các thống kê:**
 - + Tổng giá trị của mảng.
 - + Số lớn nhất và nhỏ nhất trong mảng.
 - + Đếm số chẵn và số lẻ.
- **Sắp xếp mảng:**
 - + Sắp xếp tăng dần sử dụng thuật toán sắp xếp đơn giản (Bubble Sort, Selection Sort hoặc Arrays.sort).
- **Hiển thị kết quả:**
 - + Xuất các thống kê và mảng sau khi sắp xếp.

Gợi ý triển khai: Sử dụng vòng lặp for để duyệt qua các phần tử của mảng. Tận dụng các cấu trúc điều kiện (if) để thực hiện các thống kê.

3.9. VÍ DỤ ÁP DỤNG

Triển khai case study 2 với các thành phần theo yêu cầu của bài toán:

- **Nhập dữ liệu:**
 - + Người dùng được yêu cầu nhập hai số nguyên.
 - + Dữ liệu nhập được đọc bằng Scanner.
- **Hiển thị menu:**
 - + Menu với các lựa chọn phép toán cơ bản (cộng, trừ, nhân, chia).
- **Xử lý các phép toán:**
 - + Sử dụng cấu trúc switch để chọn phép toán dựa trên đầu vào của người dùng.
 - + Kiểm tra ngoại lệ chia cho 0 bằng if và ném ngoại lệ ArithmeticException.
- **Xử lý ngoại lệ:**
 - + ArithmeticException được bắt và hiển thị thông báo khi có lỗi chia cho 0.
 - + Exception bắt các lỗi nhập liệu khác (như nhập ký tự thay vì số).
- **Tiếp tục hay kết thúc:**
 - + Sau mỗi phép tính, chương trình hỏi người dùng có muốn thực hiện tính toán khác không.
 - + Nếu người dùng nhập "yes", chương trình tiếp tục; nếu không, chương trình thoát.

Mã lệnh tương ứng:

```
import java.util.Scanner;

public class SimpleCalculator {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        boolean continueProgram = true;
        while (continueProgram) {
            try {
                // Nhập hai số nguyên từ bàn phím
                System.out.print("Enter the first number: ");
                int num1 = scanner.nextInt();

                System.out.print("Enter the second number: ");
                int num2 = scanner.nextInt();
                // Hiển thị menu lựa chọn
                System.out.println("Choose an operation:");
                System.out.println("1. Addition (+)");
                System.out.println("2. Subtraction (-)");
                System.out.println("3. Multiplication (*)");
```

```

System.out.println("4. Division (/");
System.out.print("Your choice: ");

int choice = scanner.nextInt();
double result;
// Xử lý các phép toán
switch (choice) {
case 1:
    result = num1 + num2;
    System.out.println("Result: " + result);
    break;
case 2:
    result = num1 - num2;
    System.out.println("Result: " + result);
    break;
case 3:
    result = num1 * num2;
    System.out.println("Result: " + result);
    break;
case 4:
    if (num2 == 0) {
        throw new ArithmeticException("Cannot
divide by zero.");
    }
    result = (double) num1 / num2;
    System.out.println("Result: " + result);
    break;
default:
    System.out.println("Invalid choice. Please select a
valid operation.");
}
} catch (ArithmeticException e) {
    System.out.println("Error: " + e.getMessage());
} catch (Exception e) {

```

```

        System.out.println("Invalid input. Please enter valid
numbers.");

        scanner.nextLine(); // Clear the invalid input
    }
    // Hỏi người dùng có muốn tiếp tục không
    System.out.print("Do you want to perform another calculation?
(yes/no): ");

    String userChoice = scanner.next();
    continueProgram = userChoice.equalsIgnoreCase("yes");
}
System.out.println("Program terminated. Goodbye!");
scanner.close();
}
}

```

3.10. TỔNG KẾT CHƯƠNG

Chương 3 giới thiệu các thành phần cơ bản trong lập trình Java, bao gồm:

- Các phần tử cơ sở của Java: Giới thiệu những khái niệm nền tảng của ngôn ngữ Java.
- Các kiểu dữ liệu trong Java: Trình bày các kiểu dữ liệu cơ bản như kiểu nguyên thủy (int, float, char...) và kiểu tham chiếu (String, Array...).
- Biến, hằng, biểu thức và các phép toán cơ bản: Bao gồm cách khai báo và sử dụng biến, hằng, cùng với các phép toán cơ bản như toán tử số học, so sánh và logic.
- Câu lệnh nhập xuất dữ liệu cơ bản: Hướng dẫn cách nhập dữ liệu từ bàn phím và xuất dữ liệu ra màn hình.
- Các cấu trúc điều khiển chương trình: Giới thiệu các cấu trúc điều khiển như câu lệnh rẽ nhánh (if, switch), các câu lệnh lặp (for, while, do-while) và các lệnh chuyển vị (như break, continue).
- Cấu trúc mảng và chuỗi ký tự: Giải thích cách khai báo và sử dụng mảng, chuỗi ký tự trong Java.
- Ngoại lệ và xử lý ngoại lệ: Mô tả cấu trúc xử lý ngoại lệ, bao gồm các lớp ngoại lệ, các khối lệnh try, catch, finally, cách ném ngoại lệ (throw), khai báo ngoại lệ (throws), và tự định nghĩa ngoại lệ.

3.11. CÂU HỎI TRẮC NGHIỆM

1. Từ khóa nào trong Java dùng để khai báo hằng?

- A. const
- B. static
- C. final

D. constant

2. Kiểu dữ liệu nào sau đây không phải là kiểu nguyên thủy trong Java?

A. int

B. float

C. String

D. double

3. Trong Java, từ khóa nào được dùng để nhập dữ liệu từ bàn phím?

A. Console

B. Scanner

C. Input

D. System.in

4. Phép toán nào sau đây là phép toán logic?

A. &&

B. +

C. -

D. /

5. Biến trong Java cần được khai báo với gì trước khi sử dụng?

A. Tên biến

B. Kiểu dữ liệu

C. Giá trị khởi tạo

D. Toán tử

6. Phép toán nào dùng để lấy phần dư trong Java?

A. %

B. /

C. &

D. //

7. Từ khóa nào được dùng để thoát khỏi vòng lặp?

A. exit

B. return

C. break

D. continue

8. Từ khóa switch được sử dụng trong cấu trúc nào?

A. Vòng lặp

B. Rẽ nhánh

- C. Nhập dữ liệu
- D. Xuất dữ liệu

9. Cấu trúc nào cho phép khối mã chạy ít nhất một lần dù điều kiện sai?

- A. for
- B. while
- C. if
- D. do-while

10. Từ khóa nào dùng để ném ngoại lệ trong Java?

- A. throws
- B. throw
- C. try
- D. finally

11. Từ khóa try thường đi kèm với từ khóa nào để xử lý ngoại lệ?

- A. finally
- B. catch
- C. throws
- D. throw

12. Mảng trong Java có đặc điểm gì?

- A. Kích thước thay đổi được
- B. Chỉ lưu trữ kiểu dữ liệu nguyên thủy
- C. Kích thước cố định sau khi khởi tạo
- D. Không thể chứa kiểu dữ liệu char

13. System.out.println() được sử dụng để:

- A. Đọc dữ liệu
- B. In dữ liệu ra màn hình
- C. Ghi dữ liệu vào file
- D. Thoát chương trình

14. Giá trị trả về của biểu thức $(5 > 3) \&\& (4 < 6)$ là:

- A. true
- B. false
- C. 1
- D. 0

15. Biểu thức "Hello".concat(" World") sẽ trả về giá trị nào?

- A. Hello

- B. Hello World
- C. World
- D. HelloWorld

16. Kiểu dữ liệu nào sau đây chỉ có thể chứa một ký tự?

- A. String
- B. char
- C. int
- D. float

17. Scanner sử dụng phương thức nào để đọc số nguyên?

- A. nextInt()
- B. readInt()
- C. next()
- D. getInt()

18. Phát biểu nào sau đây là đúng về ngoại lệ?

- A. Ngoại lệ không thể xử lý bằng lệnh catch
- B. finally chỉ chạy khi có ngoại lệ
- C. finally luôn được thực thi bất kể ngoại lệ xảy ra hay không
- D. Không thể tự định nghĩa ngoại lệ

19. Lệnh *continue* được sử dụng để:

- A. Thoát khỏi vòng lặp
- B. Bỏ qua phần còn lại của vòng lặp hiện tại và tiếp tục vòng lặp tiếp theo
- C. Dừng chương trình
- D. Quay lại đầu chương trình

20. Từ khóa *throws* dùng để:

- A. Bắt ngoại lệ
- B. Khai báo ngoại lệ có thể xảy ra
- C. Kết thúc chương trình
- D. Chạy khối finally

21. Xâu ký tự trong Java là đối tượng của lớp nào?

- A. Char
- B. Character
- C. String
- D. Text

22. Giá trị nào sau đây là hợp lệ cho kiểu dữ liệu *boolean*?

- A. 1
- B. 0
- C. true
- D. "true"

23. Lệnh *if* là dạng câu lệnh thuộc loại nào?

- A. Cấu trúc lặp
- B. Cấu trúc rẽ nhánh
- C. Lệnh xuất dữ liệu
- D. Lệnh nhập dữ liệu

24. Phát biểu nào là đúng về phương thức *equals()* trong Java?

- A. So sánh hai đối tượng về tham chiếu
- B. So sánh hai đối tượng về giá trị
- C. Chỉ có thể dùng cho số
- D. Chỉ có thể dùng cho chuỗi

25. Giá trị khởi tạo của biến kiểu *int* nếu không gán là:

- A. 0
- B. 1
- C. null
- D. -1

26. Lệnh *System.exit(0)*; có tác dụng gì?

- A. Kết thúc vòng lặp
- B. Kết thúc chương trình
- C. Kết thúc khối lệnh try-catch
- D. Bắt đầu chương trình

27. Từ khóa *null* biểu thị:

- A. Một biến chưa khởi tạo
- B. Một biến có giá trị 0
- C. Một tham chiếu trống
- D. Một đối tượng rỗng

28. Ngoại lệ nào xảy ra khi chia một số nguyên cho 0?

- A. ArithmeticException
- B. NullPointerException
- C. ClassCastException
- D. IOException

29. Kết quả của biểu thức $10 + "5"$ là:

- A. 15
- B. 105
- C. Lỗi biên dịch
- D. 10

30. Để kiểm tra kích thước của một mảng arr, ta dùng:

- A. arr.length()
- B. arr.size()
- C. arr.length
- D. arr.getSize()

3.12. BÀI TẬP THỰC HÀNH

Bài tập 1: Viết chương trình nhập vào 3 số nguyên từ bàn phím, chương trình sẽ in ra số nguyên có giá trị lớn nhất

Bài tập 2: Viết chương trình nhập và 1 số nguyên từ bàn phím, chương trình sẽ in ra thông tin số vừa nhập là số ‘âm’, ‘dương’ hoặc là số ‘không’

Bài tập 3: Viết chương trình cho phép nhập vào 3 số thực tương ứng với độ dài của một tam giác và kiểm tra 3 số này có phải là 3 cạnh của một tam giác hay không(nếu nó thỏa mãn điều kiện tổng của hai cạnh bất kỳ đều lớn hơn cạnh thứ 3) .

Bài tập 4: Với n nhập từ bàn phím hay viết các chương trình tính tổng các dãy sau:

1. Tính tổng dãy

$$S=1+2-3+.....(-1)^{n+1}.n$$

2. Tính tổng dãy:

$$S=1!+2!+3!+...+n!$$

3. Nếu n lẻ: tính tổng các số lẻ <n, nếu n chẵn: tính tổng các số chẵn <n.

4.
$$S = \frac{1}{2} + \frac{2}{3} + + \frac{n-1}{n}$$

Bài tập 5: Sử dụng cấu trúc try, và 2 khối catch xây dựng chương trình nhập vào 2 số nguyên a, b và in ra kết quả a/b. Yêu cầu bắt các ngoại lệ NumberFormatException khi nhập a và b, ArithmeticException khi thực hiện a/b.

Bài tập 6: Dùng mệnh đề throw để tạo ra ngoại lệ khi nhập tuổi cho nhân viên (không âm và không lớn hơn 62 tuổi)

Bài tập 7: Xây dựng hàm nhapTuoi() và dùng mệnh đề Throws để chuyển ngoại lệ ra ngoài hàm và thực hiện bắt và xử lý các ngoại lệ khi gọi hàm nhapTuoi().

Chương 4 LỚP ĐỐI TƯỢNG VÀ CÁC THÀNH PHẦN CỦA LỚP

Chương 4 giới thiệu chi tiết các vấn đề gồm:

- Lớp đối tượng
- Các thành phần của lớp đối tượng
- Cách định nghĩa, xây dựng và sử dụng lớp đối tượng

4.1. LỚP ĐỐI TƯỢNG

4.1.1. Định nghĩa lớp đối tượng và các thành phần của lớp

Lớp là mô hình khái quát đặc tính của một họ thực thể vật lý hoặc trừu tượng trong chương trình.

Định nghĩa lớp là đặc tả một kiểu dữ liệu mới và mô tả cách cài đặt kiểu dữ liệu đó.

Mỗi đối tượng là một thể hiện của lớp.

* Cú pháp định nghĩa lớp

```
[<Phạm vi hoặc thuộc tính kiểm soát truy nhập>] class <Tên lớp> [extends <Tên lớp cha>] [implements <Tên giao diện>]  
{ <Các thành phần của lớp> }
```

* Giải thích:

- *Phạm vi*: Xác định phạm vi hoặc kiểm soát truy nhập:
- *abstract*: Không thể tạo ra một đối tượng cụ thể của lớp, lớp loại này là khung cơ sở nhất để dẫn xuất ra các lớp con cháu.
- *public*: (công khai) Cho phép sử dụng mọi nơi trong hệ thống.
- *protected*: cho phép truy cập trong gói và trong các lớp thuộc gói khác nhưng được kế thừa.
- *private*: chỉ được sử dụng bên trong lớp, không cho phép truy cập và kế thừa đối với các lớp khác ở trong cùng gói và khác gói.
- *final*: (hằng) nếu nó đứng trước khai báo lớp thì nó không được sử dụng để kế thừa cho các lớp con, nếu nó đứng trước khai báo biến thì nó là “hằng” (chỉ được gán giá trị một lần duy nhất), nếu nó đứng trước các phương thức thì nó sẽ không được phép viết đè ở các lớp con.
- Nếu không khai báo (*public*, *protected*, *private*) thì gọi là các thành phần “mặc định”, khi đó phạm vi là truy cập và kế thừa là trong gói chứa lớp đó.
- *tên_lớp_cha*: Tên lớp cha được kế thừa
- *DS_tên_các_giao_diện*: Tên các giao diện (được viết phân cách nhau bởi dấu “,”) được phép sử dụng các phương thức và các hằng.
- *class*, *extends*, *implements* là các từ khoá.

Những phần trong cặp [và] là tùy chọn(có thể có hoặc có thể không).

<Các thành phần của lớp> bao gồm:

- Các biến dữ liệu thành phần(attributes): Xác định các thuộc tính (dữ liệu) của lớp, thông thường dùng để mô tả các đặc tính của đối tượng thuộc lớp đó
- Các toán tử tạo lập(constructor): Xác định cách khởi tạo một đối tượng thuộc lớp
- Các phương thức(method): Xác định hành vi của các đối tượng thuộc lớp

4.1.2. Các thuộc tính dữ liệu của lớp

Thuộc tính dữ liệu của lớp là các biến thành phần có thể thuộc kiểu nguyên thủy, kiểu tham chiếu.

* Khai báo:

<Phạm_vi> <Kiểu> <tên_biến>;

<Phạm_vi> <Kiểu> <tên_biến> = <Giá_trị_khởi_tạo>;

- *Phạm vi*: xác định phạm vi truy cập của các đối tượng đến thuộc tính. có thể chọn một trong các phạm vi:
- *static*: Khi một thuộc tính được khai báo static thì khi đó thuộc tính đó là “sở hữu” của tất cả các đối tượng của lớp đó chứ không phải nó là của riêng của từng đối tượng thuộc lớp đó.
- *public*: Có thể truy cập mọi nơi trong hệ thống (các đối tượng khác lớp có thể truy cập được)
- *protected*: cho phép truy cập trong gói và trong các lớp thuộc gói khác nhưng được kế thừa.
- *private*: của riêng đối tượng chỉ bên trong đối tượng.
- *final*: Hằng (chỉ được khởi tạo giá trị một lần duy nhất)
- *volatile*: là biến được lưu trữ trên bộ nhớ chính, được dùng với luồng kết hợp với từ khóa synchronized.

4.1.3. Định nghĩa hàm thành phần

Hành vi của các đối tượng thuộc một lớp được xác định bởi các hàm thành phần của lớp đó, hành vi thể hiện “vai trò” của đối tượng đó trong hệ thống.

Là các hàm, thủ tục thực hiện các phép xử lý trên các biến thành phần.

* Khai báo

<Phạm_vi> <Kiểu_giá_trị> <tên_hàm>(<danh_sách_các_tham_số_hình_thức>)
<các_mệnh_đề_throws> {*//Thân hàm*}

Phạm_vi:

- *public*: Cho phép truy cập mọi nơi trong hệ thống
- *private*: Chỉ được truy xuất (gọi) bên trong lớp chứa nó
- *protected*: lớp đó và lớp con của nó được truy xuất
- *final*: Không cho phép nạp chồng và ghi đè

- *static*: Nó có ý nghĩa tương tự như với thuộc tính *static*, thông thường chỉ những phương thức chung và không dùng cho một đối tượng cụ thể thì khi đó phương thức đó nó mới là phương thức tĩnh (*static*).
- *abstract*: Phương thức trừu tượng (là phương thức “mẫu”, nó chỉ nói lên là nó làm được gì mà không mô tả nó làm như thế nào), do đó nó không cài đặt nội dung ở lớp khai báo nó
- *native*: dùng cho phương thức khi cài đặt phụ thuộc môi trường trong một ngôn ngữ khác, như C hay hợp ngữ.
- *synchronized*: dùng trong lập trình đa luồng, nhằm ngăn các tác động của các phương thức khác của các luồng khác khi phương thức đang được thực hiện.
- Giá trị mặc định là *public*.

<Kiểu trả lại> có thể là kiểu nguyên thủy, kiểu lớp hoặc không có giá trị trả lại (kiểu *void*).

<Danh sách tham biến hình thức> bao gồm dãy các tham biến (kiểu và tên) phân cách với nhau bởi dấu phẩy.

* Sử dụng phương thức:

- Nếu phương thức không phải là *static* thì cần phải tạo ra một đối tượng cụ thể và gọi phương thức thông qua tên đối tượng như cấu trúc sau:

<tên đối tượng>.<tên_hàm>(< danh sách các tham số hiện thời >)

Khai báo đối tượng: <tên lớp> <tên đối tượng> ;

Khởi tạo giá trị thuộc tính cho đối tượng khi sử dụng Toán tử tạo lập lập(constructor) tự định nghĩa :

<tên đối tượng> = new <tên lớp> (<danh sách tham số hiện thời >) ;

Khai báo và khởi tạo đối tượng giá trị:

<tên lớp> <tên đối tượng> = new <tên lớp> (<danh sách tham số hiện thời >);

- Nếu phương thức là *static* thì gọi thông qua tên lớp hoặc thông qua tên đối tượng (nên gọi các phương thức tĩnh thông qua tên lớp)

<tên_lớp>.<tên_hàm_tĩnh> (<danh sách tham số hiện thời >)

- Các tham số trong các lời gọi hàm cung cấp cách thức trao đổi thông tin giữa đối tượng gửi và đối tượng nhận thông điệp.
 - + Số các tham biến của danh sách hiện thời phải bằng số các tham biến của danh sách tham biến hình thức.
 - + Kiểu của các tham biến hiện thời phải tương thích với kiểu của tham biến hình thức tương ứng.

Ví dụ 1: Tập Demo.java

```
public class Demo {
    public static void main(String args[]) {
```

```

        int i = 5;
        if (i == 0) {
            return;
        }
        output(checkValue(args.length));
    }
    static void output(int value) {
        System.out.println(value);
    }
    static int checkValue(int i) {
        if (i > 3) {
            return 1;
        } else {
            return 2;
        }
    }
}

```

Ví dụ 2: Tập Demo2.java

```

public class Demo2 {
    public static void main(String args[]) {
        int i = 5;
        Demo2 d = new Demo2();
        if (i == 0) {
            return;
        }
        output(d.checkValue(args.length));
    }
    static void output(int value) {
        System.out.println(value);
    }
    public int checkValue(int i) {
        if (i > 3) {
            return 1;
        } else {

```

```

        return 2;
    }
}

```

***Phương thức và thuộc tính static:** Phương thức và thuộc tính độc lập với đối tượng, có thể sử dụng mà không cần thông qua đối tượng, Phương thức tĩnh: không sử dụng được thuộc tính và phương thức “thông thường” (non-static) mà nó chỉ được gọi đến các phương thức và thuộc tính tĩnh khác

4.1.4. Truyền tham số và lời gọi hàm

Truyền các giá trị kiểu nguyên thủy

Trong Java, khi gọi một phương thức và truyền các giá trị nguyên thủy cho phương thức, được gọi là truyền tham trị. Việc thay đổi giá trị chỉ có hiệu lực trong phương thức được gọi, không có hiệu lực bên ngoài phương thức.

Các tham số có kiểu dữ liệu nguyên thủy là byte, short, int, long, float, double, boolean, char.

Ví dụ 3: *Truyền các giá trị nguyên thủy*

```

class KháchHang1 { // Lớp khách hàng
    public static void main(String[] arg) {
        HangSX banh = new HangSX(); // Tạo ra một đối tượng
        int giaBan = 20;
        double tien = banh.tinh(10, giaBan);
        System.out.println("Gia ban: " + giaBan); // giaBan không đổi
        System.out.println("Tien ban duoc : " + tien);
    }
}

// Lớp Hãng sản xuất
class HangSX {
    double tinh(int num, double gia) {
        gia = gia / 2;
        return num * gia; // Thay đổi gia nhưng không ảnh hưởng tới giaBan,
        số tiền bị thay đổi theo
    }
}

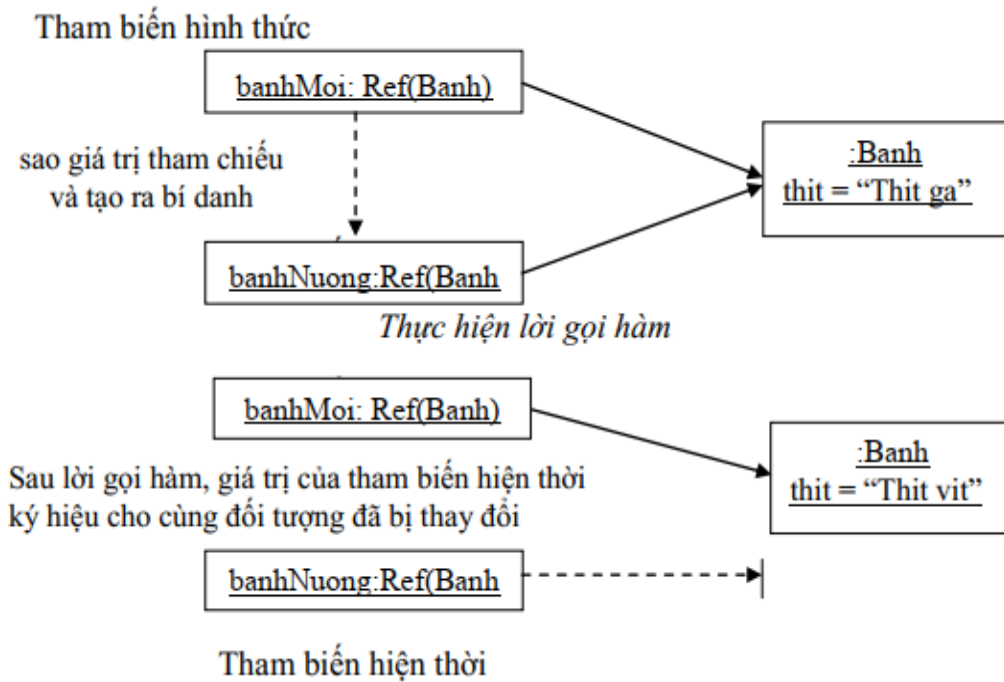
```

Truyền các giá trị tham chiếu đối tượng.

Trong Java, khi gọi một phương thức và truyền một tham chiếu cho phương thức, được gọi là truyền tham chiếu. Việc thay đổi giá trị của biến tham chiếu bên trong phương thức làm thay đổi giá trị của nó.

Trong Java, tất cả các phương thức có tham số là biến có kiểu là các lớp (class) đều là kiểu tham chiếu

```
//KhachHang2.java
class KhachHang2{ // Lớp khách hàng
    public static void main(String[] arg) {
        Banh banhMoi = new Banh(); // Tạo ra một đối tượng (1)
        System.out.println("Nhoi thit vao banh truoac khi nuong:" +
banhMoi.thit);
        nuong(banhMoi); // (2)
        System.out.println("Thit cua banh sau khi nuong:" + banhMoi.thit);
    }
    public static void nuong(Banh banhNuong) { // (3)
        banhNuong.thit = "Thit vit
        "; // đổi nhân thành thịt vịt
        banhNuong = null; // (4)
    }
}
class Banh { // Lớp Banh (5)
    String thit = "Thit ga
        "; // Quy định của hãng làm nhân bánh bằng thịt gà
}
```



Hình 4.1 Truyền các giá trị tham chiếu đối tượng

Truyền tham chiếu: đối với đối tượng thì nội dung của tham chiếu (LValue) được copy lên stack.

Truyền đối số trong dòng lệnh

```
class Pass {
    public static void main(String parameters[]) {
        System.out.println("This is what the main method received");
        System.out.println(parameters[0]);
        System.out.println(parameters[1]); System.out.println(parameters[2]);
    }
}
```

Biên dịch chương trình: **javac PassArgument.java**

Thực thi chương trình với dòng lệnh: **java PassArgument A 123 B1**

Sẽ thu được trên màn hình kết quả:

```
This is what the main method received
A
123
B1
```

Các tham biến final

Tham biến loại này được gọi là biến cuối “trắng”, nghĩa là nó không được khởi tạo giá trị (là trắng) cho đến khi nó được gán một trị nào đó và khi đã được gán trị thì giá trị đó là cuối cùng, không thay đổi được.

Ví dụ 4: *Sử dụng tham biến final*

```
//KhachHang3.java
class KhachHang3 { // Lớp khách hàng
    public static void main(String[] arg) {
        HangSX banh = new HangSX(); // Tạo ra 1 đối tượng
        int giaBan = 20;
        double tien = banh.tinh(10, giaBan);
        System.out.println("Gia ban: " + giaBan); // giaBan không đổi
        System.out.println("Tien ban duoc : " + tien);
    }
}
// Lớp Hãng sản xuất
class HangSX {
    double tinh(int num, final double gia) { // (1)
        gia = gia / 2.0; // (2)
        return num * gia; // Thay đổi gia nhưng không ảnh hưởng tới giaBan,
        số tiền vẫn bị thay đổi theo
    }
}
```

4.1.5. Toán tử tạo lập đối tượng

Mục đích chính của toán tử tạo lập (constructor): Đặt các giá trị khởi tạo cho các đối tượng khi một đối tượng được tạo ra bằng toán tử new.

Khai báo: [<phạm vi>] <Tên lớp> ([<danh sách tham biến>]) {
 // Nội dung cần tạo lập
}

Trong đó:

- Phạm_vi có thể là: public, protected, private
- Phần thân là các lệnh khởi tạo giá trị các thuộc tính của đối tượng, là thành phần tùy chọn.
- Danh sách các tham số : tùy chọn

Phân loại:

- Toán tử tạo lập mặc định không tường minh: **Tên_Lớp()** {}. Khi định nghĩa một lớp mà không xây dựng toán tử tạo lập nào thì hệ thống sẽ cung cấp toán tử tạo lập mặc định không tường minh.
- Toán tử tạo lập mặc định tường minh:

```
TênLớp(<danhsách tham biến hình thức>) {
    tên thuộc tính= giá trị;
    ..... }
```

Giá trị của các biến thành phần được khởi tạo không mặc định bằng các phép gán trong thân của toán tử bằng các giá trị được truyền vào.

Ví dụ 5: Minh họa một phương thức khởi tạo của lớp Person bằng cách gán giá trị cho các thuộc tính tên và tuổi.

```
class Person {
    public String name;
    public int age;
    // Phương thức tạo lập mặc định không tường minh
    // public Person(){}
    // Phương thức tạo lập mặc định tường minh
    public Person(){
        name = "";
        age = 16;
    }
    // Phương thức tạo lập không mặc định
    public Person(String name1, int age1){
        name = name1;
        age = age1;
    }
    // Phương thức tạo lập không mặc định
    public Person(String name1, int age1){
        name = name1;
        age = age1;
    }
    /*public Person(String name, int age){
    this.name = name;
    this.age = age;
    }*/
```

```

    public void show(){
        System.out.println( name + “ is ” + age + “ years old!”);
    }
}

```

Nạp chồng các toán tử tạo lập

Giống như các hàm thành phần, các toán tử tạo lập có thể nạp chồng với nhiều nội dung thực hiện khác nhau.

Khi tạo lập đối tượng sử dụng toán tử tạo lập nào thì toán tử đó được gọi.

Ví dụ 6:

```

class BongDen{
    // Biến thành phần (1)
    private int soWatts;
    private boolean batTat;
    private String viTri;
    // Định nghĩa toán tử tạo lập số 1 (2)
    BongDen(){
        this(40, true);
        System.out.println(“Toán tử số 1”); }
    // Định nghĩa toán tử tạo lập số 2 (3)
    BongDen(int w, boolean s){
        this(w, s, “XX”);
        System.out.println(“Toán tử số 2”); }
    //Định nghĩa toán tử tạo lập số 3 (4)
    BongDen(int soWatts, boolean batTat, String viTri){
        this.soWatts = soWatts;
        this.batTat = batTat;
        this.viTri = new String(viTri);
        System.out.println(“Toán tử số 3”); }
    }
class DenTuyp extends BongDen {
    private int doDai;
    private int mau;
    DenTuyp(int leng, int colo){
        this(leng,colo,100,true,“Chua biet”);
    }
}

```

```

    }
    DenTuyp(int leng, int colo, int soWatt, boolean bt, String noi){
        super(soWatt, bt, noi);
        this.doDai = leng;
        this.mau = colo;
    }
}

public class NhaKho{
    public static void main(String args[]){
        System.out.println("Tao ra bong đèn tuyp");
        DenTuyp d = new DenTuyp(20, 5);
    }
}

```

4.2. CÁC THUỘC TÍNH KIỂM SOÁT TRUY NHẬP CÁC THÀNH PHẦN CỦA LỚP

Khi xây dựng một lớp ta có thể hạn chế sự truy cập đến các thành viên của lớp, từ một đối tượng khác. Ta tóm tắt qua bảng sau:

Bảng 4.1 Thuộc tính kiểm soát truy nhập các thành phần của lớp

Từ khoá	Truy cập trong chính lớp đó	Truy cập trong lớp con cùng gói	Truy cập trong lớp con khác gói	Truy cập trong lớp khác cùng gói	Truy cập trong lớp khác khác gói
private	X	-	-	-	-
protected	X	X	X	X	-
public	X	X	X	X	X
default	X	X	-	X	-

Trong bảng trên thì X thể hiện cho sự truy cập hợp lệ còn – thể hiện không thể truy cập vào thành phần này.

Ví dụ 1: Các thành phần private

Các thành viên private chỉ có thể sử dụng bên trong lớp, ta không thể truy cập các thành viên private từ bên ngoài lớp này.

```

class Alpha {
    private int iamprivate;
    private void privateMethod() {
        System.out.println("privateMethod");
    }
}

```

```

    }
}
class Beta {
    void accessMethod() {
        Alpha a = new Alpha();
        a.iamprivate = 10; // không hợp lệ
        a.privateMethod(); // không hợp lệ
    }
}

```

Ví dụ 2: Các thành phần public

Các thành viên public có thể truy cập từ bất cứ đâu, ta sẽ xem ví dụ sau:

```

public class Alpha {
    public int iampublic;
    public void publicMethod() {
        System.out.println("publicMethod");
    }
}

package Roman;
import Greek.*;
class Beta {
    void accessMethod() {
        Alpha a = new Alpha();
        a.iampublic = 10; // hợp lệ
        a.publicMethod(); // hợp lệ
    }
}

```

Ví dụ 3: các thành phần có mức truy xuất gói:

Khi ta khai báo các thành viên mà không sử dụng một trong các từ public, private, protected thì java mặc định thành viên đó có mức truy cập gói.

```

class Alpha {
    int iampackage;
    void packageMethod() {
        System.out.println("packageMethod");
    }
}

```

```

    }
}
package Greek;
class Beta {
    void accessMethod() {
        Alpha a = new Alpha();
        a.iampackage = 10; // hợp lệ
        a.packageMethod(); // hợp lệ
    }
}

```

4.3. TẠO VÀ SỬ DỤNG ĐỐI TƯỢNG

Trong Java, đối tượng là một thực thể của một lớp (class). Một lớp định nghĩa các thuộc tính (attributes) và phương thức (methods), và đối tượng là phiên bản cụ thể của lớp đó, chứa dữ liệu cụ thể và có thể thực hiện các hành động do lớp định nghĩa. Để tạo và sử dụng đối tượng, đầu tiên cần định nghĩa một lớp. Một lớp là bản thiết kế mô tả các thuộc tính và hành vi của các đối tượng.

Ví dụ 1: Khai báo lớp *Car* với các thuộc tính *make* và *model*, và một phương thức *displayInfo* để in thông tin của xe.

```

public class Car {
    // Thuộc tính
    private String make;
    private String model;
    private int year;

    // Constructor (Phương thức khởi tạo)
    public Car(String make, String model, int year) {
        this.make = make;
        this.model = model;
        this.year = year;
    }

    // Phương thức hiển thị thông tin xe
    public void displayInfo() {
        System.out.println("Make: " + make);
        System.out.println("Model: " + model);
        System.out.println("Year: " + year);
    }
}

```

```
}
```

Để tạo một đối tượng từ lớp, ta sử dụng từ khóa *new* và gọi phương thức khởi tạo (constructor).

```
Car myCar = new Car("Toyota", "Corolla", 2020);
```

Khi đó:

- *myCar* là đối tượng của lớp *Car*.
- *new Car("Toyota", "Corolla", 2020)* tạo một đối tượng mới và khởi tạo các thuộc tính của nó với các giá trị *make* = "Toyota", *model* = "Corolla", và *year* = 2020.

Sử dụng đối tượng: Sau khi tạo đối tượng, bạn có thể gọi các phương thức và truy cập các thuộc tính của đối tượng đó.

```
myCar.displayInfo();
```

Kết quả:

Make: Toyota

Model: Corolla

Year: 2020

Ta cũng có thể truy cập và cập nhật các thuộc tính của đối tượng (nếu chúng không có từ khóa *private*):

```
myCar.year = 2021; // Thay đổi năm sản xuất
```

```
myCar.displayInfo();
```

Ví dụ 2:

```
public class Car {
    String make;
    String model;
    int year;
    public Car(String make, String model, int year) {
        this.make = make;
        this.model = model;
        this.year = year;
    }
    public void displayInfo() {
        System.out.println("Make: " + make);
        System.out.println("Model: " + model);
        System.out.println("Year: " + year);
    }
    public static void main(String[] args) {
        Car myCar = new Car("Toyota", "Corolla", 2020);
```

```

        myCar.displayInfo();
        // Thay đổi thuộc tính
        myCar.year = 2021;
        System.out.println("Updated Year: " + myCar.year);
    }
}

```

Kết quả:

Make: Toyota
 Model: Corolla
 Year: 2020
 Updated Year: 2021

4.4. CASE STUDY

Một thư viện cần một hệ thống để quản lý sách, người mượn sách, và các giao dịch mượn trả sách. Hệ thống phải lưu trữ thông tin sách, người mượn, và theo dõi các giao dịch mượn trả.

Phân tích và giải quyết:

- **Xác định các lớp (Classes):**
 - + Book: Đại diện cho mỗi cuốn sách, chứa thông tin như mã sách, tên sách, tác giả, và số lượng có sẵn.
 - + Member: Đại diện cho mỗi người mượn, chứa thông tin như mã người mượn, tên, và số điện thoại.
 - + Transaction: Đại diện cho mỗi giao dịch mượn trả, chứa thông tin như mã giao dịch, người mượn, sách, ngày mượn, và ngày trả.
- **Xác định các thuộc tính và phương thức:**
 - + Book:
 - **Thuộc tính:** bookID, title, author, quantity.
 - **Phương thức:** updateQuantity().
 - + Member:
 - **Thuộc tính:** memberID, name, phone.
 - **Phương thức:** Không cần phương thức đặc biệt.
 - + Transaction:
 - **Thuộc tính:** transactionID, member, book, borrowDate, returnDate.
 - **Phương thức:** borrowBook(), returnBook().
- **Mối quan hệ giữa các lớp:**
 - + Transaction có mối quan hệ nhiều-nhiều với Book.
 - + Transaction có mối quan hệ 1-nhiều với Member.

4.5. VÍ DỤ ÁP DỤNG

Lớp Book: đại diện cho một cuốn sách, với các thuộc tính như *bookID*, *title*, *author*, *quantity* và phương thức *updateQuantity()* để cập nhật số lượng sách.

```
import java.util.ArrayList;
import java.util.Date;
public class Book {
    private String bookID;
    private String title;
    private String author;
    private int quantity;
    public Book(String bookID, String title, String author, int quantity) {
        this.bookID = bookID;
        this.title = title;
        this.author = author;
        this.quantity = quantity;
    }
    // Getter và Setter cho thuộc tính
    public String getBookID() {
        return bookID;
    }
    public String getTitle() {
        return title;
    }
    public String getAuthor() {
        return author;
    }
    public int getQuantity() {
        return quantity;
    }
    // Cập nhật số lượng sách
    public void updateQuantity(int change) {
        this.quantity += change;
    }
}
```


Lớp Member đại diện cho một người mượn, với các thuộc tính *memberID*, *name*, và *phone*. Lớp này không yêu cầu phương thức đặc biệt.

```
public class Member {
    private String memberID;
    private String name;
    private String phone;
    public Member(String memberID, String name, String phone) {
        this.memberID = memberID;
        this.name = name;
        this.phone = phone;
    }
    // Getter cho thuộc tính
    public String getMemberID() {
        return memberID;
    }
    public String getName() {
        return name;
    }
    public String getPhone() {
        return phone;
    }
}
```

Lớp Transaction đại diện cho mỗi giao dịch mượn trả, với các thuộc tính như *transactionID*, *member*, *book*, *borrowDate*, và *returnDate*. Phương thức *borrowBook()* và *returnBook()* dùng để thực hiện các giao dịch mượn và trả sách.

```
public class Transaction {
    private String transactionID;
    private Member member;
    private Book book;
    private Date borrowDate;
    private Date returnDate;
    public Transaction(String transactionID, Member member, Book book, Date
borrowDate) {
        this.transactionID = transactionID;
        this.member = member;

```

```

        this.book = book;
        this.borrowDate = borrowDate;
        this.returnDate = null; // chưa trả sách
    }
    // Getter cho thuộc tính
    public String getTransactionID() {
        return transactionID;
    }
    public Member getMember() {
        return member;
    }
    public Book getBook() {
        return book;
    }
    public Date getBorrowDate() {
        return borrowDate;
    }
    public Date getReturnDate() {
        return returnDate;
    }
    // Phương thức mượn sách
    public boolean borrowBook() {
        if (book.getQuantity() > 0) {
            book.updateQuantity(-1); // giảm số lượng sách khi mượn
            System.out.println("Book borrowed: " + book.getTitle());
            return true;
        } else {
            System.out.println("Book not available: " + book.getTitle());
            return false;
        }
    }
    // Phương thức trả sách
    public void returnBook() {
        if (this.returnDate == null) {

```

```

        this.returnDate = new Date(); // Ngày trả sách là ngày hiện tại
        book.updateQuantity(1); // tăng số lượng sách khi trả
        System.out.println("Book returned: " + book.getTitle());
    } else {
        System.out.println("This book has already been returned.");
    }
}
}
}

```

Lớp LibrarySystem quản lý danh sách sách, người mượn, và giao dịch mượn trả.

```

public class LibrarySystem {
    private ArrayList<Book> books = new ArrayList<>();
    private ArrayList<Member> members = new ArrayList<>();
    private ArrayList<Transaction> transactions = new ArrayList<>();
    // Thêm sách mới
    public void addBook(Book book) {
        books.add(book);
    }
    // Thêm thành viên mới
    public void addMember(Member member) {
        members.add(member);
    }
    // Tạo giao dịch mượn sách
    public void createTransaction(String transactionID, Member member, Book
book) {
        Transaction transaction = new Transaction(transactionID, member,
book, new Date());
        if (transaction.borrowBook()) {
            transactions.add(transaction);
        }
    }
    // Xử lý trả sách
    public void returnBook(Transaction transaction) {
        transaction.returnBook();
    }
}

```

```

// In thông tin các giao dịch
public void showTransactions() {
    for (Transaction transaction : transactions) {
        System.out.println("Transaction ID: " +
transaction.getTransactionID());
        System.out.println("Member: " +
transaction.getMember().getName());
        System.out.println("Book: " +
transaction.getBook().getTitle());
        System.out.println("Borrow Date: " +
transaction.getBorrowDate());
        System.out.println("Return Date: " +
(transaction.getReturnDate() == null ? "Not Returned" :
transaction.getReturnDate()));
        System.out.println("-----");
    }
}

public static void main(String[] args) {
    LibrarySystem library = new LibrarySystem();
    // Thêm sách
    Book book1 = new Book("B001", "Java Programming", "Author A",
5);
    Book book2 = new Book("B002", "Python Programming", "Author B",
2);

    library.addBook(book1);
    library.addBook(book2);
    // Thêm thành viên
    Member member1 = new Member("M001", "John Doe",
"123456789");
    library.addMember(member1);
    // Tạo giao dịch mượn sách
    library.createTransaction("T001", member1, book1);
    // Trả sách
    Transaction transaction = library.transactions.get(0); // lấy giao dịch
đầu tiên
    library.returnBook(transaction);

```

```
// Hiển thị tất cả các giao dịch  
library.showTransactions();
```

```
}  
}
```

4.6. TỔNG KẾT CHƯƠNG

Chương 4 cung cấp kiến thức cơ bản về lớp đối tượng trong lập trình hướng đối tượng, cùng với các thành phần cấu thành và cách sử dụng lớp. Nội dung bao gồm:

1. Lớp đối tượng: Giới thiệu về lớp (class), là khuôn mẫu để tạo ra các đối tượng, chứa các thuộc tính và phương thức của đối tượng.
 - Định nghĩa lớp đối tượng và các thành phần của lớp: Định nghĩa lớp và các thành phần cấu thành như thuộc tính dữ liệu (data attributes) và các phương thức (methods).
 - Các thuộc tính dữ liệu của lớp: Giải thích các thuộc tính lưu trữ trạng thái của đối tượng, các biến thành phần đại diện cho dữ liệu của lớp.
 - Định nghĩa hàm thành phần: Giới thiệu cách viết các phương thức, là các hàm xác định hành vi của đối tượng.
 - Truyền tham số và lời gọi hàm: Giải thích cách truyền tham số vào phương thức và cách gọi phương thức của đối tượng.
 - Toán tử tạo lập đối tượng: Cách tạo đối tượng mới từ lớp thông qua toán tử đặc biệt.
2. Các thuộc tính kiểm soát truy nhập các thành phần của lớp: Đề cập đến các mức độ truy cập khác nhau của các thành phần trong lớp (như private, protected, public), giúp kiểm soát khả năng truy cập và bảo vệ dữ liệu.
3. Tạo và sử dụng đối tượng: Hướng dẫn cách khởi tạo đối tượng từ lớp, cách sử dụng các phương thức và truy cập các thuộc tính của đối tượng để thực hiện các thao tác cụ thể.

Chương này cung cấp nền tảng cần thiết để hiểu cách làm việc với các lớp và đối tượng, cũng như quản lý các thành phần của chúng, là những kiến thức cốt lõi của lập trình hướng đối tượng.

4.7. CÂU HỎI TRẮC NGHIỆM

1. Đây là cách định nghĩa một lớp trong Java?

- A. class <TênLớp> { ... }
- B. Class <TênLớp> { ... }
- C. class = <TênLớp> { ... }
- D. <TênLớp> class { ... }

2. Một lớp trong Java có thể chứa các thành phần nào?

- A. Biến
- B. Phương thức
- C. Constructor
- D. Tất cả các đáp án trên

3. Phương thức nào sau đây là phương thức khởi tạo trong Java?

- A. Một phương thức có cùng tên với lớp
 - B. Một phương thức không có giá trị trả về
 - C. Một phương thức có kiểu trả về void
 - D. Cả A và B đều đúng
- 4. Từ khóa this dùng để làm gì trong Java?**
- A. Truy cập biến toàn cục
 - B. Truy cập biến thành viên của lớp hiện tại
 - C. Truy cập phương thức của lớp cha
 - D. Truy cập phương thức static
- 5. Phương thức nào không thể gọi trực tiếp từ đối tượng của lớp?**
- A. Phương thức public
 - B. Phương thức private
 - C. Phương thức protected
 - D. Phương thức static
- 6. Phương thức nào có thể truy cập được từ các lớp con?**
- A. private
 - B. protected
 - C. public
 - D. B và C đúng
- 7. Phương thức nào sau đây trả về một đối tượng?**
- A. `public void getObject() {}`
 - B. `public Object getObject() { return new Object(); }`
 - C. `public int getObject() { return 0; }`
 - D. `public boolean getObject() { return true; }`
- 8. Trong một lớp, thuộc tính nào là thuộc tính thành viên?**
- A. Thuộc tính khai báo với từ khóa public
 - B. Thuộc tính khai báo với từ khóa private
 - C. Thuộc tính được khai báo trong lớp
 - D. Tất cả đều đúng
- 9. Phương thức nào có thể gọi mà không cần tạo đối tượng?**
- A. Phương thức public
 - B. Phương thức static
 - C. Phương thức protected
 - D. Phương thức private

10. Từ khóa nào dùng để tạo đối tượng trong Java?

- A. new
- B. class
- C. this
- D. public

11. Từ khóa *final* khi dùng với một lớp có nghĩa là gì?

- A. Lớp đó không thể bị kế thừa
- B. Lớp đó không thể có thuộc tính
- C. Lớp đó không thể có phương thức
- D. Lớp đó không thể bị khởi tạo

12. Biến *static* trong một lớp có đặc điểm nào?

- A. Chỉ có thể truy cập từ các lớp con
- B. Chỉ tồn tại khi có đối tượng
- C. Được chia sẻ giữa tất cả các đối tượng
- D. Được tạo mới mỗi lần tạo đối tượng

13. Phạm vi truy cập nào giúp thuộc tính chỉ truy cập được trong lớp?

- A. public
- B. protected
- C. private
- D. default

14. Thuộc tính hoặc phương thức của lớp nào có thể truy cập được từ bất kỳ đâu?

- A. public
- B. protected
- C. private
- D. default

15. Lệnh nào dùng để gọi phương thức của đối tượng?

- A. obj.methodName();
- B. obj->methodName();
- C. methodName().obj;
- D. methodName(obj);

16. Đây là cú pháp đúng để truyền tham số vào phương thức trong Java?

- A. methodName[tham số];
- B. methodName(tham số);
- C. methodName->(tham số);

D. methodName{tham số};

17. Phương thức khởi tạo không có tham số gọi là gì?

- A. Phương thức final
- B. Phương thức protected
- C. Phương thức default
- D. Phương thức public

18. Từ khóa *null* trong Java có ý nghĩa gì khi dùng với đối tượng?

- A. Biểu thị đối tượng rỗng
- B. Biểu thị đối tượng đang tồn tại
- C. Biểu thị lớp cha
- D. Biểu thị thuộc tính

19. Đây là cú pháp đúng để truy cập thuộc tính của đối tượng?

- A. object.property;
- B. property.object;
- C. object->property;
- D. object[property];

20. Khi nào một phương thức được gọi là nạp chồng?

- A. Khi có nhiều phương thức cùng tên nhưng khác tham số
- B. Khi phương thức có tham số giống nhau
- C. Khi phương thức có cùng tên và tham số giống nhau
- D. Khi phương thức không có tham số

21. Getter là gì?

- A. Phương thức để lấy giá trị thuộc tính
- B. Phương thức để gán giá trị thuộc tính
- C. Phương thức để khởi tạo đối tượng
- D. Phương thức để kiểm tra đối tượng

22. Đây là đặc điểm của setter trong Java?

- A. Trả về giá trị thuộc tính
- B. Thay đổi giá trị thuộc tính
- C. Khởi tạo đối tượng
- D. Tạo lớp mới

23. Constructor trong Java có kiểu trả về nào?

- A. void
- B. Không có kiểu trả về

- C. int
- D. boolean

24. Từ khóa nào ngăn cản một phương thức bị ghi đè (override)?

- A. static
- B. final
- C. private
- D. public

25. Lệnh nào sau đây là cú pháp để xóa đối tượng trong Java?

- A. delete obj;
- B. free(obj);
- C. obj = null;
- D. obj.clear();

26. Phương thức *toString()* trả về giá trị gì?

- A. Mã hash của đối tượng
- B. Chuỗi mô tả đối tượng
- C. Đối tượng mới
- D. Giá trị thuộc tính của đối tượng

27. Các thuộc tính *public* của lớp có thể được truy cập từ đâu?

- A. Chỉ trong lớp
- B. Từ lớp con
- C. Từ bất cứ đâu
- D. Chỉ từ gói (package)

28. Biến thành viên được khai báo là *private* có thể truy cập từ đâu?

- A. Từ bất cứ đâu
- B. Chỉ trong cùng lớp
- C. Từ lớp con
- D. Từ gói (package)

29. Cách nào dưới đây để kiểm tra xem hai đối tượng có bằng nhau không?

- A. object1 == object2
- B. object1.equals(object2)
- C. object1.compare(object2)
- D. object1.isEqual(object2)

30. Đây là cách khai báo một thuộc tính trong lớp?

- A. public int age;

- B. int age;
- C. private int age;
- D. Tất cả đều đúng

4.8. BÀI TẬP THỰC HÀNH

Bài tập 1: Quản lý nhân sự

Mô tả: Một công ty cần hệ thống quản lý nhân sự để theo dõi thông tin của các nhân viên, bao gồm tên, mã nhân viên, chức vụ, và lương. Hệ thống cũng cần có khả năng tính toán lương tháng dựa trên các yếu tố như ngày làm việc, số giờ làm thêm và các khoản thưởng phạt.

Gợi ý giải quyết:

- **Lớp:** Employee (Nhân viên)
- **Thuộc tính:** employeeID, name, position, salary, workDays, overtimeHours, bonus, penalty.
- **Phương thức:** calculateMonthlySalary(), addWorkDays(), addOvertimeHours().

Bài tập 2: Quản lý tài khoản ngân hàng

Mô tả: Ngân hàng cần một hệ thống để quản lý tài khoản của khách hàng, bao gồm việc mở tài khoản, nạp tiền, rút tiền, và kiểm tra số dư. Mỗi khách hàng có thể có nhiều tài khoản với số dư khác nhau.

Gợi ý giải quyết:

- **Lớp:** BankAccount (Tài khoản ngân hàng), Customer (Khách hàng).
- **Thuộc tính:** accountNumber, balance, accountType cho BankAccount; customerID, name, accounts cho Customer.
- **Phương thức:** deposit(), withdraw(), checkBalance() cho BankAccount; addAccount() cho Customer.

Bài tập 3: Quản lý cửa hàng sách

Mô tả: Một cửa hàng sách cần hệ thống để quản lý sách, tác giả, và đơn hàng. Hệ thống cần lưu trữ thông tin sách, theo dõi số lượng sách bán ra, và quản lý đơn hàng của khách hàng.

Gợi ý giải quyết:

- **Lớp:** Book (Sách), Author (Tác giả), Order (Đơn hàng).
- **Thuộc tính:** bookID, title, author, price, stock cho Book; authorID, name, booksWritten cho Author; orderID, customer, booksOrdered cho Order.
- **Phương thức:** sellBook(), updateStock() cho Book; addBook() cho Author; calculateTotalPrice() cho Order.

Bài tập 4: Quản lý hệ thống đặt phòng khách sạn

Mô tả: Một khách sạn cần một hệ thống để quản lý đặt phòng, khách hàng và thanh toán. Hệ thống phải có khả năng quản lý các phòng trống, kiểm tra tình trạng phòng, và xử lý đặt phòng cũng như thanh toán.

Gợi ý giải quyết:

- **Lớp:** Room (Phòng), Customer (Khách hàng), Booking (Đặt phòng).
- **Thuộc tính:** roomID, roomType, price, availability cho Room; customerID, name, phone cho Customer; bookingID, room, customer, checkInDate, checkOutDate cho Booking.
- **Phương thức:** checkAvailability(), bookRoom() cho Room; makePayment() cho Booking.

Bài tập 5: Quản lý hệ thống học trực tuyến

Mô tả: Một tổ chức giáo dục cần một hệ thống để quản lý các khóa học trực tuyến, học viên và giảng viên. Hệ thống cần lưu trữ thông tin về khóa học, danh sách học viên đăng ký và điểm số của họ.

Gợi ý giải quyết:

- **Lớp:** Course (Khóa học), Student (Học viên), Instructor (Giảng viên).
- **Thuộc tính:** courseID, courseName, instructor, students cho Course; studentID, name, coursesEnrolled cho Student; instructorID, name, coursesTaught cho Instructor.
- **Phương thức:** enrollStudent(), assignInstructor(), recordGrade() cho Course; registerCourse() cho Student.

Chương 5 GÓI CÁC LỚP VÀ CÁC LỚP CƠ SỞ TRONG GÓI JAVA.LANG

Chương 5 giới thiệu chi tiết các nội dung:

- Tính đóng gói trong OOP
- Lớp Collection trong Java
- Giới thiệu một số lớp cơ sở trong gói **java.lang**

5.1. GÓI

5.1.1 Tạo gói cho chương trình và cách sử dụng gói

Gói (Package) là một nhóm các lớp và giao diện được tổ chức theo cấu trúc thư mục. Các gói giúp tổ chức mã nguồn dễ quản lý hơn và tránh xung đột tên giữa các lớp.

Để tạo một gói, bạn khai báo từ khóa *package* ở đầu của tệp .java.

Ví dụ 1:

```
// Tạo gói "library"
package library;
public class Book {
    private String title;
    private String author;

    public Book(String title, String author) {
        this.title = title;
        this.author = author;
    }
    public String getTitle() {
        return title;
    }
    public String getAuthor() {
        return author;
    }
}
```

Sử dụng gói trong chương trình khác: Để sử dụng lớp *Book* từ gói *library*, bạn cần nhập gói này bằng từ khóa *import*.

Ví dụ 2:

```
import library.Book;
public class Main {
```

```

    public static void main(String[] args) {
        Book myBook = new Book("Java Programming", "Author A");
        System.out.println("Title: " + myBook.getTitle());
        System.out.println("Author: " + myBook.getAuthor());
    }
}

```

Lợi ích của việc sử dụng gói:

- **Tổ chức mã nguồn:** Giúp tổ chức các lớp theo nhóm chức năng.
- **Tránh xung đột tên:** Các lớp có cùng tên trong các gói khác nhau không gây xung đột.
- **Bảo mật:** Kiểm soát quyền truy cập đến các lớp hoặc phương thức trong gói.

5.1.2 Giới thiệu một số gói quan trọng của Java

Java cung cấp rất nhiều gói (package) tiêu chuẩn chứa các lớp và giao diện hữu ích, giúp lập trình viên dễ dàng xây dựng ứng dụng mà không cần phải viết mã từ đầu.

java.lang

Gói java.lang là một trong những gói quan trọng nhất của Java. Đây là gói mặc định mà mọi chương trình Java đều tự động import, vì vậy bạn không cần phải khai báo import java.lang.*.

- **Lớp cơ bản:** Object, lớp cha của tất cả các lớp trong Java, String, Math, Integer, Double.
- **Xử lý toán học:** Math, StrictMath cung cấp các phương thức toán học như sqrt(), pow(), abs().
- **Các lớp tiện ích:** System, Runtime dùng để quản lý tài nguyên hệ thống, Thread dùng để làm việc với luồng (thread).

java.util

Gói java.util chứa các lớp và giao diện hỗ trợ làm việc với cấu trúc dữ liệu, quản lý ngày giờ, và các tiện ích khác.

- **Collections:** List, Set, Map, Queue và các triển khai như ArrayList, LinkedList, HashSet, HashMap.
- **Lớp Date & Time:** Date, Calendar, và từ Java 8 là LocalDate, LocalTime, LocalDateTime.
- **Tiện ích khác:** Random để tạo số ngẫu nhiên, Scanner để nhận đầu vào từ người dùng.

java.io

Gói java.io hỗ trợ xử lý đầu vào/đầu ra, chủ yếu làm việc với tệp tin và luồng dữ liệu (streams).

- **Các lớp nhập xuất:** FileReader, FileWriter, BufferedReader, BufferedWriter, InputStream, OutputStream.
- **Làm việc với tệp tin:** File, FileInputStream, FileOutputStream.

- **Các lớp tiện ích:** `PrintStream`, `Serializable` hỗ trợ cho việc ghi và đọc đối tượng vào/ra tệp tin.

javax.swing

Gói `javax.swing` cung cấp các lớp để xây dựng giao diện người dùng đồ họa (GUI).

- **Thành phần GUI:** `JButton`, `JLabel`, `TextField`, `JTable`, `JPanel`, `JFrame`.
- **Quản lý sự kiện:** `ActionListener`, `MouseListener` để xử lý sự kiện từ người dùng.

java.sql

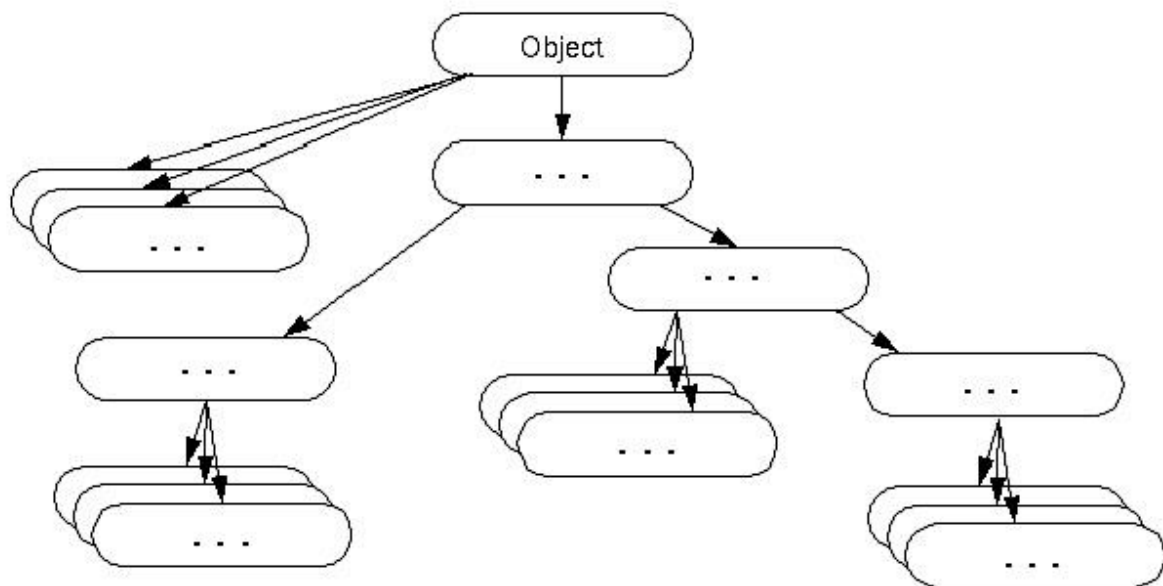
Gói `java.sql` cung cấp các lớp và giao diện để kết nối và làm việc với cơ sở dữ liệu SQL.

- **Kết nối cơ sở dữ liệu:** `Connection`, `DriverManager`.
- **Truy vấn dữ liệu:** `Statement`, `PreparedStatement`, `CallableStatement`.
- **Quản lý kết quả:** `ResultSet` để xử lý kết quả truy vấn.

5.2. CÁC LỚP CƠ SỞ TRONG GÓI JAVA.LANG

5.2.1. Lớp Object

Tất cả các lớp được xây dựng trong các chương trình Java đều hoặc là trực tiếp hoặc gián tiếp được mở rộng từ lớp `Object`. Đây là lớp cơ sở nhất, định nghĩa hầu như tất cả những phương thức phân cơ bản để các lớp con cháu của nó sử dụng trực tiếp hoặc viết đè. `Object` cung cấp các phương thức sau: `int hashCode()`



Hình 5.1 Lớp object

Khi các đối tượng được lưu vào các bảng băm (hash table), hàm này có thể sử dụng để xác định duy nhất giá trị cho mỗi đối tượng. Điều này đảm bảo tính nhất quán của hệ thống khi thực hiện chương trình.

Sử dụng lớp Object là hữu ích nếu ta muốn tham chiếu bất kỳ đối tượng nào mà ta chưa biết kiểu dữ liệu của đối tượng đó. Chú ý rằng biến tham chiếu của lớp cha có thể tham chiếu đến đối tượng của lớp con được gọi là *upcasting*.

Ví dụ 1: giả sử phương thức getObject() trả về một đối tượng nhưng nó có thể là bất kỳ kiểu nào như Employee, Student, ... chúng ta có thể sử dụng biến tham chiếu của lớp Object để tham chiếu tới đối tượng đó.

Object obj=getObject(); // chúng ta không biết đối tượng gì được trả về từ phương thức này.

Lớp Object cung cấp các phương thức như trong bảng sau:

Bảng 5.1 Các phương thức quan trọng của lớp Object

Phương thức	Mô tả
public final Class getClass()	trả về đối tượng lớp Class của đối tượng hiện tại. Từ lớp Class đó có thể lấy được các thông tin metadata của class hiện tại.
public int hashCode()	trả về số hashcode cho đối tượng hiện tại.
public boolean equals(Object obj)	so sánh đối tượng đã cho với đối tượng hiện tại.
protected Object clone() throws CloneNotSupportedException	tạo và trả về bản sao chép (clone) của đối tượng hiện tại.
public String toString()	trả về chuỗi ký tự đại diện của đối tượng hiện tại.
public final void notify()	đánh thức một luồng, đợi trình giám sát của đối tượng hiện tại.
public final void notifyAll()	đánh thức tất cả các luồng. đợi trình giám sát của đối tượng hiện tại.
public final void wait(long timeout) throws InterruptedException	làm cho Thread hiện tại đợi trong khoảng thời gian là số mili giây cụ thể, tới khi Thread khác thông báo (gọi phương thức notify() hoặc notifyAll()).
public final void wait(long timeout,int nanos)throws InterruptedException	làm cho Thread hiện tại đợi trong khoảng thời gian là số mili giây và nano giây cụ thể, tới khi Thread khác thông báo (gọi phương thức notify() hoặc notifyAll()).
public final void wait()throws InterruptedException	làm Thread hiện tại đợi, tới khi Thread khác thông báo (gọi phương thức notify() hoặc notifyAll()).
protected void finalize()throws Throwable	Được gọi bởi Garbage Collector trước khi đối tượng bị dọn rác.

5.2.2. Các lớp nguyên thủy

Các giá trị nguyên thủy không phải là đối tượng trong Java. Để có thể thao tác được trên các giá trị nguyên thủy (giá trị số, kí tự và logic) thì gói java.lang cung cấp các lớp bao gói (Wrapper) cho từng kiểu dữ liệu nguyên thủy (gọi tắt là lớp bao). Các lớp bao có những đặc tính chung sau:

Các toán tử tạo lập chung: Các lớp bao (trừ lớp Character chỉ có một toán tử tạo lập) đều có hai toán tử tạo lập:

- Toán tử tạo lập sử dụng giá trị nguyên thủy để tạo ra đối tượng tương ứng.
Character charObj = **new** Character(„a“);
Boolean boolObj = **new** Boolean(**true**);
Integer intObj = **new** Integer(2002);
Float floatObj = **new** Float(3.14F);
Double doubleObj = **new** Double(3.14);
- Toán tử thứ hai: chuyển các đối tượng lớp String biểu diễn cho các giá trị nguyên thủy về các lớp tương ứng. Các toán tử này sẽ ném ra ngoại lệ NumberFormatException khi giá trị String truyền vào Toán tử tạo lập không hợp lệ.
Boolean boolObj = **new** Boolean(**“true”**);
Integer intObj = **new** Integer(**“2002”**);
Float floatObj = **new** Float(**“3.14F”**);
Double doubleObj = **new** Double(**“3.14”**);

Các hàm tiện ích chung: *valueOf(String s)*, *toString()*, *typeValue()*, *equals()*

Mỗi lớp (trừ lớp Character) đều định nghĩa hàm static *valueOf(String s)* trả lại đối tượng tương ứng. Các hàm này ném ra ngoại lệ NumberFormatException khi giá trị String truyền vào phương thức không hợp lệ.

```
Boolean boolObj = Boolean.valueOf(“true”);  
Integer intObj = Integer.valueOf(“2002”);  
Float floatObj = Float.valueOf(“3.14F”);  
Double doubleObj = Double.valueOf(“3.14”);
```

Các lớp viết đè hàm toString() trả lại là các đối tượng String biểu diễn cho các giá trị nguyên thủy ở dạng xâu.

Các lớp viết đè hàm equals() để thực hiện so sánh bằng nhau của các đối tượng nguyên thủy.

```
Character charObj = new Character(„a“);  
boolean charTest = charObj.equals(„b“); // false  
Integer intObj1 = Integer.valueOf(“2010”);  
boolean intTest = intObj.equals(intObj1); // false
```

Lớp Boolean

Lớp này định nghĩa hai đối tượng Boolean.TRUE, Boolean.FALSE biểu diễn cho hai giá trị nguyên thủy true và false tương ứng.

Lớp Character

Lớp Character định nghĩa hai giá trị cực tiểu, cực đại Character.MIN_VALUE, Character.MAX_VALUE và các giá trị kiểu ký tự Unicode. Ngoài ra lớp này còn định nghĩa một số hàm static để xử lý trên các ký tự:

```
static boolean isLowerCase(char ch) // true nếu ch là ký tự thường
static boolean isUpperCase(char ch) // true nếu ch là ký tự viết hoa
static boolean isDigit(char ch) // true nếu ch là chữ số
static boolean isLetter(char ch) // true nếu ch là chữ cái
static boolean isLetterOrDigit(char ch) // true nếu ch là chữ hoặc là số
static char toUpperCase(char ch) // Chuyển ch về chữ viết hoa
static char toLowerCase(char ch) // Chuyển ch về chữ viết thường
static char toTitleCase(char ch) // Chuyển ch về dạng tiêu đề.
```

Các lớp bao kiểu số

Các lớp Byte, Short, Integer, Long, Float, Double là các lớp con của lớp Number. Trong các lớp này đều xác định hai giá trị:

<Lớp bao>.MIN_VALUE

<Lớp bao>.MAX_VALUE là các giới hạn của các số trong kiểu đó.

Ví dụ 2: byte minByte = Byte.MIN_VALUE;

 int maxInt = Integer.MAX_VALUE;

Trong mỗi lớp bao có hàm typeValue() để chuyển các giá trị của các đối tượng nguyên thủy về giá trị số:

```
// -128
// 2147483647
byte byteValue()
short shortValue()
int intValue()
long longValue()
float floatValue()
double doubleValue()
```

Trong mỗi lớp bao còn có hàm static parseType(String s) để chuyển các giá trị được biểu diễn dưới dạng xâu về các giá trị số:

```
byte value1 = Byte.parseByte("16");
```

```
int value2 = Integer.parseInt("2002");
```

```
double value3 = Double.parseDouble("3.14");
```

Ví dụ 3: Viết chương trình để nhập vào một dãy số tùy ý và sắp xếp theo thứ tự tăng dần.

```
import java.io.*;

public class SapXep {
    static int[] day;
    static void nhap() {
        String str;
        int n = day.length;
        DataInputStream stream = new DataInputStream(System.in);
        System.out.println("Nhập vào " + n + " số nguyên");
        for (int k = 0; k < n; k++) {
            try {
                System.out.print(k + ": ");
                str = stream.readLine();
                day[k] = Integer.valueOf(str).intValue();
            } catch (IOException e) {
                System.err.println("I/O Error!");
            }
        }
    }

    static void hienThi() {
        int n = day.length;
        for (int k = 0; k < n; k++) {
            System.out.print(day[k] + " ");
        }
        System.out.println();
    }

    static void sapXep() {
        int x, max, k;
        for (int i = day.length - 1; i > 0; i--) {
            max = day[i];
            k = i;
            for (int j = 0; j < i; j++) {
```

```

        if (max < day[j]) {
            max = day[j];
            k = j;
        }
    }
    day[k] = day[i];
    day[i] = max;
}
}

public static void main(String[] args) {
    String str;
    int n;
    DataInputStream stream = new DataInputStream(System.in);
    System.out.println("\nCho biet bao nhieu so nhap vao: ");
    try {
        str = stream.readLine();
    } catch (IOException e) {
        System.err.println("I/O Error!");
        str = "0";
    }
    n = Integer.valueOf(str).intValue();
    SapXep.day = new int[n];
    nhap();
    sapXep();
    System.out.println("Day so duoc sap xep: ");
    hienThi();
}
}

```

5.2.3. Lớp Math

Lớp Math nằm trong gói java.lang (gói mặc định) do vậy không cần phải thêm câu lệnh import ở đầu chương trình để có thể sử dụng lớp này. Các hàm này được viết là các phương thức tĩnh nên ta không cần phải tạo ra thể hiện của lớp Math. Bảng sau liệt kê một số phương thức tĩnh trong lớp Math:

Bảng 5.2 Một số phương thức tính trong lớp Math

Tên phương thức	Mô tả	Kiểu tham số	Kiểu trả về
sin(arg)	tính sin của arg	arg là một biểu thức kiểu double thể hiện một cung theo radians	double
cos(arg)	tính cos của arg	arg là một biểu thức kiểu double thể hiện một cung theo radians	double
tan(arg)	tính tang của arg	arg là một biểu thức kiểu double thể hiện một cung theo radians	double
asin(arg)	tính sin-1 (arcsin) arg	arg là một biểu thức kiểu double thể hiện một cung theo radians	double trong hệ radians
acos(arg)	tính cos-1 (arccosin) của arg	arg là một biểu thức kiểu double thể hiện một cung theo radians	double trong hệ radians
atan(arg)	tính tan-1 (arctang) của arg	arg là một biểu thức kiểu double thể hiện một cung theo radians	double trong hệ radians
atan2(arg1, arg2)	tính tan-1 (arctang) của arg1/arg2	arg1, arg2 là các biểu thức kiểu double thể hiện một cung theo radians	double trong hệ radians
abs(arg)	tính trị tuyệt đối của arg	arg là một biểu thức kiểu int, long, float, hoặc double	Nhận về kiểu cùng Kiểu với tham số
max(arg1, arg2)	Nhận về giá trị lớn trong hai tham số	arg1, arg2 là một biểu thức kiểu int, long, float, hoặc double	Nhận về kiểu cùng Kiểu với tham số
min(arg1, arg2)	Nhận về giá trị nhỏ trong hai tham số	arg1, arg2 là một biểu thức kiểu int, long, float, double	Nhận về kiểu cùng kiểu với tham số
ceil(arg)	Nhận về giá trị nguyên nhỏ hơn hoặc bằng arg	arg là biểu thức kiểu float hoặc double	double
floor(arg)	Nhận về giá trị nguyên lớn hơn hoặc bằng arg	arg là biểu thức kiểu float hoặc double	double
round(arg)	Trả về giá trị nguyên gần arg nhất, giá trị này chính là giá trị	arg là biểu thức kiểu floathoặc double	Nhận về kiểu int nếu arg kiểu float, nhận về kiểu long

Tên phương thức	Mô tả	Kiểu tham số	Kiểu trả về
	của arg sau khi đã làm tròn		nếu arg kiểu double
rint(arg)	Giống round(arg)	arg là biểu thức kiểu double	double
sqrt(arg)	tính căn bậc hai của arg	arg là biểu thức kiểu double	double
pow (arg1, arg2)	tính $arg1^{arg2}$	Cả arg1 và arg2 là các biểu thức kiểu double	double
exp(arg)	tính earg	arg là biểu thức kiểu double	double
log(arg)	tính logarithm cơ số e của arg	arg là biểu thức kiểu double	double
random()	Nhận về một số giả ngẫu nhiên nằm trong khoảng [0, 1)	Không có tham số	double

5.2.4. Lớp String

Lớp String dựng cho những chuỗi ký tự bất biến (chuỗi chỉ đọc), khi được khởi tạo giá trị đầu, giá trị đó không thay đổi được nữa.

Phương thức khởi tạo (Constructor):

String(String s)

String(<Kiểu>[] mảng)

String(StringBuffer buf)

Ví dụ 4:

```
byte[] bytes = {97, 98, 99, 100};
char[] characters = {'a', 'b', 'c', 'd', 'e'};
StringBuffer buff = new StringBuffer("abba");
String byteStr = new String(bytes); // Chuyển mảng bytes về chuỗi
String charStr = new String(characters); // charStr = "abba"
String buffStr = new String(buff);
String str = new String("abba");
String str = "Dung dong den toi";
```

Ví dụ 5: Khởi tạo chuỗi trong Java

```
public class StringInitialization {
    public static void main(String a[]) {
        String objStr_01 = "This is a string object";
    }
}
```

```

        String objStr_02 = new String("This is also string object");
        char[] arrChar = {'V', 'N', 'L', 'T', 'V', 'E', 'S'};
        String objStr_03 = new String(arrChar);
        String objStr_04 = objStr_03 + " This is another String object";
        System.out.println("objStr_01: " + objStr_01);
        System.out.println("objStr_02: " + objStr_02);
        System.out.println("objStr_03: " + objStr_03);
        System.out.println("objStr_04: " + objStr_04);
    }
}

```

Kết quả chạy chương trình:

```

objStr_01: This is a string object
objStr_02: This is also string object
objStr_03: VNLIVES
objStr_04: VNLIVES This is another String object

```

Đọc từng ký tự

int length(): kích thước xâu.

char charAt(int index): cho lại ký tự thứ *index* (bắt đầu từ 0) của xâu hiện thời ($0 \leq index < length()$). (ngoại lệ *StringIndexOutOfBoundsException*).

So sánh các xâu

boolean equals(Object obj): kiểm tra xem đối tượng lớp String và đối tượng ở tham số có cùng tập ký hiệu hay không

boolean equalsIgnoreCase(String str2) kiểm tra xem đối tượng lớp String và đối tượng ở tham số có cùng tập ký hiệu hay không (phân biệt chữ hoa và chữ thường)

boolean startWith(String str) cho kết quả true nếu xâu bắt đầu bằng đối số str.

boolean endWith(String str) cho kết quả true nếu xâu kết thúc bằng đối số str.

int compareTo(String str2)

int compareTo(Object obj)

- bằng 0 nếu xâu hiện thời bằng xâu đối số,
- nhỏ hơn 0, nếu xâu đó nhỏ hơn xâu đối số theo thứ tự từ điển,
- lớn hơn 0, nếu xâu đó lớn hơn xâu đối số theo thứ tự từ điển,

Hàm *compareTo()* thứ hai thực hiện tương tự như hàm đầu nếu đối số là chuyển về được đối tượng xâu, ngược lại sẽ cho qua để xử lý ngoại lệ *ClassCastException*.

Chuyển đổi xâu:

String toUpperCase() chuyển toàn bộ ký tự trong xâu thành chữ in hoa

String toLowerCase() chuyển toàn bộ ký tự trong xâu thành chữ in thường

Ghép xâu

String concat(String str) nối xâu *str* vào xâu hiện tại

Tìm các ký tự và các xâu con Nếu không tìm được các hàm này sẽ cho kết quả là -1:

int indexOf(int ch) Tìm chỉ số của lần xuất hiện đầu tiên của *ch* trong xâu.

int indexOf(int ch, int fromIndex) Tìm chỉ số của lần xuất hiện đầu tiên của *ch* trong xâu bắt đầu từ *fromIndex*.

int indexOf(String str) Tìm chỉ số của lần xuất hiện đầu tiên của xâu con *str* trong xâu hiện thời.

int indexOf(String str, int fromIndex) Tìm chỉ số của lần xuất hiện đầu tiên của xâu con *str* bắt đầu từ *fromIndex* trong xâu hiện thời.

int lastIndexOf(int ch) Tìm chỉ số của lần xuất hiện cuối cùng của ký tự *ch* trong xâu.

int lastIndexOf(int ch, int fromIndex) Tìm chỉ số của lần xuất hiện cuối cùng của *ch* bắt đầu từ *fromIndex* trong xâu.

int lastIndexOf(String str) Tìm chỉ số của lần xuất hiện cuối cùng của xâu con *str* trong xâu hiện thời.

int lastIndexOf(String str, int fromIndex) Tìm chỉ số của lần xuất hiện cuối cùng của xâu con *str* bắt đầu từ *fromIndex* trong xâu hiện thời.

String replace(char cu, char moi) Thay tất cả các lần xuất hiện của ký tự *cu* bằng ký tự *moi* trong xâu.

Trích ra các xâu con

String trim() tạo một xâu mới, các ký tự “trắng” (những ký tự có giá trị nhỏ hơn giá trị dấu cách „ „) ở trong xâu đều bị loại bỏ.

String substring(int startIndex) Cho kết quả là một xâu con được triết ra từ vị trí *startIndex* đến cuối của xâu.

String substring(int startIndex, int endIndex) Cho kết quả là một xâu con được triết ra từ vị trí *startIndex* đến vị trí *endIndex* của xâu. (ngoại lệ *StringIndexOutOfBoundsException*.)

Chuyển các đối tượng của Object về String

static String valueOf(Object obj) được nạp chồng để chuyển đối tượng *obj* về xâu ký tự.

static String valueOf(char[] characters) được nạp chồng để chuyển mảng các ký tự *characters* về xâu ký tự.

static String valueOf(boolean b) được nạp chồng để chuyển hai giá trị boolean: true, false về xâu ký tự “true”, “false” tương ứng.

static String valueOf(char c) được nạp chồng để chuyển ký tự *c* về xâu ký tự tương ứng.

static String valueOf(int i) được nạp chồng để chuyển số nguyên kiểu *int* về xâu ký tự tương ứng.

static String valueOf(long l) được nạp chồng để chuyển số nguyên kiểu *long* về chuỗi ký tự tương ứng.

static String valueOf(float f) được nạp chồng để chuyển số thực kiểu *float* về chuỗi ký tự tương ứng.

static String valueOf(double d) được nạp chồng để chuyển số thực kiểu *double* về chuỗi ký tự tương ứng.

Ví dụ 6: Sử dụng các hàm cơ bản của chuỗi ký tự.

```
public class Main {  
    public static void main(String[] args) {  
        String s = "Hello, World!";  
        // Độ dài của chuỗi  
        System.out.println("Length: " + s.length()); // Kết quả: 13  
        // Trả về ký tự tại vị trí chỉ định  
        System.out.println("Char at index 1: " + s.charAt(1)); // Kết quả: 'e'  
        // Kiểm tra chuỗi có bắt đầu bằng "Hello" không  
        System.out.println("Starts with 'Hello': " + s.startsWith("Hello")); // Kết quả:  
true  
        // Chuyển chuỗi thành chữ hoa  
        System.out.println("Uppercase: " + s.toUpperCase()); // Kết quả: "HELLO,  
WORLD!"  
        // Tạo chuỗi con  
        System.out.println("Substring: " + s.substring(7, 12)); // Kết quả: "World"  
        // Thay thế ký tự  
        System.out.println("Replace 'l' with 'x': " + s.replace('l', 'x')); // Kết quả:  
"Hexxo, Worxd!"  
    }  
}
```

5.2.5. Lớp StringBuffer

Lớp StringBuffer được sử dụng đối với những chuỗi ký tự động, có thể thay đổi nội dung.

Tạo lập đối tượng StringBuffer

StringBuffer(String str) Tạo ra đối tượng mới nội dung giống như chuỗi đối số *str* kích thước bằng kích thước của chuỗi *str* + 16.

StringBuffer(int length) Tạo ra đối tượng mới chưa có nội dung và đặt kích thước bằng đối số *length*, nếu đối số lớn hơn 0.

StringBuffer() Tạo ra đối tượng mới chưa có nội dung và đặt kích thước của xâu buffer là 16.

Đọc và thay đổi các ký tự trong StringBuffer

int length() trả về số ký tự trong xâu.

char charAt(int index) trả về ký tự tại vị trí *index*

void setCharAt(int index, char ch) thay thế ký tự tại vị trí *index* bằng ký tự *ch*

Hệ thống sẽ cho qua ngoại lệ *StringIndexOutOfBoundsException* nếu các chỉ số khụng tương thích.

Chuyển StringBuffer sang String

Lớp *StringBuffer* nạp chồng hàm *toString()* để chuyển xâu buffer về xâu của lớp cố định *String*.

Hàm *append()* bổ sung các ký tự vào cuối của xâu.

StringBuffer append(Object obj) đối tượng *obj* được chuyển về xâu (bằng hàm *String.valueOf()*) và sau đó bổ sung vào xâu buffer.

StringBuffer append(String str) bổ sung xâu ký tự *str* vào cuối của xâu.

StringBuffer append(char[] str) bổ sung mảng ký tự *str* vào cuối của xâu.

StringBuffer append(char[] str, int offset, int len) bổ sung một phần mảng ký tự *str* (bắt đầu từ vị trí *offset* với độ dài *len*) vào cuối của xâu.

StringBuffer append(char c) bổ sung ký tự *c* vào cuối của xâu.

Các mảng ký tự *str* và ký tự *c* được chuyển về xâu và sau đó bổ sung vào xâu *buffer*

StringBuffer append(boolean b) bổ sung chuỗi biểu diễn của đối số boolean *b* vào cuối của xâu.

StringBuffer append(int i) bổ sung chuỗi biểu diễn số nguyên kiểu *int* vào cuối của xâu.

StringBuffer append(long l) bổ sung chuỗi biểu diễn số nguyên kiểu *long* vào cuối của xâu.

StringBuffer append(float f) bổ sung chuỗi biểu diễn số thực kiểu *float* vào cuối của xâu.

StringBuffer append(double d) bổ sung chuỗi biểu diễn số thực kiểu *double* vào cuối của xâu.

Các giá trị nguyên thủy được chuyển về xâu và sau đó bổ sung vào xâu *buffer*.

StringBuffer insert(int offset, Object obj)

StringBuffer insert(int offset, String str)

StringBuffer insert(int offset, char[] str)

StringBuffer insert(int offset, char c)

StringBuffer insert(int offset, boolean b)

StringBuffer insert(int offset, int i)

StringBuffer insert(int offset, long l)

StringBuffer insert(int offset, float f)

StringBuffer insert(int offset, double d)

chèn đối số thứ hai (sau khi được chuyển về dạng xâu bằng cách sử dụng hàm *String.valueOf()*) vào từ vị trí *offset* trong xâu *buffer*.

StringBuffer delete(int index)StringBuffer delete(int start, int end)

Hàm đầu xóa đi ký tự ở vị trí *index*, hàm sau xóa đi một xâu con kể từ vị trí *start* đến *end*.

StringBuffer reverse() đảo ngược xâu

Kiểm soát kích thước của StringBuffer

int capacity(): Cho lại kích thước (khả năng chứa) của xâu *buffer*.

void ensureCapacity(int minCap): Đảm bảo đủ chỗ cho ít nhất *minCap* ký tự của xâu *buffer*.

void setLength(int newLen): Đặt độ dài của xâu *buffer* là *newLen*.

Ví dụ 7:

```
public class Main {  
    public static void main(String[] args) {  
        StringBuffer sb = new StringBuffer("Hello");  
        // Thêm chuỗi vào cuối  
        sb.append(" World");  
        System.out.println("After append: " + sb); // Kết quả: "Hello World"  
        // Chèn chuỗi tại vị trí chỉ định  
        sb.insert(5, ",");  
        System.out.println("After insert: " + sb); // Kết quả: "Hello, World"  
        // Thay thế chuỗi  
        sb.replace(6, 11, "Java");  
        System.out.println("After replace: " + sb); // Kết quả: "Hello, Java"  
        // Xóa một phần của chuỗi  
        sb.delete(5, 6);  
        System.out.println("After delete: " + sb); // Kết quả: "Hello Java"  
        // Đảo ngược chuỗi  
        sb.reverse();  
        System.out.println("After reverse: " + sb); // Kết quả: "avaJ olleH"  
    }  
}
```

5.3. CASE STUDY

Case Study 1: Thư viện của một trường học hoặc doanh nghiệp cần quản lý danh sách các cuốn sách với thông tin như mã sách, tên sách, tác giả và thể loại. Người quản lý cần một chương trình để:

- Nhập thông tin sách mới vào hệ thống.
- Tìm kiếm sách theo mã sách.
- Hiện thị toàn bộ danh sách sách hiện có để dễ dàng theo dõi.

Yêu cầu chi tiết:

- **Tổ chức chương trình bằng gói (package):**

Phân chia các thành phần của chương trình thành các lớp trong một gói duy nhất (library.management) để dễ bảo trì và mở rộng.

- **Chức năng cần có:**

- + **Thêm sách:** Người dùng nhập thông tin sách để thêm vào danh sách.
- + **Tìm kiếm sách:** Tìm kiếm sách theo mã sách, trả về thông tin chi tiết của sách (nếu tìm thấy).
- + **Hiện thị danh sách:** In ra toàn bộ thông tin sách theo định dạng dễ đọc.

- **Chức năng mở rộng:** Thư viện có thể mở rộng thêm chức năng cho mượn, trả sách hoặc quản lý người đọc dựa trên nền tảng này.

Case Study 2: Một công ty giao hàng cần quản lý danh sách các đơn hàng và thời gian giao hàng dự kiến. Mỗi đơn hàng chứa thông tin như mã đơn, tên hàng, giá trị, và ngày giao dự kiến. Hệ thống cần tự động đọc dữ liệu từ file và hiện thị thông tin rõ ràng cho người quản lý.

Yêu cầu chi tiết:

- **Tổ chức danh sách đơn hàng:**

- + Sử dụng ArrayList để lưu trữ nhiều đơn hàng.
- + Mỗi đơn hàng là một đối tượng Order chứa thông tin chi tiết như mã đơn, tên hàng, giá trị, và ngày giao.

- **Tính toán thời gian:**

- + Sử dụng gói java.time để kiểm tra xem các đơn hàng có được giao đúng thời gian dự kiến không.

- **Chức năng cần có:**

- + Hiện thị toàn bộ đơn hàng.
- + Tính tổng giá trị các đơn hàng.
- + Kiểm tra đơn hàng có giao đúng ngày hay không (có thể thêm tính năng kiểm tra ngày hiện tại).

- **Chức năng mở rộng:** Chương trình này có thể mở rộng thành hệ thống quản lý giao hàng trực tuyến với các tính năng như theo dõi đơn hàng, gửi thông báo, và quản lý khách hàng.

Case Study 3: Một trường đại học cần quản lý thông tin sinh viên, bao gồm mã sinh viên, họ tên, và điểm trung bình. Hệ thống cần so sánh sinh viên dựa trên mã, hiển thị thông tin chi tiết, và lưu trữ dữ liệu hiệu quả trong các cấu trúc dữ liệu.

Yêu cầu chi tiết:

- **Sử dụng lớp Object:** Ghi đè (override) các phương thức từ lớp Object như toString(), equals(), và hashCode().
- **Chức năng cần có:**
 - + So sánh hai sinh viên xem có cùng mã không (equals()).
 - + In thông tin sinh viên dưới định dạng dễ đọc (toString()).
 - + Lưu trữ sinh viên trong các cấu trúc dữ liệu như HashSet hoặc HashMap để đảm bảo không trùng lặp.
- **Chức năng mở rộng:** Hệ thống có thể mở rộng để quản lý thêm các thông tin khác như khóa học, điểm chi tiết, hoặc lịch học của sinh viên.

Case Study 4: Một trò chơi đoán số ngẫu nhiên giúp người chơi luyện tư duy logic. Trò chơi cần kiểm tra xem người dùng nhập dữ liệu có hợp lệ hay không, và đưa ra kết quả so sánh giữa số ngẫu nhiên và số người chơi đoán.

Yêu cầu chi tiết:

- **Tạo số ngẫu nhiên:** Dùng lớp Math để tạo ra một số nguyên ngẫu nhiên từ 1 đến 100.
- **Kiểm tra nhập dữ liệu:**
 - + Dùng lớp String để kiểm tra chuỗi nhập vào từ người dùng.
 - + Xác định chuỗi có phải là số hợp lệ không, và nếu không thì thông báo lỗi.
- **Chức năng cần có:**
 - + Hiển thị kết quả đoán (đúng, lớn hơn, hoặc nhỏ hơn số ngẫu nhiên).
 - + Cho phép người dùng chơi lại nếu muốn.
- **Chức năng mở rộng:** Trò chơi này có thể mở rộng thành ứng dụng di động hoặc tích hợp thêm tính năng nhiều người chơi.

Case Study 5: Một phần mềm xử lý văn bản cần cung cấp các tính năng chỉnh sửa văn bản cơ bản như thêm, xóa, chèn, và đảo ngược nội dung. Người dùng có thể thao tác dễ dàng với các chuỗi văn bản mà không lo vấn đề hiệu suất.

Yêu cầu chi tiết:

- **Sử dụng StringBuffer:** Tận dụng hiệu suất cao của StringBuffer để thực hiện các thao tác chỉnh sửa văn bản.
- **Chức năng cần có:**
 - + Thêm chuỗi vào cuối văn bản.
 - + Chèn chuỗi tại một vị trí chỉ định.
 - + Xóa một đoạn văn bản dựa trên vị trí bắt đầu và kết thúc.

- + Đảo ngược toàn bộ văn bản.
- **Chức năng mở rộng:** Hệ thống có thể tích hợp thêm giao diện đồ họa để xây dựng một ứng dụng xử lý văn bản đơn giản.

5.4. VÍ DỤ ÁP DỤNG

Ví dụ ứng dụng **Case Study 5: Xử lý văn bản** bằng StringBuffer trong Java. Chúng ta sẽ xây dựng một chương trình đơn giản cho phép người dùng chỉnh sửa văn bản (thêm, xóa, chèn, và đảo ngược nội dung) bằng cách sử dụng lớp StringBuffer. Người dùng có thể nhập một chuỗi văn bản và thực hiện các thao tác như:

1. **Thêm văn bản** vào cuối chuỗi.
2. **Chèn văn bản** tại một vị trí chỉ định.
3. **Xóa một đoạn văn bản** từ một vị trí cụ thể.
4. **Đảo ngược văn bản**.

```
import java.util.Scanner;

public class StringBufferExample {
    public static void main(String[] args) {
        // Khởi tạo StringBuffer với văn bản ban đầu
        StringBuffer text = new StringBuffer("Học lập trình Java cơ bản.");
        Scanner scanner = new Scanner(System.in);
        // In ra văn bản ban đầu
        System.out.println("Văn bản hiện tại: " + text);
        while (true) {
            // Hiển thị menu lựa chọn
            System.out.println("\nChọn thao tác:");
            System.out.println("1. Thêm văn bản");
            System.out.println("2. Chèn văn bản tại vị trí");
            System.out.println("3. Xóa văn bản");
            System.out.println("4. Đảo ngược văn bản");
            System.out.println("5. Thoát");
            System.out.print("Lựa chọn của bạn: ");

            int choice = scanner.nextInt();
            scanner.nextLine(); // Đọc dòng trống còn lại trong bộ đệm
            switch (choice) {
                case 1: // Thêm văn bản
                    System.out.print("Nhập văn bản để thêm: ");
```

```

        String appendText = scanner.nextLine();
        text.append(appendText); // Thêm vào cuối chuỗi
        System.out.println("Văn bản hiện tại: " + text);
        break;
    case 2: // Chèn văn bản tại vị trí
        System.out.print("Nhập vị trí chèn (số nguyên): ");
        int position = scanner.nextInt();
        scanner.nextLine(); // Đọc dòng trống còn lại trong bộ đệm
        System.out.print("Nhập văn bản để chèn: ");
        String insertText = scanner.nextLine();
        text.insert(position, insertText); // Chèn văn bản vào vị trí
        System.out.println("Văn bản hiện tại: " + text);
        break;
    case 3: // Xóa văn bản
        System.out.print("Nhập vị trí bắt đầu để xóa: ");
        int start = scanner.nextInt();
        System.out.print("Nhập vị trí kết thúc để xóa: ");
        int end = scanner.nextInt();
        text.delete(start, end); // Xóa đoạn văn bản
        System.out.println("Văn bản hiện tại: " + text);
        break;
    case 4: // Đảo ngược văn bản
        text.reverse(); // Đảo ngược chuỗi
        System.out.println("Văn bản sau khi đảo ngược: " + text);
        break;
    case 5: // Thoát
        System.out.println("Thoát chương trình.");
        scanner.close();
        return;
    default:
        System.out.println("Lựa chọn không hợp lệ! Vui lòng chọn
        lại.");
    }
}

```

```
}  
}
```

Trong ví dụ này:

- StringBuffer giúp thực hiện các thao tác thay đổi chuỗi mà không tạo ra nhiều đối tượng mới, tiết kiệm bộ nhớ và tăng hiệu suất khi làm việc với chuỗi dài hoặc nhiều thao tác.
- Các phương thức append(), insert(), delete(), và reverse() của StringBuffer rất hữu ích trong các ứng dụng chỉnh sửa văn bản hoặc xử lý chuỗi.

5.5. TỔNG KẾT CHƯƠNG

Chương 5 cung cấp cái nhìn tổng quan về khái niệm gói (package) và giới thiệu các lớp cơ bản trong gói java.lang – một gói quan trọng mà Java cung cấp sẵn.

- Tạo gói cho chương trình và cách sử dụng gói: Gói là tập hợp các lớp và giao diện liên quan, giúp tổ chức mã nguồn hợp lý và dễ quản lý. Phần này hướng dẫn cách tạo gói cho chương trình và cách sử dụng chúng. Gói cũng giúp tránh xung đột tên giữa các lớp.
- Giới thiệu một số gói quan trọng của Java: Java cung cấp nhiều gói chuẩn, trong đó java.lang là gói mặc định luôn được tự động nhập vào các chương trình Java, chứa các lớp và hàm cơ bản nhất.
- Lớp Object: Lớp Object là lớp gốc của mọi lớp trong Java, cung cấp các phương thức cơ bản như equals(), hashCode(), và toString() để xử lý đối tượng.
- Các lớp nguyên thủy: Java không có kiểu dữ liệu nguyên thủy nào là đối tượng, nhưng các lớp bao bọc (wrapper classes) như Integer, Double, Boolean,... cho phép xử lý các kiểu dữ liệu nguyên thủy dưới dạng đối tượng.
- Lớp Math: Lớp Math cung cấp các phương thức toán học hữu ích như sqrt(), pow(), abs(),... hỗ trợ thực hiện các phép toán thông dụng.
- Lớp String: String là lớp để lưu trữ và thao tác chuỗi ký tự, với đặc tính bất biến (immutable). Các phương thức của String cho phép xử lý chuỗi như so sánh, cắt chuỗi, tìm kiếm,...
- Lớp StringBuffer: StringBuffer là lớp hỗ trợ tạo và chỉnh sửa chuỗi có thể thay đổi (mutable). Nó hiệu quả hơn khi cần thay đổi chuỗi liên tục mà không tạo ra các đối tượng mới.

5.6. CÂU HỎI TRẮC NGHIỆM

1. Gói trong Java giúp:

- A. Tăng tốc độ thực thi chương trình.
- B. Tổ chức mã nguồn, tránh xung đột tên lớp.
- C. Giảm bộ nhớ sử dụng.
- D. Tự động kiểm tra lỗi.

2. Lệnh nào dùng để khai báo một gói mới trong Java?

- A. new package

- B. create package
- C. package
- D. import package

3. Lớp nào trong Java là lớp cha của tất cả các lớp khác?

- A. java.lang.Class
- B. java.lang.System
- C. java.lang.Object
- D. java.lang.String

4. Gói nào được tự động nhập vào mọi chương trình Java?

- A. java.io
- B. java.util
- C. java.lang
- D. java.net

5. Lệnh nào dùng để nhập một lớp trong gói khác vào chương trình?

- A. include
- B. import
- C. load
- D. add

6. Phương thức *equals()* trong lớp *Object* dùng để:

- A. So sánh hai đối tượng có cùng tham chiếu không.
- B. So sánh nội dung của hai đối tượng.
- C. In ra địa chỉ của đối tượng.
- D. Chuyển đổi đối tượng thành chuỗi.

7. Để tính căn bậc hai của một số *x*, bạn dùng phương thức nào trong lớp *Math*?

- A. Math.sqrt(x)
- B. Math.pow(x, 2)
- C. Math.cbrt(x)
- D. Math.square(x)

8. Phương thức nào trong lớp *Object* được gọi tự động khi một đối tượng bị xóa?

- A. finalize()
- B. dispose()
- C. delete()
- D. destroy()

9. Lớp nào trong *java.lang* dùng để thao tác với chuỗi ký tự bất biến?

- A. String
- B. StringBuffer
- C. StringBuilder
- D. CharArray

10. Để thay đổi một chuỗi mà không tạo ra đối tượng mới, bạn nên dùng:

- A. String
- B. StringBuffer
- C. StringBuilder
- D. Cả B và C

11. Phương thức *hashCode()* trong Object trả về:

- A. Mã băm của đối tượng.
- B. Giá trị Boolean.
- C. Địa chỉ bộ nhớ của đối tượng.
- D. Chuỗi ký tự của đối tượng.

12. Integer là lớp bao bọc của kiểu dữ liệu nào?

- A. float
- B. double
- C. int
- D. char

13. Phương thức *toUpperCase()* của lớp String trả về:

- A. Chuỗi chữ thường.
- B. Chuỗi chữ hoa.
- C. Chuỗi đảo ngược.
- D. Chuỗi gốc.

14. Để nối chuỗi với một StringBuffer, ta dùng:

- A. concat()
- B. append()
- C. add()
- D. merge()

15. Lớp bao bọc của kiểu dữ liệu *double* là:

- A. Double
- B. Float
- C. Int
- D. Char

16. Math.abs(-10) trả về giá trị nào?

- A. -10
- B. 0
- C. 10
- D. Lỗi

17. Character là lớp bao bọc của:

- A. char
- B. String
- C. int
- D. boolean

18. Phương thức nào kiểm tra một chuỗi có trống không?

- A. isEmpty()
- B. isBlank()
- C. length()
- D. size()

19. Để tạo một chuỗi mới từ StringBuffer, dùng phương thức:

- A. toString()
- B. newString()
- C. create()
- D. getString()

20. Để chuyển một *String* thành *int*, dùng:

- A. Integer.parseInt()
- B. Integer.toString()
- C. Integer.valueOf()
- D. Int.convert()

21. System.out.println() thuộc về lớp nào?

- A. System
- B. PrintStream
- C. Output
- D. Console

22. Lệnh nào tạo một đối tượng StringBuffer với dung lượng mặc định?

- A. new StringBuffer()
- B. new StringBuffer("")
- C. StringBuffer.create()

D. new StringBuffer(0)

23. Math.PI trả về:

- A. Giá trị π .
- B. Giá trị e.
- C. Số nguyên gần nhất của π .
- D. Không giá trị nào.

24. Để so sánh hai chuỗi bất kể chữ hoa hay thường, dùng:

- A. equals()
- B. equalsIgnoreCase()
- C. compareTo()
- D. compare()

25. Phương thức *capacity()* trong StringBuffer trả về:

- A. Dung lượng hiện tại.
- B. Độ dài chuỗi.
- C. Độ dài tối đa của chuỗi.
- D. Giá trị mặc định.

26. System.gc() gợi ý JVM thực hiện:

- A. Thu hồi bộ nhớ.
- B. Tắt chương trình.
- C. Tạo đối tượng mới.
- D. Thực thi phương thức finalize.

27. Để chuyển một *String* thành *double*, dùng:

- A. Double.parseDouble()
- B. Double.toString()
- C. Double.valueOf()
- D. Double.convert()

28. Character.isDigit('9') trả về:

- A. true
- B. false
- C. 9
- D. '9'

29. Math.pow(2, 3) trả về:

- A. 6
- B. 8

- C. 2
- D. 9

30. StringBuffer khác String ở:

- A. Tính bất biến.
- B. Tốc độ thao tác.
- C. Khả năng thay đổi nội dung.
- D. Cả A và C.

5.7. BÀI TẬP THỰC HÀNH

Bài tập 1: Quản lý danh sách sinh viên bằng lớp String

Mô tả: Tạo một chương trình quản lý danh sách sinh viên, mỗi sinh viên có các thuộc tính id, name, dob (ngày sinh). Sử dụng lớp String để thao tác với tên và ngày sinh của sinh viên, bao gồm các chức năng tìm kiếm sinh viên theo tên, sắp xếp sinh viên theo ngày sinh, và thay đổi tên của sinh viên.

Yêu cầu:

- Tạo lớp Student với các thuộc tính: id (kiểu int), name (kiểu String), dob (kiểu String).
- Sử dụng phương thức String.equals() để tìm kiếm sinh viên theo tên.
- Sắp xếp danh sách sinh viên theo ngày sinh (dùng String.compareTo()).
- Cập nhật tên sinh viên bằng cách thay đổi giá trị của thuộc tính name sử dụng phương thức String.replace().

Bài tập 2: Xử lý chuỗi lớn với StringBuffer và StringBuilder

Mô tả: Tạo một chương trình có thể nối hàng triệu chuỗi lại với nhau và tính toán hiệu suất sử dụng String, StringBuffer và StringBuilder. Chạy ba thử nghiệm riêng biệt để đo thời gian thực hiện khi nối các chuỗi bằng các lớp khác nhau.

Yêu cầu:

- Dùng một vòng lặp để nối 1 triệu chuỗi ("Hello") vào một chuỗi duy nhất với ba cách:
 - + Dùng String.
 - + Dùng StringBuffer.
 - + Dùng StringBuilder.
- Đo thời gian thực hiện mỗi cách và in ra kết quả so sánh thời gian.
- Giải thích kết quả về sự khác biệt giữa String, StringBuffer, và StringBuilder khi làm việc với chuỗi lớn.

Bài tập 3: Xây dựng lớp ComplexNumber với các phép toán

Mô tả: Tạo lớp ComplexNumber để đại diện cho số phức với phần thực và phần ảo. Cung cấp các phương thức để thực hiện các phép toán với số phức, bao gồm cộng, trừ, nhân và chia. Sử dụng lớp Math để tính toán các phép toán.

Yêu cầu:

- Tạo lớp ComplexNumber với các thuộc tính: real (kiểu double) và imaginary (kiểu double).
- Cung cấp các phương thức:
 - + add(ComplexNumber other)
 - + subtract(ComplexNumber other)
 - + multiply(ComplexNumber other)
 - + divide(ComplexNumber other)
- Sử dụng lớp Math để tính toán các phép toán, ví dụ: tính module của số phức bằng công thức $\text{Math.sqrt}(\text{real}^2 + \text{imaginary}^2)$.
- Tạo đối tượng số phức và thực hiện các phép toán trên.

Bài tập 4: Thực hiện các phép toán số học với Math

Mô tả: Tạo một lớp StatisticsCalculator để tính toán các giá trị thống kê cơ bản từ một mảng số thực, bao gồm giá trị trung bình, phương sai, độ lệch chuẩn và giá trị lớn nhất, nhỏ nhất. Sử dụng các phương thức từ lớp Math để tính toán.

Yêu cầu:

- Tạo lớp StatisticsCalculator với một mảng các số thực (kiểu double[]).
- Cung cấp các phương thức:
 - + mean(): Tính giá trị trung bình.
 - + variance(): Tính phương sai.
 - + standardDeviation(): Tính độ lệch chuẩn.
 - + max(): Tìm giá trị lớn nhất.
 - + min(): Tìm giá trị nhỏ nhất.
- Sử dụng phương thức Math.sqrt() để tính độ lệch chuẩn và Math.max(), Math.min() để tìm giá trị lớn nhất và nhỏ nhất.

Bài tập 5: Đảo ngược chuỗi với điều kiện sử dụng StringBuffer

Mô tả: Tạo một lớp ReverseWithCondition cho phép đảo ngược chuỗi đầu vào, nhưng chỉ đảo ngược chuỗi con nằm trong dấu ngoặc đơn. Nếu có nhiều cặp dấu ngoặc đơn trong chuỗi, tất cả các chuỗi con trong dấu ngoặc đơn đều cần được đảo ngược.

Yêu cầu:

- Sử dụng StringBuffer để đảo ngược chuỗi con trong dấu ngoặc đơn.
- Phân tích chuỗi đầu vào và tìm tất cả các chuỗi con nằm trong dấu ngoặc đơn.
- Đảo ngược các chuỗi con này và kết hợp lại với phần còn lại của chuỗi.
- Ví dụ: Đầu vào "abc(def)ghi(jkl)" sẽ cho kết quả "abc(fed)ghi(lkj)".

Bài tập 6: Sắp xếp danh sách đối tượng với compareTo

Mô tả: Tạo một lớp Book với các thuộc tính title (kiểu String) và price (kiểu double). Cung cấp phương thức compareTo() để sắp xếp danh sách sách theo giá (giá thấp đến cao) và theo tên sách (alphabetically).

Yêu cầu:

- Tạo lớp Book với các thuộc tính title và price.
- Cung cấp phương thức compareTo() để so sánh hai cuốn sách theo price hoặc title.
- Trong lớp Main, tạo danh sách sách và sắp xếp theo giá hoặc tên sách, sử dụng Collections.sort().

Bài tập 7: Thực hiện phép toán số học phức tạp với lớp Math

Mô tả: Tạo một chương trình tính toán giá trị của biểu thức toán học phức tạp bằng cách sử dụng các phép toán trong lớp Math, ví dụ: $3 * \sin(x) + \sqrt{y}$.

Yêu cầu:

- Tạo lớp MathExpression với phương thức evaluate(double x, double y) để tính giá trị của biểu thức $3 * \sin(x) + \sqrt{y}$.
- Sử dụng các phương thức trong lớp Math như Math.sin(), Math.sqrt() để tính giá trị.
- Cho phép người dùng nhập các giá trị của x và y và tính toán kết quả.

Bài tập 8: Xây dựng hệ thống quản lý sản phẩm với String và Object

Mô tả: Tạo một chương trình quản lý sản phẩm trong kho, mỗi sản phẩm có các thuộc tính id, name, và quantity. Cung cấp các phương thức để thêm, xóa, tìm kiếm sản phẩm theo tên và cập nhật số lượng sản phẩm.

Yêu cầu:

- Tạo lớp Product với các thuộc tính id (kiểu int), name (kiểu String), và quantity (kiểu int).
- Cung cấp các phương thức:
 - + addProduct(Product product): Thêm sản phẩm vào danh sách.
 - + removeProduct(int id): Xóa sản phẩm theo id.
 - + findProductByName(String name): Tìm sản phẩm theo tên.
 - + updateQuantity(int id, int quantity): Cập nhật số lượng sản phẩm.
- Sử dụng phương thức equals() và toString() để so sánh và hiển thị thông tin sản phẩm.

Bài tập 9: Đánh giá hiệu suất của các phép toán số học với Math

Mô tả: Tạo một chương trình so sánh hiệu suất của các phép toán cơ bản (cộng, trừ, nhân, chia) giữa các số nguyên và số thực trong một số lượng lớn vòng lặp.

Yêu cầu:

- Dùng vòng lặp để thực hiện 1 triệu phép toán cộng, trừ, nhân, chia giữa hai số nguyên và hai số thực.
- Đo thời gian thực hiện của mỗi phép toán và in ra kết quả so sánh giữa các phép toán cho số nguyên và số thực.

Chương 6 KẾ THỪA

Chương này sẽ cung cấp một số khái niệm quan trọng của lập trình hướng đối tượng, đó là KẾ THỪA, VIẾT ĐỀ, CÂY KẾ THỪA & các quan hệ HAS – A, IS – A trong thiết kế quan hệ giữa các lớp. Trong đó, khái niệm KẾ THỪA sẽ đặc biệt được nhấn mạnh trong chương này. Kế thừa là một loại quan hệ giữa các lớp, cho phép lớp con (hoặc lớp dẫn xuất) kế thừa các thuộc tính và phương thức từ lớp cha (lớp cơ sở). Do đó, nó không những cho phép dùng lại mã nguồn một cách hiệu quả mà nó còn giúp tạo ra các lớp đối tượng có cấu trúc, có quan hệ rõ ràng, và dễ bảo trì.

Sau khi học xong chương này, người học có thể:

- Hiểu được các quan hệ “kế thừa”, hay quan hệ “is -a”, và quan hệ “has -a”.
- Xây dựng được lớp cơ sở, lớp dẫn xuất kế thừa từ lớp cơ sở sử dụng ngôn ngữ lập trình Java.
- Hiểu được các bước xây dựng cây kế thừa.
- Biết cách viết đề phương thức được kế thừa từ lớp cơ sở trong lớp dẫn xuất.

6.1. QUAN HỆ KẾ THỪA

Trong lập trình hướng đối tượng, kế thừa là một khái niệm quan trọng để thể hiện quan hệ giữa các lớp đối tượng. Kế thừa cho phép một lớp (còn được gọi là lớp con, hay lớp dẫn xuất) kế thừa các thuộc tính và phương thức từ một lớp khác (được gọi là lớp cha, hay lớp cơ sở).

Lớp con có thể kế thừa các phương thức và thuộc tính không là riêng tư của lớp cha và bổ sung thêm các thuộc tính, phương thức của riêng mình. Khi đó quan hệ kế thừa ta còn gọi là “*kế thừa và mở rộng (extends)*”. Trong giáo trình này, khi đề cập đến khái niệm kế thừa, chúng ta cần ngầm hiểu đó là quan hệ “*kế thừa mở rộng*”.

Bảng 6.1 Một số ví dụ về lớp cơ sở và lớp dẫn xuất

Lớp cơ sở	Lớp dẫn xuất
Người	Sinh viên, giảng viên, nhân viên văn phòng
Thiết bị điện tử	Tủ lạnh, điều hòa, máy giặt, TV, ...
Động vật	Chó, mèo, ...

Khi sử dụng kế thừa trong Java, chúng ta có thể xây dựng cấu trúc chương trình một cách có tổ chức, giảm độ lặp trong mã nguồn, cho phép dùng lại mã của lớp cơ sở (lớp cha) trong lớp con (lớp dẫn xuất) mà lớp con không cần phải khai báo, định nghĩa lại các thuộc tính, phương thức được kế thừa từ lớp cha. Ngoài ra, lớp con có thể mở rộng bổ sung thêm các thuộc tính, phương thức của riêng nó, đồng thời có thể thay đổi các thuộc tính, phương thức được kế thừa từ lớp cha (viết đề) sao cho phù hợp với yêu cầu cụ thể của nó mà không làm ảnh hưởng đến lớp cha và các lớp con khác kế thừa từ lớp cha này. Khi thay đổi lớp cha, mặc nhiên các lớp con sẽ được thừa hưởng những sự thay đổi này.

Kế thừa trong lập trình hướng đối tượng là khái niệm rất gần gũi với quan hệ kế thừa trong thế giới thực. Điều này giúp các chương trình hướng đối tượng giải quyết các bài toán thực tế dễ dàng hơn.

Để thể hiện quan hệ kế thừa trong chương trình Java, chúng ta sử dụng từ khóa *extends*.

Ví dụ: Quan hệ kế thừa giữa các lớp.

```
// Lớp cha
class Animal {
    int age;
    float weight;
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

// Lớp con kế thừa từ lớp cha
class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}

//Lớp con khác kế thừa từ lớp cha
class Cat extends Animal {
    void sound() {
        System.out.println("Cat meows");
    }
}
```

Trong ví dụ này, **Dog** và **Cat** là lớp con (các lớp dẫn xuất) kế thừa từ lớp cha **Animal**. Các lớp này đều có các thuộc tính tuổi (*age*), cân nặng (*weight*) và phương thức *sound()* được kế thừa từ lớp **Animal**, nhưng chúng viết đè phương thức *sound()* theo cách riêng phù hợp với ngữ cảnh của từng lớp.

Lưu ý: một số điểm quan trọng khi sử dụng kế thừa trong Java

1. Lớp con có thể kế thừa các thuộc tính và phương thức không phải là private của lớp cha.
2. Kế thừa chỉ hoạt động theo hướng từ lớp cha đến lớp con, nghĩa là lớp con có thể mở rộng (hoặc thay đổi) các tính năng của lớp cha, nhưng ngược lại không được phép.
3. Một lớp chỉ có thể kế thừa từ một lớp cha duy nhất, không hỗ trợ kế thừa đa lớp trực tiếp. Tuy nhiên, Java hỗ trợ đa kế thừa thông qua các interface.

6.2. THIẾT KẾ CÂY THỪA KẾ

Cây kế thừa là một cách biểu diễn trực quan mối quan hệ kế thừa giữa các lớp, cấu trúc của từng lớp trong lập trình hướng đối tượng. Nó thể hiện cách thức các lớp con được tạo ra từ lớp cha và chúng mở rộng hoặc thay đổi hành vi được kế thừa từ lớp cha như thế nào.

Ví dụ 1: Xét chương trình Java sau đây:

```
// Lớp cơ sở (Superclass)
class Vehicle{
    void drive(){
        System.out.println("Phương tiện có thể được lái.");
    }
}

//Lớp con kế thừa từ lớp cơ sở
class Car extends Vehicle {
    void honk() {
        System.out.println("Ô tô đang kéo còi. ");
    }
}

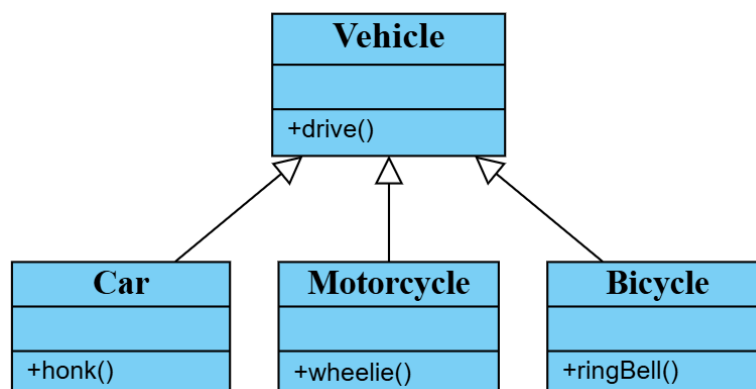
//Lớp con khác kế thừa từ lớp cơ sở
class Motorcycle extends Vehicle {
    void wheelie() {
        System.out.println("Xe máy đang đi bằng một bánh.");
    }
}

//Lớp con khác kế thừa từ lớp cơ sở
class Bicycle extends Vehicle {
    void ringBell() {
        System.out.println("Xe đạp đang rung chuông. ");
    }
}
```

Trong ví dụ này:

- Lớp **Vehicle** là lớp cơ sở chứa một phương thức *drive()*.
- Lớp **Car** kế thừa từ lớp **Vehicle** và có thêm phương thức *honk()*.
- Lớp **Motorcycle** kế thừa từ **Vehicle** và có thêm phương thức riêng là *wheelie()*.
- Lớp **Bicycle** cũng kế thừa từ **Vehicle** và có một phương thức riêng là *ringBell()*.

Cây kế thừa trong trường hợp này có dạng:



Hình 6.1 Cây kế thừa.

Như vậy, để thiết kế cây kế thừa trong Java, ta cần làm theo các bước sau:

Bước 1: Xác định các lớp cơ sở

- Xác định các tính chất chung hoặc hành vi chung mà tất cả các lớp con sẽ có.
- Tạo một lớp cơ sở chứa các thuộc tính và phương thức này.

Bước 2: Xác định các lớp con

- Xác định các đặc điểm riêng của mỗi lớp con.
- Tạo các lớp con kế thừa từ lớp cơ sở, bằng cách sử dụng từ khóa extends.

Bước 3: Xác định các thuộc tính và phương thức cho mỗi lớp

- Đảm bảo mỗi lớp con kế thừa tất cả hoặc một phần các thuộc tính và phương thức từ lớp cha.
- Thêm các thuộc tính và phương thức mới vào mỗi lớp con (nếu cần).

Bước 4: Xây dựng cây kế thừa

- Xây dựng cây kế thừa bằng cách sắp xếp các lớp con dưới dạng cây phân cấp, trong đó mỗi lớp con là một nhánh của lớp cha.

Ví dụ 2: Xét phần mềm quản lý người lao động trong công ty phần mềm ABC. Có hai vị trí công việc trong công ty bao gồm: người quản lý (manager) và phát triển viên (developer).

Phát triển viên và nhân viên đều cần phải quản lý các thông tin chung bao gồm: họ tên (name), số hiệu nhân viên (id), lương (salary). Với người quản lý, cần quản lý thêm thông tin phòng ban (department). Phát triển viên cần quản lý thêm thông tin về các ngôn ngữ lập trình mà họ có thể dùng để viết mã. Với vị trí là người quản lý, họ có quyền phê chuẩn việc nghỉ phép của phát triển viên. Phát triển viên cần thực hiện công việc như viết code. Cả phát triển viên và người quản lý đều có thể thực hiện việc tính lương và xem chi tiết thông tin cá nhân của họ.

Sau khi trải qua các bước thiết kế cây kế thừa như trên ta thu được kết quả:

Bước 1: Xác định được lớp cơ sở: *Employee*

- Thuộc tính: *name, id, salary*
- Phương thức: *calculateSalary(), displayDetails()*

Bước 2 & Bước 3: Xác định các lớp con & các thành phần bổ sung của chúng

+ Lớp con: **Manager**

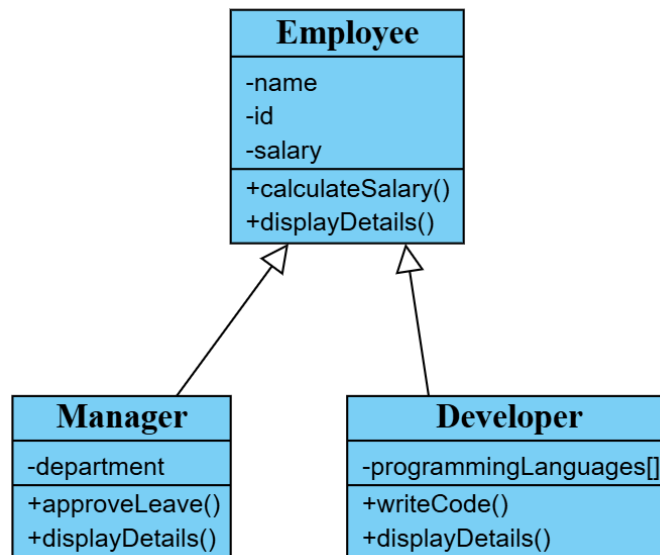
- Thuộc tính: *department*
- Phương thức: *approveLeave()*, *displayDetails()* (ghi đè để hiển thị thông tin cụ thể cho quản lý)

+ Lớp con: **Developer**

- Thuộc tính: *programmingLanguages[]*
- Phương thức: *writeCode()*, *displayDetails()* (ghi đè để hiển thị thông tin cụ thể cho nhà phát triển)

Bước 4: Xây dựng cây kế thừa.

=> Hình 6.2 là hình ảnh cây kế thừa được xây dựng.



Hình 6.2 Xây dựng cây kế thừa

Trong hình này, lớp **Manager** và **Developer** đều kế thừa từ lớp **Employee**, nghĩa là chúng sẽ có các thuộc tính và phương thức của lớp **Employee**, và ghi đè các phương thức *displayDetails()* để hiển thị thông tin cá nhân chi tiết tương ứng với từng vai trò, vị trí công việc trong công ty.

6.3. VIẾT ĐÈ

Viết đè (Overriding) trong Java là khái niệm mô tả tình huống: lớp con cung cấp một triển khai mới cho phương thức được kế thừa từ lớp cha. Nó cho phép lớp con thay đổi hoặc mở rộng hành vi của phương thức kế thừa từ lớp cha.

Ví dụ: Chương trình Java minh họa việc viết đè phương thức:

```
// Lớp cơ sở (Superclass)
class Animal {
    void makeSound() {
```

```

        System.out.println("Animal makes a sound.");
    }
}
//Lớp con kế thừa từ lớp cơ sở và viết đè phương thức make Sound()
class Dog extends Animal {
    // Phương thức makeSound() được viết đè
    @Override
    void makeSound() {
        System.out.println("Dog go go.");
    }
}
//Lớp con khác kế thừa từ lớp cơ sở và viết đè phương thức makeSound()
class Cat extends Animal {
    // Phương thức makeSound() được viết đè
    @Override
    void makeSound() {
        System.out.println("Cat meows.");
    }
}
public class Main {
    public static void main(String[] args) {
        Animal animal1 = new Dog();
        Animal animal2 = new Cat();
        animal1.makeSound(); // Kết quả: "Dog go go."
        animal2.makeSound(); // Kết quả: "Cat meows."
    }
}

```

Trong ví dụ này:

- Lớp **Animal** là lớp cơ sở, trong nó có một phương thức *makeSound()*.
- Lớp **Dog** và lớp **Cat** kế thừa từ lớp **Animal** và viết đè phương thức *makeSound()* để cung cấp triển khai mới cho hành vi âm thanh của chúng.
- Trong phương thức *main()*, một đối tượng của lớp **Dog** và lớp **Cat** được tạo và ta gọi phương thức *makeSound()* trên các đối tượng này, Java sẽ gọi phương thức đã được viết đè tương ứng của lớp con và trả về kết quả.

Lưu ý:

Khi một phương thức trong lớp con có cùng tên, cùng kiểu trả về và cùng các tham số với một phương thức trong lớp cha, nó được gọi là viết đè.

6.4. QUAN HỆ “IS – A” VÀ “HAS – A”

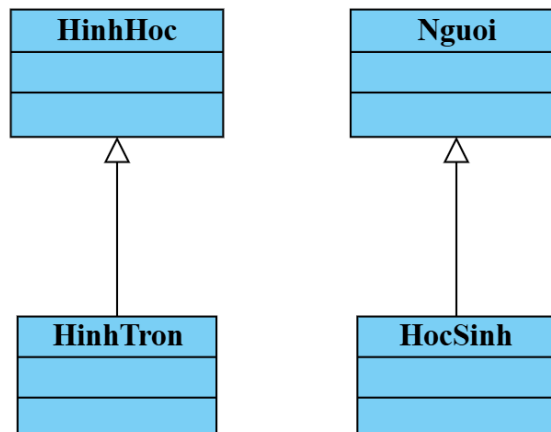
“is – a” và “has – a” là khái niệm dùng để mô tả hai loại quan hệ giữa các đối tượng trong lập trình hướng đối tượng.

6.4.1 Quan hệ “is – a”

Mỗi quan hệ "is-a" thường được biểu diễn thông qua quan hệ kế thừa giữa các lớp. Nếu một đối tượng ngoài việc nó là thể hiện của lớp đối tượng cụ thể mô tả nó (lớp A), nó còn thuộc một loại đối tượng (chứa nó) được biểu diễn bởi lớp khác (lớp B), thì chúng ta nói rằng có mối quan hệ “is – a” giữa lớp A và lớp B, hay A kế thừa B.

Ví dụ 1: Một đối tượng hình tròn, ngoài việc nó là thể hiện của lớp Hình tròn (lớp A), nó còn là một loại hình học, do đó nó cũng thuộc loại “Hình học” (lớp B). Tương tự một học sinh là ngoài việc bạn ấy là một học sinh (lớp Học Sinh), đồng thời bạn ấy cũng là một con người (lớp Người).

Mỗi quan hệ kế thừa (quan hệ “is – a”) giữa hai lớp được biểu diễn mũi tên có chiều hướng từ lớp dẫn xuất, đến lớp cơ sở. Hình 5.3 Biểu diễn quan hệ này (quan hệ giữa lớp *HìnhTron* và lớp *HìnhHoc*; quan hệ giữa lớp *HocSinh* và lớp *Ngươi*).



Hình 6.3 Quan hệ “is – a”, hay quan hệ kế thừa

Như vậy, có thể xem đối tượng thuộc lớp dẫn xuất là một trường hợp cụ thể của lớp cơ sở, hay "B is-a A", nghĩa là mọi đối tượng của lớp B cũng là đối tượng của lớp A, và chúng có thể được sử dụng trong mọi ngữ cảnh mà một đối tượng của lớp A có thể được sử dụng.

Một số ví dụ minh họa về mối quan hệ "is-a":

- Hình tròn "is-a" hình học.
- Chó "is-a" động vật.
- Sinh viên "is-a" người.

Tóm lại:

Mối quan hệ "is-a" trong Java thường được ánh xạ sang quan hệ kế thừa. Nó diễn đạt một mối quan hệ giữa hai lớp, trong đó một lớp được coi là một phiên bản cụ thể của lớp khác. Mối quan hệ "is-a" thường được sử dụng khi lớp con được coi là một loại đặc biệt hoặc chi tiết hơn của lớp cha.

6.4.2 Quan hệ “has – a”

"has-a" trong thiết kế hướng đối tượng diễn đạt mối quan hệ khi một đối tượng (là thể hiện của lớp A) chứa/sở hữu một hay nhiều đối tượng thuộc lớp khác (lớp B) như là thành phần của nó. Khi đó ta có qua hệ “A has a B”. Mối quan hệ "has-a" thường được sử dụng để biểu diễn mối quan hệ hợp thành, hoặc quan hệ sở hữu giữa các đối tượng.

Ví dụ 2: lớp A có một thuộc tính là một đối tượng thuộc lớp B, điều này có nghĩa là "A has-a B", đối tượng là một thể hiện của lớp A sở hữu hoặc chứa đối tượng là thể hiện của lớp B.

Dưới đây là một số ví dụ về mối quan hệ "has-a":

- Một ô tô "has-a" động cơ (một ô tô có/sở hữu một động cơ)
- Một ngôi nhà "has-a" phòng ngủ, phòng khách, phòng bếp, ... (một ngôi nhà có một hoặc nhiều phòng ngủ, phòng khách, và phòng bếp)
- Một người "has-a" địa chỉ (một người sở hữu/có một địa chỉ)

Ví dụ 3: mã nguồn thể hiện quan hệ giữa hai lớp: *Oto* & *DongCo*:

```
class DongCo {  
    // mã lệnh ở đây  
}  
  
class Oto {  
    private DongCo dongCo; // dongCo là thể hiện của lớp DongCo  
    // mã lệnh ở đây  
}
```

Như vậy, mối quan hệ "has-a" giữa các lớp đối tượng diễn đạt một mối quan hệ sở hữu hoặc hợp thành, trong đó một lớp đối tượng được cho là chứa hoặc sở hữu một hoặc nhiều đối tượng là thể hiện của lớp khác.

Tùy theo mức độ quan hệ từ lỏng lẻo đến chặt chẽ của “has – a”, nó có thể được chia thành ba mối quan hệ cụ thể hơn:

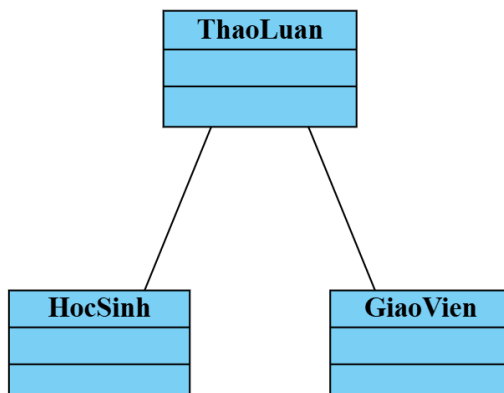
Quan hệ kết hợp (Association)

Quan hệ kết hợp là mối quan hệ (lỏng lẻo nhất) giữa hai đối tượng, trong đó đối tượng thuộc lớp này (lớp A) có thể tương tác với đối tượng thuộc lớp khác (lớp B) và ngược lại. Tương tác có thể hiểu là trong lớp A sẽ có các khai báo, hoặc sử dụng đối tượng thuộc lớp B, và ngược lại.

Ví dụ 4: một chủ đề thảo luận sẽ có giáo viên và học sinh cùng tham gia thảo luận. Như vậy lớp *ThaoLuan* sẽ có **quan hệ kết hợp** với lớp *HocSinh* và lớp *GiaoVien* và ngược lại.

Quan hệ này thường được biểu diễn bởi đoạn thẳng nối giữa hai lớp tương ứng.

Hình 6.4 thể hiện mối quan hệ kết hợp giữa các lớp này:



Hình 6.4 Ví dụ minh họa quan hệ “Association”

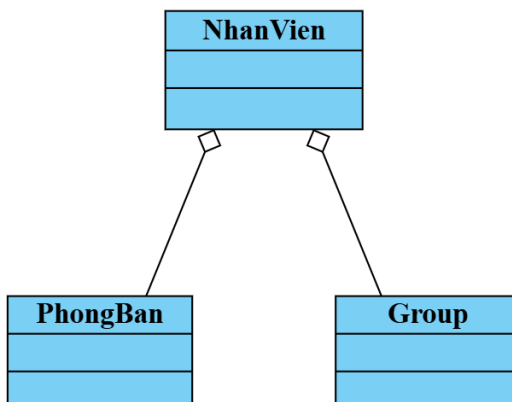
Quan hệ kết tập (Aggregation)

Quan hệ kết tập là một dạng quan hệ chặt chẽ hơn quan hệ kết hợp, trong đó một đối tượng chứa nhiều đối tượng thuộc lớp khác. Đối tượng chứa không chịu trách nhiệm về sự tồn tại của đối tượng con và đối tượng con có thể tồn tại mà không cần đến đối tượng chứa. Đối tượng con này có thể nằm trong đối tượng chứa khác. Do đó quan hệ này đôi khi còn được gọi là quan hệ hợp thành có chia sẻ (chia sẻ đối tượng con).

Ví dụ 5:

Một phòng ban (lớp **PhongBan**) có thể chứa nhiều nhân viên (lớp **NhanVien**), và trong một nhóm người đồng hương (**Group**) có thể chứa đối tượng lớp nhân viên (**NhanVien**). Như vậy, **PhongBan** có quan hệ kết tập với **NhanVien**, **Group** cũng có quan hệ kết tập với **NhanVien**. Khi hủy lớp **Phòng ban**, hoặc lớp **Group**, lớp **NhanVien** vẫn tồn tại.

Mỗi quan hệ kết tập thường được biểu diễn bằng một đường nối với một hình “Diamond” trên đỉnh. Hình 6.5 biểu diễn các quan hệ trong ví dụ này.



Hình 6.5 Quan hệ “Aggregation”

Trong Hình 6.5, lớp **PhongBan** chứa một danh sách các đối tượng thuộc lớp **NhanVien**, Lớp **Group** cũng chứa một danh sách các đối tượng thuộc lớp **NhanVien**.

Quan hệ hợp thành (Composition)

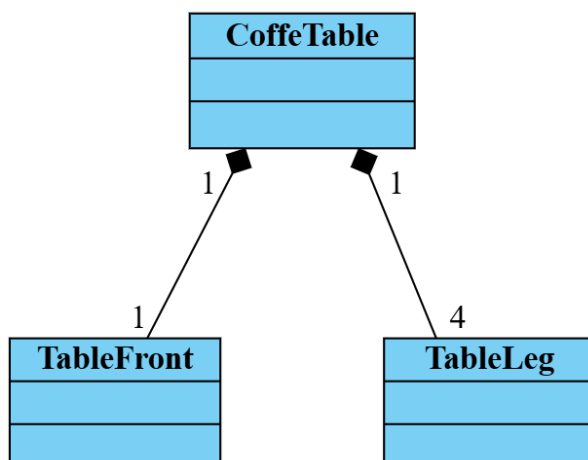
Quan hệ hợp thành là một dạng quan hệ chặt chẽ nhất, chặt chẽ hơn quan hệ kết tập. Trong đó, đối tượng thành phần được tạo và quản lý bởi đối tượng chứa (đối tượng tổng thể). Nếu đối tượng chứa bị hủy (lớp chứa bị hủy) thì đối tượng thành phần cũng bị hủy theo (lớp thành phần bị hủy theo). Trong chương trình Java, để mã hóa quan hệ này ta thường tạo lớp thành phần (inner class) bên trong lớp chứa.

Quan hệ này thường được biểu diễn bởi đường nối với một hình “Diamond đặc” trên đỉnh.

Ví dụ 6:

Một chiếc bàn cà phê (thuộc lớp **CoffeTable**) sẽ được cấu thành từ một mặt bàn (là thể hiện của lớp thành phần **TableFront** và bốn chiếc chân bàn (là các thể hiện của lớp thành phần **TableLeg**). Như vậy, ta có thể nói: **CoffeTable** được hợp thành từ **TableFront** và **TableLeg**; hoặc **TableFront** và **TableLeg** là một phần cấu tạo của **CoffeeTable**.

Khi viết code, **TableFront** và **TableLeg** thường sẽ được khai báo lồng bên trong **CoffeeTable**. Hình 6.6 biểu diễn các quan hệ cho ví dụ này:



Hình 6.6 Ví dụ về quan hệ hợp thành (composition)

6.5. CASE STUDY

Thư viện XYZ cần một phần mềm để quản lý thông tin về các loại sách trong bộ sưu tập của họ. Các loại sách cần quản lý bao gồm: Tiểu thuyết, sách tham khảo, sách giáo trình, sách kỹ thuật, và sách hướng dẫn.

Yêu cầu:

- Xác định các thông tin cần quản lý của từng loại sách;
- Xác định lớp cơ sở và các lớp dẫn xuất;
- Thiết kế cây kế thừa;
- Viết chương trình nhập vào tất cả các cuốn sách trong bộ sưu tập và in ra các cuốn sách đã sưu tập trong ba tháng gần đây.

6.6. Ví dụ áp dụng

Xét chương trình quản lý giáo viên và sinh viên tham gia các khóa học tại một trung tâm đào tạo.

Yêu cầu: Viết chương trình Java khai báo các lớp **Person**, **Student** và **Teacher**:

- **Person** là lớp cơ sở, **Student** và **Teacher** là các lớp con của **Person**.
- Mỗi lớp có các phương thức để nhập, xuất thông tin và thực hiện các chức năng cụ thể.
- Trong hàm main của lớp chính **TrainingCenterManagement**, phép người dùng nhập vào một danh sách các sinh viên, và các giáo viên tham gia khóa học, đồng thời hiển thị thông tin của tất cả các thành viên tham gia.

=> Chương trình minh họa:

```
import java.util.ArrayList;
import java.util.Scanner;
class Person {
    protected String name;
    protected int age;
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
    public void displayInfo() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}
class Student extends Person {
    private ArrayList<Double> grades;
    public Student(String name, int age) {
        super(name, age);
        grades = new ArrayList<>();
    }
    public void addGrade(double grade) {
        grades.add(grade);
    }
    public double calculateAverageGrade() {
        double sum = 0;
        for (double grade : grades) {
```

```

        sum += grade;
    }
    return sum / grades.size();
}

public void displayInfo() {
    super.displayInfo();
    System.out.println("Average Grade: " + calculateAverageGrade());
}
}

class Teacher extends Person {
    private String course;
    public Teacher(String name, int age, String course) {
        super(name, age);
        this.course = course;
    }
    public void displayInfo() {
        super.displayInfo();
        System.out.println("Course: " + course);
    }
}

public class TrainingCenterManagement {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        ArrayList<Person> participants = new ArrayList<>();
        while (true) {
            System.out.println("1. Add Student");
            System.out.println("2. Add Teacher");
            System.out.println("3. Display Information");
            System.out.println("4. Exit");
            System.out.print("Choose an option: ");
            int option = scanner.nextInt();
            switch (option) {
                case 1:
                    System.out.print("Enter student name: ");

```

```

String studentName = scanner.next();
System.out.print("Enter student age: ");
int studentAge = scanner.nextInt();
Student student = new Student(studentName, studentAge);
System.out.print("Enter number of grades: ");
int numOfGrades = scanner.nextInt();
System.out.println("Enter grades:");
for (int i = 0; i < numOfGrades; i++) {
    double grade = scanner.nextDouble();
    student.addGrade(grade);
}
participants.add(student);
break;
case 2:
    System.out.print("Enter teacher name: ");
    String teacherName = scanner.next();
    System.out.print("Enter teacher age: ");
    int teacherAge = scanner.nextInt();
    System.out.print("Enter course: ");
    String course = scanner.next();
    Teacher teacher = new Teacher(teacherName, teacherAge,
course);

    participants.add(teacher);
    break;
case 3:
    for (Person person : participants) {
        System.out.println("-----");
        person.displayInfo();
    }
    break;
case 4:
    System.out.println("Exiting program...");
    System.exit(0);
    break;

```

default:

```
        System.out.println("Invalid option. Please choose again.");
    }
}
}
}
```

6.7. TỔNG KẾT CHƯƠNG

Chương này, chúng ta đã tìm hiểu về:

1. Quan hệ kế thừa trong Java:

- Kế thừa cho phép một lớp (lớp con) kế thừa các thuộc tính và phương thức từ một lớp khác (lớp cơ sở hoặc lớp cha).
- Lớp con được phép mở rộng từ lớp cơ sở, cho phép tái sử dụng mã nguồn từ lớp cha, và bổ sung thêm các thành phần mới phù hợp với ngữ cảnh của nó.

2. Quan hệ "is-a" và "has-a":

- Quan hệ "is-a" trong lập trình hướng đối tượng biểu diễn một đối tượng là một loại đặc biệt/cụ thể của một loại khác. Ví dụ: chó là một loại động vật, người là một loại động vật cấp cao, động vật cấp cao lại là một động vật.
- Quan hệ "has-a" biểu diễn mối quan hệ sở hữu hoặc chứa/có, ví dụ: "Lớp học có/chứa một hoặc nhiều sinh viên".

3. Viết đè (Override):

- Khi một phương thức trong lớp con có cùng tên, cùng kiểu trả về và cùng các tham số với một phương thức trong lớp cha, nó được gọi là viết đè.
- Viết đè cho phép lớp con cung cấp một triển khai cụ thể cho phương thức mà nó kế thừa từ lớp cha.

4. Thiết kế cây kế thừa trong Java:

- Thiết kế cây kế thừa là quá trình tổ chức các lớp thành một cấu trúc phân cấp, trong đó mỗi lớp con kế thừa từ một lớp cha.
- Cây kế thừa cho phép tái sử dụng mã và tổ chức mã nguồn một cách có cấu trúc, trực quan và dễ dàng quản lý, bảo trì.

6.8. CÂU HỎI TRẮC NGHIỆM

1. Quan hệ "is-a" trong lập trình hướng đối tượng biểu thị điều gì?

- A. Một đối tượng chứa một đối tượng khác.
- B. Một đối tượng là một loại con của một loại khác.
- C. Một đối tượng sở hữu một đối tượng khác.
- D. Tất cả các phương án trên.

2. Trong Java, quan hệ "has-a" biểu thị điều gì?

- A. Một lớp sở hữu một lớp khác như là một thuộc tính của nó.
 - B. Một lớp có một lớp khác bên trong nó.
 - C. Một lớp là một loại con của lớp khác.
 - D. Tất cả các phương án trên.
- 3. Quan hệ kế thừa trong Java cho phép ...?**
- A. Tạo ra các lớp mới từ một lớp hiện có.
 - B. Tái sử dụng mã nguồn và tổ chức mã nguồn một cách hiệu quả.
 - C. Tạo ra các lớp con từ các lớp cha.
 - D. Tất cả các phương án trên.
- 4. Viết đè (Override) trong Java cho phép ...?**
- A. Thay đổi tên của một phương thức trong lớp con.
 - B. Ghi đè một thuộc tính trong lớp cha bằng cách triển khai lại nó trong lớp con.
 - C. Gọi một phương thức của lớp cha trong lớp con.
 - D. Tất cả các phương án trên.
- 5. Trong quan hệ kế thừa giữa hai lớp, lớp nào là lớp cơ sở?**
- A. Lớp cha
 - B. Lớp con
 - C. Lớp con trực tiếp kế thừa từ lớp cha
 - D. Lớp không liên quan đến quan hệ kế thừa
- 6. Quan hệ "is-a" được triển khai như thế nào trong Java?**
- A. Sử dụng từ khóa "is-a" trong khai báo lớp.
 - B. Sử dụng kỹ thuật kế thừa giữa các lớp.
 - C. Sử dụng kỹ thuật ghi đè phương thức.
 - D. Sử dụng kỹ thuật has-a giữa các lớp.
- 7. Quan hệ "has-a" được hiểu như thế nào trong Java?**
- A. Một lớp là một loại đặc biệt của lớp khác.
 - B. Một lớp chứa một lớp khác như là một phần của nó.
 - C. Một lớp sở hữu một lớp khác như là một thuộc tính của nó.
 - D. Một lớp là cha của lớp khác.
- 8. Trong lập trình hướng đối tượng, kế thừa nghĩa là gì?**
- A. Một lớp con sử dụng lại các thuộc tính và phương thức của một lớp cha.
 - B. Một lớp cha sử dụng lại các thuộc tính và phương thức của một lớp con.
 - C. Một lớp con nằm trong lớp cha.
 - D. Tất cả các phương án trên.

- 9. Trong Java, từ khóa nào được sử dụng để thể hiện quan hệ kế thừa và mở rộng?**
- A. inherit
 - B. extends
 - C. override
 - D. implement
- 10. Khi viết đè một phương thức trong Java, mục đích chính là gì?**
- A. Thay đổi tên của một phương thức trong lớp con.
 - B. Ghi đè một phương thức trong lớp cha để có thể sử dụng nó trong lớp con.
 - C. Thay đổi phạm vi truy cập của một phương thức trong lớp cha.
 - D. Tất cả các phương án trên.
- 11. Trong Java, một lớp có thể kế thừa từ nhiều lớp cha không?**
- A. Có, Java hỗ trợ đa kế thừa.
 - B. Không, Java không hỗ trợ đa kế thừa.
 - C. Có, nhưng chỉ khi sử dụng từ khóa "implements".
 - D. Có, nhưng chỉ khi sử dụng từ khóa "extend".
- 12. Trong quan hệ kế thừa giữa hai lớp, lớp nào là lớp dẫn xuất?**
- A. Lớp cha
 - B. Lớp con
 - C. Lớp con trực tiếp kế thừa từ lớp cha
 - D. Lớp không liên quan đến quan hệ kế thừa
- 13. Trong lập trình hướng đối tượng, mục đích chính của việc sử dụng kế thừa là gì?**
- A. Tạo ra các lớp con mới từ các lớp hiện có.
 - B. Tái sử dụng mã nguồn và tổ chức mã nguồn một cách hiệu quả.
 - C. Tạo ra mối quan hệ "is-a" giữa các lớp.
 - D. Tất cả các phương án trên.
- 14. Biểu đồ lớp đối tượng trong lập trình hướng đối tượng được sử dụng để làm gì?**
- A. Mô tả cấu trúc và mối quan hệ giữa các lớp trong một hệ thống.
 - B. Mô tả dữ liệu được sử dụng trong một hệ thống.
 - C. Mô tả luồng điều khiển của chương trình.
 - D. Mô tả các phương thức của một lớp cụ thể.
- 15. Trong biểu đồ lớp đối tượng, mối quan hệ "is-a" được biểu diễn bằng cách nào?**
- A. Một mũi tên đặc chỉ hướng từ lớp con đến lớp cha.
 - B. Một mũi tên chỉ hướng từ lớp cha đến lớp con.
 - C. Một đường thẳng vẽ từ lớp đến lớp khác.

- D. Một hình chữ nhật với một tiêu đề "has-a" được vẽ giữa hai lớp.
- 16. Trong biểu đồ lớp đối tượng, mối quan hệ hợp thành "composition" được biểu diễn như thế nào?**
- A. Một mũi tên chỉ hướng từ lớp cha đến lớp con.
 - B. Một mũi tên chỉ hướng từ lớp con đến lớp cha.
 - C. Một đường thẳng vẽ từ lớp này đến lớp kia.
 - D. Đường nối với một hình "Diamond đặc" trên đỉnh phía lớp tổng thể.
- 17. Trong biểu đồ lớp đối tượng, mối quan hệ kết hợp "aggregation" được biểu diễn như thế nào?**
- A. Một mũi tên chỉ hướng từ lớp cha đến lớp con.
 - B. Đường nối với một hình "Diamond" trên đỉnh
 - C. Một đường thẳng vẽ từ lớp này đến lớp kia.
 - D. Đường nối với một hình "Diamond đặc" trên đỉnh phía lớp tổng thể.
- 18. Trong biểu đồ lớp đối tượng, có thể mối quan hệ nào giữa các lớp?**
- A. Quan hệ kế thừa.
 - B. Quan hệ kết hợp.
 - C. Quan hệ cấu thành.
 - D. Tất cả A, B, C đều đúng
- 19. Thiết kế cây kế thừa trong lập trình hướng đối tượng có tác dụng gì?**
- A. Giảm sự phức tạp của mã nguồn.
 - B. Tăng tính tái sử dụng của mã nguồn.
 - C. Dễ dàng quản lý và mở rộng hệ thống.
 - D. Tất cả các phương án trên.
- 20. Xét hai lớp có quan hệ kế thừa, lớp nào là lớp cơ sở, lớp nào là lớp dẫn xuất?**
- A. Lớp cha là lớp cơ sở, lớp con là lớp dẫn xuất
 - B. Lớp con là lớp cơ sở, lớp cha là lớp dẫn xuất.
 - C. Lớp cha và lớp con đều có thể là lớp cơ sở.
 - D. Lớp cha và lớp con đều là lớp dẫn xuất.
- 21. Trong Java, quan hệ giữa lớp và giao diện từ khóa nào để thể hiện?**
- A. Sử dụng từ khóa "implements"
 - B. Sử dụng từ khóa "extends"
 - C. Sử dụng từ khóa "inherits"
 - D. Sử dụng từ khóa "inherits from"
- 22. Lớp con có thể kế thừa bao nhiêu lớp cha trong Java?**

- A. Chỉ một lớp cha
- B. Nhiều hơn một lớp cha
- C. Lớp con không thể kế thừa từ lớp cha
- D. Tùy thuộc vào cách hiện thực hóa

23. Trong Java, đâu là một loại quan hệ "has-a"?

- A. Kế thừa
- B. Mở rộng
- C. Composition
- D. "is -a"

24. Trong Java, lớp nào không cho phép kế thừa?

- A. lớp final
- B. lớp abstract
- C. lớp static
- D. Tất cả đều có thể được kế thừa

25. Khi một lớp kế thừa từ một lớp khác, nó thừa hưởng những gì từ lớp cha?

- A. Chỉ các phương thức
- B. Chỉ các thuộc tính
- C. Tất cả phương thức và thuộc tính
- D. Chỉ các phương thức, thuộc tính không là private của lớp cha.

26. Sự khác biệt chính giữa quan hệ "has - a" và quan hệ "is - a" là gì?

- A. "Has - a" là quan hệ composition trong khi "is - a" là quan hệ inheritance.
- B. "Has - a" là quan hệ inheritance trong khi "is - a" là quan hệ composition.
- C. Không có sự khác biệt, hai quan hệ này đều giống nhau.
- D. "Has - a" là quan hệ mở rộng trong khi "is - a" là quan hệ inheritance.

27. Khi sử dụng quan hệ "has - a", việc tạo đối tượng thành viên nên được thực hiện ở đâu?

- A. Trong lớp cha
- B. Trong lớp con
- C. Trong lớp có quan hệ "has - a"
- D. Tạo đối tượng ở đâu không quan trọng.

28. Quan hệ kế thừa trong lập trình hướng đối tượng có tầm quan trọng như thế nào?

- A. Quan trọng để tạo ra các đối tượng mới từ các đối tượng hiện có, giúp tái sử dụng mã và giảm sự phức tạp của mã.
- B. Không quan trọng, vì kế thừa chỉ tạo ra sự phụ thuộc giữa các lớp và gây ra sự phức tạp không cần thiết.

C. Kế thừa chỉ có tầm quan trọng trong một số trường hợp cụ thể và không phải là một nguyên tắc cần thiết trong lập trình hướng đối tượng.

D. Quan trọng để tạo ra các tượng có khả năng truyền thông điệp

29. Kế thừa trong lập trình hướng đối tượng đóng vai trò gì?

A. Kế thừa cho phép các lớp con có thể kế thừa các thuộc tính và phương thức từ các lớp cha của chúng.

B. Kế thừa là một khái niệm không cần thiết và ít được sử dụng trong lập trình hướng đối tượng.

C. Kế thừa chỉ là một thuộc tính của ngôn ngữ lập trình và không có tác dụng thực tế trong việc thiết kế chương trình.

D. Kế thừa chỉ là một phần của quan hệ “has - a” và không có ý nghĩa độc lập.

30. Tại sao quan hệ kế thừa và mở rộng lại quan trọng trong việc phát triển chương trình lớn?

A. Giúp tạo ra các đối tượng có cấu trúc, dễ hiểu và dễ bảo trì hơn.

B. Tạo ra sự phụ thuộc giữa các lớp, làm tăng độ phức tạp của chương trình.

C. Không quan trọng, vì các ngôn ngữ lập trình hiện đại không cần thiết kế theo mô hình kế thừa và mở rộng.

D. Chỉ quan trọng trong việc phát triển các ứng dụng nhỏ và không quan trọng trong các dự án lớn.

6.9. BÀI TẬP THỰC HÀNH

Bài tập 1: Tính diện tích của các hình học:

Xây dựng một lớp cơ sở *Shape* có một phương thức *calculateArea()* để tính diện tích. Tạo các lớp con ***Circle***, ***Rectangle***, và ***Triangle***, kế thừa từ lớp ***Shape*** và ghi đè phương thức *calculateArea()* để tính diện tích riêng cho mỗi loại hình.

Bài tập 2: Quản lý nhân viên

Viết một chương trình Java để quản lý thông tin của các nhân viên trong một công ty.

- Mỗi nhân viên có các thông tin chung cần quản lý gồm: Họ và tên, Tuổi, Mức lương cơ bản.
- Công ty có hai loại nhân viên: Nhân viên văn phòng và Nhân viên sản xuất.
- Nhân viên văn phòng có các thuộc tính bổ sung: Số ngày làm việc trong tháng, Hệ số phụ cấp.
- Nhân viên sản xuất có các thuộc tính bổ sung: Số sản phẩm đã sản xuất trong tháng, Giá bán của mỗi sản phẩm.

Yêu cầu:

- Xây dựng lớp nhân viên (***Staff***) với các thuộc tính chung và một phương thức để tính toán lương.
- Xây dựng lớp nhân viên văn phòng (***OfficeStaff***) kế thừa từ lớp ***Staff***, thêm các thuộc tính và phương thức cần thiết.

- Xây dựng lớp nhân viên sản xuất (***ProductionStaff***) kế thừa từ lớp ***Staff***, thêm các thuộc tính và phương thức cần thiết.
- Viết một chương trình chính để kiểm tra các lớp đã xây dựng, nhập thông tin của một số nhân viên văn phòng và nhân viên sản xuất, sau đó hiển thị thông tin và lương của từng nhân viên.

Chương 7 ĐA HÌNH VÀ LỚP TRỪU TƯỢNG

Chương 7, chúng ta sẽ khám phá hai khía cạnh quan trọng trong lập trình hướng đối tượng – ĐA HÌNH & LỚP TRỪU TƯỢNG.

- ĐA HÌNH là một khái niệm mô tả khả năng truyền một thông điệp chung đến nhiều đối tượng và mỗi đối tượng sẽ phản hồi lại thông điệp này; hoặc nó mô tả khả năng truyền một thông điệp đến một đối tượng và đối tượng này sẽ phản hồi theo các cách khác nhau tùy thuộc vào ngữ cảnh của nó. Do vậy đa hình mang lại tính linh hoạt cao trong việc mô tả cùng một hành vi nhưng khi hành vi này được thực hiện bởi đối tượng khác nhau, kết quả trả về sẽ khác nhau.
- LỚP TRỪU TƯỢNG là một khái niệm hỗ trợ cho đa hình, tạo điều kiện cho việc ghi đè được thuận lợi.

Sau khi học xong chương này, người học có thể:

- Hiểu được khái niệm ĐA HÌNH, tình huống sử dụng.
- Cài đặt được tính đa hình thông qua việc ghi đè, hoặc nạp chồng trong chương trình Java.
- Hiểu được khái niệm LỚP TRỪU TƯỢNG & làm việc được với nó trong chương trình Java.

7.1. ĐA HÌNH

Đa hình (polymorphism) là một khái niệm quan trọng của lập trình hướng đối tượng. Đa hình mô tả khả năng: (1) truyền một thông điệp chung đến nhiều đối tượng và mỗi đối tượng sẽ phản hồi lại thông điệp theo cách riêng của nó; hoặc (2) nó mô tả khả năng truyền một thông điệp đến một đối tượng và đối tượng này sẽ phản hồi theo các cách khác nhau tùy thuộc vào ngữ cảnh của nó (ngữ cảnh được hiểu thông qua các tham số truyền đi).

Do đó, trong lập trình hướng đối tượng, có hai loại đa hình: (1) đa hình tĩnh; (2) đa hình động.

7.1.1. Đa hình tĩnh

Đa hình tĩnh còn được gọi là **compile-time polymorphism**, hay **overloading polymorphism**.

Đa hình tĩnh mô tả khả năng cùng một hành vi được triệu gọi trên đối tượng, với ngữ cảnh khác nhau, đối tượng này sẽ thực hành vi đó theo các cách khác nhau (~ đối tượng đa hình thái)

Đa hình tĩnh thể hiện thông qua việc cho phép trong một lớp đối tượng, ta có thể nạp chồng các phương thức/hành vi; nghĩa là ta có thể khai báo nhiều phương thức trùng tên (~ thông điệp), nhưng danh sách các đối số của các phương thức này là khác nhau (~ ngữ cảnh khác nhau) và nội dung thực thi của các phương thức này cũng khác nhau, phụ thuộc vào ngữ cảnh (~ phản hồi lại thông điệp gửi đi theo các cách khác nhau tùy theo ngữ cảnh cụ thể).

Java sẽ chọn phương thức thích hợp dựa trên số lượng và kiểu dữ liệu của tham số được truyền vào trong lúc biên dịch (dựa trên ngữ cảnh).

Ví dụ 1: Đa hình tĩnh (nạp chồng)

Xét chương trình Java sau đây:

```
class Calculator {  
    // Phương thức tính tổng cho hai số nguyên
```

```

public int add(int x, int y) {
    return x + y;
}
// Phương thức tính tổng cho ba số nguyên
public int add(int x, int y, int z) {
    return x + y + z;
}
// Phương thức tính tổng cho hai số thực
public double add(double x, double y) {
    return x + y;
}
}
class Main {
    public static void main(String[] args) {
        Calculator calc = new Calculator();
        // Gọi phương thức add() với hai số nguyên
        System.out.println("Sum of 5 and 10: " + calc.add(5, 10));
        // Gọi phương thức add() với ba số nguyên
        System.out.println("Sum of 5, 10, and 15: " + calc.add(5, 10, 15));
        // Gọi phương thức add() với hai số thực
        System.out.println("Sum of 3.5 and 2.5: " + calc.add(3.5, 2.5));
    }
}

```

Kết quả:

Sum of 5 and 10: 15

Sum of 5, 10, and 15: 30

Sum of 3.5 and 2.5: 6.0

Trong ví dụ này, lớp **Calculator** có ba phương thức *add()* được nạp chồng, mỗi phương thức có số lượng tham số khác nhau, kiểu giá trị khác nhau. Khi gọi phương thức *add()* với các đối số tương ứng trong hàm main, Java sẽ quyết định phương thức được gọi dựa trên kiểu và số lượng tham số được truyền vào trong lời gọi, ví dụ: khi ta truyền vào 2 số nguyên thì phương thức *add(int, int)* sẽ được thực thi. Nếu ta truyền vào 2 số thực thì phương thức *add(double, double)* sẽ được thực thi.

7.1.2. Đa hình động

Đa hình động, còn được gọi là đa hình động, hay **runtime polymorphism**, hoặc **overriding polymorphism**.

Đa hình động mô tả khả năng truyền một thông điệp chung (triệu gọi một phương thức của đối tượng) đến nhiều đối tượng khác nhau (các đối tượng thuộc lớp con, chứa phương thức có tên trùng với tên phương thức trong thông điệp), mỗi đối tượng nhận được thông điệp này sẽ phản hồi lại theo cách riêng của nó (phương thức cùng tên được triệu trong các lớp con khác nhau sẽ có cách triển khai khác nhau).

Đa hình động thể hiện thông qua việc trong các lớp con ta có thể ghi đè một phương thức chung kế thừa từ lớp cha. Khi gọi phương thức này thông qua các đối tượng thuộc lớp con, mỗi phương thức sẽ xử lý theo cách riêng. Ví dụ: truyền thông điệp chung *vẽ()* đến đối tượng *hình tròn*, nó sẽ vẽ hình tròn; truyền thông điệp *vẽ()* đến đối tượng *hình vuông*, nó sẽ vẽ hình vuông, ...

Khái niệm ghi đè ta đã được học trong các chương trước đó. Bạn đọc cũng cần phân biệt rõ ghi đè và nạp chồng phương thức khác nhau ở những điểm nào.

Khi gọi phương thức trên một đối tượng, hệ thống sẽ xác định phương thức cụ thể được thực thi dựa trên loại của đối tượng tại thời điểm *runtime*.

Ví dụ 2: Đa hình động (viết đè)

Xét chương trình Java chứa các lớp sau:

- Lớp cơ sở ***Shape***
- Các lớp con: ***Circle***, ***Rectangle***, và ***Triangle*** kế thừa từ ***Shape***.

Các lớp con kế thừa hành vi/phương thức “*draw()*” của lớp cha, nhưng viết đè nó để tạo ra khả năng đa hình.

```
public class Shape {  
    public void draw() {  
        System.out.println("Drawing a shape...");  
    }  
}  
  
class Circle extends Shape {  
    @Override  
    public void draw() {  
        System.out.println("Drawing a circle...");  
    }  
}  
  
class Rectangle extends Shape {  
    @Override  
    public void draw() {  
        System.out.println("Drawing a rectangle...");  
    }  
}
```

```

class Triangle extends Shape {
    @Override
    public void draw() {
        System.out.println("Drawing a triangle...");
    }
}

public class Main{
    public static void main(String args[]){
        Circle o1= new Circle();
        Rectangle o2 =new Rectangle();
        Triangle o3=new Triangle();
        o1.draw();
        o2.draw();
        o3.draw();
    }
}

```

Kết quả:

```

Drawing a circle...
Drawing a rectangle...
Drawing a triangle...

```

Trong tình huống này, ta truyền thông điệp “draw” đến ba đối tượng *o1*, *o2*, *o3* thông qua ba lời gọi: *o1.draw()*; *o2.draw()*; *o3.draw()*. Kết quả trả về từ ba đối tượng này là khác nhau: *o1* trả về thông báo “Drawing a circle...”; *o2* trả về: “Drawing a rectangle ...”; *o3* trả về: “Drawing a traingle ...”.

Như vậy, đa hình runtime để xác định phương thức *draw()* được triệu gọi dựa trên loại của đối tượng nhận thông điệp. Ví dụ, khi chúng ta gọi phương thức *draw()* trên một đối tượng ***o1***, tương tự với phương thức *draw()* tương ứng với loại đối tượng ***Rectangle***, ***Triangle***.

7.1.3. Đặc điểm của đa hình

Đa hình trong lập trình hướng đối tượng có các đặc điểm sau:

- **Tính linh hoạt:** cho phép một đối tượng, cùng hành vi nhưng kết quả hành vi là khác nhau tùy thuộc vào ngữ cảnh cụ thể. Điều này tạo ra sự linh hoạt trong mã nguồn, cho mở rộng, điều chỉnh hành vi để thích ứng với các yêu cầu mới.
- **Tính tái sử dụng:** Thay vì phải viết lại mã nguồn hành vi/phương thức của lớp cha, đa hình cho phép kế thừa hành vi/phương thức trong lớp cha và ghi đè nó trong các lớp con để tạo ra các ứng xử khác nhau phụ của mỗi lớp con.

- **Tính trừu tượng:** Đa hình cho phép trừu tượng hóa hành vi/xử lý của các đối tượng từ tổng quát (khai báo trong lớp cho) đến chi tiết (viết đè trong lớp con).
- **Chia làm hai loại:** tĩnh và động.

7.2. LỚP TRỪU TƯỢNG

7.2.1. Khái niệm

Trong Java, lớp trừu tượng (abstract class) là lớp chứa ít nhất một phương thức trừu tượng. Phương thức trừu tượng là phương thức chưa được triển khai, nó chỉ có phần nguyên mẫu và có tính chất trừu tượng (abstract).

Mục đích chính của lớp trừu tượng là cung cấp một cấu trúc cơ sở cho các lớp con thông qua việc định nghĩa các phương thức mà các lớp con cần ghi đè (override).

Một số điểm chính của lớp trừu tượng:

- **Không thể tạo đối tượng:** Không thể tạo một đối tượng cụ thể từ một lớp trừu tượng bằng cách sử dụng toán tử new. Điều này làm cho lớp trừu tượng chỉ có thể được sử dụng thông qua kế thừa.
- **Chứa phương thức trừu tượng:** Một lớp trừu tượng cần chứa ít nhất một phương thức trừu tượng, tức là một phương thức mà chỉ có khai báo (declaration) mà không có cài đặt (implementation). Các lớp con kế thừa lớp trừu tượng cần triển khai (implement) các phương thức trừu tượng này.
- **Có thể chứa phương thức không trừu tượng:** Ngoài các phương thức trừu tượng, lớp trừu tượng cũng có thể chứa các phương thức không trừu tượng, có cài đặt nhưng không bắt buộc.

Ví dụ: xét chương trình java chứa khai báo lớp trừu tượng

```
// Lớp trừu tượng Animal
abstract class Animal {
    // Phương thức trừu tượng makeSound
    public abstract void makeSound();
    // Phương thức không trừu tượng sleep
    public void sleep() {
        System.out.println("Sleeping...");
    }
}

//Lớp con Dog kế thừa từ lớp Animal và triển khai phương thức make Sound
class Dog extends Animal {
    @Override
    public void makeSound() {
        System.out.println("Dog barks");
    }
}
```

```

    }
}
//Lớp con Cat kế thừa từ lớp Animal và triển khai phương thức makeSound
class Cat extends Animal {
    @Override
    public void makeSound() {
        System.out.println("Cat meows");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal dog = new Dog(); // Tạo một đối tượng Dog và gán cho một tham chiếu
        dog.makeSound(); // Gọi phương thức makeSound của đối tượng Dog
        dog.sleep(); // Gọi phương thức sleep của đối tượng Dog
        Animal cat = new Cat(); // Tạo một đối tượng Cat và gán cho một tham chiếu
        cat.makeSound(); // Gọi phương thức makeSound của đối tượng Cat
        cat.sleep(); // Gọi phương thức sleep của đối tượng Cat
    }
}

```

Kết quả:

```

Dog barks
Sleeping...
Cat meows
Sleeping...

```

Trong ví dụ trên, lớp **Animal** là một lớp trừu tượng chứa một phương thức trừu tượng *makeSound()* và một phương thức không trừu tượng *sleep()*. Cả hai lớp con **Dog** và **Cat** đều kế thừa từ lớp **Animal** và triển khai phương thức *makeSound()* tương ứng của chúng.

7.2.2. Lớp trừu tượng và lớp cụ thể

Dưới đây là sự khác biệt chính giữa lớp trừu tượng và lớp cụ thể trong chương trình Java:

- **Khả năng tạo đối tượng:**
 - + Lớp trừu tượng không thể tạo ra các đối tượng cụ thể (instances) từ nó. Điều này có nghĩa là không thể sử dụng từ khóa *new* để tạo một đối tượng của lớp trừu tượng.

- + Lớp cụ thể, hay còn gọi là lớp thường, có thể được sử dụng để tạo ra các đối tượng cụ thể bằng cách sử dụng từ khóa new.
- **Phương thức trừu tượng:**
 - + Lớp trừu tượng có thể chứa các phương thức trừu tượng (abstract methods), tức là các phương thức chỉ có khai báo (declaration) mà không có cài đặt (implementation). Các lớp con phải triển khai (implement) các phương thức trừu tượng này.
 - + Lớp cụ thể không thể chứa các phương thức trừu tượng. Tất cả các phương thức trong lớp cụ thể đều phải có cài đặt (implementation).
- **Tính linh hoạt:**
 - + Lớp trừu tượng cung cấp một cấu trúc cơ sở cho các lớp con và cho phép chúng kế thừa các thuộc tính và phương thức từ lớp trừu tượng. Điều này giúp tạo ra sự linh hoạt trong thiết kế và cho phép các lớp con thực hiện các chức năng cụ thể của chúng.
 - + Lớp cụ thể thường đại diện cho các đối tượng cụ thể và cung cấp các chi tiết cụ thể về cách một đối tượng hoạt động.
- **Sử dụng trong mã nguồn:**
 - + Lớp trừu tượng thường được sử dụng khi cần xác định một giao diện chung cho một nhóm các lớp con. Nó đóng vai trò như một bản thiết kế cho các lớp con.
 - + Lớp cụ thể được sử dụng khi cần tạo ra các đối tượng cụ thể để sử dụng trong ứng dụng.

7.2.3. Phương thức trừu tượng

Phương thức trừu tượng trong Java là một phương thức mà không có cài đặt (implementation) cụ thể trong lớp mà nó được khai báo, chỉ có khai báo (declaration) và không có cặp dấu ngoặc nhọn {}.

Mục đích của phương thức trừu tượng là để tạo ra một giao diện hành vi chung mà các lớp con phải triển khai (implement) theo cách riêng của chúng. Phương thức trừu tượng là phương tiện để triển khai đa hình (polymorphism) trong Java, cho phép các lớp con cung cấp cài đặt cụ thể của phương thức theo cách riêng của chúng.

Xét ví dụ về một hệ thống tài khoản ngân hàng trong Java. Trong hệ thống này, mỗi tài khoản có thể thực hiện các hoạt động như gửi tiền, rút tiền và kiểm tra số dư. Tuy nhiên, cách thức thực hiện các hoạt động này có thể khác nhau tùy thuộc vào loại tài khoản (tài khoản tiết kiệm, tài khoản thanh toán).

```
// Lớp trừu tượng Account định nghĩa các phương thức gửi tiền, rút tiền và kiểm tra số dư
abstract class Account {
    private double balance;
    public void deposit(double amount) {
        balance += amount;
    }
}
```

```

    public void withdraw(double amount) {
        if (balance >= amount) {
            balance -= amount;
        } else {
            System.out.println("Insufficient funds");
        }
    }

    public double getBalance() {
        return balance;
    }
    // Phương thức trừu tượng kiểm tra lãi suất (mỗi loại tài khoản có mức lãi suất khác
    nhau)
    public abstract double getInterestRate();
}
//Lớp con Saving Account triển khai phương thức trừu tượng getInterestRate
class SavingAccount extends Account {
    @Override
    public double getInterestRate() {
        return 0.05; // Lãi suất 5% cho tài khoản tiết kiệm
    }
}
//Lớp con Checking Account triển khai phương thức trừu tượng getInterestRate
class CheckingAccount extends Account {
    @Override
    public double getInterestRate() {
        return 0.01; // Lãi suất 1% cho tài khoản thanh toán
    }
}

public class Main {
    public static void main(String[] args) {
        Account savings = new SavingAccount();
        Account checking = new CheckingAccount();
    }
}

```

```

        System.out.println("Interest rate for saving account: " +
savings.getInterestRate());
        System.out.println("Interest rate for checking account:" +
checking.getInterestRate());
    }
}

```

Trong ví dụ trên, lớp trừu tượng **Account** định nghĩa một phương thức trừu tượng *getInterestRate()* để trả về lãi suất cho mỗi loại tài khoản. Các lớp con **SavingAccount** và **CheckingAccount** triển khai phương thức này theo cách thức riêng của chúng. Điều này cho phép chúng ta sử dụng đa hình để tính toán lãi suất cho mỗi loại tài khoản một cách linh hoạt.

7.2.4. Lớp Object

Trong Java, lớp **Object** là lớp cơ sở cho tất cả các lớp khác trong cây phân cấp các lớp của ngôn ngữ. Mọi lớp trong Java tự động kế thừa từ lớp **Object**, ngay cả khi không khai báo quan hệ kế thừa một cách rõ ràng. Điều này có nghĩa là mọi đối tượng trong Java đều là đối tượng của lớp **Object** và kế thừa tất cả các phương thức mà **Object** cung cấp.

Dưới đây là một số phương thức quan trọng mà lớp **Object** cung cấp:

- **toString():** Phương thức này trả về chuỗi biểu diễn đối tượng. Mặc định, nó trả về tên của lớp, theo sau bởi ký tự '@' và mã hex của đối tượng.
- **equals(Object obj):** Phương thức này so sánh hai đối tượng. Mặc định, nó so sánh các tham chiếu đến đối tượng chứ không phải nội dung của đối tượng.
- **hashCode():** Phương thức này trả về mã hash của đối tượng. Mã hash là một số nguyên duy nhất được gán cho mỗi đối tượng và được sử dụng trong các cấu trúc dữ liệu như bảng băm.
- **getClass():** Phương thức này trả về lớp của đối tượng.
- **clone():** Phương thức này tạo và trả về một bản sao của đối tượng.
- **finalize():** Phương thức này được gọi bởi bộ dọn rác Java (garbage collector) trước khi một đối tượng bị thu hồi bộ nhớ.

Dưới đây là một ví dụ đơn giản về việc sử dụng lớp Object trong Java:

```

public class Example {
    public static void main(String[] args) {
        Object obj1 = new Object();
        Object obj2= new Object();
        // Sử dụng phương thức toString()
        System.out.println("obj1: " + obj1.toString());
        // Sử dụng phương thức equals()
        System.out.println("obj1 equals obj2:" + obj1.equals(obj2));
        // Sử dụng phương thức hashCode()
    }
}

```

```

        System.out.println("Hash code of obj1: " + obj1.hashCode());
        // Sử dụng phương thức getClass()
        System.out.println("Class of obj1: " + obj1.getClass());
    }
}

```

Kết quả:

```

obj1: java.lang.Object@dcf3e99
obj1 equals obj2:false
Hash code of obj1: 231685785
Class of obj1: class java.lang.Object

```

Trong ví dụ này, chúng ta tạo một đối tượng là thể hiện của lớp **Object**, sau đó sử dụng các phương thức *toString()*, *equals()*, *hashCode()*, *getClass()* để làm việc với đối tượng này.

7.3. CASE STUDY

Trong ứng dụng quản lý người lao động, chúng ta sẽ sử dụng lớp trừu tượng và đa hình để quản lý các loại nhân viên khác nhau như nhân viên toàn thời gian, nhân viên bán thời gian và nhân viên hợp đồng.

Yêu cầu: Viết chương trình Java

- Khai báo lớp trừu tượng **Employee** với các thuộc tính chung như *id*, *name*, *salary*. Định nghĩa phương thức trừu tượng *calculatePay()* để tính lương cho nhân viên.
- Khai báo các lớp:
 - + Lớp **FullTimeEmployee**, **PartTimeEmployee**, và **ContractEmployee** là các lớp con của **Employee**.
 - + Mỗi lớp con sẽ triển khai phương thức *calculatePay()* theo cách riêng của nó, tùy thuộc vào loại nhân viên.
- Sử dụng đa hình:
 - + Trong ứng dụng, chúng ta có thể tạo một mảng các đối tượng **Employee**, mỗi đối tượng có thể là một loại nhân viên khác nhau.
 - + Bằng cách sử dụng đa hình, chúng ta có thể gọi phương thức *calculatePay()* trên mỗi đối tượng của mảng mà không cần quan tâm đến loại nhân viên cụ thể của đối tượng đó.
 - + Quan sát kết quả chạy.

7.4. VÍ DỤ ÁP DỤNG

Trong ứng dụng quản lý hình học, chúng ta sẽ sử dụng khái niệm đa hình và lớp trừu tượng để quản lý các loại hình học như hình tròn, hình vuông và hình tam giác.

- **Lớp trừu tượng Shape:**

- + Định nghĩa một lớp trừu tượng **Shape** với phương thức trừu tượng *calculateArea()* để tính diện tích của hình. Phương thức này phải là trừu tượng, do ta chưa biết đây là hình gì nên chưa thể triển khai việc tính diện tích của hình.
- + Mỗi lớp con của **Shape** sẽ triển khai phương thức *calculateArea()* theo cách riêng của nó.
- **Các lớp con cụ thể:**
 - + **Circle, Square,** và **Triangle** là các lớp con của **Shape**.
 - + Mỗi lớp con sẽ triển khai phương thức *calculateArea()* để tính diện tích của hình dạng tương ứng.
- **Sử dụng đa hình:**
 - + Trong ứng dụng, chúng ta có thể tạo một mảng các đối tượng **Shape**, mà mỗi đối tượng có thể là một hình tròn, hình vuông hoặc hình tam giác.
 - + Bằng cách sử dụng đa hình, chúng ta có thể gọi phương thức *calculateArea()* trên mỗi đối tượng mà không cần quan tâm đến loại hình học cụ thể của đối tượng đó.

Chương trình java tương ứng:

```
// Lớp trừu tượng Shape định nghĩa phương thức trừu tượng calculate Area()
abstract class Shape {
    public abstract double calculateArea();
}

//Lớp con Circle kế thừa từ lớp Shape và triển khai phương thức calculateArea()
class Circle extends Shape {
    private double radius;
    public Circle(double radius) {
        this.radius =radius;
    }
    @Override
    public double calculateArea() {
        return Math.PI*radius*radius; }
}

// Lớp con Rectangle kế thừa từ lớp Shape và triển khai phương thức calculateArea()
class Rectangle extends Shape {
    private double width;
    private double height;
    public Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }
}
```

```

    }
    @Override
    public double calculateArea() {
        return width*height;
    }
}

public class Example {
    public static void main(String[] args) {
        // Tạo một mảng các đối tượng Shape
        Shape[] shapes = new Shape[2];
        shapes[0] = new Circle(5); // Hình tròn có bán kính 5
        shapes[1] = new Rectangle(4, 6); // HCN có chiều dài 4, chiều rộng 6
        // Tính toán diện tích của từng hình
        for (Shape shape: shapes) {
            System.out.println("Area: " + shape.calculateArea());
        }
    }
}

```

Kết quả:

Area: 78.53981633974483

Area: 24.0

7.5. TỔNG KẾT CHƯƠNG

Chương này chúng ta đã tìm hiểu về:

- **Đa hình (Polymorphism):**
 - + Đa hình là một khái niệm quan trọng trong lập trình hướng đối tượng, cho phép một đối tượng có thể có nhiều hình dạng với cùng một hành vi.
 - + Có hai loại đa hình trong Java: đa hình tĩnh (compile-time polymorphism) và đa hình động (runtime polymorphism).
 - + Đa hình tĩnh xảy ra khi chúng ta sử dụng overloading, trong đó cho phép nhiều phương thức có cùng tên gọi trong phạm vi một lớp đối tượng.
 - + Đa hình động xảy ra khi chúng ta sử dụng overriding, trong đó một phương thức được triển khai lại trong các lớp con.
- **Lớp trừu tượng (Abstract Class):**
 - + Lớp trừu tượng là một lớp không có thể hiện.
 - + Lớp trừu tượng là lớp chứa một hoặc nhiều phương thức trừu tượng.
- **Lớp cụ thể (Concrete Class):**

- + Lớp cụ thể là một lớp mà có thể tạo ra các đối tượng cụ thể từ đó.
- + Lớp cụ thể triển khai (implements) tất cả các phương thức trừu tượng trong lớp trừu tượng mà nó kế thừa.
- **Phương thức trừu tượng (Abstract Method):**
 - + Phương thức trừu tượng là một phương thức không có cài đặt và chỉ có khai báo.
 - + Lớp chứa phương thức trừu tượng phải được đánh dấu là trừu tượng (abstract), và các lớp con phải triển khai tất cả các phương thức trừu tượng này.
- **Lớp Object:**
 - + Lớp Object là lớp cơ sở cho tất cả các lớp khác trong Java.
 - + Mọi lớp trong Java tự động kế thừa từ lớp Object, ngay cả khi không được khai báo một cách rõ ràng.
 - + Lớp Object cung cấp các phương thức như toString(), equals(), hashCode(), getClass() để làm việc với đối tượng.

Trên cơ sở các khái niệm trên, Java cho phép xây dựng các ứng dụng phức tạp và linh hoạt, sử dụng đa hình, lớp trừu tượng và các khái niệm khác để tạo ra mã dễ đọc, bảo trì và mở rộng.

7.6. CÂU HỎI TRẮC NGHIỆM

1. Lớp trừu tượng trong Java là gì?

- A. Lớp mà không thể tạo ra các đối tượng cụ thể từ nó.
- B. Lớp mà có thể tạo ra các đối tượng cụ thể từ nó.
- C. Lớp chứa mọi phương thức đều phải được cài đặt.
- D. Lớp không thể kế thừa từ các lớp khác.

2. Phương thức trừu tượng là gì?

- A. Phương thức không có cài đặt và chỉ có khai báo.
- B. Phương thức có cài đặt nhưng không có khai báo.
- C. Phương thức không có cài đặt và không có khai báo.
- D. Phương thức có cài đặt và có khai báo.

3. Lớp nào được sử dụng để triển khai đa hình trong Java?

- A. Lớp cụ thể
- B. Lớp trừu tượng
- C. Lớp Interface
- D. Tất cả đều đúng

4. Đa hình trong Java có bao nhiêu loại?

- A. Một loại
- B. Hai loại
- C. Ba loại
- D. Bốn loại

5. Trong Java, lớp Object là lớp nào?

- A. Lớp cơ sở cho tất cả các lớp khác.
- B. Lớp có cài đặt cho tất cả các phương thức.
- C. Lớp chỉ có một phương thức toString().
- D. Lớp chỉ chứa các thuộc tính và không có phương thức.

6. Trong Java, đa hình tĩnh xảy ra khi nào?

- A. Khi trong một lớp có nhiều phương thức cùng tên & khác nhau về danh sách đối số
- B. Khi một phương thức có một tên duy nhất.
- C. Khi một phương thức được triển khai lại trong lớp con.
- D. Khi một phương thức không có cài đặt.

7. Trong Java, đa hình động xảy ra khi nào?

- A. Khi trong một lớp có nhiều phương thức trùng cùng tên.
- B. Khi mỗi phương thức có một tên duy nhất.
- C. Khi một phương thức được triển khai lại trong lớp con.
- D. Khi một phương thức không có cài đặt.

8. Lớp nào không thể chứa phương thức trừu tượng?

- A. Lớp trừu tượng
- B. Lớp cụ thể
- C. Lớp đa hình
- D. Lớp interface

9. Đa hình động trong Java thường được thực hiện bằng cách nào?

- A. Overloading
- B. Overriding
- C. Abstraction
- D. Inheritance

10. Phương thức toString() trong lớp Object trả về điều gì?

- A. Mã hash của đối tượng
- B. Tên của lớp đối tượng
- C. Nội dung của đối tượng dưới dạng chuỗi
- D. Chữ ký của đối tượng

11. Đa hình biên còn có tên khác là gì trong Java?

- A. Compile-time polymorphism
- B. Runtime polymorphism
- C. Dynamic polymorphism

- D. Static polymorphism
- 12. Lớp trừu tượng có thể chứa phương thức không trừu tượng không?**
- A. Có
 - B. Không
- 13. Phương thức trừu tượng có cần khai báo với tính chất abstract không?**
- A. Có, nếu nó nằm trong lớp trừu tượng
 - B. Không, nếu nó nằm trong lớp trừu tượng
- 14. Đa hình có thể xảy ra dựa trên ... trong chương trình Java?**
- A. Tên phương thức
 - B. Số lượng tham số phương thức
 - C. Kiểu dữ liệu tham số phương thức
 - D. Tất cả các phương án trên
- 15. Lớp nào được sử dụng để tạo ra các đối tượng cụ thể trong Java?**
- A. Lớp trừu tượng
 - B. Lớp cụ thể
 - C. Lớp giao diện
 - D. Lớp abstract
- 16. Phương thức trừu tượng trong Java được khai báo bằng cách nào?**
- A. Sử dụng từ khóa "abstract" trước tên phương thức.
 - B. Sử dụng từ khóa "interface" trước tên phương thức
 - C. Sử dụng từ khóa "final" trước tên phương thức
 - D. Sử dụng từ khóa "virtual" trước tên phương thức.
- 17. Phương thức trừu tượng trong Java có thể chứa thân phương thức (implementations) không?**
- A. Có, phải có thân phương thức để biên dịch được.
 - B. Không, phương thức trừu tượng không có thân.
 - C. Chỉ nếu sử dụng từ khóa "virtual" trước tên phương thức là được.
 - D. Phương thức trừu tượng không được sử dụng trong Java.
- 18. Lớp nào có thể chứa phương thức trừu tượng?**
- A. Lớp cha (superclass)
 - B. Lớp con (subclass)
 - C. Lớp giao diện (interface)
 - D. Tất cả các lớp đều có thể chứa phương thức trừu tượng

19. Trong Java, khi lớp con kế thừa lớp cha có phương thức trừu tượng, lớp con phải làm gì?

- A. Không cần làm gì cả, lớp con tự động kế thừa phương thức trừu tượng từ lớp cha.
- B. Viết đè các phương thức trừu tượng trong lớp con
- C. Lớp con không thể kế thừa phương thức trừu tượng từ lớp cha.
- D. Sử dụng từ khóa "override" trước tên phương thức trừu tượng để kế thừa.

20. Phương thức trừu tượng trong Java thường được sử dụng để làm gì?

- A. Thực thi logic của phương thức trong một lớp cụ thể.
- B. Cung cấp một giao diện hành vi chung để các lớp con triển khai.
- C. Tạo ra các đối tượng mới từ các đối tượng hiện có.
- D. Phương thức trừu tượng không có mục đích cụ thể.

21. Đa hình (polymorphism) trong lập trình hướng đối tượng thường được thực hiện thông qua:

- A. Kế thừa
- B. Gói (package)
- C. Ghi đè (override)
- D. Mở rộng (extension)

22. Quan hệ giữa kế thừa và đa hình là gì?

- A. Đa hình cho phép các đối tượng của lớp con được xử lý như các đối tượng của lớp cha.
- B. Kế thừa là cách thức một lớp con có thể kế thừa các thuộc tính và phương thức từ một lớp cha.
- C. Đa hình là quá trình tạo ra các lớp con từ các lớp cha.
- D. Kế thừa và đa hình không có quan hệ với nhau.

23. Đa hình (polymorphism) có thể được triển khai như thế nào trong Java?

- A. Thông qua việc sử dụng từ khóa "extends" để kế thừa từ một lớp cha.
- B. Thông qua việc sử dụng các giao diện (interfaces) và triển khai các phương thức của chúng trong các lớp con.
- C. Chỉ qua việc sử dụng các phương thức static.
- D. Đa hình không thể được triển khai trong Java.

24. Trong đa hình tĩnh, việc xác định phương thức nào sẽ được gọi được quyết định tại ...

- A. Thời điểm biên dịch.
- B. Thời điểm chương trình chạy.
- C. Thời điểm triển khai.
- D. Thời điểm chương trình được viết.

25. Có thể khai báo phương thức trừu tượng trong lớp static không?

- A. Có, phương thức trừu tượng có thể khai báo trong lớp tĩnh.
- B. Không, phương thức trừu tượng không thể được khai báo trong lớp tĩnh.
- C. Chỉ khi phương thức trừu tượng được triển khai trong lớp tĩnh.
- D. Chỉ khi lớp tĩnh là lớp công khai.

26. Phương thức tĩnh (static) thường được sử dụng để làm gì?

- A. Tạo ra các đối tượng mới từ các đối tượng hiện có.
- B. Cung cấp một giao diện chung cho các lớp con để triển khai.
- C. Thực thi logic của phương thức trong một lớp cụ thể.
- D. Thực hiện các tác vụ liên quan đến lớp mà không cần tạo ra đối tượng của lớp đó.

27. Phương thức trừu tượng có thể được triển khai (implement) trong lớp nào?

- A. Chỉ có thể triển khai trong lớp trừu tượng.
- B. Chỉ có thể triển khai trong lớp không trừu tượng.
- C. Cả hai lựa chọn trên đều đúng.
- D. Không thể triển khai trong bất kỳ lớp nào.

28. Khi nào nên sử dụng phương thức trừu tượng thay vì phương thức tĩnh?

- A. Khi muốn có một phương thức không thể gọi được từ các đối tượng.
- B. Khi muốn định nghĩa một phương thức mà các lớp con phải triển khai.
- C. Khi muốn tạo ra một phương thức có thể gọi trực tiếp từ tên lớp.
- D. Khi muốn ghi đè (override) một phương thức từ lớp cha.

29. Phương thức nào sau đây trong lớp Object được sử dụng để so sánh hai đối tượng?

- A. toString()
- B. equals()
- C. hashCode()
- D. getClass()

30. Phương thức nào trong lớp Object được sử dụng để chuyển đổi đối tượng thành chuỗi và thường được gọi tự động khi in một đối tượng ra màn hình?

- A. toString()
- B. equals()
- C. hashCode()
- D. getClass()

7.7. BÀI TẬP THỰC HÀNH

Bài tập 1: Quản lý danh sách sản phẩm:

Yêu cầu: Tạo một lớp trừu tượng Product với các thuộc tính như id, name và price. Triển khai lớp con Book và Electronics kế thừa từ Product để quản lý thông tin của sách và sản phẩm điện tử.

- Đầu vào: Thông tin về sản phẩm bao gồm id, tên và giá.
- Đầu ra: Thông tin chi tiết về sản phẩm.

Bài tập 2: Quản lý danh sách xe:

Yêu cầu: Tạo một lớp trừu tượng Vehicle với các phương thức trừu tượng như start() và stop(). Triển khai lớp con Car và Motorbike kế thừa từ Vehicle để quản lý hoạt động của ô tô và xe máy.

- Đầu vào: None
- Đầu ra: Thông báo khi xe bắt đầu và kết thúc hoạt động.

Chương 8 GIAO DIỆN VÀ CÁC LỚP TRONG

Chương 8 giới thiệu và trình bày các vấn đề gồm:

- Giao diện (*interfaces*)
- Lớp trong (*inner classes*)

Sau khi học xong chương này, người học có thể:

- Hiểu được khái niệm giao diện và khai báo, làm việc được với giao diện (*interface*) trong chương trình Java.
- Hiểu, khai báo và sử dụng được các lớp lồng lớp trong chương trình Java và các tình huống áp dụng.

8.1. GIAO DIỆN (INTERFACES)

8.1.1. Định nghĩa

Để đảm bảo cấu trúc chương trình đơn giản (dạng cây phân cấp kế thừa). Java không hỗ trợ quan hệ kế thừa bội (tức kế thừa đa, nghĩa là một lớp con có thể kế thừa từ nhiều lớp cha). Nếu cần thiết phải áp dụng quan hệ kế thừa đa, ta có thể triển khai loại kế thừa này thông qua giao diện. Do vậy, giao diện có thể xem là một sự mở rộng quan hệ kế thừa trong Java.

Khi định nghĩa giao diện trong chương trình, java cho phép tạo một giao diện kế thừa từ nhiều giao diện. Một lớp đối tượng có thể triển khai một hoặc nhiều giao diện. Vậy giao diện là gì?

Giao diện tương tự như lớp đối tượng, tuy nhiên điểm khác biệt của nó là: (1) các thuộc tính của giao diện là các thông tin chung cho, cố định (~ thuộc tính hằng) cho mọi lớp con kế thừa & triển khai nó. Các thuộc tính này mặc định có tính chất: *public static final*; (2) các phương thức của giao diện là trừu tượng (chỉ có phần nguyên mẫu), những lớp kế thừa giao diện phải triển khai (implement) tất cả các phương thức này. Giao diện không chứa bất kỳ phần cài đặt cụ thể nào cho các phương thức của nó; nó chỉ định ra các phương thức mà một lớp triển khai phải có.

Cú pháp khai báo giao diện (interface) trong chương trình Java:

```
[phạm vi truy cập] interface <TenGiaoDien> [danh sách các giao diện cơ sở] {  
    [//khai báo các thuộc tính hằng; mặc định có tính chất public static final]  
    [// Khai báo các phương thức: mặc định có tính chất public abstract]  
}
```

Trong đó: những phần được đặt trong cặp dấu [] là tùy chọn.

Ví dụ 1: xét chương trình Java phối hợp giao diện, lớp trừu tượng và lớp cụ thể để cộng tác cùng làm việc với nhau.

```
interface PhuongTienGT {  
    String tenHang = "Jamaha";  
    String diaChi = "Số 98, Nguyễn Trãi, Hà nội";  
    String dienThoaiLienHe = "0986 865 543";  
}
```

```

    void diChuyen(int cheDo); // chế độ: tiến hoặc lùi
    void khoiDong(); // khởi động động cơ
    void keoCo(); //kéo cò
    void dung(); //dừng, đỗ
    void baoDuong(); // bảo dưỡng phương tiện
}

abstract class PhuongTienDuongBo {
    String banhXe[]; // số lượng bánh xe
    String dongCo; // động cơ
    abstract void tuanThuLuatGiaoThong(); // lái tự động cần tuân thủ luật giao thông
}

abstract class PhuongTienDuongThuy {
    String chieuDaiThan; // chiều dài thân phương tiện
    String dongCoThuyLuc; // động cơ thủy lực
    abstract void tuanThuTuyenDuong(); // Tuân thủ tuyến đường
    abstract void canhGiacMoiTruong(); // Cảnh giác với MT: thời tiết, sóng lớn,...
}

abstract class PhuongTienDuongKhong {
    String rotor; // cánh hoặc rotor để tạo sức nâng hoặc lực đẩy
    String dongCo; // động cơ đốt trong hoặc động cơ phản lực để tạo ra sức đẩy
    String banhXe[]; // số lượng bánh xe
    abstract void tuanThuLuatKhongPhan(); //tuân thủ luật ko phạm, hạn chế ko gian
    bay
    abstract void dieuKhien(); //Điều khiển an toàn khi lái, cất cánh, hạ cánh
}

class Oto extends PhuongTienDuongBo implements PhuongTienGT {
    Oto() {
        banhXe = new String[4];
        dongCo = "Động cơ ô tô Jamaha";
    }
    // 1. triển khai phương thức trừu tượng của lớp trừu tượng PhuongTienDuongBo
    void tuanThuLuatGiaoThong() {
        System.out.println(" Lái ô tô phải tuân thủ luật giao thông ");
    }
}

```

// 2. Triển khai tất cả các phương thức của giao diện *PhuongTienGiaoThong*

```
public void diChuyen(int x) {  
    System.out.println("Ô tô đang di chuyển");  
}  
public void keoCo() {  
    System.out.println("Ô tô đang kéo cò");  
}  
public void khaiDong() {  
    System.out.println("Bạn đang khởi động ô tô");  
}  
public void dung() {  
    System.out.println("Ô tô đang dừng/đỗ");  
}  
public void baoDuong() {  
    System.out.println("Ô tô đang được bảo dưỡng");  
}  
}  
public class Main {  
    public static void main(String arg[]) {  
        Oto o = new Oto();  
        o.khaiDong();  
        o.diChuyen(0);  
        o.keoCo();  
        o.dung();  
        o.baoDuong();  
    }  
}
```

Trong ví dụ này, lớp **Oto** kế thừa cả hai, gồm lớp trừu tượng **PhuongTienDuongBo** và giao diện **PhuongTienGiaoThong**. Nghĩa **Oto** đang triển khai loại hình kế thừa đa (bội).

=> Kết quả thực thi:

Bạn đang khởi động ô tô
Ô tô đang di chuyển
Ô tô đang kéo cò
Ô tô đang dừng/đỗ

Ô tô đang được bảo dưỡng

Lưu ý:

- Tất cả các thuộc tính của giao diện đều là hằng (mặc định có tính chất public static final, lưu trữ những thông tin chung, cố định cho mọi lớp dẫn xuất từ nó), đây là điểm khác biệt với các thuộc tính của lớp trừu tượng.
- Tất cả các phương thức của giao diện đều là trừu tượng (mặc định có tính chất public abstract, do đó khi viết đề phương thức này trong các lớp con, cần lưu ý tính chất public phải có ở phương thức viết đề). Đây là điểm khác biệt thứ 2 so với lớp trừu tượng (phương thức trừu tượng có thể có phạm vi truy cập khác *public*, không nhất thiết mọi phương thức trong lớp trừu tượng đều phải là trừu tượng).

8.1.2. Các tính chất của giao diện

Dưới đây là các tính chất quan trọng của giao diện trong Java:

- **Tính đa kế thừa:** Trong Java, một lớp có thể kế thừa và triển khai nhiều giao diện. Điều này cho phép một lớp nhận các đặc điểm từ nhiều nguồn khác nhau.
- **Tính linh hoạt:** Giao diện cung cấp một cách linh hoạt để tạo ra các lớp có khả năng tương tác với nhau theo chuẩn, mà không cần quan tâm đến cấu trúc nội bộ của lớp.
- **Triển khai bằng từ khóa "implements":** Để triển khai một giao diện trong Java, lớp phải sử dụng từ khóa "implements". Lớp triển khai phải cung cấp cài đặt cho tất cả các phương thức trừu tượng được định nghĩa trong giao diện.
- **Phương thức mặc định:** Từ Java 8 trở đi, giao diện cũng có thể chứa phương thức mặc định (default methods), những phương thức đã được triển khai (hành vi chung, cố định) và có thể được sử dụng trực tiếp bởi các lớp kế thừa giao diện mà không cần phải viết đè (override) chúng.
- **Tính trừu tượng hóa:** Sử dụng giao diện giúp tăng tính trừu tượng hóa và giảm thiểu sự phụ thuộc giữa các thành phần của ứng dụng.

Giao diện là một công cụ mạnh mẽ trong Java, được sử dụng rộng rãi trong việc thiết kế các ứng dụng lớn và phức tạp, đặc biệt là khi cần tính linh hoạt và tái sử dụng cao.

8.1.3. Sử dụng giao diện

Trong Java, giao diện (interface) là một cấu trúc quan trọng để định nghĩa các hành vi mà các lớp cụ thể có thể triển khai.

Dưới đây là các bước kèm ví dụ minh họa về việc sử dụng giao diện:

Ví dụ 2: xây dựng một ứng dụng quản lý các hình học cho phép tính diện tích và chu vi của các hình học khác nhau như hình tròn, hình vuông và hình chữ nhật.

Bước 1: Xác định các giao diện cần tạo trong ứng dụng, giả sử giao diện *Shape*:

// Giao diện Shape định nghĩa các hành vi chung của hình học

public interface Shape {

double calculateArea(); // Tính diện tích của hình


```
        double calculatePerimeter(); // Tính chu vi của hình
    }
```

Bước 2: Xác định và định nghĩa các lớp sẽ kế thừa và triển khai các giao diện đã tạo

Ở ví dụ này, với mỗi loại hình ta xác định được một lớp, cụ thể *Circle*, *Square*, *Rectangle*. Các lớp này kế thừa giao diện *Shape* để triển khai các hành vi cụ thể cho chúng

//Lớp Square triển khai giao diện Shape

```
class Square implements Shape {
    private double side;
    public Square(double side) {
        this.side = side;
    }
    @Override
    public double calculateArea() {
        return side * side;
    }
    @Override
    public double calculatePerimeter() {
        return 4 * side;
    }
}
```

//Lớp Rectangle triển khai giao diện Shape

```
class Rectangle implements Shape {
    private double length;
    private double width;
    public Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }
    @Override
    public double calculateArea() {
        return length * width;
    }
    @Override
    public double calculatePerimeter() {
```

```

        return 2 * (length + width);
    }
}

```

Bước 3: Xây dựng lớp chính chứa hàm main để thực thi chương trình

Ở bước cuối cùng này, chúng ta có thể sử dụng các đối tượng của các lớp ở bước 2 trong hàm main để tính diện tích và chu vi của từng hình học:

```

public class Main {
    public static void main(String[] args) {
        Shape circle = new Circle(5);
        Shape square = new Square(4);
        Shape rectangle = new Rectangle(4, 6);
        System.out.println("Area of Circle: " + circle.calculateArea());
        System.out.println("Perimeter of Circle: " + circle.calculatePerimeter());
        System.out.println("Area of Square: " + square.calculateArea());
        System.out.println("Perimeter of Square: " + square.calculatePerimeter());
        System.out.println("Area of Rectangle: " + rectangle.calculateArea());
        System.out.println("Perimeter of Rectangle: " +
rectangle.calculatePerimeter());
    }
}

```

Kết quả chạy: diện tích và chu vi của mỗi hình học được tính toán và hiển thị kết quả ra màn hình.

```

Area of Square: 16.0
Perimeter of Square: 16.0
Area of Rectangle: 24.0
Perimeter of Rectangle: 20.0

```

Lưu ý:

Giao diện (interface) và lớp trừu tượng (abstract class) đều là các cơ chế quan trọng trong lập trình hướng đối tượng trong Java, nhưng chúng khác nhau về bản chất, về cách thức triển khai và các tình huống áp dụng:

- Giao diện cho phép các lớp khác nhau về bản chất cùng kế thừa và triển khai các phương thức chung đã khai báo trong giao diện.

Ví dụ 3: lớp *Chim*, lớp *Máy bay* khác nhau về bản chất (một là động vật, một là phương tiện máy móc) nhưng đều có hành vi *bay*. Do đó các lớp này đều có thể kế thừa và triển khai giao diện **Hành vi** chứa phương thức *bay*.

- Lớp trừu tượng cho phép các lớp cùng bản chất (cùng loài, cùng loại) kế thừa và triển khai các phương thức trừu tượng.

Ví dụ 4: lớp trừu tượng *HinhHoc* cho phép các lớp con có cùng bản chất là các hình học khác nhau như hình đa giác, tứ giác, hình tròn, hình vuông, .. kế thừa và triển khai lớp trừu tượng *HinhHoc* chứa phương thức trừu tượng *veHinh()*.

8.2. LỚP TRONG (INNER CLASSES)

8.2.1. Định nghĩa

Trong Java, lớp nội (nested class) là một lớp được định nghĩa bên trong một lớp khác. Lớp nội được sử dụng để nhóm các lớp lại với nhau nhằm tăng cường tính tổ chức và tính bảo mật của mã nguồn. Nó cũng có thể được sử dụng để cung cấp một cơ chế đóng gói cho mã, giúp giảm thiểu sự truy cập từ bên ngoài.

8.2.2. Tính chất của lớp nội

Các tính chất của lớp nội (nested class) trong Java bao gồm:

- **Tính đóng gói (Encapsulation):** Lớp nội có thể được sử dụng để nhóm các hành vi, dữ liệu lại với nhau, giữ cho mã được tổ chức và bảo mật hơn, giảm thiểu sự truy cập từ bên ngoài.
- **Tính bảo mật (Security):** Lớp nội cung cấp một cấu trúc bảo mật bằng cách ẩn thông tin của lớp nội khỏi các truy cập từ bên ngoài lớp chứa.
- **Tính đồng nhất (Consistency):** Lớp nội có thể được truy cập và sử dụng bên trong lớp chứa nó.
- **Tính phân tách (Separation):** Lớp nội có thể giúp tách biệt các thành phần của ứng dụng, làm cho mã trở nên dễ đọc và dễ bảo trì.
- **Tính tự nhiên (Naturalness):** Sử dụng lớp nội có thể làm cho mã trở nên tự nhiên hơn trong việc phát triển ứng dụng, đặc biệt là khi các lớp chỉ có thể được sử dụng trong phạm vi nhất định (biểu diễn quan hệ composition giữa các lớp).
- **Tính sạch sẽ (Cleanliness):** Khi sử dụng lớp nội một cách hợp lý, mã có thể trở nên sạch sẽ hơn và ít phức tạp hơn, do đó dễ dàng bảo trì và mở rộng.

Lưu ý: Mặc dù lớp nội có nhiều ưu điểm, nhưng sử dụng chúng một cách cẩn thận để tránh tình trạng mã trở nên phức tạp và khó hiểu.

8.2.3. Phân loại lớp nội

Trong Java, lớp nội (nested class) được chia thành hai loại: lớp nội tĩnh (static nested class) và lớp nội phi tĩnh (non-static nested class, inner class).

★ Lớp nội tĩnh (Static Nested Class)

Lớp nội tĩnh là một lớp có tính chất *static* được khai báo bên trong một lớp. Với tính chất này, lớp nội sẽ không được phép truy cập các thành phần không là tĩnh của lớp chứa (lớp bao bọc, lớp ngoài). Tuy nhiên nó có thể truy cập các thành phần tĩnh của lớp chứa.

Lớp nội tĩnh có thể được sử dụng khi ta cần nhóm các tác vụ lại với nhau, các tác vụ mà không cần truy cập các thành phần không tĩnh của lớp bao bọc. Do đó, nó thích hợp để gom các tác vụ độc lập không phụ thuộc vào trạng thái của lớp bao bọc.

Cú pháp khai báo:

```
public class OuterClass {  
  
    static class StaticNestedClass {  
        // ...  
    }  
}
```

Ví dụ 1: sử dụng lớp nội tĩnh trong Java để gom các tác vụ bảo mật, độc lập với trạng thái của lớp chứa.

```
public class SecurityManager {  
    public static void main(String[] args) {  
        String username = "exampleUser";  
        String password = "examplePassword";  
        // Kiểm tra tính hợp lệ của tên người dùng và mật khẩu  
        if (SecurityValidator.isValidUsername(username) &&  
            SecurityValidator.isValidPassword(password)) {  
            System.out.println("Đăng nhập thành công.");  
        } else {  
            System.out.println("Đăng nhập thất bại.");  
        }  
    }  
    static class SecurityValidator {  
        public static boolean isValidUsername(String username) {  
            // Kiểm tra tính hợp lệ của tên người dùng  
            return username.matches("[a-zA-Z0-9]+");  
        }  
        public static boolean isValidPassword(String password) {  
            // Kiểm tra tính hợp lệ của mật khẩu  
            return password.length() >= 8;  
        }  
    }  
}
```

Trong ví dụ này, lớp **SecurityManager** sử dụng lớp nội tĩnh **SecurityValidator** để kiểm tra tính hợp lệ của tên người dùng và mật khẩu. Cả hai phương thức `isValidUsername()` và `isValidPassword()` trong **SecurityValidator** là các tác vụ bảo mật, độc lập và không phụ thuộc vào trạng thái của lớp **SecurityManager**.

Khi chạy **SecurityManager.main()**, nó sẽ kiểm tra tính hợp lệ của tên người dùng và mật khẩu mà không cần tạo ra một đối tượng của lớp **SecurityValidator**.

★ Lớp nội phi tĩnh (Inner Class)

Lớp nội phi tĩnh là một lớp không có tính chất *static* được khai báo bên trong một lớp khác. Với đặc điểm này, lớp nội phi tĩnh có thể truy cập trực tiếp tất cả các thành phần (thuộc tính, phương thức) của lớp bao bọc mà không phải chỉ rõ tên của lớp bao bọc.

Lớp nội phi tĩnh có tính chất bao bọc, nghĩa là nó thích hợp cho việc gom các tác vụ có liên quan chặt chẽ với lớp chứa để tạo ra một cấu trúc được bảo vệ và ẩn thông tin. Các tác vụ trong lớp nội phụ thuộc vào trạng thái của lớp bao bọc nên có thể truy cập tất cả các trường và phương thức của lớp bao bọc, bao gồm các trường và phương thức *private*, *protected* và *public*.

Cú pháp khai báo:

```
public class OuterClass {  
    class InnerClass { }  
    // ...  
}
```

Ví dụ 2: Dưới đây là một ví dụ về cách sử dụng lớp nội phi tĩnh để tăng tính linh hoạt và tái sử dụng mã trong Java:

```
// Lớp chứa các phương thức tính toán  
class Calculator {  
    // Phương thức tính tổng  
    static int add(int a, int b) {  
        return a + b;  
    }  
    // Phương thức tính hiệu  
    static int subtract(int a, int b) {  
        return a - b;  
    }  
    // Phương thức tính tích  
    static int multiply(int a, int b) {  
        return a * b;  
    }  
}
```

```

// Lớp nội tĩnh chứa các phương thức tính toán mở rộng
static class AdvancedCalculator {
    // Phương thức tính lũy thừa
    static double power(double base, double exponent) {
        return Math.pow(base, exponent);
    }
    // Phương thức tính căn bậc hai
    static double squareRoot(double number) {
        return Math.sqrt(number);
    }
}

public class Main {
    public static void main(String[] args) {
        int result1 = Calculator.add(5, 3);
        System.out.println("5 + 3 = " + result1);
        int result2 = Calculator.subtract(10, 4);
        System.out.println("10 - 4 = " + result2);
        int result3 = Calculator.multiply(6, 2);
        System.out.println("6 * 2 = " + result3);
        double result4 = Calculator.AdvancedCalculator.power(2, 3);
        System.out.println("2 ^ 3 = " + result4);
        double result5 = Calculator.AdvancedCalculator.squareRoot(25);
        System.out.println("√25 = " + result5);
    }
}

```

Trong ví dụ này, lớp **Calculator** chứa các phương thức tính toán cơ bản như cộng, trừ, nhân. Lớp nội tĩnh **AdvancedCalculator** chứa các phương thức tính toán mở rộng như lũy thừa và căn bậc hai.

Khi chạy hàm *main*, chúng ta có thể sử dụng cả phương thức của **Calculator** và **AdvancedCalculator**. Điều này giúp tăng tính linh hoạt và tái sử dụng của mã. Đồng thời, việc định nghĩa **AdvancedCalculator** là một lớp nội tĩnh giúp làm cho cấu trúc mã trở nên gọn gàng hơn và không cần phải tạo ra một lớp mới. Kết quả khi thực hiện chương trình:

$$10 - 4 = 6$$

$$6 * 2 = 12$$

$$2^3 = 8.0$$

$$\sqrt{25} = 5.0$$

Lưu ý:

- Cả hai loại lớp nội đều có thể truy cập các thành phần (fields và methods) của lớp bao bọc, nhưng cách truy cập có thể khác nhau.
- Lớp nội phi tĩnh thường được sử dụng nhiều hơn và cung cấp tính linh hoạt cao hơn trong việc truy cập và sử dụng các thành phần của lớp bao bọc.

8.3. CASE STUDY

Trong ứng dụng quản lý trường học, chúng ta cần quản lý thông tin về học sinh và giáo viên. Mỗi học sinh có tên, tuổi và lớp học, trong khi mỗi giáo viên có tên, tuổi và môn học giảng dạy.

Yêu cầu: sử dụng giao diện và lớp nội để quản lý thông tin một cách hiệu quả:

- Sử dụng giao diện **Person** để định nghĩa các phương thức cơ bản như **getName()** và **getAge()**.
- Triển khai hai lớp con **Student** và **Teacher** từ giao diện **Person**, và thêm các thuộc tính và phương thức cụ thể cho mỗi loại. Sử dụng lớp nội để triển khai các lớp con này trong lớp chứa **SchoolManager**.

=> Chương trình java mẫu:

```
// Giao diện Person
interface Person {
    String getName();
    int getAge();
}

// Lớp nội
class SchoolManager {
    // Lớp nội tĩnh để quản lý học sinh
    static class Student implements Person {
        private String name;
        private int age;
        private String className;
        // Constructors, getters, and setters
        public int getAge() {
            return 10;
        }
        public String getName(){
            return this.name;
        }
    }
}
```

```

        Student(String ten, int tuoi, String tenLop){
            name=ten;
            age=tuoi;
            className=tenLop;
        }
        String getClassName(){
            return className;
        }
    }

    // Lớp nội phi tĩnh để quản lý giáo viên
    class Teacher implements Person {
        private String name;
        private int age;
        private String subject;
        // Constructors, getters, and setters
        Teacher(String ten, int tuoi, String monHoc){
            name=ten;
            age=tuoi;
            subject=monHoc;
        }
        public int getAge() {
            return 10;
        }
        public String getName(){
            return this.name;
        }
        public String getSubject(){
            return subject;
        }
    }
}

// Sử dụng
public class Main {
    public static void main(String[] args) {

```



```

        SchoolManager.Student student = new SchoolManager.Student("John", 15,
"10A");

        SchoolManager.Teacher teacher = new SchoolManager().new Teacher("Ms.
Smith", 35, "Math");

        System.out.println(student.getName() + " is in class " +
student.getClassName());

        System.out.println(teacher.getName() + " teaches " + teacher.getSubject());
    }
}

```

8.4. VÍ DỤ ÁP DỤNG

Ví dụ 1: Xét ví dụ về việc sử dụng giao diện trong Java để cho phép các lớp khác nhau về bản chất kế thừa và triển khai cùng một hành vi. Trong ví dụ này, chúng ta sẽ sử dụng giao diện **Animal** để định nghĩa hành vi **move()**, và triển khai nó cho hai loài động vật khác nhau: chim và cá.

```

// Giao diện Animal
interface Animal {
    void move();
}

// Lớp Bird triển khai giao diện Animal
class Bird implements Animal {
    @Override
    public void move() {
        System.out.println("Bird is flying.");
    }
}

// Lớp Fish triển khai giao diện Animal
class Fish implements Animal {
    @Override
    public void move() {
        System.out.println("Fish is swimming.");
    }
}

// Lớp Main sử dụng các đối tượng của Bird và Fish
public class Main {
    public static void main(String[] args) {
        Animal bird = new Bird();

```

```

        Animal fish = new Fish();
        bird.move(); // Output: Bird is flying.
        fish.move(); // Output: Fish is swimming.
    }
}

```

Trong ví dụ này, *Bird* và *Fish* là hai loài khác về môi trường sống, nhưng chúng đều có cùng hành vi là bay di chuyển, *move()*, được định nghĩa bởi giao diện *Animal*.

Ví dụ 2: Xét ví dụ minh họa về việc sử dụng lớp nội trong Java để triển khai một chức năng quản lý người dùng:

```

// Lớp UserManagement chứa danh sách người dùng và các phương thức quản lý
import java.util.ArrayList;
import java.util.List;
public class UserManagement {
    private List<User> users;
    public UserManagement() {
        users = new ArrayList<>();
    }
    // thêm người dùng vào danh sách
    public void addUser(String username, String email, String pass) {
        User user = new User(username, email, pass);
        users.add(user);
        System.out.println("User added: " + username);
    }
    //Lớp nội User để định nghĩa một người dùng
    private class User {
        private String username;
        private String email;
        private String password;
        public User(String username, String email, String password) {
            this.username = username;
            this.email = email;
            this.password = password;
        }
        public String getUsername() {

```

```

        return username;
    }
    public String getEmail() {
        return email;
    }
    public String getPassword() {
        return password;
    }
}

// Phương thức để hiển thị tất cả người dùng
public void displayUsers() {
    System.out.println("List of Users:");
    for (User user : users) {
        System.out.println("Username:" + user.getUsername() + ", Email: " +
user.getEmail());
    }
}

// Main method để kiểm tra
public static void main(String[] args) {
    UserManagement manager = new UserManagement();
    manager.addUser("John", "john@example.com", "123456");
    manager.addUser("Alice", "alice@example.com", "654321");
    manager.displayUsers();
}
}

```

Trong ví dụ này, lớp *UserManagement* chứa một danh sách các người dùng và cung cấp các phương thức để quản lý chúng, bao gồm *addUser()* để thêm người dùng mới; *displayUsers()* để hiển thị danh sách người dùng. Lớp nội *User* được sử dụng để định nghĩa một người dùng với các thuộc tính như *name*, *email* và *password*. Khi phương thức *addUser()* được gọi, một đối tượng *User* mới được tạo và thêm vào danh sách người dùng. Khi phương thức *displayUsers()* được gọi, thông tin của tất cả người dùng được hiển thị ra màn hình.

8.5. TỔNG KẾT CHƯƠNG

Trong chương này, chúng ta đã học về:

1) Giao Diện (Interface):

Giao diện là một khối xác định các phương thức mà một lớp có thể triển khai. Nó chỉ định hành vi mà một đối tượng có thể thực hiện mà không cung cấp các chi tiết về cách nó được thực hiện.

Đặc Điểm:

- Không thể khởi tạo đối tượng từ một giao diện.
- Các lớp triển khai giao diện phải triển khai tất cả các phương thức được định nghĩa trong giao diện.
- Giao diện có thể mở rộng từ một hoặc nhiều giao diện khác.

Tính Chất:

- Linh hoạt: Cho phép các lớp không liên quan triển khai cùng một giao diện.
- Dễ mở rộng: Cho phép thêm các tính năng mới mà không cần thay đổi các lớp hiện có.
- Tách biệt: Tách biệt hành vi trừu tượng và các chi tiết triển khai cụ thể..

2) Lớp Nội (Inner Class):

Lớp nội là một lớp được định nghĩa bên trong một lớp khác. Nó có thể được sử dụng để tăng tính đóng gói và làm cho code dễ đọc và dễ bảo trì.

Phân Loại:

- Lớp nội phi tĩnh (Non-static Inner Class): Có quyền truy cập vào tất cả các thành phần của lớp chứa nó và có thể truy cập đến cả biến phi tĩnh.
- Lớp nội tĩnh (Static Inner Class): Không có quyền truy cập đến các thành phần phi tĩnh của lớp chứa nó, nó không thể tham chiếu trực tiếp đến các biến phi tĩnh hoặc phương thức của lớp chứa nó.

Tính Chất:

- Tăng tính đóng gói: Cho phép lớp nội truy cập các thành viên của lớp bên ngoài mà không cần phải tiết lộ chúng.
- Tạo cấu trúc logic: Các lớp nội có thể được sử dụng để tạo ra các cấu trúc logic phức tạp mà không làm rối code.

Như vậy, giao diện và lớp nội là hai khái niệm quan trọng trong Java giúp tạo ra các cấu trúc code linh hoạt và dễ bảo trì. Sử dụng chúng một cách đúng đắn có thể giúp cải thiện hiệu suất và tăng tính bảo mật của ứng dụng.

8.6. CÂU HỎI TRẮC NGHIỆM

1. Giao diện (interface) trong Java là gì?

- A. Một lớp cụ thể.
- B. Một khối mã chứa các biến và phương thức.
- C. Một tập hợp các hằng số.
- D. Một khối xác định các thuộc tính hằng và các phương thức mà một lớp có thể triển khai.

2. Lớp nội (inner class) trong Java là gì?

- A. Một lớp được định nghĩa bên trong một hàm.
- B. Một lớp được định nghĩa bên trong một giao diện.
- C. Một lớp được định nghĩa bên trong một lớp khác.
- D. Một lớp không thể được định nghĩa bên trong một lớp khác.

3. Lớp nội tĩnh (static inner class) trong Java có điều gì đặc biệt?

- A. Nó có thể truy cập các thành phần static của lớp bên ngoài.
- B. Nó không thể được khởi tạo ở lớp bên ngoài.
- C. Nó có thể truy cập vào các biến phi tĩnh của lớp bên ngoài.
- D. Nó có thể truy cập vào các thuộc tính và phương thức phi tĩnh của lớp bên ngoài.

4. Giao diện trong Java có thể kế thừa từ bao nhiêu giao diện khác?

- A. Chỉ một.
- B. Bắt buộc từ hai giao diện khác trở lên.
- C. Không thể kế thừa giao diện.
- D. Không có giới hạn.

5. Cách khởi tạo một đối tượng từ lớp nội trong Java?

- A. `Outer.Inner innerObject = outerObject.new Inner();`
- B. `Inner innerObject = new Inner();`
- C. `Inner innerObject = new Outer.Inner();`
- D. `Outer outerObject = new Outer(); Inner innerObject = outerObject.createInner();`

5. Đâu không phải là đặc điểm của giao diện trong Java?

- A. Giao diện không thể chứa các trường (fields).
- B. Một lớp có thể triển khai nhiều giao diện.
- C. Giao diện không thể khởi tạo đối tượng.
- D. Giao diện không được phép kế thừa từ giao diện khác.

7. Lớp nội tĩnh được sử dụng trong Java khi nào?

- A. Khi bạn muốn truy cập các thành phần tĩnh của lớp bên ngoài.
- B. Khi bạn muốn tạo nhiều đối tượng của lớp nội mà không cần tạo đối tượng của lớp bên ngoài.
- C. Khi bạn muốn tạo ra một hàm riêng lẻ.
- D. Khi bạn muốn triển khai một giao diện.

8. Điều nào sau đây là đúng về quyền truy cập của lớp nội phi tĩnh trong Java?

- A. Nó có thể truy cập vào tất cả các thành viên của lớp bên ngoài, bao gồm cả các thành viên private.
- B. Nó chỉ có thể truy cập vào các thành viên public của lớp bên ngoài.

- C. Nó không thể truy cập vào các thành viên của lớp bên ngoài.
- D. Nó chỉ có thể truy cập vào các thành phần static của lớp bên ngoài.

9. Trong Java, tại sao nên sử dụng kế thừa giao diện thay vì kế thừa từ lớp cha?

- A. Để tránh xung đột giữa các lớp.
- B. Để tạo ra các lớp kế thừa đa năng.
- C. Để cho phép một lớp triển khai nhiều giao diện.
- D. Tất cả các phương án trên.

10. Điểm khác biệt chính giữa giao diện và lớp trừu tượng trong Java là gì?

- A. Giao diện không thể chứa các phương thức đã được triển khai, trong khi lớp trừu tượng có thể.
- B. Lớp trừu tượng không thể chứa các biến instance, trong khi giao diện có thể.
- C. Giao diện không thể được kế thừa, trong khi lớp trừu tượng có thể.
- D. Lớp trừu tượng không thể có các phương thức abstract, trong khi giao diện có thể.

11. Điểm giống nhau giữa giao diện và lớp trừu tượng trong Java là gì?

- A. Lớp trừu tượng và giao diện đều có thể kế thừa nhiều lớp.
- B. Tất cả các phương thức trong lớp trừu tượng và trong giao diện đều là trừu tượng.
- C. Tất cả các thuộc trong giao diện và trong lớp trừu tượng đều là abstract
- D. Lớp trừu tượng và giao diện đều chứa phương thức tượng.

12. Phát biểu nào sau đây là đúng trong Java?

- A. Giao diện có thể chứa các trường (fields), trong khi lớp trừu tượng không thể.
- B. Lớp trừu tượng có thể được khởi tạo bằng từ khóa new, trong khi giao diện không thể.
- C. Một lớp có thể triển khai nhiều giao diện, nhưng chỉ có thể kế thừa một lớp trừu tượng.
- D. Giao diện có thể chứa các phương thức static, trong khi lớp trừu tượng không thể.

13. Trong Java, lớp trừu tượng có thể được định nghĩa bằng từ khóa nào?

- A. abstract
- B. interface
- C. final
- D. extends

14. Giao diện có thể chứa các thành phần nào sau đây?

- A. Các thuộc tính hằng.
- B. Chỉ phương thức final.
- C. Các lớp cụ thể.
- D. Không có biến nào, chỉ các phương thức final.

15. Trong Java, khi một lớp triển khai một giao diện, nó chỉ cần triển khai một vài phương thức trong giao diện đó.

- A. Đúng
- B. Sai

16. Một giao diện có thể kế thừa từ một hoặc nhiều giao diện khác trong Java. Phát biểu này là:

- A. Đúng, Java cho phép kế thừa đa giao diện.
- B. Sai, giao diện không thể kế thừa từ giao diện khác.
- C. Chỉ đúng khi tất cả các giao diện được kế thừa là trừu tượng.
- D. Chỉ đúng khi tất cả các giao diện được kế thừa là không trừu tượng.

17. Một lớp triển khai các phương thức của giao diện phải làm gì?

- A. Phải bổ sung từ khóa abstract cho phương thức của giao diện đó.
- B. Chỉ cần triển khai một số phương thức cần thiết của giao diện đó.
- C. Không cần phải triển khai phương thức nào cả.
- D. Một lớp không thể triển khai giao diện.

18. Trong Java, quan hệ giữa lớp triển khai giao diện và giao diện là quan hệ gì?

- A. Kế thừa và triển khai
- B. Kế thừa đơn thuần
- C. Hợp thành.
- D. Liên kết.

19. Thuộc tính trong giao diện có thể được ghi đè (override) trong lớp triển khai không?

- A. Có, thuộc tính trong giao diện có thể được ghi đè trong lớp triển khai.
- B. Không, thuộc tính trong giao diện không thể được ghi đè trong lớp triển khai.
- C. Chỉ khi thuộc tính được khai báo là static thì mới có thể được ghi đè.
- D. Chỉ khi thuộc tính được khai báo là final thì mới có thể được ghi đè.

20. Thuộc tính trong giao diện có tính chất mặc định là gì?

- A. static.
- B. final.
- C. public static final.
- D. không có tính chất mặc định.

21. Phương thức trong giao diện (interface) có tính chất mặc định nào sau đây?

- A. Tự động triển khai (implement) trong lớp con.
- B. public abstract và không có cơ thể (body).
- C. Tự động ghi đè (override) trong lớp triển khai.

D. Phải có giá trị trả về.

22. Phạm vi truy cập mặc định của phương thức trong giao diện là gì?

- A. private
- B. protected
- C. public
- D. default (package-private)

23. Trong giao diện, có thể khai báo phương thức có phần thân (body) không?

- A. Có, phương thức trong giao diện có thể có phần thân.
- B. Không, phương thức trong giao diện không thể có phần thân.
- C. Chỉ khi phương thức được khai báo là static thì mới được phép có phần thân.
- D. Chỉ khi phương thức được khai báo là final thì mới được phép có phần thân.

24. Lớp triển khai giao diện có thể thừa kế (extends) một lớp khác không?

- A. Có, lớp triển khai giao diện có thể thừa kế một lớp khác.
- B. Không, lớp triển khai giao diện không thể thừa kế một lớp khác.
- C. Chỉ nếu lớp triển khai giao diện không chứa phương thức trừu tượng thì mới có thể thừa kế.
- D. Chỉ nếu lớp triển khai giao diện là một lớp trừu tượng thì mới có thể thừa kế.

25. Một lớp có thể triển khai (implement) nhiều giao diện không?

- A. Có, một lớp có thể triển khai nhiều giao diện.
- B. Không, một lớp chỉ có thể triển khai một giao diện.
- C. Chỉ khi các giao diện được kế thừa từ cùng một giao diện cha thì mới có thể triển khai nhiều giao diện.
- D. Chỉ khi mỗi giao diện được triển khai trong một lớp con riêng biệt thì mới có thể triển khai nhiều giao diện.

26. Cú pháp khai báo giao diện trong Java là gì?

- A. interface MyInterface { }
- B. class MyInterface { }
- C. abstract MyInterface { }
- D. interface class MyInterface { }

27. Giao diện có phải là sự mở rộng quan hệ kế thừa trong Java không?

- A. Có
- B. Không.

28. Từ khóa nào được dùng để khai báo một lớp triển khai giao diện trong chương trình Java?

- A. extends

- B. implements
- C. abstract
- D. interface

29. Quan hệ kế thừa và mở rộng được hiểu là?

- A. Dùng lại những thành phần private của lớp cho và bổ sung thêm các thành phần mới.
- B. Dùng lại những thành phần không private của lớp cho và bổ sung thêm các thành phần mới trong lớp con.
- C. Mở rộng phạm vi sử dụng của đối tượng.
- D. cả A và B

30. Lớp nào trong Java được tất cả các lớp khác kế thừa theo mặc định?

- A. Lớp Abstract
- B. Lớp Interface
- C. Lớp Object
- D. Lớp Main

8.7. BÀI TẬP THỰC HÀNH

Bài tập 1: Giao Diện và Lớp Nội

Viết chương trình Java khai báo giao diện có tên là Drawable với phương thức draw() và một lớp nội tĩnh trong lớp chứa giao diện Drawable có tên là Square. Trong lớp Square, triển khai phương thức draw() để in ra xâu "Vẽ hình vuông".

Bài tập 2: Sử Dụng Giao Diện trong Mảng

Viết chương trình Java chứa một mảng các đối tượng có kiểu là giao diện Drawable, bao gồm các đối tượng là hình tròn và hình chữ nhật. Duyệt qua mảng và gọi phương thức draw() cho từng đối tượng.

Bài tập 3: Triển Khai Giao Diện

Viết chương trình Java xây dựng lớp đối tượng triển khai giao diện Comparable. Sử dụng phương thức compareTo() để so sánh hai đối tượng và in ra kết quả.

Bài tập 4: Sử dụng lớp nội trong phương thức

Viết chương trình java khai báo một phương thức trong một lớp bất kỳ và tạo một lớp nội tĩnh bên trong phương thức đó. Trong lớp nội tĩnh, viết một phương thức để in ra thông điệp: *"Hello, I am demo inner classes"*

Bài tập 5: Giao diện & lớp nội kết hợp làm việc cùng nhau

Viết chương trình Java tạo một giao diện có tên là Operation với phương thức calculate() và một lớp nội trong lớp chứa giao diện Operation có tên là Addition. Trong lớp Addition, triển khai phương thức calculate() để thực hiện phép cộng.

Chương 9 MỘT SỐ CHỦ ĐỀ NÂNG CAO

Chương 9 giới thiệu và trình bày các vấn đề gồm:

- Một số cấu trúc dữ liệu đặc biệt trong Java
- Các luồng vào/ra dữ liệu trong Java
- Tổ chức mã nguồn theo mô hình hướng đối tượng

Sau khi học xong chương này, người học có thể:

- Hiểu được hiểu và làm việc được với một số cấu trúc dữ liệu đặc biệt trong Java như: *ArrayList*, *LinkedList*, *HashMap*, ...
- Hiểu và sử dụng được các luồng vào/ra dữ liệu trong Java và các tình huống áp dụng.
- Hiểu và vận dụng được cách tổ chức mã nguồn theo mô hình hướng đối tượng

9.1. MỘT SỐ CẤU TRÚC DỮ LIỆU TRONG JAVA

9.1.1. ArrayList

ArrayList là một lớp triển khai *List* sử dụng mảng động để lưu trữ các phần tử. Đặc điểm của *ArrayList* là hỗ trợ truy cập phần tử nhanh chóng theo chỉ số nhưng việc thêm/xóa phần tử ở giữa mảng sẽ tốn thời gian vì cần dịch chuyển các phần tử còn lại.

Cơ chế hoạt động: Khi thêm phần tử vào *ArrayList*, nó sẽ kiểm tra xem dung lượng của mảng có đủ không. Nếu không, *ArrayList* sẽ tạo một mảng mới có dung lượng gấp 1.5 lần mảng cũ, sau đó sao chép các phần tử vào mảng mới này.

Ưu điểm:

- Truy cập nhanh phần tử qua chỉ số ($O(1)$).
- Tương đối hiệu quả khi thêm phần tử vào cuối danh sách.

Nhược điểm:

- Chèn và xóa phần tử ở giữa mảng khá tốn kém ($O(n)$) vì cần phải di chuyển các phần tử.
- Nếu dung lượng không phù hợp, *ArrayList* sẽ phải thực hiện việc cấp phát lại bộ nhớ.

Ứng dụng: Thích hợp cho các trường hợp truy xuất ngẫu nhiên nhiều và ít cần chèn/xóa phần tử ở giữa.

Các phương thức quan trọng của *ArrayList*

Bảng 9.1 Các phương thức quan trọng của *ArrayList*

Phương Thức	Mô Tả
<code>add(E element)</code>	Thêm phần tử vào cuối danh sách.
<code>add(int index, E element)</code>	Thêm phần tử vào vị trí chỉ định trong danh sách.
<code>get(int index)</code>	Lấy phần tử tại vị trí chỉ định.

Phương Thức	Mô Tả
set(int index, E element)	Gán phần tử mới tại vị trí chỉ định.
remove(int index)	Xóa phần tử tại vị trí chỉ định.
remove(Object o)	Xóa phần tử đầu tiên xuất hiện trong danh sách bằng với phần tử o.
size()	Trả về số lượng phần tử trong danh sách.
contains(Object o)	Kiểm tra danh sách có chứa phần tử o hay không.
indexOf(Object o)	Trả về chỉ số của lần xuất hiện đầu tiên của phần tử o.
clear()	Xóa tất cả phần tử trong danh sách.
isEmpty()	Kiểm tra danh sách có rỗng hay không.

Ví dụ 1: Giả sử ta có một danh sách *ArrayList* lưu trữ tên của các sinh viên và muốn thêm tên mới, truy cập tên theo chỉ số, và xóa tên khi cần.

```
import java.util.ArrayList;

public class ArrayListExample {
    public static void main(String[] args) {
        // Tạo một ArrayList để lưu trữ tên sinh viên
        ArrayList<String> students = new ArrayList<>();
        // Thêm sinh viên vào ArrayList
        students.add("Alice");
        students.add("Bob");
        students.add("Charlie");
        // Truy cập sinh viên theo chỉ số
        System.out.println("Sinh viên thứ nhất: " + students.get(0)); // Alice

        // Cập nhật sinh viên ở vị trí thứ hai
        students.set(1, "Bobby");
        // Xóa sinh viên ở chỉ số 2
        students.remove(2); // Xóa "Charlie"
        // Duyệt qua ArrayList
        System.out.println("Danh sách sinh viên:");
        for (String student : students) {
            System.out.println(student);
        }
    }
}
```

```

    }
}
}

```

Kết quả:

Sinh viên thứ nhất: Alice

Danh sách sinh viên:

Alice

Bobby

9.1.2. LinkedList

LinkedList là một lớp triển khai *List* dưới dạng danh sách liên kết đôi, lưu trữ các phần tử dưới dạng các nút (*node*), mỗi nút chứa dữ liệu và tham chiếu đến nút trước và sau.

Cơ chế hoạt động:

- *LinkedList* sử dụng các đối tượng *Node* để lưu trữ giá trị và tham chiếu tới nút liền trước và liền sau nó.
- Không giống như *ArrayList*, *LinkedList* không cần phải di chuyển dữ liệu khi chèn hoặc xóa phần tử ở giữa danh sách.

Ưu điểm:

- Chèn và xóa phần tử ở đầu hoặc cuối danh sách nhanh chóng ($O(1)$).
- Có thể sử dụng như một *Queue* hoặc *Deque* (double-ended queue) nhờ vào các phương thức của nó.

Nhược điểm:

- Truy cập phần tử chậm ($O(n)$) vì phải duyệt qua từng nút từ đầu đến cuối.
- Chiếm bộ nhớ nhiều hơn *ArrayList* vì phải lưu trữ cả tham chiếu đến các nút.

Ứng dụng: Thích hợp cho các trường hợp thêm/xóa phần tử thường xuyên ở bất kỳ vị trí nào.

Bảng 9.2 Các phương thức quan trọng của LinkList

Phương Thức	Mô Tả
add(E element)	Thêm phần tử vào cuối danh sách.
addFirst(E element)	Thêm phần tử vào đầu danh sách.
addLast(E element)	Thêm phần tử vào cuối danh sách.
removeFirst()	Xóa và trả về phần tử đầu tiên.
removeLast()	Xóa và trả về phần tử cuối cùng.
getFirst()	Trả về phần tử đầu tiên mà không xóa.
getLast()	Trả về phần tử cuối cùng mà không xóa.
contains(Object o)	Kiểm tra danh sách có chứa phần tử o.

Phương Thức	Mô Tả
size()	Trả về số lượng phần tử trong danh sách.
clear()	Xóa tất cả phần tử trong danh sách.

Ví dụ 2: Một danh sách chứa tên các môn học và thêm, xóa môn học từ đầu hoặc cuối danh sách mà không quan tâm đến tốc độ truy cập chỉ số.

```
import java.util.LinkedList;
public class LinkedListExample {
    public static void main(String[] args) {
        // Tạo một LinkedList để lưu trữ môn học
        LinkedList<String> subjects = new LinkedList<>();
        // Thêm môn học vào cuối danh sách
        subjects.add("Math");
        subjects.add("Physics");
        subjects.add("Chemistry");
        // Thêm môn học vào đầu danh sách
        subjects.addFirst("English");
        // Truy cập và in ra môn học đầu tiên
        System.out.println("Môn học đầu tiên: " + subjects.getFirst());
        // Xóa môn học cuối cùng
        subjects.removeLast();
        // Duyệt qua LinkedList
        System.out.println("Danh sách môn học:");
        for (String subject : subjects) {
            System.out.println(subject);
        }
    }
}
```

Kết quả:

Môn học đầu tiên: English

Danh sách môn học:

English

Math

Physics

9.1.3. HashMap

HashMap là một cấu trúc dữ liệu ánh xạ (*Map*) lưu trữ các cặp *key-value*, trong đó mỗi *key* là duy nhất và ánh xạ tới một *value*. *HashMap* sử dụng bảng băm (*hash table*) để lưu trữ các phần tử, cho phép truy cập nhanh theo *key*.

Cơ chế hoạt động:

- *HashMap* tính toán chỉ số băm từ *key*, sau đó lưu trữ *value* vào vị trí tương ứng.
- Nếu có xung đột (tức hai *key* khác nhau có chỉ số băm giống nhau), *HashMap* sử dụng danh sách liên kết hoặc phương pháp khác để lưu trữ các phần tử cùng vị trí.

Ưu điểm:

- Truy xuất nhanh phần tử theo *key* ($O(1)$ trong điều kiện tốt).
- Không có thứ tự các phần tử (trừ khi sử dụng *LinkedHashMap* hoặc *TreeMap*).

Nhược điểm:

- Không đảm bảo thứ tự các phần tử khi duyệt.
- Cần sử dụng đúng *key* để truy xuất phần tử.

Ứng dụng: Thích hợp cho các trường hợp cần ánh xạ nhanh chóng giữa các cặp *key-value*.

Bảng 9.3 Các phương thức quan trọng của HashMap

Phương Thức	Mô Tả
put(K key, V value)	Thêm cặp key-value vào map.
get(Object key)	Trả về giá trị tương ứng với key.
remove(Object key)	Xóa cặp key-value theo key chỉ định.
containsKey(Object key)	Kiểm tra map có chứa key.
containsValue(Object value)	Kiểm tra map có chứa value.
keySet()	Trả về tập hợp các key trong map.
values()	Trả về collection của các value.
entrySet()	Trả về tập hợp các cặp key-value.
size()	Trả về số lượng cặp key-value trong map.
clear()	Xóa tất cả cặp key-value khỏi map.

Ví dụ 3: Giả sử ta muốn lưu trữ thông tin điểm số của học sinh với tên học sinh là *key* và điểm số là *value*.

```
import java.util.HashMap;

public class HashMapExample {

    public static void main(String[] args) {
        // Tạo một HashMap để lưu trữ điểm số
        HashMap<String, Integer> scores = new HashMap<>();
```

```

// Thêm điểm số vào HashMap
scores.put("Alice", 90);
scores.put("Bob", 85);
scores.put("Charlie", 92);
// Truy xuất điểm của Alice
System.out.println("Điểm của Alice: " + scores.get("Alice"));
// Cập nhật điểm của Bob
scores.put("Bob", 88);
// Duyệt qua HashMap
System.out.println("Danh sách điểm số:");
for (String name : scores.keySet()) {
    System.out.println(name + ": " + scores.get(name));
}
}
}

```

Kết quả:

Danh sách điểm số:

Bob: 88

Alice: 90

Charlie: 92

9.1.4. TreeSet

TreeSet là một cấu trúc dữ liệu lưu trữ các phần tử dưới dạng tập hợp (*Set*) không trùng lặp và được sắp xếp theo thứ tự tự nhiên hoặc theo bộ so sánh tùy chỉnh. *TreeSet* được triển khai dựa trên cây nhị phân cân bằng.

Cơ chế hoạt động:

- *TreeSet* sử dụng một cây cân bằng (red-black tree) để sắp xếp và lưu trữ phần tử theo thứ tự tự nhiên.
- Các phần tử được chèn theo một thứ tự xác định và không cho phép phần tử trùng lặp.

Ưu điểm:

- Sắp xếp phần tử tự động, không cần sắp xếp thủ công.
- Duyệt các phần tử theo thứ tự sắp xếp nhanh chóng ($O(\log n)$).

Nhược điểm:

- Truy cập phần tử có thể chậm hơn *HashSet* do phải duy trì cân bằng cây.
- Không cho phép phần tử trùng lặp.

Ứng dụng: Thích hợp khi cần tập hợp các phần tử không trùng lặp và yêu cầu sắp xếp tự động.

Bảng 9.4 Các phương thức chính của TreeSet

Phương Thức	Mô Tả
add(E e)	Thêm phần tử e vào tập hợp.
remove(Object o)	Xóa phần tử o khỏi tập hợp.
first()	Trả về phần tử đầu tiên (nhỏ nhất) theo thứ tự tự nhiên.
last()	Trả về phần tử cuối cùng (lớn nhất).
higher(E e)	Trả về phần tử nhỏ nhất lớn hơn e.
lower(E e)	Trả về phần tử lớn nhất nhỏ hơn e.
contains(Object o)	Kiểm tra tập hợp có chứa phần tử o.
size()	Trả về số lượng phần tử trong tập hợp.
clear()	Xóa tất cả phần tử khỏi tập hợp.

Ví dụ 4: Giả sử ta có một tập hợp số cần duyệt theo thứ tự tăng dần mà không cho phép số trùng lặp.

```
import java.util.TreeSet;

public class TreeSetExample {

    public static void main(String[] args) {
        // Tạo một TreeSet để lưu trữ số nguyên
        TreeSet<Integer> numbers = new TreeSet<>();
        // Thêm các số vào TreeSet
        numbers.add(5);
        numbers.add(2);
        numbers.add(8);
        numbers.add(1);
        numbers.add(5); // Số trùng lặp, sẽ không được thêm
        // Duyệt qua TreeSet
        System.out.println("Các số trong TreeSet:");
        for (Integer number : numbers) {
            System.out.println(number);
        }
    }
}
```

Kết quả:

Các số trong TreeSet:

1
2
5
8

9.1.5. Stack và Queue

Stack là một cấu trúc dữ liệu dạng ngăn xếp, hoạt động theo nguyên tắc LIFO (Last In, First Out - vào sau ra trước). Các phần tử chỉ được thêm và xóa ở đầu ngăn xếp.

- **Cơ chế hoạt động:** Stack cho phép thêm phần tử mới vào đỉnh ngăn xếp và lấy ra phần tử từ đỉnh, nghĩa là phần tử cuối cùng được thêm vào sẽ là phần tử đầu tiên bị lấy ra.
- **Ưu điểm:** Cấu trúc đơn giản, dễ sử dụng cho các bài toán LIFO. Thích hợp cho các bài toán liên quan đến ngăn xếp và đảo ngược.
- **Nhược điểm:** Khả năng truy cập phần tử bị giới hạn (chỉ truy cập đỉnh ngăn xếp).
- **Ứng dụng:** Dùng trong tính toán biểu thức, kiểm tra dấu ngoặc, hệ thống điều hướng (back/forward), quản lý lịch sử trình duyệt.

Bảng 9.5 Các phương thức chính của Stack

Phương Thức	Mô Tả
push(E e)	Đẩy phần tử e vào đỉnh ngăn xếp.
pop()	Lấy và xóa phần tử ở đỉnh ngăn xếp.
peek()	Trả về phần tử ở đỉnh ngăn xếp mà không xóa.
isEmpty()	Kiểm tra ngăn xếp có rỗng không.
search(Object o)	Trả về vị trí của phần tử o từ đỉnh ngăn xếp.

Ví dụ 5: Triển khai Stack để quản lý trình tự duyệt lùi trong trình duyệt web.

```
import java.util.Stack;

public class StackExample {
    public static void main(String[] args) {
        Stack<String> history = new Stack<>();
        // Thêm trang web vào ngăn xếp
        history.push("Trang 1");
        history.push("Trang 2");
        history.push("Trang 3");
        // Lấy trang web gần nhất và quay lại
        System.out.println("Quay lại: " + history.pop()); // Trang 3
        System.out.println("Trang hiện tại: " + history.peek()); // Trang 2
    }
}
```

```
}
```

Kết quả:

Quay lại: Trang 3

Trang hiện tại: Trang 2

Queue là một cấu trúc dữ liệu dạng hàng đợi, hoạt động theo nguyên tắc FIFO (First In, First Out - vào trước ra trước). Các phần tử được thêm vào cuối hàng đợi và lấy ra từ đầu hàng đợi.

- **Cơ chế hoạt động:** Queue thêm phần tử vào cuối hàng đợi (*offer*) và lấy phần tử ra từ đầu hàng đợi (*poll*).
- **Ưu điểm:** Cấu trúc đơn giản cho các bài toán FIFO. Dễ dàng quản lý các yêu cầu xếp hàng.
- **Nhược điểm:** Không hỗ trợ truy cập phần tử giữa một cách ngẫu nhiên.
- **Ứng dụng:** Quản lý hàng đợi trong xử lý tác vụ, lập lịch, hệ thống xử lý yêu cầu.

Bảng 9.6 Các phương thức chính của Queue

Phương Thức	Mô Tả
offer(E e)	Thêm phần tử vào cuối hàng đợi.
poll()	Lấy và xóa phần tử đầu tiên của hàng đợi.
peek()	Trả về phần tử đầu tiên của hàng đợi mà không xóa.
isEmpty()	Kiểm tra hàng đợi có rỗng không.
size()	Trả về số lượng phần tử trong hàng đợi.

Ví dụ 6: Giả sử ta xây dựng một hệ thống quản lý hàng đợi đặt chỗ.

```
import java.util.LinkedList;
import java.util.Queue;
public class QueueExample {
    public static void main(String[] args) {
        Queue<String> queue = new LinkedList<>();
        // Thêm người vào hàng đợi
        queue.offer("Khách 1");
        queue.offer("Khách 2");
        queue.offer("Khách 3");
        // Lấy người đầu tiên trong hàng đợi
        System.out.println("Phục vụ: " + queue.poll()); // Khách 1
        System.out.println("Người kế tiếp: " + queue.peek()); // Khách 2
    }
}
```

Kết quả:

Phục vụ: Khách 1

Người kế tiếp: Khách 2

9.1.6. Lớp Collection

Lớp Collection trong Java là một phần của Java Collections Framework - bộ khung cấu trúc dữ liệu tích hợp, cung cấp các lớp và giao diện để quản lý và thao tác dữ liệu như List, Set, Queue, và Map.

Java Collection Framework bao gồm ba phần chính:

- **Interfaces:** Định nghĩa các hành vi chung cho các cấu trúc dữ liệu, bao gồm Collection, List, Set, Queue, và Map.
- **Implementations:** Các lớp triển khai các giao diện nói trên, như ArrayList, LinkedList, HashSet, LinkedHashSet, HashMap, v.v.
- **Algorithms:** Các phương thức để thao tác dữ liệu như sắp xếp, tìm kiếm, và thao tác với các phần tử.

Collection là giao diện cha của các cấu trúc dữ liệu cơ bản trong Java (trừ Map), cung cấp các phương thức phổ biến để thao tác dữ liệu, ví dụ như add, remove, size, clear, v.v.

Ví dụ 7:

```
Collection<Integer> numbers = new ArrayList<>();  
numbers.add(1);  
numbers.add(2);  
numbers.add(3);  
System.out.println("Size: " + numbers.size()); // Kết quả: 3
```

Set là giao diện mở rộng của *Collection*, đại diện cho tập hợp các phần tử không trùng lặp và không có thứ tự cụ thể. Không thể truy cập phần tử theo chỉ số trong *Set*.

Các lớp triển khai phổ biến:

- **HashSet:** Lưu trữ phần tử không theo thứ tự.
- **LinkedHashSet:** Bảo toàn thứ tự chèn.
- **TreeSet:** Sắp xếp phần tử theo thứ tự tự nhiên hoặc theo bộ so sánh.

Ví dụ 8:

```
Set<String> uniqueNames = new HashSet<>();  
uniqueNames.add("Alice");  
uniqueNames.add("Bob");  
uniqueNames.add("Alice"); // Không thêm vì trùng lặp  
System.out.println("Set of names: " + uniqueNames); // Kết quả: [Alice, Bob]
```

Queue đại diện cho hàng đợi với thứ tự FIFO (First In, First Out). *Queue* phù hợp cho các ứng dụng yêu cầu xử lý dữ liệu theo thứ tự nhập vào.

Các lớp triển khai phổ biến:

- **LinkedList**: Được sử dụng như một hàng đợi nhờ các phương thức offer, poll, peek.
- **PriorityQueue**: Hàng đợi ưu tiên, sắp xếp phần tử dựa trên thứ tự tự nhiên hoặc theo bộ so sánh.

Ví dụ 9:

```
Queue<String> queue = new LinkedList<>();
queue.offer("Alice");
queue.offer("Bob");
System.out.println("Peek: " + queue.peek()); // Kết quả: Alice
System.out.println("Poll: " + queue.poll()); // Kết quả: Alice
System.out.println("Queue after poll: " + queue); // Kết quả: [Bob]
```

Map lưu trữ các cặp *key-value*, trong đó *key* là duy nhất và có thể liên kết với một giá trị (*value*). *Map* không phải là một *Collection* vì nó lưu trữ theo cặp.

Các lớp triển khai phổ biến:

- **HashMap**: Không bảo toàn thứ tự, hỗ trợ null key và null value.
- **LinkedHashMap**: Bảo toàn thứ tự chèn.
- **TreeMap**: Sắp xếp các key theo thứ tự tự nhiên hoặc theo bộ so sánh.
- **Hashtable**: Đồng bộ hóa, không hỗ trợ null key hoặc null value.

Ví dụ 10:

```
Map<String, Integer> ages = new HashMap<>();
ages.put("Alice", 25);
ages.put("Bob", 30);
System.out.println("Age of Alice: " + ages.get("Alice")); // Kết quả: 25
```

Các phương thức quan trọng trong Collection Framework của Java, bao gồm các giao diện *Collection*, *List*, *Set*, *Queue* và *Map*.

Bảng 9.7 Các phương thức quan trọng trong Collection Framework

Phương Thức	Mô Tả	Áp Dụng
add(E e)	Thêm phần tử vào Collection.	Collection, List, Set, Queue
addAll(Collection<? extends E> c)	Thêm tất cả phần tử của Collection c vào Collection hiện tại.	Collection, List, Set, Queue
remove(Object o)	Xóa một phần tử khỏi Collection nếu tồn tại.	Collection, List, Set, Queue
removeAll(Collection<?> c)	Xóa tất cả các phần tử trong Collection hiện tại có trong Collection c.	Collection, List, Set

Phương Thức	Mô Tả	Áp Dụng
retainAll(Collection<?> c)	Giữ lại các phần tử trong Collection hiện tại có trong Collection c, xóa các phần tử khác.	Collection, List, Set
clear()	Xóa tất cả phần tử trong Collection.	Collection, List, Set, Queue, Map
contains(Object o)	Kiểm tra Collection có chứa phần tử o không.	Collection, List, Set
containsAll(Collection<?> c)	Kiểm tra Collection có chứa tất cả phần tử của Collection c không.	Collection, List, Set
isEmpty()	Kiểm tra Collection có rỗng không.	Collection, List, Set, Queue, Map
size()	Trả về số lượng phần tử trong Collection.	Collection, List, Set, Queue, Map
toArray()	Trả về mảng chứa tất cả các phần tử trong Collection.	Collection, List, Set
iterator()	Trả về Iterator để duyệt các phần tử trong Collection.	Collection, List, Set
get(int index)	Trả về phần tử tại chỉ số index trong List.	List
set(int index, E element)	Gán phần tử element tại vị trí index trong List.	List
indexOf(Object o)	Trả về chỉ số của phần tử o trong List, hoặc -1 nếu không tìm thấy.	List
lastIndexOf(Object o)	Trả về chỉ số cuối cùng của phần tử o trong List, hoặc -1 nếu không tìm thấy.	List
offer(E e)	Thêm phần tử vào cuối hàng đợi (Queue).	Queue
poll()	Trả về và loại bỏ phần tử đầu hàng đợi (Queue).	Queue
peek()	Trả về phần tử đầu hàng đợi (Queue) nhưng không loại bỏ.	Queue
put(K key, V value)	Thêm cặp key-value vào Map.	Map
get(Object key)	Trả về giá trị tương ứng với key trong Map.	Map
remove(Object key)	Xóa cặp key-value khỏi Map theo key.	Map
containsKey(Object key)	Kiểm tra Map có chứa key không.	Map
containsValue(Object value)	Kiểm tra Map có chứa giá trị value không.	Map
keySet()	Trả về tập hợp các key trong Map.	Map
values()	Trả về Collection của các giá trị trong Map.	Map
entrySet()	Trả về tập hợp các cặp key-value trong Map.	Map

Phương Thức	Mô Tả	Áp Dụng
putAll(Map<? extends K, ? extends V> m)	Thêm tất cả cặp key-value từ Map m vào Map hiện tại.	Map
replace(K key, V value)	Thay thế giá trị cũ của key bằng value mới nếu key tồn tại.	Map

Duyệt Collection:

- Sử dụng *for-each*: *for-each* có thể duyệt qua các Collection mà không cần biết đến chỉ số. Ví dụ:

```
List<String> names = new ArrayList<>();
names.add("Alice");
names.add("Bob");
for (String name : names) {
    System.out.println(name);
}
```

- Sử dụng *Iterator*: *Iterator* cung cấp cách duyệt qua *Collection* và cho phép loại bỏ phần tử khi duyệt mà không gặp lỗi. Ví dụ:

```
Iterator<String> iterator = names.iterator();
while (iterator.hasNext()) {
    System.out.println(iterator.next());
}
```

Các thuật toán trong Collections

- Sắp xếp: Collections.sort(List<T> list).
- Tìm kiếm: Collections.binarySearch(List<T> list, T key).
- Đảo ngược: Collections.reverse(List<?> list).
- Trộn ngẫu nhiên: Collections.shuffle(List<?> list).

Ví dụ 11: sắp xếp

```
List<Integer> numbers = new ArrayList<>(Arrays.asList(3, 1, 4, 1, 5));
Collections.sort(numbers);
System.out.println("Sorted numbers: " + numbers); // Kết quả: [1, 1, 3, 4, 5]
```

Ưu điểm của Collection:

- Tính đồng nhất**: Các giao diện chuẩn cho phép lập trình viên dễ dàng thay đổi lớp triển khai mà không ảnh hưởng đến mã.
- Dễ dàng mở rộng**: Các giao diện có thể được mở rộng để tạo các cấu trúc dữ liệu tùy chỉnh.

- **Thuận tiện và an toàn:** Các phương thức chuẩn giúp quản lý dữ liệu nhất quán và bảo mật trong các ứng dụng đa luồng (với `Collections.synchronizedList`, `Collections.synchronizedMap`).

9.2. CÁC LUỒNG VÀO/RA DỮ LIỆU TRONG JAVA

9.2.1. Khái niệm

Stream: Luồng vào ra trong Java hay Input/Output (I/O) trong Java được sử dụng để xử lý đầu vào và đầu ra trong Java. Stream là một dòng liên tục, có thứ tự các bytes dữ liệu chảy giữa chương trình và thiết bị ngoại vi. Nó là khái niệm trừu tượng giúp giảm bớt các thao tác vào ra phức tạp đối với người lập trình. Nó cho phép kết nối nhiều thiết bị ngoại vi khác nhau với chương trình.

Trong java, một stream bao gồm các byte và có 3 stream được tạo cho chúng ta một cách tự động. Tất cả các stream này được gắn với console.

- `System.out`: output stream tiêu chuẩn
- `System.in`: input stream tiêu chuẩn
- `System.err`: error stream tiêu chuẩn

Ví dụ 1: in thông báo *output* và *error* tới console.

```
System.out.println("simple message");
```

```
System.err.println("error message");
```

Ví dụ 2: lấy giá trị *input* từ console.

```
int i=System.in.read();//tra ve ma ASCII cua ky tu dau tien
```

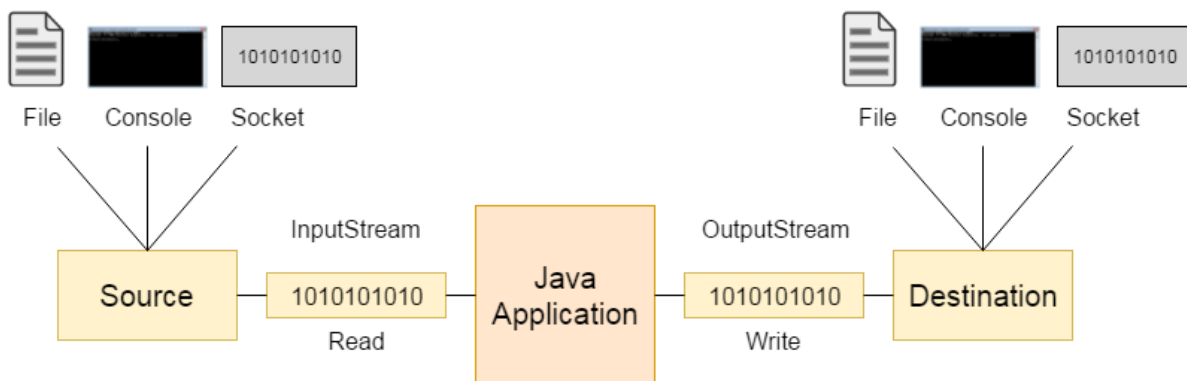
```
System.out.println((char)i);//in ky tu lay duoc ra man hinh
```

Nếu dòng dữ liệu trong Stream có hướng chảy từ thiết bị ngoại vi vào chương trình thì ta nói đây là Stream nhập (Input Stream), ngược lại gọi là Stream xuất (Output Stream).

OutputStream Ứng dụng Java sử dụng một output stream để ghi dữ liệu đến đích, nó có thể là một tệp tin, một mảng, thiết bị ngoại vi hoặc socket.

InputStream Ứng dụng Java sử dụng một input stream để đọc dữ liệu từ một nguồn, nó có thể là một tệp tin, một mảng, thiết bị ngoại vi hoặc socket.

Hoạt động của Java OutputStream và InputStream được mô tả trong hình dưới đây.



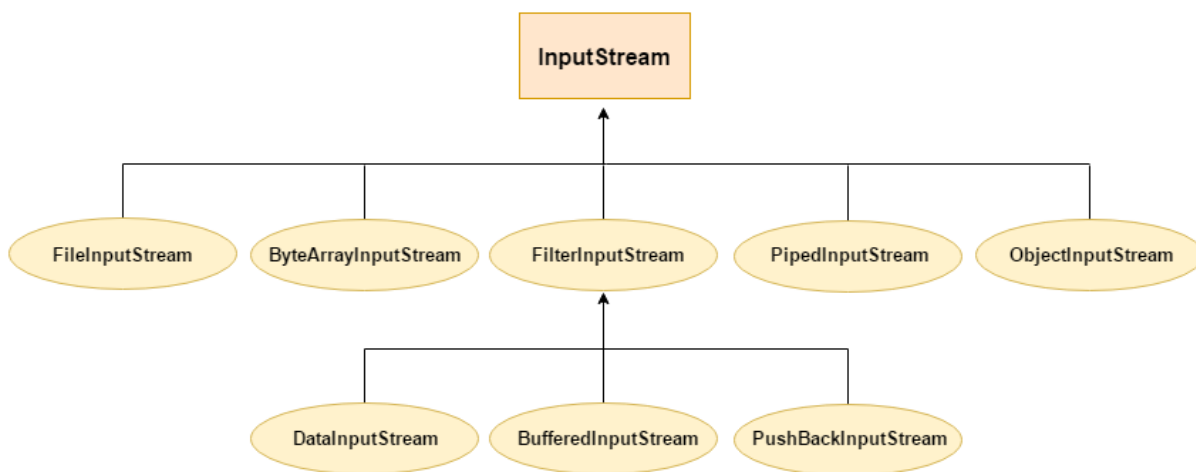
Hình 9.1 OutputStream và InputStream

Để giao tiếp với thiết bị ngoại vi, chương trình trước tiên phải lấy được Stream nhập/ Stream xuất gắn với thiết bị ngoại vi này. Sau đó chương trình sẽ gửi dữ liệu ra thiết bị ngoại vi bằng thao tác ghi (write) vào Stream xuất của thiết bị ngoại vi. Ngược lại, chương trình có thể nhận dữ liệu từ thiết bị ngoại vi bằng thao tác đọc (read) Stream nhập của thiết bị ngoại vi đó. Nói cách khác, chương trình chỉ làm việc trên các Stream nhập và Stream xuất của thiết bị ngoại vi mà không cần quan tâm đến đặc điểm riêng biệt của thiết bị ngoại vi.

Gói **java.io** chứa tất cả các lớp cần thiết cho hoạt động input và output của Stream.

9.2.2. Input Streams

Khái niệm: *InputStream* là lớp trừu tượng (abstract class) đại diện cho luồng dữ liệu đầu vào dưới dạng byte. Các luồng kế thừa *InputStream* đọc dữ liệu từ các nguồn khác nhau như tệp tin, thiết bị đầu vào, hoặc từ mạng. Các lớp con của lớp *InputStream* được mô tả như hình 9.2 dưới đây.



Hình 9.2 InputStream Hierarchy

Cách hoạt động:

- Luồng *InputStream* đọc dữ liệu từ nguồn byte, trả về dữ liệu dưới dạng byte hoặc một mảng byte.

- Khi đọc đến cuối nguồn dữ liệu, phương thức *read()* trả về giá trị -1 để báo hiệu không còn dữ liệu.

Ưu điểm:

- Phù hợp cho xử lý dữ liệu nhị phân như hình ảnh, âm thanh, và video.
- Dễ dàng đọc dữ liệu từ nhiều nguồn khác nhau (file, mạng).

Nhược điểm:

- Chỉ làm việc với byte, không hỗ trợ trực tiếp dữ liệu dạng ký tự.
- Không tự động chuyển đổi dữ liệu byte sang định dạng dễ đọc (như chuỗi ký tự).

Các phương thức chính:

- *int read()*: Đọc một *byte* dữ liệu, trả về -1 nếu không còn dữ liệu.
- *int read(byte[] b)*: Đọc các *byte* dữ liệu vào mảng *b*, trả về số *byte* đã đọc hoặc -1 nếu hết dữ liệu.
- *int available()*: Trả về số *byte* còn lại có thể đọc mà không bị chặn.
- *void close()*: Đóng luồng để giải phóng tài nguyên.

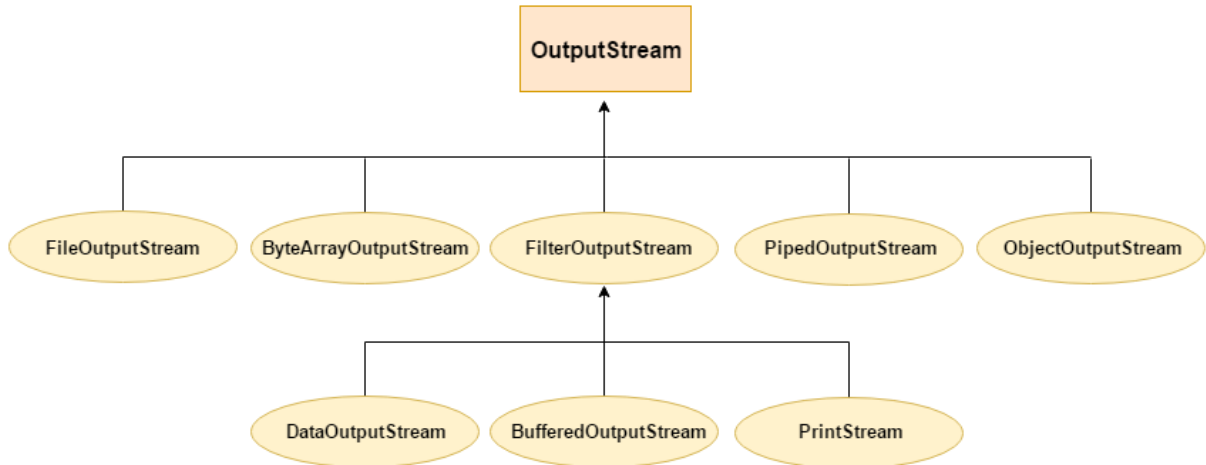
Ví dụ 3:

```
import java.io.FileInputStream;
import java.io.IOException;
public class InputStreamExample {
    public static void main(String[] args) {
        try (FileInputStream fis = new FileInputStream("input.txt")) {
            int data;
            while ((data = fis.read()) != -1) { // Đọc từng byte từ file
                System.out.print((char) data); // Ép kiểu byte sang ký tự để in
                ra
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Trong ví dụ trên *FileInputStream* được sử dụng để đọc dữ liệu từ file "input.txt" dưới dạng byte. Phương thức *fis.read()* đọc một byte từ file mỗi lần và trả về giá trị byte. *while ((data = fis.read()) != -1)* tiếp tục đọc byte từ file cho đến khi không còn dữ liệu, và kết quả sẽ in ra các ký tự có trong file "input.txt"

9.2.2. Output Streams

Khái niệm: *OutputStream* là lớp trừu tượng đại diện cho luồng dữ liệu đầu ra dưới dạng byte. Các luồng kế thừa *OutputStream* ghi dữ liệu byte vào các nguồn như tệp tin, thiết bị đầu ra hoặc mạng. Các lớp con của lớp *OutputStream* được mô tả như hình 9.3 dưới đây.



Hình 9.3 OutputStream Hierarchy

Cách hoạt động:

- Luồng *OutputStream* ghi dữ liệu byte vào nguồn, có thể là từng byte, mảng byte, hoặc một phần của mảng byte.
- Ta có thể sử dụng phương thức *flush()* để đẩy các byte còn lại trong bộ đệm vào nguồn ra trước khi đóng luồng.

Ưu điểm:

- Phù hợp với dữ liệu nhị phân như hình ảnh và âm thanh.
- Linh hoạt để ghi dữ liệu đến nhiều nguồn khác nhau.

Nhược điểm:

- Chỉ làm việc với byte, không hỗ trợ trực tiếp dữ liệu dạng ký tự.
- Cần thao tác thủ công để xử lý bộ đệm trong một số trường hợp.

Các phương thức:

- *void write(int b)*: Ghi một byte dữ liệu.
- *void write(byte[] b)*: Ghi một mảng byte vào luồng.
- *void flush()*: Xả (flush) các byte còn lại trong bộ đệm vào nguồn đầu ra.
- *void close()*: Đóng luồng để giải phóng tài nguyên.

Ví dụ 4:

```
import java.io.FileOutputStream;
import java.io.IOException;
public class OutputStreamExample {
    public static void main(String[] args) {
```

```

    try (FileOutputStream fos = new FileOutputStream("output.txt")) {
        String data = "Hello, World!";
        fos.write(data.getBytes()); // Chuyển chuỗi thành byte và ghi vào file
    } catch (IOException e) {
        e.printStackTrace();
    }
}

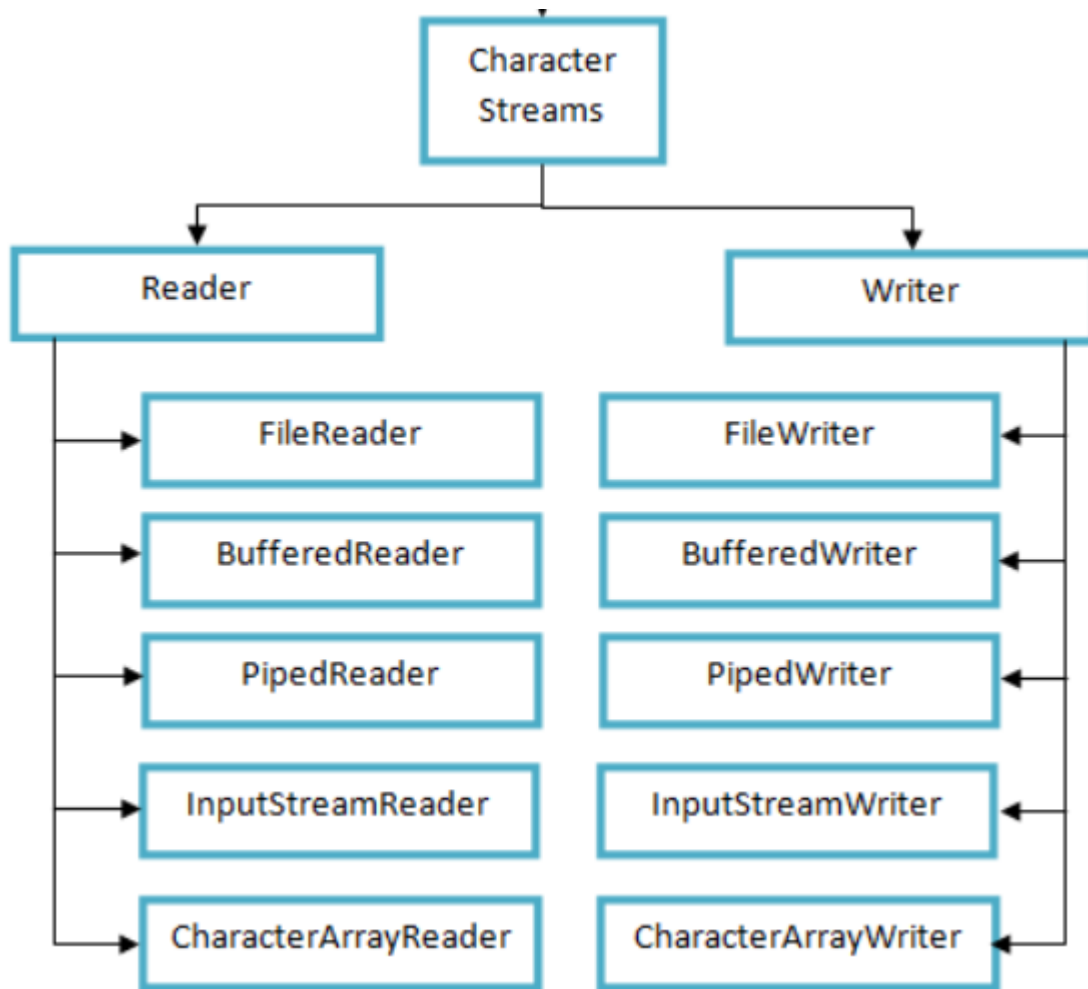
```

Trong ví dụ trên *FileOutputStream* được sử dụng để ghi chuỗi "Hello, World!" vào file "output.txt" dưới dạng byte.

9.2.3. Character Streams

Khái niệm: *Character Streams* là các luồng vào/ra chuyên xử lý dữ liệu dạng ký tự Unicode (16-bit), giúp làm việc với các file văn bản. *Character Streams* chuyển byte thành ký tự khi đọc và ngược lại khi ghi, dựa vào bộ mã hóa ký tự (character encoding), điều này làm cho chúng thích hợp khi làm việc với dữ liệu văn bản, vì chúng hỗ trợ tốt các ký tự Unicode, giúp xử lý các ngôn ngữ đa dạng dễ dàng hơn.

Các lớp liên quan đến *Character Streams* trong *Java* là các lớp con của hai lớp trừu tượng *Reader* và *Writer*. Các lớp con của lớp *Character Streams* được mô tả như hình vẽ 9.4



Hình 9.4 Character Streams Hierarchy

Cách hoạt động: Các luồng *Character Stream* hoạt động theo cách tự động chuyển đổi byte thành ký tự (và ngược lại) bằng cách sử dụng một mã hóa ký tự (character encoding). Khi đọc hoặc ghi dữ liệu ký tự, Character Streams sẽ làm việc với chuỗi ký tự, thay vì làm việc trực tiếp với byte như các luồng vào/ra byte.

- Mã hóa ký tự: Java sử dụng bộ mã hóa như UTF-8, UTF-16, và các chuẩn khác để chuyển đổi byte thành ký tự khi đọc và ngược lại khi ghi.
- Dễ dàng làm việc với văn bản: Khi làm việc với văn bản, Character Streams giúp xử lý các ngôn ngữ và ký tự đặc biệt mà không cần phải quan tâm đến mã hóa hay chuyển đổi thủ công.

Các phương thức:

- **Reader Class:**
 - + `int read()`: Đọc một ký tự từ luồng và trả về giá trị int (mã Unicode của ký tự).
 - + `int read(char[] cbuf)`: Đọc một mảng các ký tự và trả về số lượng ký tự đã đọc.
 - + `boolean ready()`: Kiểm tra xem luồng có sẵn sàng đọc hay không.
 - + `void close()`: Đóng luồng và giải phóng tài nguyên.

– **Writer Class:**

- + `void write(int c)`: Ghi một ký tự vào luồng.
- + `void write(char[] cbuf)`: Ghi một mảng các ký tự vào luồng.
- + `void write(String str)`: Ghi một chuỗi ký tự vào luồng.
- + `void flush()`: Xả dữ liệu từ bộ đệm ra ngoài luồng.
- + `void close()`: Đóng luồng và giải phóng tài nguyên.

Sử dụng `BufferedReader` và `BufferedWriter`: Một trong những vấn đề khi làm việc với các luồng ký tự là hiệu suất, vì mỗi lần đọc hoặc ghi một ký tự sẽ mất một thời gian. Để tối ưu hóa điều này, Java cung cấp `BufferedReader` và `BufferedWriter`.

- `BufferedReader`: Đọc dữ liệu từ luồng đầu vào (`Reader`) nhưng không đọc một ký tự một lần mà đọc dữ liệu theo khối (`buffer`). Điều này giúp giảm số lần I/O và cải thiện hiệu suất khi đọc.
- `BufferedWriter`: Tương tự như `BufferedReader` nhưng dành cho ghi dữ liệu. Nó sẽ ghi các ký tự vào bộ đệm trước và chỉ ghi vào nguồn đầu ra khi bộ đệm đầy hoặc khi gọi `flush()`.

Ví dụ 5: Đọc từ tệp với `BufferedReader`

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
public class BufferedReaderExample {
    public static void main(String[] args) {
        try (BufferedReader reader = new BufferedReader(new
FileReader("example.txt"))) {
            String line;
            while ((line = reader.readLine()) != null) { // Đọc một dòng
                System.out.println(line); // In dòng ra màn hình
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Trong ví dụ trên `BufferedReader` sử dụng bộ đệm để đọc dữ liệu từ tệp "example.txt". Phương thức `readLine()` đọc một dòng văn bản tại một thời điểm, thay vì từng ký tự. Đây là cách hiệu quả khi làm việc với các tệp văn bản có nhiều dòng.

Ví dụ 6: Ghi vào tệp với `BufferedWriter`

```

import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
public class BufferedWriterExample {
    public static void main(String[] args) {
        try (BufferedWriter writer = new BufferedWriter(new
FileWriter("output.txt"))) {
            writer.write("Hello, World!"); // Ghi chuỗi vào tệp
            writer.newLine(); // Thêm một dòng mới
            writer.write("Goodbye!");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

Trong ví dụ trên *BufferedWriter* ghi dữ liệu vào tệp "output.txt" với bộ đệm. Phương thức *write("Hello, World!")* ghi chuỗi vào tệp, và *newLine()* thêm một dòng mới. Sử dụng bộ đệm giúp cải thiện hiệu suất ghi dữ liệu, đặc biệt khi ghi một lượng lớn dữ liệu.

9.2.4. Object Streams

Khái niệm: *Object Streams* trong Java là một nhóm các luồng được thiết kế để ghi và đọc các đối tượng từ hoặc vào tệp hoặc truyền qua mạng. Chúng cho phép "tuần tự hóa" đối tượng (*serialization*), tức là chuyển đổi đối tượng thành một chuỗi byte để có thể lưu trữ hoặc truyền tải, và "giải tuần tự hóa" đối tượng (*deserialization*), tức là tái tạo lại đối tượng từ chuỗi byte.

Tuần tự hóa (*Serialization*) là quá trình chuyển đổi đối tượng Java thành một chuỗi byte mà có thể lưu trữ trong tệp hoặc truyền qua mạng. Ngược lại, giải tuần tự hóa (*Deserialization*) là quá trình tái tạo lại đối tượng từ chuỗi byte đã được tuần tự hóa.

Object Streams chủ yếu bao gồm các lớp *ObjectInputStream* và *ObjectOutputStream*:

- *ObjectInputStream*: Dùng để đọc (giải tuần tự hóa) đối tượng từ luồng. Các phương thức của lớp *ObjectInputStream*:
 - + *writeObject(Object obj)*: Tuần tự hóa đối tượng và ghi vào luồng.
 - + *flush()*: Xả bộ đệm, ghi tất cả các đối tượng vào luồng.
 - + *close()*: Đóng luồng và giải phóng tài nguyên.
- *ObjectOutputStream*: Dùng để ghi (tuần tự hóa) đối tượng vào luồng. Các phương thức của lớp *ObjectOutputStream*:
 - + *readObject()*: Đọc đối tượng đã được tuần tự hóa từ luồng và giải tuần tự hóa thành đối tượng gốc.
 - + *close()*: Đóng luồng và giải phóng tài nguyên.

Cách sử dụng Object Streams:

- Lưu trữ trạng thái của đối tượng: Khi ta muốn lưu trạng thái của các đối tượng vào tệp, hoặc truyền đối tượng qua mạng, ta cần tuần tự hóa đối tượng.
- Truyền tải đối tượng qua mạng: Khi gửi các đối tượng qua mạng (ví dụ: trong các ứng dụng phân tán), ta cần chuyển các đối tượng thành các byte để có thể gửi đi và sau đó giải tuần tự hóa tại máy nhận.
- Công cụ hỗ trợ cho các đối tượng phức tạp: Với Object Streams, ta có thể tuần tự hóa các đối tượng phức tạp có liên quan đến các đối tượng con (bao gồm các đối tượng tham chiếu) mà không cần phải xử lý thủ công các mối quan hệ này.

Yêu cầu khi sử dụng Object Streams: Để một đối tượng có thể được tuần tự hóa, lớp của đối tượng đó phải *implement* interface *Serializable*. Interface này không có phương thức nào, chỉ là một đánh dấu (marker interface), để thông báo cho Java biết rằng đối tượng của lớp này có thể được tuần tự hóa.

Ví dụ 7:

```
import java.io.Serializable;

class Person implements Serializable {
    private String name;
    private int age;
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
    public String getName() {
        return name;
    }
    public int getAge() {
        return age;
    }
}
```

Trong ví dụ trên lớp *Person* implement interface *Serializable*, đối tượng của lớp *Person* có thể được tuần tự hóa và giải tuần tự hóa.

Ví dụ 8: Ghi đối tượng vào tệp với ObjectOutputStream

```
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectOutputStream;
public class ObjectStreamExample {
```

```

public static void main(String[] args) {
    // Ghi đối tượng vào file
    try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("person.dat"))) {
        Person person = new Person("Alice", 25);
        oos.writeObject(person); // Tuần tự hóa đối tượng và ghi vào tệp
    } catch (IOException e) {
        e.printStackTrace();
    }
}

```

Trong ví dụ trên với lớp *Person* đã xây dựng ở ví dụ 1, lớp *ObjectOutputStream* được sử dụng để tuần tự hóa đối tượng *person* và ghi vào tệp *person.dat*. Đối tượng *person* có hai trường: *tên* và *tuổi*. Chúng được tuần tự hóa thành chuỗi byte và ghi vào tệp. *writeObject()* là phương thức dùng để tuần tự hóa đối tượng và ghi nó vào luồng.

Ví dụ 9: Đọc đối tượng từ tệp với *ObjectInputStream*

```

import java.io.*;
public class ObjectStreamExample {
    public static void main(String[] args) {
        // Đọc đối tượng từ file
        try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream("person.dat"))) {
            Person person = (Person) ois.readObject(); // Giải tuần tự hóa đối tượng
            System.out.println("Name: " + person.getName());
            System.out.println("Age: " + person.getAge());
        } catch (IOException | ClassNotFoundException e) {
            e.printStackTrace();
        }
    }
}

```

Trong ví dụ trên với lớp *Person* đã xây dựng ở ví dụ 1, và thực hiện như ví dụ 2, lớp *ObjectInputStream* được sử dụng để đọc đối tượng đã tuần tự hóa từ tệp *person.dat*. *readObject()* đọc đối tượng từ luồng và giải tuần tự hóa lại thành đối tượng *person*. Sau khi đọc, ta có thể truy cập các thuộc tính của đối tượng (như *getName()* và *getAge()*).

Kết quả:

Name: Alice

Age: 25

Ưu điểm của Object Streams:

- Lưu trữ và truyền tải đối tượng: Object Streams là công cụ lý tưởng để lưu trữ và truyền tải các đối tượng Java phức tạp, bao gồm các đối tượng tham chiếu, giúp tiết kiệm thời gian và công sức khi làm việc với các cấu trúc dữ liệu phức tạp.
- Khả năng tuần tự hóa và giải tuần tự hóa: Java cung cấp phương thức đơn giản để tuần tự hóa đối tượng và sau đó giải tuần tự hóa lại, giúp việc lưu trữ và truyền tải dữ liệu trở nên dễ dàng.
- Hỗ trợ đối tượng liên kết: Khi bạn tuần tự hóa một đối tượng, mọi đối tượng mà nó tham chiếu cũng sẽ được tuần tự hóa (nếu những đối tượng này cũng *implement Serializable*), đảm bảo rằng mối quan hệ giữa các đối tượng được bảo toàn.

Nhược điểm của Object Streams:

- Khả năng tương thích phiên bản: Khi bạn thay đổi cấu trúc của lớp (thêm hoặc xóa các trường), có thể gây ra vấn đề khi giải tuần tự hóa đối tượng cũ. Tuy nhiên, Java cung cấp các cơ chế để kiểm soát phiên bản thông qua thuộc tính *serialVersionUID* trong lớp *Serializable*.
- Hiệu suất: Tuần tự hóa và giải tuần tự hóa có thể không nhanh bằng các phương pháp khác (như làm việc với dữ liệu nhị phân thông qua *InputStream* và *OutputStream*). Đặc biệt khi làm việc với đối tượng lớn hoặc phức tạp.

9.3. TỔ CHỨC MÃ NGUỒN THEO MÔ HÌNH HƯỚNG ĐỐI TƯỢNG

9.3.1. Nhóm các lớp

Việc tổ chức mã nguồn theo gói (packages) là rất quan trọng trong phát triển phần mềm, giúp tổ chức và quản lý mã nguồn hiệu quả, đặc biệt trong các dự án phần mềm lớn. Việc sử dụng gói (package) mang lại nhiều lợi ích như hỗ trợ quản lý mã nguồn, tăng tính rõ ràng và khả năng bảo trì hệ thống, tránh xung đột tên lớp, và cải thiện khả năng tái sử dụng mã. Các lớp trong Java được phân loại theo chức năng, theo vai trò hoặc theo tầng (layer) và được tổ chức dựa trên các nguyên tắc như Single Responsibility Principle (SRP) và Separation of Concerns (SoC). Theo chuẩn Java, gói được xây dựng bằng tên miền ngược với cấu trúc phân cấp rõ ràng. Gói là một không gian tên dùng để nhóm các lớp, giao diện và các thành phần liên quan, mang lại lợi ích như tránh xung đột tên, dễ dàng quản lý mã nguồn thông qua cấu trúc thư mục rõ ràng, hỗ trợ bảo mật nhờ các mức truy cập, và thúc đẩy tái sử dụng mã nguồn khi tổ chức hợp lý. Cách tổ chức mã nguồn theo gói mang lại nhiều lợi ích thiết thực:

- **Quản lý mã nguồn hiệu quả:**
 - + Gói giúp tổ chức mã nguồn thành các thư mục có cấu trúc rõ ràng, hỗ trợ việc tìm kiếm, chỉnh sửa và kiểm soát mã trong dự án dễ dàng hơn.
 - + Khi dự án mở rộng, việc phân chia mã thành các gói nhỏ gọn theo chức năng hoặc tầng (layer) sẽ giúp lập trình viên tập trung hơn vào từng phần của hệ thống.

- **Tránh xung đột tên lớp:** Trong các dự án lớn hoặc khi sử dụng thư viện bên ngoài, việc trùng tên lớp là điều thường gặp. Gói cho phép các lớp có cùng tên tồn tại ở các không gian tên khác nhau, giảm thiểu rủi ro xung đột. Việc nhóm các lớp bằng cách sử dụng gói giúp giảm khả năng xung đột tên lớp hoặc các thành phần khác trong mã nguồn. Nếu hai lớp có cùng tên nhưng thuộc về các gói khác nhau, chúng có thể tồn tại song song mà không gây ra lỗi.
- **Tăng tính rõ ràng và khả năng bảo trì:** Khi mã được tổ chức theo cấu trúc gói, việc bảo trì và cập nhật mã trở nên dễ dàng hơn. Các thay đổi chỉ cần được thực hiện trong gói cụ thể mà không làm ảnh hưởng đến toàn bộ hệ thống.
 - + Việc phân loại các lớp theo chức năng, vai trò hoặc tầng giúp hệ thống dễ hiểu và dễ dàng mở rộng.
 - + Áp dụng các nguyên tắc như Single Responsibility Principle (SRP) và Separation of Concerns (SoC) đảm bảo mỗi gói, mỗi lớp chỉ chịu trách nhiệm một nhiệm vụ cụ thể, giúp mã nguồn dễ bảo trì và nâng cấp.
- **Tăng khả năng tái sử dụng mã nguồn:** Các phần mã trong các gói có thể được tái sử dụng trong nhiều ứng dụng khác nhau mà không cần viết lại. Các lớp và gói được tổ chức hợp lý có thể được sử dụng lại trong các dự án khác mà không cần chỉnh sửa nhiều, tiết kiệm thời gian và công sức phát triển.
- **Hỗ trợ bảo mật:** Gói cung cấp cơ chế bảo mật với các mức truy cập (public, protected, private, default) giúp kiểm soát phạm vi truy cập giữa các lớp, cho phép các nhà phát triển xác định mức độ truy cập cho các lớp bên trong và bên ngoài. Điều này giúp bảo vệ thông tin và giảm thiểu lỗi do truy cập không mong muốn.
- **Hỗ Trợ Quá Trình Phát Triển Nhóm:** Trong môi trường phát triển nhóm, việc phân chia mã thành các gói giúp các thành viên trong nhóm có thể làm việc độc lập trên các phần khác nhau của dự án mà không gây ra xung đột.

Phân loại và tổ chức các lớp trong gói dựa trên các tiêu chí cụ thể như:

- **Chức năng:** Các lớp có cùng mục đích (ví dụ: xử lý dữ liệu, giao tiếp với cơ sở dữ liệu) được nhóm vào cùng một gói.
- **Vai trò:** Phân biệt các lớp dựa trên nhiệm vụ chính của chúng, chẳng hạn như lớp điều khiển (Controller), lớp dịch vụ (Service), lớp mô hình dữ liệu (Model).
- **Tầng (Layer):** Gói được sắp xếp dựa trên kiến trúc tầng như Presentation (giao diện), Business Logic (logic nghiệp vụ), và Data Access (truy cập dữ liệu).

Ví dụ 1: tổ chức gói cho một ứng dụng quản lý thư viện:

```
com.library
├── entity    // Các lớp biểu diễn thực thể dữ liệu
│   ├── Book.java
│   ├── User.java
│   └── BorrowCard.java
├── service  // Các lớp xử lý logic nghiệp vụ
│   ├── BookService.java
│   ├── UserService.java
│   └── BorrowService.java
├── repository // Các lớp lưu trữ và truy vấn dữ liệu
│   ├── BookRepository.java
│   ├── UserRepository.java
│   └── BorrowCardRepository.java
├── util     // Các lớp tiện ích dùng chung
│   ├── DateUtil.java
│   └── ValidationUtil.java
├── main     // Điểm bắt đầu của ứng dụng
└── App.java
```

Trong ví dụ trên việc phân loại và tổ chức các gói giúp xác định rõ ràng chức năng của từng gói, dễ dàng bảo trì và nâng cấp từng tầng và tăng khả năng mở rộng của hệ thống (khi thêm một chức năng mới, chỉ cần tạo gói mới).

Trong Java, các gói thường được đặt tên dựa trên tên miền ngược của tổ chức phát triển phần mềm và dựa theo các yếu tố sau:

- **Tên gói (Package Name):** Tên gói thường được viết bằng chữ thường và sử dụng dấu chấm (.) để phân tách các phần. Ví dụ: com.example.project.
- **Cấu trúc thư mục:** Thư mục phải phản ánh cấu trúc tên gói. Ví dụ, gói com.example.project sẽ được tổ chức trong thư mục như sau: com/example/project/.
- **Tệp tin Java:** Các tệp tin mã nguồn Java (.java) sẽ nằm trong thư mục tương ứng với gói chứa chúng.
- **Chỉ định gói:** Để chỉ định gói cho lớp trong tệp tin Java, sử dụng câu lệnh package ở đầu tệp. Ví dụ: **package** com.example.project;
- **Gói mặc định:** Nếu không chỉ định gói, lớp sẽ thuộc về gói mặc định, không khuyến nghị cho các dự án lớn.
- **Thư viện bên ngoài:** Các gói bên ngoài có thể được thêm vào thông qua cấu hình build (như Maven hoặc Gradle).

9.3.2. Áp dụng các nguyên tắc hướng đối tượng

Các nguyên tắc hướng đối tượng (Object-Oriented Programming - OOP) là nền tảng quan trọng trong phát triển phần mềm hiện đại. Việc áp dụng OOP không chỉ giúp hiểu rõ hơn về cách thiết kế và xây dựng hệ thống phần mềm mà còn mang đến khả năng tạo ra mã nguồn chất lượng cao, dễ bảo trì và mở rộng. Mục tiêu chính gồm:

- **Hiểu sâu các nguyên tắc OOP:** sẽ nắm vững và sử dụng hiệu quả các khái niệm cốt lõi như:
 - + **Encapsulation** (Đóng gói): Bảo vệ dữ liệu và đảm bảo tính toàn vẹn của đối tượng.
 - + **Inheritance** (Kế thừa): Tái sử dụng mã nguồn và tổ chức cấu trúc lớp hợp lý.
 - + **Polymorphism** (Đa hình): Tăng tính linh hoạt thông qua việc thay đổi hành vi của các đối tượng.
 - + **Abstraction** (Trừu tượng hóa): Định nghĩa hành vi chung và giảm sự phức tạp khi làm việc với các đối tượng.
- **Tạo hệ thống dễ mở rộng và bảo trì:** Áp dụng OOP để xây dựng các ứng dụng có tính linh hoạt cao, dễ dàng bổ sung tính năng mới mà không làm ảnh hưởng đến phần hiện tại. Đồng thời, mã nguồn trở nên rõ ràng và có thể tái sử dụng trong nhiều bối cảnh khác nhau.
- **Encapsulation** (Đóng gói): giúp bảo vệ dữ liệu bên trong lớp và ngăn chặn sự can thiệp không mong muốn từ bên ngoài. Các thuộc tính của lớp thường được khai báo với mức truy cập *private*, chỉ có thể truy cập thông qua các phương thức *getter* và *setter*.

Ví dụ 2:

```
public class Book {  
    private String title;  
    private boolean available;  
    public String getTitle() {  
        return title;  
    }  
    public void setTitle(String title) {  
        this.title = title;  
    }  
    public boolean isAvailable() {  
        return available;  
    }  
}
```

- **Inheritance (Kế thừa):** cho phép một lớp con kế thừa các thuộc tính và phương thức từ lớp cha. Điều này giúp tái sử dụng mã nguồn và xây dựng các lớp có quan hệ logic với nhau. Điều này sẽ làm giảm trùng lặp mã và cấu trúc tổ chức lớp rõ ràng hơn.

Ví dụ 3:

```

public class User {
    private String name;
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
}

public class Librarian extends User {
    private String employeeId;

    public String getEmployeeId() {
        return employeeId;
    }
    public void setEmployeeId(String employeeId) {
        this.employeeId = employeeId;
    }
}

```

- **Polymorphism (Đa hình):** cho phép các đối tượng thuộc cùng một kiểu cha có thể hành xử khác nhau dựa trên lớp con cụ thể. Điều này đạt được thông qua tính năng ghi đè phương thức (method overriding), giúp tăng tính linh hoạt và khả năng mở rộng của hệ thống và xử lý các đối tượng theo cách thống nhất nhưng vẫn đảm bảo tính đặc thù.

Ví dụ 4:

```

public class User {
    public void showRole() {
        System.out.println("Generic User");
    }
}

public class Librarian extends User {
    @Override
    public void showRole() {
        System.out.println("Librarian");
    }
}

```

```
}
```

- **Abstraction (Trừu tượng hóa):** tập trung vào việc định nghĩa các hành vi chung mà nhiều lớp con có thể triển khai. Điều này được thực hiện thông qua **giao diện (interface)** hoặc **lớp trừu tượng (abstract class)**, làm giảm sự phụ thuộc giữa các thành phần và cung cấp một khung làm việc rõ ràng cho các lớp triển khai.

Ví dụ 5:

```
public class User {  
    public void showRole() {  
        System.out.println("Generic User");  
    }  
}
```

```
public class Librarian extends User {  
    @Override  
    public void showRole() {  
        System.out.println("Librarian");  
    }  
}
```

Các nguyên tắc hướng đối tượng (OOP) – Encapsulation, Inheritance, Polymorphism, và Abstraction – là những thành phần cốt lõi giúp lập trình viên xây dựng hệ thống phần mềm chất lượng cao. Chúng không chỉ là nền tảng lý thuyết mà còn mang lại nhiều lợi ích thực tế, từ việc cải thiện cấu trúc mã nguồn đến tối ưu hóa quá trình phát triển và bảo trì hệ thống. Việc áp dụng hiệu quả các nguyên tắc OOP giúp lập trình viên tạo ra mã nguồn dễ đọc, dễ bảo trì, đồng thời nâng cao tính tái sử dụng và linh hoạt. Điều này đặc biệt quan trọng trong các dự án phần mềm lớn, nơi yêu cầu về mở rộng và bảo trì luôn tồn tại. Thông qua việc thực hành và hiểu rõ các nguyên tắc OOP, sinh viên và lập trình viên sẽ có nền tảng vững chắc để phát triển các ứng dụng phần mềm hiện đại, đáp ứng tốt nhu cầu công việc và thị trường.

9.3.3. Sử dụng JavaDoc

JavaDoc là một công cụ quan trọng trong hệ sinh thái Java, cho phép lập trình viên tài liệu hóa mã nguồn một cách chi tiết và chuyên nghiệp. Với mục tiêu chính là giúp người đọc mã dễ hiểu, dễ bảo trì và tái sử dụng mã nguồn, JavaDoc đã trở thành một phần không thể thiếu trong quy trình phát triển phần mềm. Công cụ này không chỉ giúp các nhà phát triển hiện tại mà còn hỗ trợ cho các đội nhóm phát triển trong tương lai, tạo ra một bộ tài liệu tham chiếu rõ ràng và dễ truy cập.

Trong lĩnh vực phát triển phần mềm, tài liệu hóa mã nguồn đóng vai trò cực kỳ quan trọng. Khi dự án trở nên phức tạp, việc hiểu rõ cấu trúc và chức năng của từng phần trong mã nguồn là một thách thức lớn, đặc biệt khi có sự tham gia của nhiều lập trình viên. Việc ghi chú và tài liệu hóa rõ ràng giúp:

- **Giảm chi phí bảo trì mã nguồn:** Một mã nguồn được tài liệu hóa tốt sẽ giúp tiết kiệm thời gian khi bảo trì hoặc mở rộng.
- **Tăng tính minh bạch và rõ ràng:** Người đọc mã dễ dàng hiểu được ý nghĩa của từng thành phần trong hệ thống.
- **Hỗ trợ cộng tác hiệu quả:** Trong các dự án nhóm, tài liệu hóa giúp các thành viên phối hợp nhịp nhàng hơn.

JavaDoc hỗ trợ ghi chú trực tiếp trong mã nguồn, giúp mô tả chi tiết về từng lớp, phương thức, biến, và các tham số đi kèm. Đây là một cách tiếp cận hiệu quả để kết hợp tài liệu và mã trong cùng một nơi.

JavaDoc sử dụng cú pháp `/** */`, cho phép lập trình viên ghi chú về ý nghĩa và chức năng của các thành phần trong mã nguồn.

- **Ghi chú cho lớp:** JavaDoc cho phép bạn mô tả mục đích của một lớp hoặc giao diện, giúp người dùng nhanh chóng hiểu được vai trò của lớp trong hệ thống:

```
/**
 * Class đại diện cho một quyển sách trong thư viện.
 * Chứa thông tin cơ bản như tên sách và các phương thức quản lý.
 */
public class Book {
    // Nội dung lớp
}
```

- **Ghi chú cho thuộc tính:** Các thuộc tính (variables) cũng có thể được ghi chú để giải thích vai trò của chúng:

```
/**
 * Tên sách.
 */
private String title;
```

- **Ghi chú cho phương thức:** Phương thức được ghi chú rõ ràng với mô tả chức năng, giá trị trả về (nếu có), và các tham số đầu vào:

```
/**
 * Lấy tên sách.
 *
 * @return tên sách
 */
public String getTitle() {
    return title;
}
```

```

/**
 * Đặt tên sách.
 *
 * @param title tên sách
 */
public void setTitle(String title) {
    this.title = title;
}

```

- **Sử dụng các thẻ JavaDoc phổ biến:** JavaDoc hỗ trợ nhiều thẻ để mô tả chi tiết các thành phần, chi tiết có được trình bày trong bảng 9.8.

Bảng 9.8 Các thẻ JavaDoc phổ biến

Thẻ	Miêu tả	Cú pháp
@author	Thêm author của một lớp	@author name-text
{@code}	Hiển thị text trong code font mà không thông dịch text như là các thẻ HTML markup hoặc javadoc được lồng.	{@code text}
{@docRoot}	Biểu diễn relative path tới thư mục root của tài liệu từ các trang được tạo	{@docRoot}
@deprecated	Thêm một comment chỉ dẫn rằng API này không còn được sử dụng nữa	@deprecated deprecated-text
@exception	Thêm một Throws subheading tới documentation được tạo, với tên lớp và text mô tả	@exception class-name Miêu tả
{@inheritDoc}	Kế thừa một comment từ lớp có thể kế thừa gần nhất hoặc interface có thể triển khai	Kế thừa một comment từ một lớp cha trung gian.
{@link}	Chèn một in-line link với nhãn text có thể nhìn thấy mà chỉ tới Documentation cho package, lớp hoặc tên member đã xác định của một lớp tham chiếu	{@link package.class#member label}
{@linkplain}	Giống {@link}, ngoại trừ label của link được hiển thị ở dạng plain text thay vì code font.	{@linkplain package.class#member label}

Thẻ	Miêu tả	Cú pháp
@param	Thêm một tham số với parameter-name đã xác định được theo sau bởi description đã xác định tới khu vực "Parameters".	@param parameter-name Miêu tả
@return	Thêm một khu vực "Returns" với Miêu tả	@return Miêu tả
@see	Thêm một "See Also" heading với một link hoặc text mà chỉ tới reference	@see reference
@serial	Được sử dụng trong doc comment cho một trường có thể xếp thứ tự.	@serial field-description include exclude
@serialData	Thu thập thông tin dữ liệu được viết bởi phương thức writeObject() hoặc writeExternal()	@serialData data-Miêu tả
@serialField	Thu thập thông tin một thành phần ObjectOutputStreamField	@serialField field-name field-type field-Miêu tả
@since	Thêm một "Since" heading với since-text đã xác định tới Documentation đã tạo	@since release
@throws	Thẻ @throws và @exception là như nhau	@throws class-name Miêu tả
{@value}	Khi {@value} được sử dụng trong doc comment của một trường static, nó hiển thị giá trị của hằng số đó	{@value package.class#field}
@version	Thêm một "Version" subheading với version-text đã xác định tới Documentation đã tạo khi tùy chọn -version được sử dụng	@version version-text

Ví dụ 6: biểu diễn Java comment:

```
/**
 * <h1>Hello, World!</h1>
 * Chương trình HelloWorld đơn giản sau triển khai một ứng dụng mà
 * hiển thị "Hello World!" tới standard output.
 * <p>
 * Việc cung cấp comment thích hợp trong chương trình giúp nó trở nên
 * thân thiện với người dùng hơn.
 *
 * @author CNTT&TT
```

```

    * @version 1.0
    * @since 15-12-2024
    */

    public class HelloWorld {
        public static void main(String[] args) {
            /**
             * In dòng chu Hello, World! tren standard output.
             */
            System.out.println("Hello World!");
        }
    }

```

Tạo tài liệu JavaDoc: JavaDoc không chỉ dừng lại ở việc ghi chú trong mã nguồn, mà còn cung cấp khả năng tự động tạo ra tài liệu tham chiếu ở định dạng HTML. Điều này rất hữu ích khi cần chia sẻ tài liệu với đồng nghiệp hoặc tạo hướng dẫn sử dụng API.

Để tạo tài liệu JavaDoc, ta sử dụng lệnh sau trong terminal hoặc command prompt:

```
javadoc -d doc -sourcepath src com.library
```

Trong đó:

- **-d doc:** Chỉ định thư mục đầu ra (output directory) cho các tệp tài liệu.
- **-sourcepath src:** Chỉ định đường dẫn tới mã nguồn.
- **com.library:** Chỉ định gói (package) cần tạo tài liệu.

Sau khi chạy lệnh, một tập hợp các tệp HTML sẽ được tạo ra trong thư mục đầu ra. Các tệp này chứa thông tin chi tiết về từng lớp, phương thức, và tham số, với giao diện thân thiện và dễ tra cứu.

Class HelloWorld

java.lang.Object[Ⓔ]
HelloWorld

```
public class HelloWorld  
    extends ObjectⒺ
```

Hello, World!

Chương trình HelloWorld đơn giản sau triển khai một ứng dụng mà hiển thị "Hello World!" tới standard output.

Việc cung cấp comment thích hợp trong chương trình giúp nó trở nên thân thiện với người dùng hơn.

Since:

15-12-2024

Version:

1.0

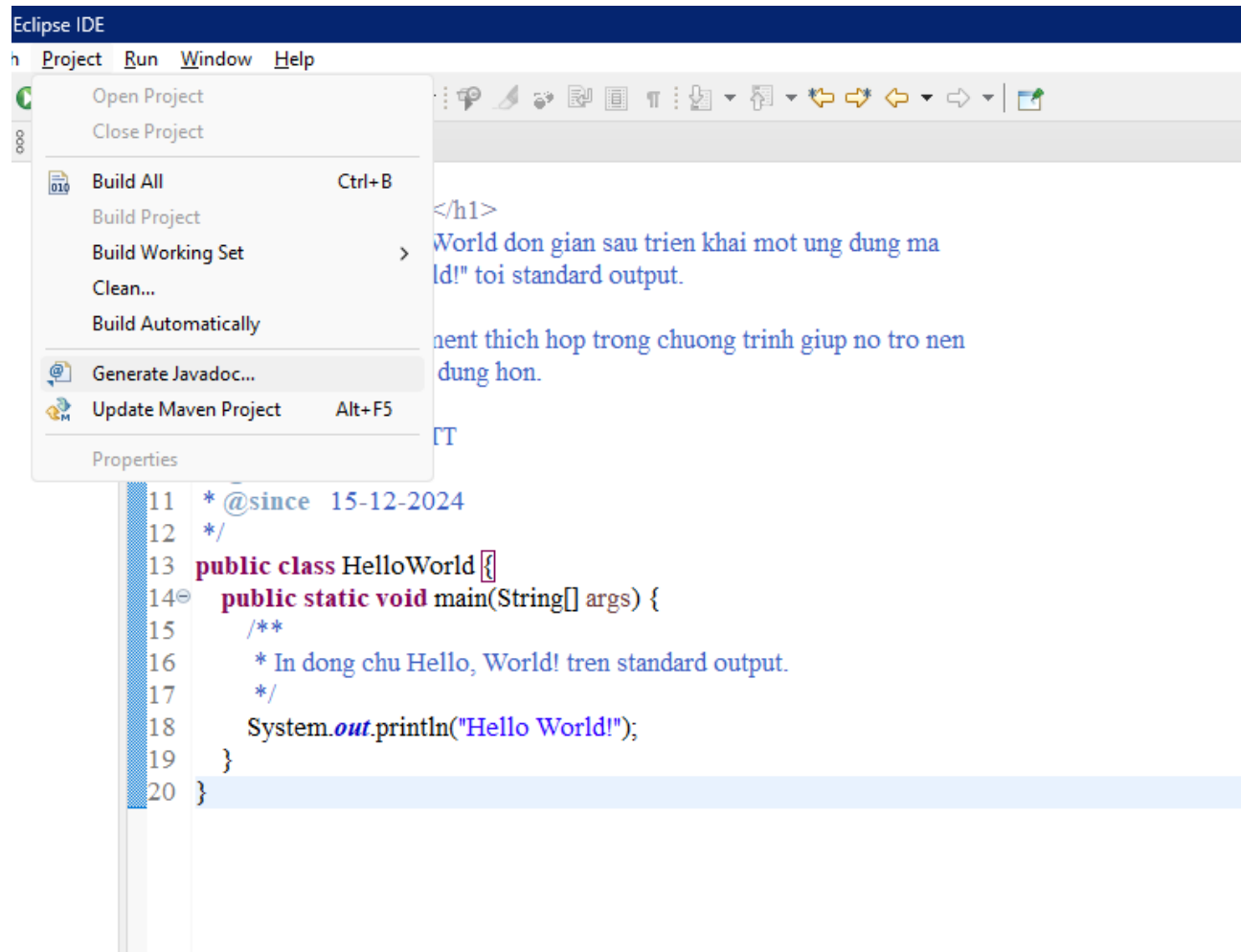
Author:

CNTT&TT

Hình 9.5 Ví dụ minh họa về sử dụng JavaDoc

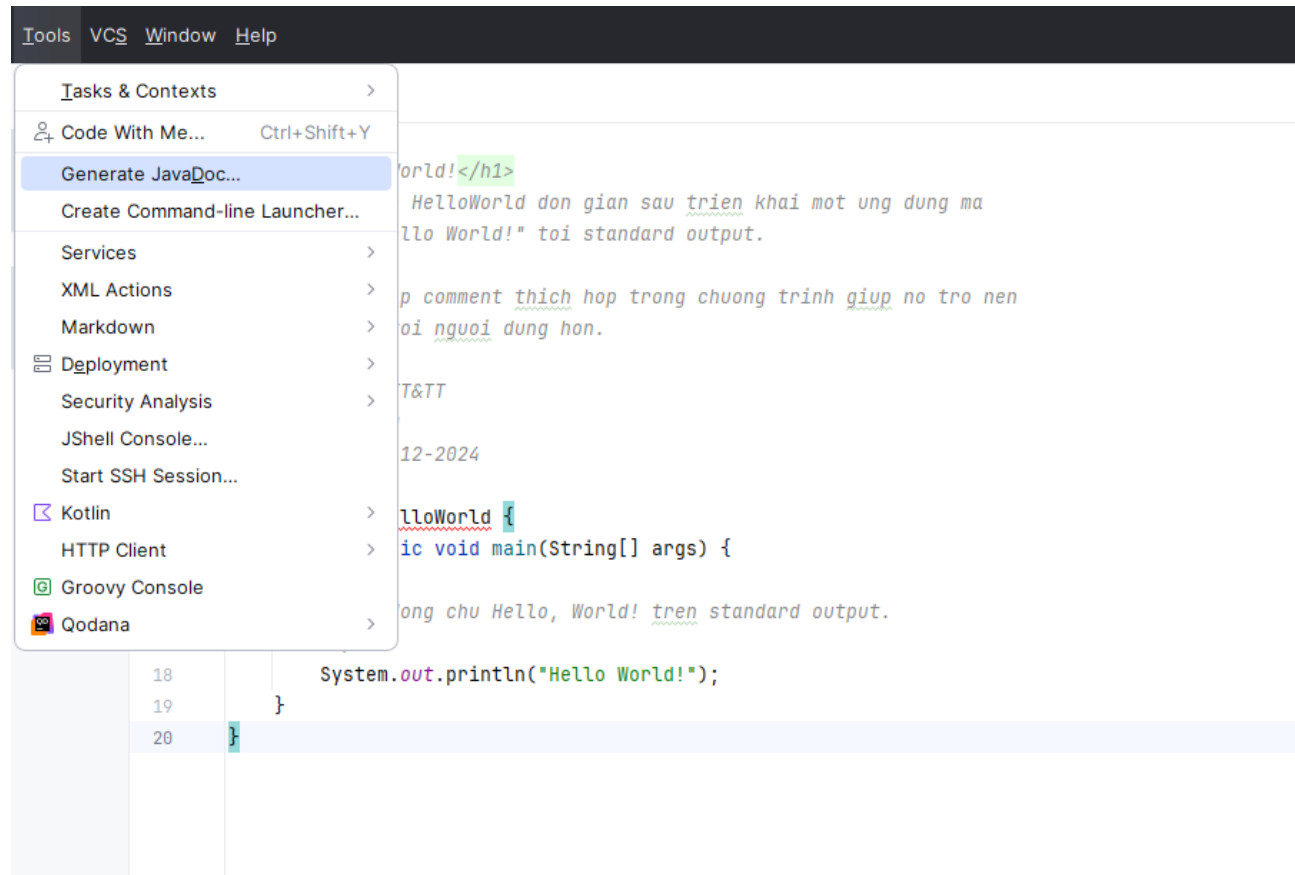
Hầu hết các IDE như IntelliJ IDEA, Eclipse, hoặc NetBeans đều hỗ trợ tạo JavaDoc trực tiếp từ giao diện. Bạn chỉ cần chọn tùy chọn "Generate JavaDoc" và cấu hình các tham số cần thiết.

Với Eclipse: Với project đang mở và có một vài *Doc Comment* đã được định nghĩa trong source code. Chọn theo menu **Project > Generate Javadoc....**



Hình 9.6 Tạo Javadoc với Eclipse IDE

Với **IntelliJ**: Với project đang mở và có một vài **Doc Comment** đã được định nghĩa trong source code. Chọn theo menu **Tools > Generate JavaDoc....**



Hình 9.7 Tạo JavaDoc với IntelliJ IDE

Lợi ích của việc sử dụng JavaDoc:

- **Tăng tính chuyên nghiệp cho dự án:** JavaDoc giúp dự án chuyên nghiệp hơn, đặc biệt khi chia sẻ với khách hàng hoặc cộng đồng lập trình. Một dự án có tài liệu rõ ràng sẽ dễ dàng thu hút sự quan tâm và hợp tác từ các lập trình viên khác.
- **Dễ dàng tra cứu và sử dụng:** Tài liệu JavaDoc giúp lập trình viên nhanh chóng tìm thấy thông tin cần thiết về các lớp và phương thức, mà không cần phải đọc toàn bộ mã nguồn. Điều này đặc biệt hữu ích khi làm việc với các API hoặc thư viện lớn.
- **Hỗ trợ bảo trì và phát triển:** Mã nguồn được tài liệu hóa tốt giúp giảm thời gian và công sức cần thiết khi bảo trì hoặc mở rộng dự án. Lập trình viên mới tham gia dự án cũng có thể nhanh chóng nắm bắt hệ thống.
- **Chuẩn hóa tài liệu:** JavaDoc cung cấp một tiêu chuẩn nhất quán cho việc ghi chú, giúp tất cả các thành viên trong nhóm tuân thủ cùng một phong cách tài liệu hóa.

JavaDoc không chỉ là một công cụ mà còn là một phương pháp thực hành tốt trong lập trình. Nó giúp tài liệu hóa mã nguồn rõ ràng, hỗ trợ bảo trì dễ dàng và tạo ra một môi trường làm việc chuyên nghiệp. Bằng cách sử dụng JavaDoc, bạn không chỉ cải thiện chất lượng mã nguồn mà còn

nâng cao khả năng cộng tác và tái sử dụng trong dự án của mình. Việc đầu tư thời gian để viết tài liệu JavaDoc ngay từ đầu sẽ mang lại lợi ích lâu dài cho toàn bộ vòng đời của phần mềm.

9.4. CASE STUDY

Case Study 1: Quản lý thư viện sách

Mục tiêu: Áp dụng các cấu trúc dữ liệu đặc biệt trong Java.

Mô tả chi tiết: Một thư viện quản lý các đầu sách dựa trên các tiêu chí: mã sách, tên sách, tác giả, và năm xuất bản.

Chức năng yêu cầu:

- **Thêm, sửa, xóa sách:**
 - + Người dùng nhập thông tin sách (mã, tên, tác giả, năm xuất bản) để thêm vào danh sách.
 - + Sửa thông tin sách dựa trên mã sách.
 - + Xóa sách khỏi danh sách.
- **Tìm kiếm sách:**
 - + Tìm kiếm sách theo mã sử dụng HashMap.
 - + Tìm kiếm sách theo tên hoặc tác giả bằng cách duyệt qua ArrayList.
- **Sắp xếp sách:**
 - + Sắp xếp danh sách sách theo tên hoặc năm xuất bản bằng TreeSet.
- **Quản lý lịch sử mượn/trả sách:**
 - + Lưu danh sách sách mượn sử dụng Stack.
 - + Xếp hàng trả sách theo thứ tự nhập hàng chờ sử dụng Queue.

Hướng dẫn triển khai:

- Sử dụng ArrayList để quản lý danh sách sách chính.
- Dùng HashMap với key là mã sách, value là đối tượng sách để tăng tốc độ tìm kiếm.
- Kết hợp TreeSet với Comparator để sắp xếp sách theo yêu cầu.

Case Study 2: Quản lý file nhân viên

Mục tiêu: Thực hành các luồng vào/ra dữ liệu trong Java.

Mô tả chi tiết: Một chương trình quản lý danh sách nhân viên của công ty, mỗi nhân viên có các thông tin: mã, tên, vị trí và lương.

Chức năng yêu cầu:

- **Nhập dữ liệu từ file:**
 - + Đọc dữ liệu nhân viên từ file văn bản bằng Character Streams. Mỗi dòng trong file chứa thông tin một nhân viên được phân cách bởi dấu ",".
- **Ghi dữ liệu ra file nhị phân:**
 - + Ghi danh sách nhân viên vào file nhị phân bằng Object Streams.

- **Sao chép file báo cáo:**
 - + Sử dụng InputStream và OutputStream để sao chép file báo cáo từ nguồn này sang nguồn khác.
- **Hiển thị danh sách nhân viên:**
 - + Đọc dữ liệu từ file nhị phân và hiển thị thông tin chi tiết ra màn hình.

Hướng dẫn triển khai:

- Tạo lớp Employee với các thuộc tính mã, tên, vị trí, và lương.
- Triển khai các phương thức ghi/đọc dữ liệu với BufferedReader, BufferedWriter, và ObjectOutputStream.

Case Study 3: Quản lý hệ thống điểm sinh viên

Mục tiêu: Tổ chức mã nguồn theo mô hình hướng đối tượng và áp dụng các nguyên tắc OOP.

Mô tả chi tiết: Hệ thống quản lý điểm sinh viên gồm các thông tin: sinh viên, môn học và điểm số.

Yêu cầu thực hiện:

- **Tổ chức các lớp:**
 - + **Student:** Mã, tên, lớp.
 - + **Subject:** Mã môn, tên môn, số tín chỉ.
 - + **Grade:** Mã sinh viên, mã môn, điểm thi.
- **Chức năng quản lý:**
 - + Thêm, sửa, xóa sinh viên, môn học, và điểm.
 - + Tính điểm trung bình của từng sinh viên.
 - + Tìm sinh viên có điểm cao nhất trong từng môn học.
- **JavaDoc:**
 - + Viết tài liệu cho từng lớp và phương thức.

Hướng dẫn triển khai:

- Sử dụng các nguyên tắc SOLID: tách rời logic tính toán và quản lý dữ liệu.
- Viết JavaDoc với mô tả chi tiết và ví dụ minh họa.

Case Study 4: Quản lý đơn hàng trong hệ thống thương mại điện tử

Mục tiêu: Áp dụng các cấu trúc dữ liệu kết hợp và xử lý luồng dữ liệu.

Mô tả chi tiết: Một hệ thống thương mại điện tử quản lý các đơn hàng gồm: mã đơn hàng, tên khách hàng, danh sách sản phẩm, và tổng tiền.

Chức năng yêu cầu:

- **Quản lý danh sách sản phẩm:**
 - + Dùng LinkedList để lưu trữ các sản phẩm trong mỗi đơn hàng.
- **Quản lý đơn hàng:**

- + Dùng HashMap với mã đơn hàng làm key, và thông tin đơn hàng làm value.
- **Lịch sử giao dịch:**
 - + Lưu thông tin đơn hàng vào file nhị phân.
 - + Đọc và hiển thị danh sách đơn hàng từ file.

Chức năng mở rộng:

- Tìm kiếm đơn hàng theo mã.
- Tính tổng doanh thu trong ngày.
- Hiển thị sản phẩm bán chạy nhất.

Hướng dẫn triển khai:

- Xây dựng các lớp Order, Product, và Customer.
- Sử dụng các cấu trúc dữ liệu phù hợp để quản lý và xử lý dữ liệu hiệu quả.

Case Study 5: Ứng dụng quản lý dự án

Mục tiêu: Kết hợp tổ chức mã nguồn, cấu trúc dữ liệu, và xử lý file.

Mô tả chi tiết: Một công ty phần mềm cần quản lý các dự án, mỗi dự án có: mã dự án, tên dự án, danh sách nhân viên tham gia, và tiến độ hoàn thành.

Chức năng yêu cầu:

- **Tổ chức các lớp:**
 - + **Project:** Mã dự án, tên dự án, tiến độ hoàn thành.
 - + **Employee:** Mã nhân viên, tên, vị trí.
- **Chức năng quản lý:**
 - + Thêm/xóa/sửa dự án.
 - + Thêm/xóa nhân viên vào dự án.
 - + Tính số lượng nhân viên tham gia từng dự án.
- **Xử lý file JSON:**
 - + Ghi thông tin dự án và nhân viên tham gia vào file JSON.
 - + Đọc và hiển thị dữ liệu từ file JSON.

Chức năng mở rộng:

- Tìm kiếm dự án có tiến độ cao nhất.
- Xuất báo cáo dự án thành file văn bản.

Hướng dẫn triển khai:

- Sử dụng thư viện JSON như Gson hoặc Jackson để ghi/đọc file JSON.
- Tổ chức mã nguồn rõ ràng và sử dụng các cấu trúc dữ liệu phù hợp.

9.5. VÍ DỤ ÁP DỤNG

Áp dụng với **Case Study 3: Quản lý hệ thống điểm sinh viên** ta tổ chức mã nguồn theo cấu trúc thư mục sau:


```

com.studentmanagement
├── entity    // Các lớp biểu diễn thực thể dữ liệu
│   ├── Student.java
│   ├── Subject.java
│   └── Grade.java
├── service  // Các lớp xử lý logic nghiệp vụ
│   ├── StudentService.java
│   ├── SubjectService.java
│   └── GradeService.java
├── repository // Các lớp lưu trữ và truy vấn dữ liệu
│   ├── StudentRepository.java
│   ├── SubjectRepository.java
│   └── GradeRepository.java
├── util     // Các lớp tiện ích dùng chung
│   ├── MathUtil.java
│   └── ValidationUtil.java
├── main     // Điểm bắt đầu của ứng dụng
└── App.java

```

Từ cách tổ chức mã nguồn theo cây trên ta xây dựng các lớp tương ứng và lưu trữ mã nguồn ở các vị trí gói tương ứng với cây trên.

Ví dụ xây dựng lớp **Student**: để quản lý thông tin của sinh viên và lưu mã nguồn ở **entity/Student.java**

```

package com.studentmanagement.entity;

/**
 * Class đại diện cho thông tin của một sinh viên.
 */

public class Student {
    private String studentId;
    private String name;
    private String className;

    /**
     * Constructor để khởi tạo đối tượng Student.
     * @param studentId Mã sinh viên.
     * @param name Tên sinh viên.

```

```

    * @param className Lớp sinh viên.
    */
    public Student(String studentId, String name, String className) {
        this.studentId = studentId;
        this.name = name;
        this.className = className;
    }
    // Getters và Setters
    public String getStudentId() {
        return studentId;
    }
    public void setStudentId(String studentId) {
        this.studentId = studentId;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getClassName() {
        return className;
    }
    public void setClassName(String className) {
        this.className = className;
    }
    @Override
    public String toString() {
        return "Student{" +
            "studentId=" + studentId + "\" +
            ", name=" + name + "\" +
            ", className=" + className + "\" +
            '}'";
    }
}

```

```
}
```

Ví dụ xây dựng lớp **StudentService** để xử lý nghiệp vụ liên quan đến sinh viên và lưu theo cấu trúc **service/StudentService.java**

```
package com.studentmanagement.service;
import com.studentmanagement.entity.Student;
import java.util.*;

/**
 * Lớp xử lý nghiệp vụ liên quan đến sinh viên.
 */

public class StudentService {
    private List<Student> students = new ArrayList<>();

    /**
     * Thêm sinh viên vào danh sách.
     * @param student Đối tượng sinh viên cần thêm.
     */

    public void addStudent(Student student) {
        students.add(student);
    }

    /**
     * Hiển thị danh sách sinh viên.
     */

    public void displayStudents() {
        for (Student student : students) {
            System.out.println(student);
        }
    }
}
```

Các lớp còn lại có thể triển khai tương tự, phân chia rõ ràng các chức năng và tuân thủ nguyên tắc tổ chức mã nguồn hướng đối tượng.

9.6. TỔNG KẾT CHƯƠNG

Trong chương này, chúng ta đã học về:

1) Một số cấu trúc dữ liệu đặc biệt trong Java

Phần này giới thiệu các cấu trúc dữ liệu quan trọng trong Java và ứng dụng:

- **ArrayList**: Là một danh sách động, hỗ trợ truy cập nhanh theo chỉ mục và cho phép thay đổi kích thước linh hoạt.
- **LinkedList**: Cấu trúc danh sách liên kết, phù hợp với các thao tác chèn, xóa phần tử thường xuyên.
- **HashMap**: Một cấu trúc ánh xạ khóa-giá trị, tối ưu cho việc tra cứu dữ liệu nhờ cơ chế hash.
- **TreeSet**: Cung cấp tập hợp các phần tử không trùng lặp, được sắp xếp theo thứ tự tự nhiên hoặc theo comparator tùy chỉnh.
- **Stack và Queue**: Hai cấu trúc dữ liệu cổ điển; Stack hoạt động theo nguyên tắc LIFO (Last In, First Out), trong khi Queue tuân theo FIFO (First In, First Out).
- **Lớp Collection**: Đóng vai trò như một công cụ hỗ trợ quản lý và thao tác trên các cấu trúc dữ liệu khác nhau, cung cấp các phương thức hữu ích như sắp xếp, tìm kiếm, và chuyển đổi.

2) Các luồng vào/ra dữ liệu trong Java

Java cung cấp các cơ chế mạnh mẽ để xử lý dữ liệu vào/ra, được chia thành nhiều loại luồng:

- **Khái niệm**: Các luồng trong Java giúp quản lý dữ liệu từ các nguồn khác nhau, như tệp, bộ nhớ, hoặc mạng.
- **Input Streams**: Luồng nhập được dùng để đọc dữ liệu dưới dạng byte, ví dụ FileInputStream.
- **Output Streams**: Luồng xuất dùng để ghi dữ liệu dưới dạng byte, như FileOutputStream.
- **Character Streams**: Xử lý dữ liệu dưới dạng ký tự, tiện lợi hơn khi làm việc với văn bản, ví dụ FileReader và FileWriter.
- **Object Streams**: Được dùng để đọc/ghi các đối tượng, hỗ trợ việc tuần tự hóa và phi tuần tự hóa dữ liệu.

3) Tổ chức mã nguồn theo mô hình hướng đối tượng

Phần cuối của chương tập trung vào việc áp dụng các nguyên tắc hướng đối tượng để tổ chức mã nguồn hiệu quả hơn:

- **Nhóm các lớp**: Hướng dẫn cách tổ chức các lớp thành nhóm hoặc gói (package) để dễ quản lý và tái sử dụng.
- **Áp dụng các nguyên tắc hướng đối tượng**: Nhấn mạnh các nguyên tắc như đóng gói, kế thừa, đa hình và giao diện trong thiết kế phần mềm.
- **Sử dụng JavaDoc**: Giới thiệu cách sử dụng JavaDoc để tạo tài liệu API tự động, giúp cải thiện tính dễ hiểu và khả năng bảo trì mã nguồn.

9.7. CÂU HỎI TRẮC NGHIỆM

1. ArrayList trong Java là gì?

- Một cấu trúc dữ liệu tĩnh.
- Một danh sách có kích thước thay đổi.
- Một bảng băm.
- Một cây nhị phân.

2. **Cấu trúc dữ liệu nào trong Java hỗ trợ truy cập phần tử theo chỉ mục?**
 - A. LinkedList.
 - B. HashMap.
 - C. ArrayList.
 - D. TreeSet.
3. **Cấu trúc dữ liệu nào trong Java là danh sách liên kết?**
 - A. ArrayList.
 - B. LinkedList.
 - C. HashSet.
 - D. Stack.
4. **HashMap trong Java sử dụng phương pháp nào để lưu trữ các phần tử?**
 - A. Chia đôi danh sách.
 - B. Ánh xạ khóa và giá trị.
 - C. Sắp xếp phần tử theo thứ tự.
 - D. Liên kết các phần tử theo dạng cây.
5. **TreeSet trong Java lưu trữ các phần tử theo cách nào?**
 - A. Không sắp xếp.
 - B. Sắp xếp theo thứ tự tự nhiên hoặc comparator.
 - C. Ánh xạ khóa và giá trị.
 - D. Sắp xếp theo giá trị hash.
6. **Cấu trúc dữ liệu nào trong Java tuân theo nguyên tắc LIFO (Last In, First Out)?**
 - A. Queue.
 - B. Stack.
 - C. TreeSet.
 - D. LinkedList.
7. **Cấu trúc dữ liệu nào trong Java tuân theo nguyên tắc FIFO (First In, First Out)?**
 - A. Queue.
 - B. Stack.
 - C. HashMap.
 - D. TreeSet.
8. **Lớp Collection trong Java có chức năng gì?**
 - A. Cung cấp các phương thức để xử lý dữ liệu kiểu số.
 - B. Hỗ trợ quản lý và thao tác với các cấu trúc dữ liệu như List, Set.
 - C. Xử lý các luồng vào/ra.

- D. Tạo tài liệu API tự động.
- 9. Trong Java, luồng vào (Input Stream) dùng để làm gì?**
- A. Đọc dữ liệu từ một nguồn.
 - B. Ghi dữ liệu vào một tệp tin.
 - C. Xử lý các đối tượng Java.
 - D. Cả A và B.
- 10. Luồng Output Stream trong Java được sử dụng để làm gì?**
- A. Đọc dữ liệu từ nguồn.
 - B. Ghi dữ liệu vào đích.
 - C. Sử dụng ký tự để lưu trữ dữ liệu.
 - D. Xử lý dữ liệu nhị phân.
- 11. Character Streams trong Java xử lý dữ liệu như thế nào?**
- A. Dữ liệu dưới dạng byte.
 - B. Dữ liệu dưới dạng ký tự.
 - C. Dữ liệu đối tượng Java.
 - D. Dữ liệu trong tệp nhị phân.
- 12. Object Streams trong Java cho phép làm gì?**
- A. Đọc và ghi các đối tượng Java.
 - B. Chuyển đổi dữ liệu từ byte sang ký tự.
 - C. Tạo các luồng xử lý dữ liệu nhị phân.
 - D. Đọc các dòng văn bản từ tệp.
- 13. Trong Java, phương thức nào được sử dụng để đọc một byte từ InputStream?**
- A. read().
 - B. write().
 - C. next().
 - D. get().
- 14. Khi sử dụng JavaDoc, mục đích chính là gì?**
- A. Tạo giao diện người dùng.
 - B. Tạo tài liệu API tự động từ mã nguồn.
 - C. Xử lý các luồng vào/ra.
 - D. Tạo lớp đối tượng.
- 15. Để nhóm các lớp lại với nhau trong Java, ta sử dụng gì?**
- A. Package.
 - B. Interface.

- C. Class.
 - D. Method.
- 16. Nguyên tắc nào trong hướng đối tượng liên quan đến việc ẩn thông tin và chỉ cung cấp các phương thức truy cập?**
- A. Kế thừa.
 - B. Đóng gói
 - C. Đa hình.
 - D. Trừu tượng.
- 17. Trong Java, nguyên tắc nào cho phép một lớp kế thừa các thuộc tính và phương thức của lớp khác?**
- A. Đóng gói.
 - B. Kế thừa.
 - C. Đa hình.
 - D. Trừu tượng.
- 18. Lớp nào trong Java có thể triển khai một hoặc nhiều giao diện (interfaces)?**
- A. Class.
 - B. Abstract Class.
 - C. Interface.
 - D. Cả A and B.
- 19. Phương thức nào trong Java được sử dụng để ghi một byte vào OutputStream?**
- A. read().
 - B. write().
 - C. append().
 - D. push().
- 20. Câu nào sau đây là đúng về ArrayList trong Java?**
- A. ArrayList có thể thay đổi kích thước động.
 - B. ArrayList chỉ lưu trữ các phần tử duy nhất.
 - C. ArrayList không thể chứa null.
 - D. ArrayList luôn sắp xếp các phần tử.
- 21. LinkedList trong Java sử dụng cấu trúc dữ liệu gì để lưu trữ các phần tử?**
- A. Mảng.
 - B. Danh sách liên kết.
 - C. Bảng băm.
 - D. Cây nhị phân.

22. Để duy trì thứ tự của các phần tử trong TreeSet, Java sử dụng phương pháp gì?

- A. Sắp xếp tự động theo thứ tự tự nhiên hoặc comparator.
- B. Ánh xạ khóa và giá trị.
- C. Lưu trữ theo kiểu cây nhị phân.
- D. Lưu trữ theo thứ tự nhập vào.

23. Java hỗ trợ xử lý tệp tin nào dưới dạng byte?

- A. FileInputStream và FileOutputStream.
- B. FileReader và FileWriter.
- C. BufferedReader và BufferedWriter.
- D. ObjectInputStream và ObjectOutputStream.

24. Khái niệm nào không phải là một thành phần của JavaDoc?

- A. Thêm mô tả cho lớp và phương thức.
- B. Tự động tạo tài liệu API từ mã nguồn.
- C. Xử lý dữ liệu nhị phân trong tệp.
- D. Ghi chú cho các tham số và giá trị trả về.

25. Stack trong Java có đặc điểm gì?

- A. Dữ liệu được lưu trữ theo thứ tự nhập vào.
- B. Dữ liệu được truy xuất theo chỉ mục.
- C. Dữ liệu được truy xuất theo nguyên tắc LIFO.
- D. Dữ liệu được lưu trữ theo khóa và giá trị.

26. Khi nào bạn sử dụng Object Streams trong Java?

- A. Khi bạn cần đọc và ghi các đối tượng Java.
- B. Khi làm việc với các tệp văn bản.
- C. Khi cần xử lý các luồng dữ liệu nhị phân.
- D. Khi bạn muốn lưu trữ dữ liệu trong bộ nhớ.

27. Java sử dụng cấu trúc dữ liệu nào để tổ chức các đối tượng vào các gói?

- A. Class.
- B. Interface.
- C. Package.
- D. Method.

28. Java hỗ trợ cơ chế nào để đảm bảo tính an toàn trong xử lý luồng vào/ra?

- A. Kết nối mạng.
- B. Đồng bộ hóa.
- C. Giao tiếp qua giao diện.

D. Ánh xạ khóa và giá trị.

29. Lớp nào trong Java cho phép bạn thao tác với các đối tượng có thể thay đổi kích thước?

- A. ArrayList.
- B. HashMap.
- C. TreeSet.
- D. Stack.

30. Lệnh nào được sử dụng để tạo tài liệu API tự động từ mã nguồn?

- A. javadoc.
- B. writeDoc().
- C. generateDoc().
- D. createAPI().

9.8. BÀI TẬP THỰC HÀNH

Bài tập 1: Sử dụng ArrayList

Mô tả: Viết một chương trình sử dụng ArrayList để lưu trữ và thao tác với một danh sách các số nguyên. Hãy thực hiện các thao tác sau:

- Thêm 5 phần tử vào ArrayList.
- In ra danh sách các phần tử.
- Xóa phần tử đầu tiên và in lại danh sách.
- Sắp xếp danh sách theo thứ tự tăng dần và in ra.

Bài tập 2: Sử dụng LinkedList

Mô tả: Viết một chương trình sử dụng LinkedList để thực hiện các thao tác sau:

- Thêm 5 phần tử vào LinkedList.
- In ra danh sách các phần tử.
- Chèn một phần tử mới ở vị trí thứ 2 và in lại danh sách.
- Xóa phần tử cuối cùng và in lại danh sách.

Bài tập 3: Sử dụng HashMap

Mô tả: Viết một chương trình sử dụng HashMap để lưu trữ thông tin sinh viên (mã sinh viên, tên sinh viên) và thực hiện các thao tác sau:

- Thêm 3 sinh viên vào HashMap.
- In ra thông tin tất cả sinh viên.
- Tìm kiếm một sinh viên theo mã sinh viên và in ra thông tin.
- Xóa một sinh viên và in lại toàn bộ thông tin.

Bài tập 4: Sử dụng TreeSet

Mô tả: Viết một chương trình sử dụng TreeSet để lưu trữ một tập hợp các số nguyên và thực hiện các thao tác sau:

- Thêm 5 số nguyên vào TreeSet.
- In ra các số nguyên trong TreeSet (phải theo thứ tự tăng dần).
- Thử thêm các số trùng lặp và in lại kết quả.

Bài tập 5: Sử dụng Stack

Mô tả: Viết một chương trình sử dụng Stack để thực hiện các thao tác sau:

- Đẩy 5 phần tử vào Stack.
- In ra phần tử đầu tiên.
- Thực hiện một thao tác pop (xóa phần tử đầu tiên) và in ra phần tử vừa bị xóa.
- In lại các phần tử còn lại trong Stack.

Bài tập 6: Sử dụng Queue

Mô tả: Viết một chương trình sử dụng Queue (có thể dùng LinkedList làm Queue) để thực hiện các thao tác sau:

- Thêm 5 phần tử vào Queue.
- In ra phần tử đầu tiên.
- Thực hiện một thao tác dequeue (xóa phần tử đầu tiên) và in ra phần tử vừa bị xóa.
- In lại các phần tử còn lại trong Queue.

Bài tập 7: Đọc dữ liệu từ tệp sử dụng InputStream

Mô tả: Viết một chương trình Java đọc một tệp văn bản sử dụng FileInputStream và in ra nội dung của tệp đó.

- Tạo một tệp văn bản với nội dung bất kỳ.
- Sử dụng FileInputStream để đọc nội dung tệp và in ra màn hình.

Bài tập 8: Ghi dữ liệu vào tệp sử dụng OutputStream

Mô tả: Viết một chương trình Java ghi một chuỗi văn bản vào một tệp sử dụng FileOutputStream.

- Tạo một chuỗi văn bản.
- Sử dụng FileOutputStream để ghi chuỗi đó vào một tệp.
- Kiểm tra lại nội dung tệp sau khi ghi.

Bài tập 9: Sử dụng ObjectOutputStream và ObjectInputStream

Mô tả: Viết một chương trình Java sử dụng ObjectOutputStream để ghi một đối tượng vào tệp và ObjectInputStream để đọc lại đối tượng đó từ tệp.

- Tạo một lớp SinhVien với các thuộc tính như tên và mã sinh viên.
- Ghi đối tượng SinhVien vào tệp.
- Đọc lại đối tượng từ tệp và in thông tin của sinh viên ra màn hình.

Bài tập 10: Tổ chức mã nguồn theo mô hình hướng đối tượng

Mô tả: Viết một chương trình Java để quản lý thông tin sinh viên, bao gồm các lớp và giao diện (interface) sau:

- Lớp SinhVien với các thuộc tính mã sinh viên, tên, điểm.
- Giao diện QuảnLýSinhVien với các phương thức: thêm sinh viên, xóa sinh viên, tìm kiếm sinh viên theo mã.
- Lớp QuanLySinhVienImpl triển khai giao diện QuảnLýSinhVien và thực hiện các thao tác trên danh sách sinh viên (dùng ArrayList).
- Sử dụng JavaDoc để tạo tài liệu API cho các lớp và phương thức.

TÀI LIỆU THAM KHẢO

- [1]. Phạm Văn Trung, Phạm Thị Minh Thương, Bùi Công Thành, Phạm Văn Thọ (2023), *Lập trình Java căn bản*, NXB Xây Dựng.
- [2]. Nguyễn Văn Hậu, Đào Anh Hiền, Nguyễn Hữu Đông (2024), *Giáo trình Lập trình Hướng đối tượng*, NXB Khoa học và Kỹ thuật.
- [3]. Matt Weisfeld (2019), *Object-Oriented Thought Process, 5th Edition*, Addison-Wesley Professional.
- [4]. Cay S. Horstmann (2024), *Core Java Volume I - Fundamentals, 12th Edition*, Pearson Kindle Edition.
- [5]. Joshua Bloch (2018), *Effective Java 3rd Edition*, Addison-Wesley Professional.
- [6]. Kathy Sierra, Bert Bates, Trisha Gee (2022), *Head First Java: A Brain-Friendly Guide 3rd Edition*, O'Reilly Media